Evaluating Machine Learning Models

How to measure, compare and trust models before and after they reach real users

MEASURE

Splits, metrics, calibration

PROVE

Statistics, slices, robustness

TRUST

Fairness, online tests, monitoring

GETTING STARTED

Why evaluation deserves a whole tutorial



A model is only as trustworthy as the evidence behind it. Evaluation is how that evidence is created.

The problem

Teams often report one impressive number that hides leakage, bias or a weak baseline.

The cost

Poorly evaluated models fail quietly in production, harming users and wasting investment.

The goal

Give you a rigorous, practical method for measuring models honestly from first split to live traffic.

GETTING STARTED

1 Data scientists

Who choose metrics, run experiments and must defend their results.

2 ML engineers

Who build evaluation pipelines, quality gates and monitoring.

3 Analysts

Who design A/B tests and interpret experiment results.

4 Product managers

Who must judge whether a model is good enough to launch.

5 Reviewers and auditors

Who assess fairness, robustness and compliance evidence.

6 Students

Who want to move beyond accuracy to professional evaluation practice.

GETTING STARTED

What you should already know

- What training and prediction mean
- The difference between classification and regression
- Basic Python and a library such as sklearn
- Reading simple charts and tables
- The idea of averages and percentages
- Helpful but optional: basic probability
- Helpful but optional: experiment tracking tools
- Helpful but optional: SQL for slicing data

GETTING STARTED

Learning outcomes

1 **Design sound splits**

Create training, validation and test sets that reflect real use and avoid leakage.

2 **Choose the right metric**

Match classification, regression, ranking and forecasting metrics to business goals.

3 **Judge probabilities**

Evaluate thresholds, calibration and the cost of different errors.

4 **Prove differences**

Use confidence intervals and significance testing to compare models honestly.

5 **Look deeper**

Run error analysis, slicing, fairness, robustness and uncertainty checks.

6 **Evaluate live**

Plan A/B tests and production monitoring that confirm real value.

GETTING STARTED

Roadmap of the twenty one sections

No.	Section	No.	Section
1	Foundations of evaluation	12	Evaluating LLMs and generative models
2	Data splitting and validation	13	Statistical rigour
3	Data leakage	14	Error analysis and slicing
4	Classification metrics	15	Fairness evaluation
5	Thresholds, probabilities, calibration	16	Robustness and uncertainty
6	Regression metrics	17	Interpretability checks
7	Imbalanced data	18	Online evaluation and A/B tests
8	Ranking and recommendation	19	Evaluation in production
9	Forecasting	20	End to end case study
10	Clustering and anomalies	21	Checklists, glossary and quiz
11	Language, vision and speech		



Each section opens with a divider and closes with a recap, an exercise and a preview of what comes next.

GETTING STARTED

How the slides are designed to be read

Definition slides	Explain one term with an example and a memory hook.
Comparison slides	Place two options side by side and end with a verdict.
Flow slides	Show a process as numbered boxes connected by arrows.
Practice slides	Code, checklists, pitfalls and knowledge checks.
Recap slides	Summarise, set an exercise and preview the next section.



Reading guide

Blue boxes carry core ideas. Orange boxes warn about traps. Yellow sticky notes hold tips worth remembering. Dark boxes give verdicts and next steps.

SECTION 1

01

Foundations of evaluation

What we are really measuring, why, and the mindset that keeps evaluation honest.

IN THIS SECTION

- What evaluation means
- Offline versus online
- Generalisation
- Overfitting and underfitting
- Baselines
- Linking metrics to value

SECTION 1 · FOUNDATIONS

What is model evaluation?

DEFINITION

Model evaluation is the systematic process of estimating how well a model will perform on the data and decisions it will face in real use, and whether that performance is good enough.

EXAMPLE

Before replacing a manual triage process, a hospital team checks how often the model ranks urgent cases correctly on recent records it has never seen, compared with current practice.

REMEMBER

Evaluation estimates future performance. It never just describes the past.

SECTION 1 · FOUNDATIONS

Why we evaluate

1 Select
Choose between candidate models, features and settings.

2 Decide
Judge whether a model is good enough to launch at all.

3 Communicate
Give stakeholders honest evidence about benefits and risks.

4 Protect
Detect harm to users or groups before it reaches them.

5 Improve
Find specific weaknesses to fix in the next iteration.

6 Monitor
Confirm performance holds once the model is live.

SECTION 1 · FOUNDATIONS

Offline versus online evaluation

OFFLINE

Measured on stored data

- Fast, cheap and repeatable
- Uses historical labels
- No risk to real users
- May not reflect behaviour change caused by the model

ONLINE

Measured with real users

- Captures true business impact
- Slower and more expensive
- Carries some user risk
- Needs careful experiment design

VERDICT Use offline evaluation to filter and refine candidates, then confirm the winner online. Neither replaces the other.

SECTION 1 · FOUNDATIONS

Generalisation

DEFINITION

Generalisation is a model's ability to perform well on new data drawn from the same situation it will be used in, not just on the examples it learned from.

EXAMPLE

A spam filter that scores perfectly on its training emails but misses half of next week's spam has memorised rather than generalised.

REMEMBER

Every evaluation decision exists to estimate generalisation honestly.

SECTION 1 · FOUNDATIONS

Overfitting and underfitting



Good fit

Low error on training data and on new data. The goal.

Overfitting

Low training error but high error on new data. The model memorised noise.

Leakage warning

Suspiciously low error everywhere. Check for leaked information.

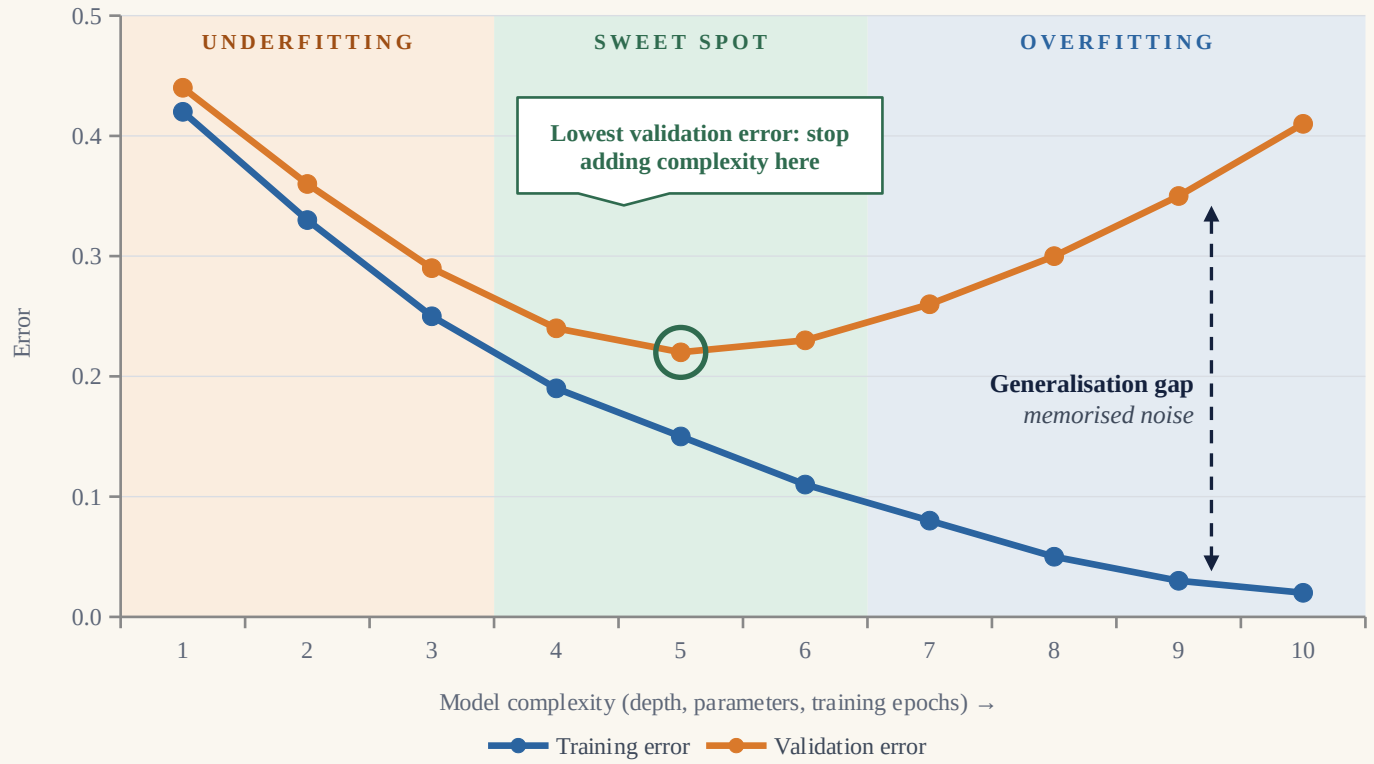
Underfitting

High error on both. The model is too simple or features are weak.

Vertical axis: Error on training data (low at top)

Horizontal axis: Error on new data (low at left)

Overfitting, drawn as two error curves



THE EXAM ANALOGY

UNDERFITTING

Skimmed the notes

Fails the practice papers and the real exam. The model is too simple for the pattern.

GOOD FIT

Understood the ideas

Scores well on practice papers and on questions never seen before.

OVERFITTING

Memorised past papers

Perfect on familiar questions, lost on new ones. The training score is lying.

! Watch the validation curve, not the training curve. Once it turns upward, extra complexity is buying memorisation, not skill.

SECTION 1 · FOUNDATIONS

Always start with a baseline

A metric means nothing in isolation. A baseline shows what performance is achievable with no model or a very simple one.

Trivial baselines

Predict the most common class, the average value or yesterday’s value.

Simple models

Logistic regression, a small decision tree or a rule written by an expert.

Current process

The existing human or rules based decision, which the model must beat to be worth it.

SECTION 1 · FOUNDATIONS

From business goal to evaluation metric

01

Goal

State the outcome the business wants, such as fewer missed frauds.

02

Decision

Describe the action the prediction triggers.

03

Costs

Estimate the cost of each type of mistake.

04

Metric

Pick metrics that reflect those costs and actions.

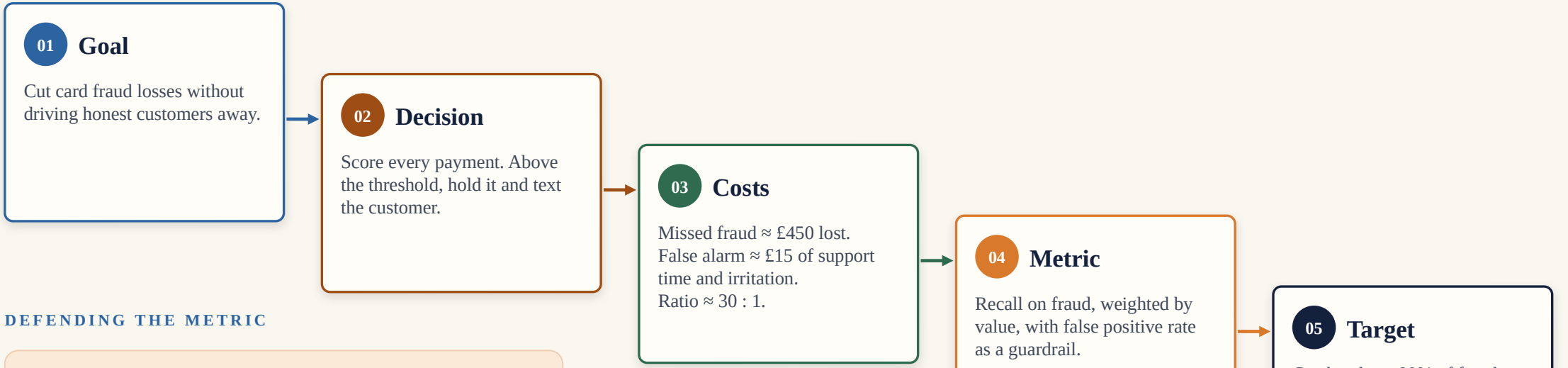
05

Target

Agree the level that counts as good enough.

If you cannot explain why a metric matters to the business, it is probably the wrong metric.

Goal to metric, worked through for a fraud model



DEFENDING THE METRIC

Stakeholder: Why report recall instead of accuracy? Accuracy is 99.6%.

You: Fraud is under 1% of payments, so 'approve everything' also scores 99%. A missed fraud costs about 30 times a false alarm, so what we must maximise is the share of fraud we catch.

! Put the cost ratio on the slide. That one line justifies the metric, the threshold and the target.

Levels of evaluation

Technical	Does the model fit the data? Loss and training curves.
Predictive	How accurate is it on unseen data? Core metrics.
Behavioural	Does it behave sensibly across slices and edge cases?
Responsible	Is it fair, robust, calibrated and explainable?
Business	Does it improve the outcome that matters, measured live?



Climb the ladder

Good teams evaluate at every level. A model can top the leaderboard at the lowest level while failing badly at the highest.

SECTION 1 · FOUNDATIONS

Foundational mistakes

PITFALL	BETTER PRACTICE
⚠ Reporting one headline metric only.	✓ Report several metrics, a baseline and confidence intervals.
⚠ Choosing the metric after seeing results.	✓ Agree metrics and targets before running experiments.
⚠ Evaluating on data the model has seen.	✓ Hold out data that stays untouched until final testing.
⚠ Ignoring the cost of different errors.	✓ Weigh mistakes by their real world consequences.

SECTION 1 · FOUNDATIONS

Knowledge check

? A model reaches 99 percent accuracy at detecting a disease that affects 1 percent of patients. Is it impressive?

MODEL ANSWER

Not necessarily. A model that always predicts no disease also scores 99 percent accuracy while finding nobody. You must compare against this baseline and look at how many real cases it detects and how many alarms are false.

WHY IT MATTERS

It shows why baselines and the right metric matter more than a big number.

SECTION 1 · FOUNDATIONS

Section recap

KEY TAKEAWAYS

- Evaluation estimates performance on future, unseen data
- Offline evaluation filters, online evaluation confirms
- Watch for overfitting, underfitting and leakage
- Always compare against simple baselines
- Derive metrics from business goals and error costs
- Evaluate at technical, predictive, responsible and business levels

TRY THIS

For a model you know, write its business goal, the decision it drives, the cost of each error and the metric you would choose.

NEXT UP

Section 2: splitting data and validating models correctly.

SECTION 2

02

Data splitting and validation

Designing the data you test on so it honestly represents the data you will face.

IN THIS SECTION

- Training, validation, test
- Random and stratified splits
- Time based splits
- Group splits
- Cross validation
- Choosing split sizes

SECTION 2 · SPLITTING AND VALIDATION

Three sets, three jobs

Training set	Used to fit model parameters. The model sees it many times.
Validation set	Used to compare models, tune settings and choose thresholds.
Test set	Used once to estimate final performance on unseen data.
Optional extra	A fresh, later sample or external dataset to test transfer.

Golden rule

The test set is opened once, at the end. Every time you look at it and adjust, it becomes a validation set and stops being an honest estimate.

Three sets, explained as one student's school year



TRAINING

The textbook exercises

Used to fit the parameters. The model works through these again and again.

SEEN THOUSANDS OF TIMES

VALIDATION

The mock exams

Used to compare models, tune settings and pick thresholds. Every choice leaks a little of it into the model.

SEEN MANY TIMES

TEST

The sealed final exam

Opened once, at the very end, to estimate how the model will do on data it has never met.

OPENED ONCE

OPTIONAL EXTRA

The external examiner

A later or outside sample that checks the result still holds somewhere new.

ONCE, LATER

RULE Each time a test score changes a decision, the test set quietly becomes a validation set. When that happens, collect a fresh one.

Proportions are typical, not fixed: what matters is enough examples of the rarest important case in each set.

SECTION 2 · SPLITTING AND VALIDATION

Random versus stratified splits

RANDOM

Shuffle and cut

- Each row assigned at random
- Fine for large, balanced datasets
- Rare classes may vanish from small sets
- Simple and widely used

STRATIFIED

Preserve class proportions

- Each set keeps the same class balance
- Essential for imbalanced problems
- More stable metric estimates
- Easy option in most libraries

VERDICT Stratify by the target for classification by default, especially when any class is rare.

Time based splits

DEFINITION

A time based split trains on earlier data and tests on later data, so evaluation mimics how the model will be used: predicting a future it has not seen.

EXAMPLE

A demand model is trained on January to September sales and tested on October to December. A random split would let it learn from December while predicting November.

REMEMBER

If the world changes over time, random splits flatter your model.

Three ways to cut one year of sales data



! Make the test window look like the period the model will serve, including its seasonality and promotions.

SECTION 2 · SPLITTING AND VALIDATION

Group splits

DEFINITION

A group split keeps every record belonging to one entity, such as a patient, customer or device, in the same set, so the model cannot recognise individuals instead of learning patterns.

EXAMPLE

A medical image model sees several scans per patient. If one patient’s scans appear in both training and test, the model can succeed by recognising that patient.

REMEMBER

Split by the unit you will predict for in the future.

How k fold cross validation works

01

Divide

Split the data into k equal parts, called folds.

02

Hold one

Keep one fold aside for validation.

03

Train

Fit the model on the remaining folds.

04

Repeat

Rotate so every fold is used once for validation.

05

Summarise

Report the average and spread across folds.

Five or ten folds are common. The spread across folds is as informative as the average: it shows how stable the model is.

Five folds, five rounds, one picture

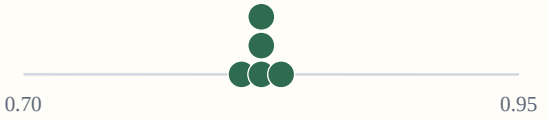
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Score
Round 1	validate	train	train	train	train	0.81
Round 2	train	validate	train	train	train	0.84
Round 3	train	train	validate	train	train	0.79
Round 4	train	train	train	validate	train	0.86
Round 5	train	train	train	train	validate	0.82

Mean 0.824
spread (SD) ± 0.027

Every row is a fresh model trained from scratch on the four blue folds.

SAME MEAN, DIFFERENT STORY

Model A 0.82 ± 0.007



Tight cluster: dependable.

Model B 0.83 ± 0.08



Wide scatter: the 0.92 fold is luck.

! Report the mean and the spread. When spreads overlap this much, a 0.01 gap in means is not a win.

Cross validation variants

Variant	What it does	Use when
Stratified k fold	Keeps class balance in each fold	Classification, imbalanced data
Group k fold	Keeps each group in one fold	Repeated entities like patients
Time series split	Folds move forward in time	Forecasting and drifting data
Repeated k fold	Repeats with new shuffles	Small data, unstable estimates
Leave one out	Each row is its own fold	Very small datasets only
Nested	Inner loop tunes, outer loop scores	Tuning plus honest estimates

SECTION 2 · SPLITTING AND VALIDATION

Why nested cross validation matters

When you tune hyperparameters with cross validation and then report that same score, the score is optimistic because tuning chose the luckiest settings.

Inner loop

Searches for the best settings using only the outer training folds.

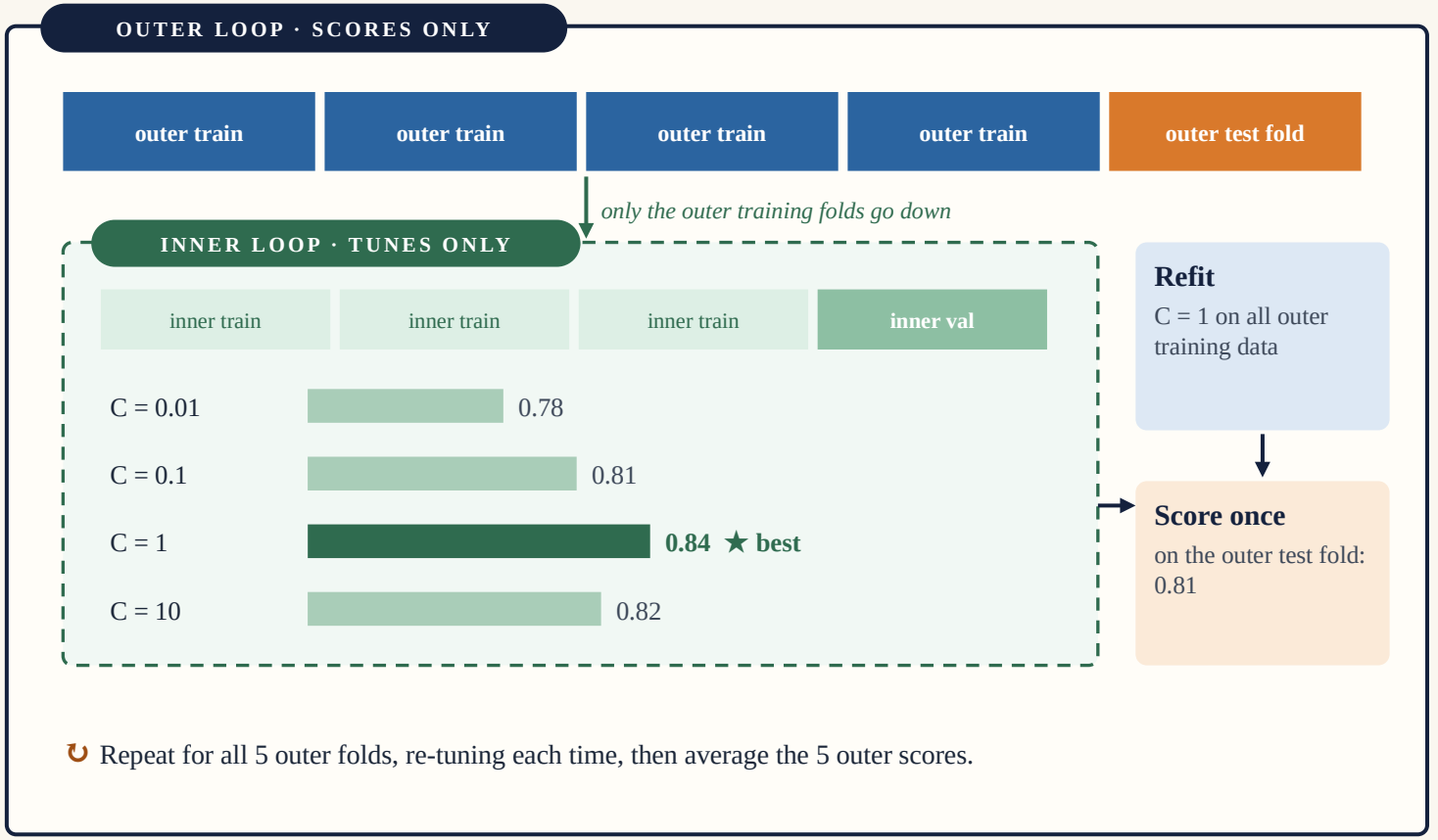
Outer loop

Scores the tuned model on folds the tuning never saw.

Result

An honest estimate of how the whole tuning procedure will perform.

Nested cross validation: a loop inside a loop



WHAT YOU WOULD REPORT

NON-NESTED

0.84

The best inner score: tuning picked the luckiest setting.

NESTED

0.81

Average of outer folds that tuning never touched.

REMEMBER

Nested CV scores the whole tuning procedure, not one fixed setting. The 0.03 gap is optimism.

Choosing split sizes

Dataset size	Typical approach	Reason
Hundreds of rows	Repeated cross validation	Every row is precious
Thousands	Cross validation plus small test set	Balance stability and honesty
Hundreds of thousands	Holdout of a few percent	Even small shares are large
Millions or more	Fixed holdout and fresh samples	Compute cost dominates



What matters is the number of examples of the rarest important case in each set, not the percentage.

Splitting pitfalls

PITFALL	BETTER PRACTICE
⚠ Random splits on time ordered data.	→ ✓ Train on the past and test on the future.
⚠ Duplicates spread across training and test.	→ ✓ Remove duplicates or split by group before evaluating.
⚠ Tuning repeatedly against the test set.	→ ✓ Tune on validation folds and open the test set once.
⚠ Preprocessing fitted on all data before splitting.	→ ✓ Fit scalers and encoders only on training folds.

Knowledge check

? A churn model uses three years of monthly customer snapshots. How should you split the data?

MODEL ANSWER

Use a time based split with groups in mind: train on earlier months, validate on a later period and test on the most recent months. Make sure a customer’s future snapshots never inform the model when scoring their past, and check results on customers new to the test period.

WHY IT MATTERS

It combines time and group thinking, which real data usually needs.

Section recap

KEY TAKEAWAYS

- Training, validation and test sets have distinct jobs
- Stratify for imbalanced classification
- Split by time when the future differs from the past
- Split by group to stop memorising entities
- Use cross validation for stable estimates
- Nest it when tuning to stay honest

TRY THIS

Review a past project's split. Identify any time, group or duplicate issue and estimate how it might have inflated results.

NEXT UP

Section 3: data leakage, the silent killer of evaluations.

SECTION 3

03

Data leakage

The most common reason a model looks brilliant in testing and fails in the real world.

IN THIS SECTION

- What leakage is
- Target leakage
- Train test contamination
- Temporal leakage
- Detecting leakage
- Prevention habits

“

If your results look too good to be true, assume leakage until you prove otherwise.

☆ Leakage gives the model access during evaluation to information it will not have when making real predictions.

SECTION 3 · DATA LEAKAGE

Target leakage

DEFINITION

Target leakage happens when a feature contains information created after, or because of, the outcome you are trying to predict.

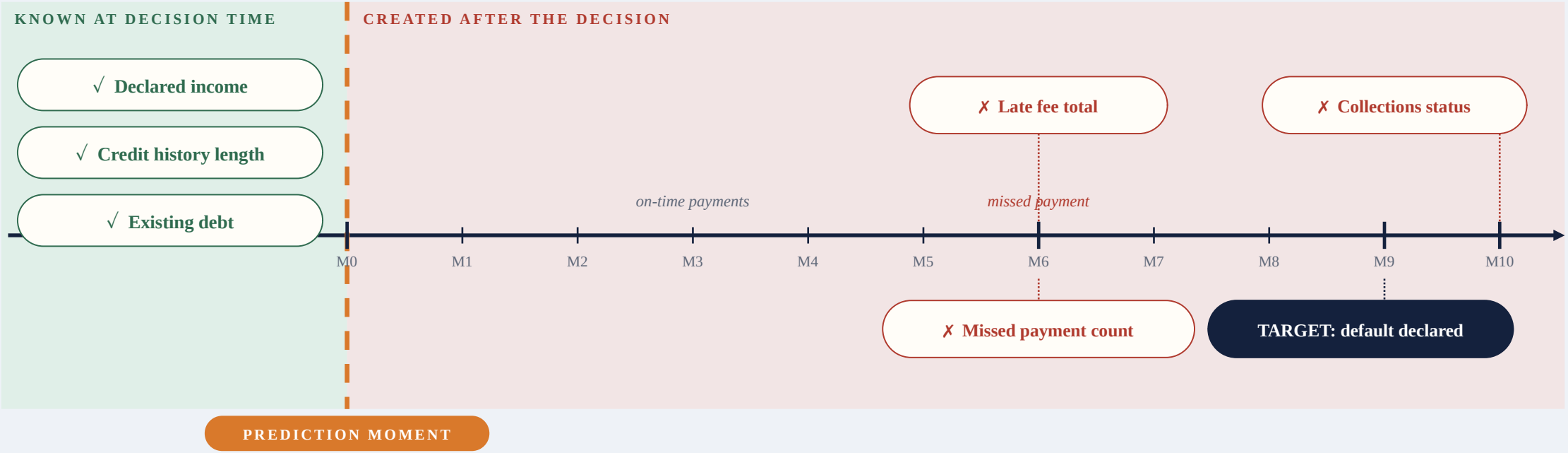
EXAMPLE

A model predicting loan default uses a field called collections status. Accounts only enter collections after default, so the feature reveals the answer.

REMEMBER

Ask of every feature: would I truly know this at prediction time?

Target leakage on the life of one loan



THE AVAILABILITY TEST

Would this value exist, with this exact content, at the prediction moment?

YES → KEEP

NO → DROP

Check the timestamp the value was written, not the date it describes.

WHY IT LEAKS

Accounts enter collections only after default. The feature is the label in disguise.

SECTION 3 · DATA LEAKAGE

Train test contamination

DEFINITION

Contamination happens when information from the test data influences training, for example through shared preprocessing, duplicates or feature selection done on all data.

EXAMPLE

A team scales features using the mean of the full dataset, then splits it. The test set has quietly shaped the training data.

REMEMBER

Split first. Fit every transformation only on training data.

SECTION 3 · DATA LEAKAGE

Temporal leakage

Temporal leakage uses information from the future relative to the moment a prediction would actually be made.

Future aggregates

A customer’s average spend computed over the whole year, including months after the prediction date.

Late corrections

Data fixed or backfilled later than it would have been available in real time.

Fix

Compute every feature as of a point in time, using only data available then.

SECTION 3 · DATA LEAKAGE

Warning signs of leakage

1 Near perfect scores

Performance far above published results or expert intuition.

2 One dominant feature

A single feature carries most of the predictive power.

3 Big offline online gap

Strong offline results collapse once the model goes live.

4 Strange feature names

Fields like status, outcome, resolution or final date.

5 Unstable across time

Performance drops sharply on newer periods.

6 Identifiers predict well

IDs or timestamps help the model, hinting at hidden structure.

SECTION 3 · DATA LEAKAGE

A leakage audit

1. **Map the timeline.** Mark when each feature is created relative to the prediction moment.
2. **Inspect top features.** Review the most important features with a domain expert.
3. **Drop and compare.** Remove suspicious features and see how much performance falls.
4. **Check duplicates.** Search for identical or near identical rows across sets.
5. **Recreate pipeline.** Confirm preprocessing is fitted inside each training fold only.
6. **Test forward.** Score on a later period that the model could not have seen.

Preprocessing inside the pipeline

no_leak_cv.py

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold

pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc")
print(scores.mean(), scores.std())
```

Inside each fold

The scaler is refit on training folds only, so validation data never shapes it.

Report spread

The standard deviation across folds shows stability, not just the average.

Same for selection

Feature selection and resampling also belong inside the pipeline.

Where you fit the scaler decides whether the test is honest

LEAKY ORDER

1. All rows together
2. **Fit the scaler on every row (test rows included)**
3. Split into training and test
4. Train and score: the test set has already shaped the features

TINY EXAMPLE

Training values
10, 20, 30

Test value
100

Mean used for scaling
40

≠

SAFE ORDER

1. Split into training and test first
2. Fit the scaler on training rows only
3. Transform the test set with the training mean and SD
4. Train and score: the test set stays unseen

TINY EXAMPLE

Training values
10, 20, 30

Test value
100

Mean used for scaling
20

```
make_pipeline(StandardScaler(), LogisticRegression())
```
← *cross_val_score refits the scaler inside every fold, so the safe order happens automatically*

FIT INSIDE EACH TRAINING FOLD, NEVER BEFORE THE SPLIT

Imputation · Scaling · Encoding · Feature selection · Resampling · Target encoding

SECTION 3 · DATA LEAKAGE

Leakage pitfalls

PITFALL	BETTER PRACTICE
⚠ Oversampling before splitting the data.	✓ Resample only inside training folds.
⚠ Selecting features using the whole dataset.	✓ Run feature selection within each fold.
⚠ Target encoding with the full target column.	✓ Use out of fold target encoding.
⚠ Trusting a feature because it is in the warehouse.	✓ Confirm when each value becomes available.

SECTION 3 · DATA LEAKAGE

Section recap

KEY TAKEAWAYS

- Leakage lets evaluation see information unavailable in real use
- Target leakage uses features caused by the outcome
- Contamination comes from shared preprocessing and duplicates
- Temporal leakage borrows from the future
- Suspiciously good results demand an audit
- Pipelines keep transformations inside each fold

TRY THIS

Take your current model's top ten features and write, for each, exactly when its value becomes available.

NEXT UP

Section 4: classification metrics.

SECTION 4

04

Classification metrics

Counting right and wrong answers in the ways that actually matter for decisions.

IN THIS SECTION

- The confusion matrix
- Accuracy and its limits
- Precision and recall
- Specificity and F1
- Balanced metrics
- Multiclass averaging

SECTION 4 · CLASSIFICATION

The confusion matrix

	Model predicts positive	Model predicts negative
Truly positive	True positive: correct alarm	False negative: missed case
Truly negative	False positive: false alarm	True negative: correct quiet

Almost every classification metric is a different way of summarising these four counts. Always look at the counts themselves, not just the summaries.

SECTION 4 · CLASSIFICATION

One confusion matrix, six metrics, all worked out

1,000 patients · 100 truly have the condition

	Predicted +	Predicted −
Truly +	TP **80** correct alarms	FN **20** missed cases
Truly −	FP **45** false alarms	TN **855** correct quiet

Colours in each formula match the cells they use.

Precision — 64%

TP ÷ (TP + FP)

80 ÷ 125

Of 125 alarms, 80 were real

Recall — 80%

TP ÷ (TP + FN)

80 ÷ 100

Of 100 real cases, 80 were caught

F1 score — 0.71

2 × P × R ÷ (P + R)

2 × 0.64 × 0.80 ÷ 1.44

Pulled toward the weaker of P and R

Accuracy — 93.5%

(TP + TN) ÷ all

935 ÷ 1,000

Flattered by 855 easy negatives

Specificity — 95%

TN ÷ (TN + FP)

855 ÷ 900

Healthy people correctly left alone

False positive rate — 5%

FP ÷ (FP + TN)

45 ÷ 900

Equal to 1 − specificity

TAKEAWAY Accuracy says 93.5%, yet 45 of 125 alarms are false. Always show the four counts beside the summaries.

SECTION 4 · CLASSIFICATION

Accuracy

DEFINITION

Accuracy is the share of all predictions that are correct, counting both positives and negatives together.

EXAMPLE

A model makes 1,000 predictions and gets 920 right, so its accuracy is 92 percent.

REMEMBER

Accuracy is misleading when one class is much more common than the other.

SECTION 4 · CLASSIFICATION

Precision

DEFINITION

Precision is the share of items the model flagged as positive that really are positive. It answers: when the model raises an alarm, how often is it right?

EXAMPLE

A fraud model flags 200 payments and 150 are genuinely fraudulent, so precision is 75 percent. The other 50 are genuine customers inconvenienced.

REMEMBER

High precision means few false alarms.

SECTION 4 · CLASSIFICATION

Recall, also called sensitivity

DEFINITION

Recall is the share of all truly positive items that the model successfully found. It answers: of everything we needed to catch, how much did we catch?

EXAMPLE

There were 300 fraudulent payments and the model caught 150, so recall is 50 percent. Half of the fraud slipped through.

REMEMBER

High recall means few missed cases.

SECTION 4 · CLASSIFICATION

The precision and recall trade off

FAVOUR PRECISION

When false alarms are costly

- Spam filters hiding real email
- Automatic account suspensions
- Expensive manual investigations
- Recommending products to buy

FAVOUR RECALL

When misses are costly

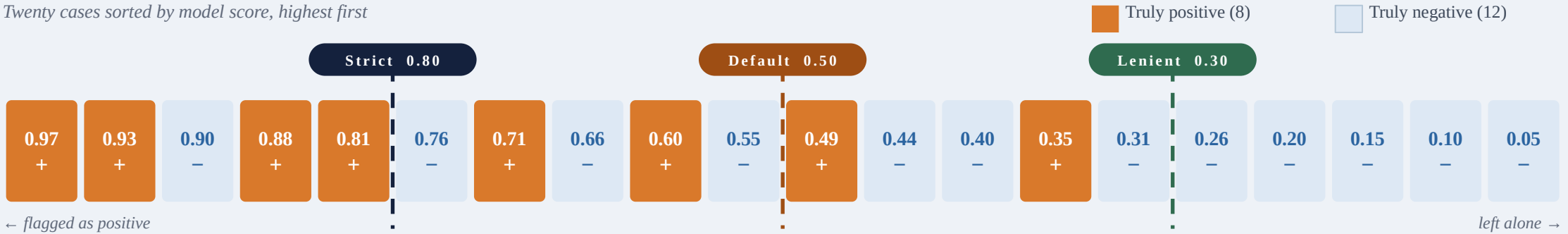
- Cancer screening
- Fraud and security threats
- Safety defects in manufacturing
- Child safety content detection

VERDICT You rarely get both for free. Raising the decision threshold usually lifts precision and lowers recall, so let error costs choose the balance.

SECTION 4 · CLASSIFICATION

Slide the threshold and watch precision and recall swap places

Twenty cases sorted by model score, highest first



Strict 0.80 — flags 5 · catches 4 of 8

Precision	4 ÷ 5	**80%**
Recall	4 ÷ 8	**50%**

Default 0.50 — flags 10 · catches 6 of 8

Precision	6 ÷ 10	**60%**
Recall	6 ÷ 8	**75%**

Lenient 0.30 — flags 15 · catches 8 of 8

Precision	8 ÷ 15	**53%**
Recall	8 ÷ 8	**100%**

TRADE OFF Moving the line right raises recall (50% → 100%) and lowers precision (80% → 53%). The cost of each error type decides where it stops.

SECTION 4 · CLASSIFICATION

More metrics from the confusion matrix

1 **Specificity**

Share of true negatives correctly left alone. The recall of the negative class.

2 **False positive rate**

Share of negatives wrongly flagged. One minus specificity.

3 **F1 score**

A single score that balances precision and recall, punishing either being low.

4 **F beta score**

Like F1 but weights recall more or less heavily than precision.

5 **Balanced accuracy**

Average of recall on each class, fair to rare classes.

6 **Matthews correlation**

Uses all four cells, robust when classes are imbalanced.

SECTION 4 · CLASSIFICATION

Understanding the F1 score

F1 combines precision and recall into one number that is high only when both are high. It behaves like a cautious average that is pulled down by the weaker of the two.

Strength

Useful when you care about the positive class and need one number for comparison.

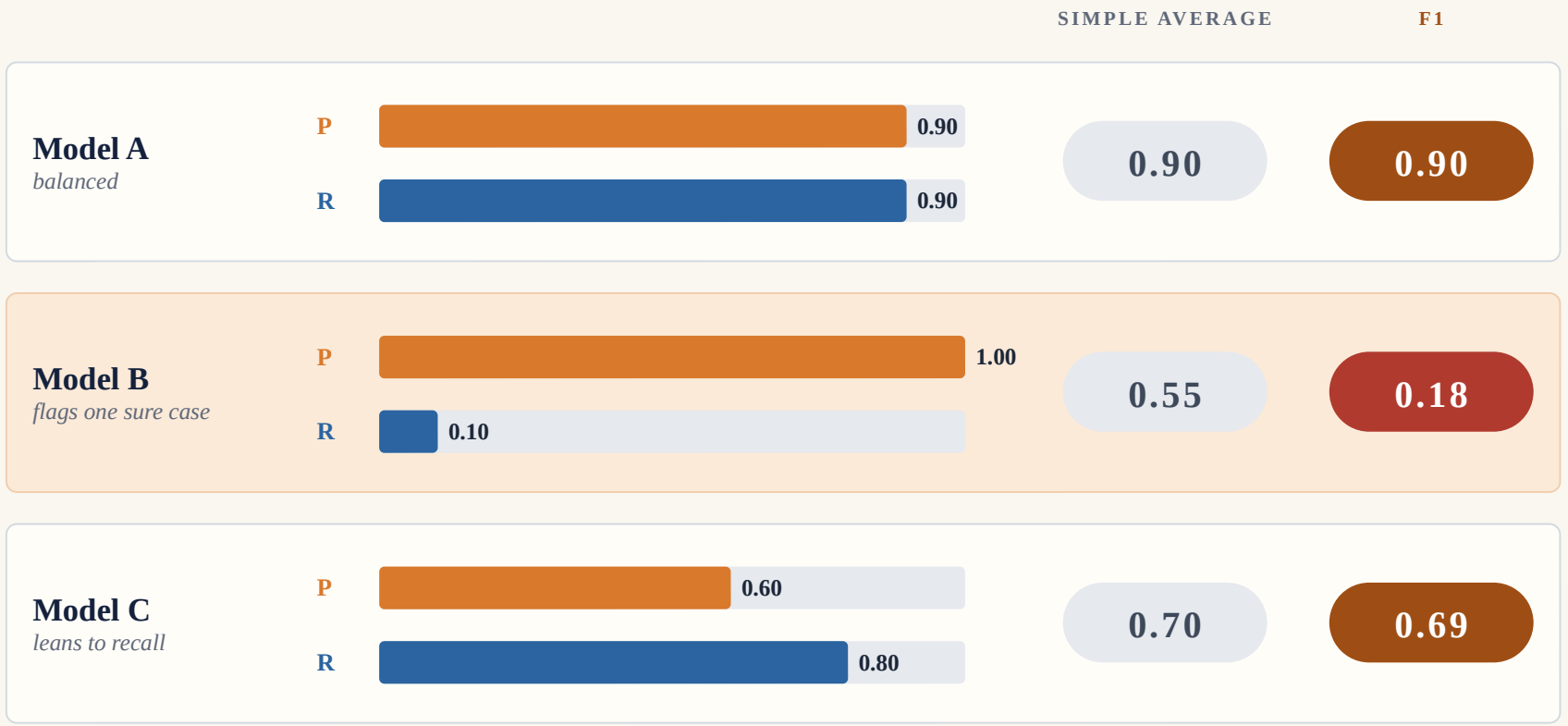
Weakness

Ignores true negatives and assumes precision and recall matter equally.

Tip

Report precision and recall beside F1 so readers can see which one is weak.

Why F1 uses a harmonic mean, not a simple average



Equal P and R: both averages agree. One collapses: only F1 notices.

THE FORMULA

F1 = 2PR ÷ (P + R)

Model B: 2 × 1.0 × 0.1 ÷ 1.1 = 0.18

NEED A TILT?

Use F beta

β = 2 counts recall twice as much as precision. β = 0.5 favours precision.

REMEMBER

Print P and R beside F1, so readers can see which one is weak.

SECTION 4 · CLASSIFICATION

Cohen’s kappa and agreement

Kappa measures how much a model agrees with the true labels beyond what chance agreement alone would produce.

Why useful

Corrects for the agreement you would get by guessing in proportion to class frequencies.

Also for humans

Commonly used to check how consistently human annotators label data.

Read it as

Near zero means chance level agreement. Near one means near perfect agreement.

SECTION 4 · CLASSIFICATION

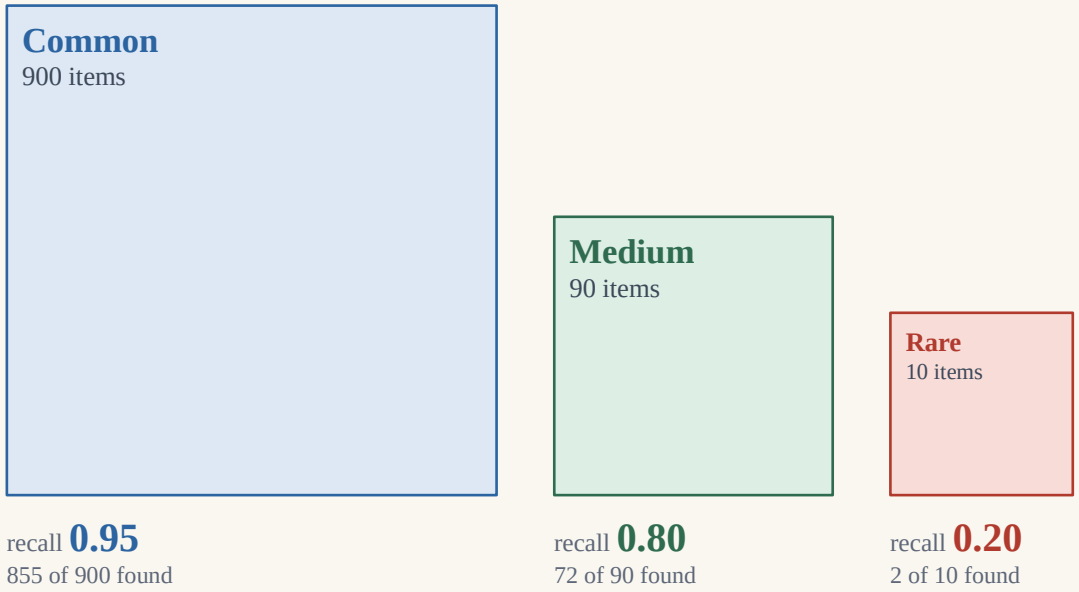
Averaging metrics across many classes

Averaging	How it works	Best when
Macro	Compute per class, then average equally	Every class matters equally
Weighted	Average per class weighted by class size	Reflecting overall population
Micro	Pool all decisions, then compute once	Overall decision quality matters
Per class	Report each class separately	Always, alongside any average

Macro averages expose weak performance on rare classes that micro and weighted averages can hide.

Macro, weighted and micro averages on one three-class example

BOX SIZE SHOWS HOW MANY ITEMS EACH CLASS HAS



recall **0.95**
855 of 900 found

recall **0.80**
72 of 90 found

recall **0.20**
2 of 10 found

REMEMBER

Macro is the only average here that notices the rare class failing.

Micro

Pool every decision, then compute once

(855 + 72 + 2) ÷ 1,000

0.929

Equals accuracy in single-label tasks

Weighted

Average per class, weighted by class size

0.90 × 0.95 + 0.09 × 0.80 + 0.01 × 0.20

0.929

Mirrors the population mix

Macro

Average per class, every class counts equally

(0.95 + 0.80 + 0.20) ÷ 3

0.65

Drags the rare-class failure into view

! Always print the per-class table beside any average: 0.95, 0.80 and 0.20 tell the real story.

SECTION 4 · CLASSIFICATION

Multilabel and top k accuracy

Some tasks allow several correct labels per item, or let the user choose among several suggestions, so plain accuracy is not enough.

Multilabel

Evaluate each label as its own yes or no task, then average, and report exact match rate.

Top k accuracy

Counts a prediction correct if the true label appears among the k most likely options.

Hamming loss

Share of individual label decisions that are wrong across all items.

Choosing a classification metric

Situation	Sensible primary metric
Balanced classes, equal error costs	Accuracy plus per class recall
False alarms are expensive	Precision at an agreed recall level
Missed cases are dangerous	Recall at an agreed precision level
Rare positive class	Precision recall curve area, F1, MCC
Many classes of equal importance	Macro averaged F1
Users choose from suggestions	Top k accuracy

SECTION 4 · CLASSIFICATION

Classification metric pitfalls

PITFALL	BETTER PRACTICE
Reporting accuracy on imbalanced data.	Report precision, recall and a baseline.
Quoting F1 without precision and recall.	Show both components alongside F1.
Using micro averages to hide weak classes.	Add macro averages and per class results.
Comparing metrics at different thresholds.	Fix the threshold or compare full curves.

SECTION 4 · CLASSIFICATION

Section recap

KEY TAKEAWAYS

- The confusion matrix underlies most classification metrics
- Precision limits false alarms, recall limits misses
- Error costs decide the right balance
- F1, balanced accuracy and MCC summarise imbalanced results
- Choose macro, micro or weighted averages deliberately
- Always report per class results

TRY THIS

Compute the confusion matrix, precision, recall, F1 and balanced accuracy for one of your models, then explain each number in one sentence.

NEXT UP

Section 5: thresholds, probabilities and calibration.

SECTION 5

05

Thresholds, probabilities and calibration

Evaluating the scores behind decisions, not just the decisions themselves.

IN THIS SECTION

- Scores and thresholds
- ROC curves and AUC
- Precision recall curves
- Choosing a threshold
- Log loss and Brier score
- Calibration

SECTION 5 · PROBABILITIES

Scores versus decisions

Most classifiers output a score or probability. A threshold turns that score into a yes or no decision, and different thresholds give different metrics.

Ranking quality

How well the scores order positives above negatives, regardless of threshold.

Decision quality

How good the yes or no choices are at one chosen threshold.

Probability quality

Whether a score of 70 percent really means 7 in 10 such cases are positive.

SECTION 5 · PROBABILITIES

The ROC curve

DEFINITION

The ROC curve plots the true positive rate against the false positive rate as the decision threshold moves from strict to lenient, showing every trade off the model offers.

EXAMPLE

At a strict threshold a screening model catches 40 percent of cases with 2 percent false alarms. At a lenient one it catches 90 percent with 20 percent false alarms.

REMEMBER

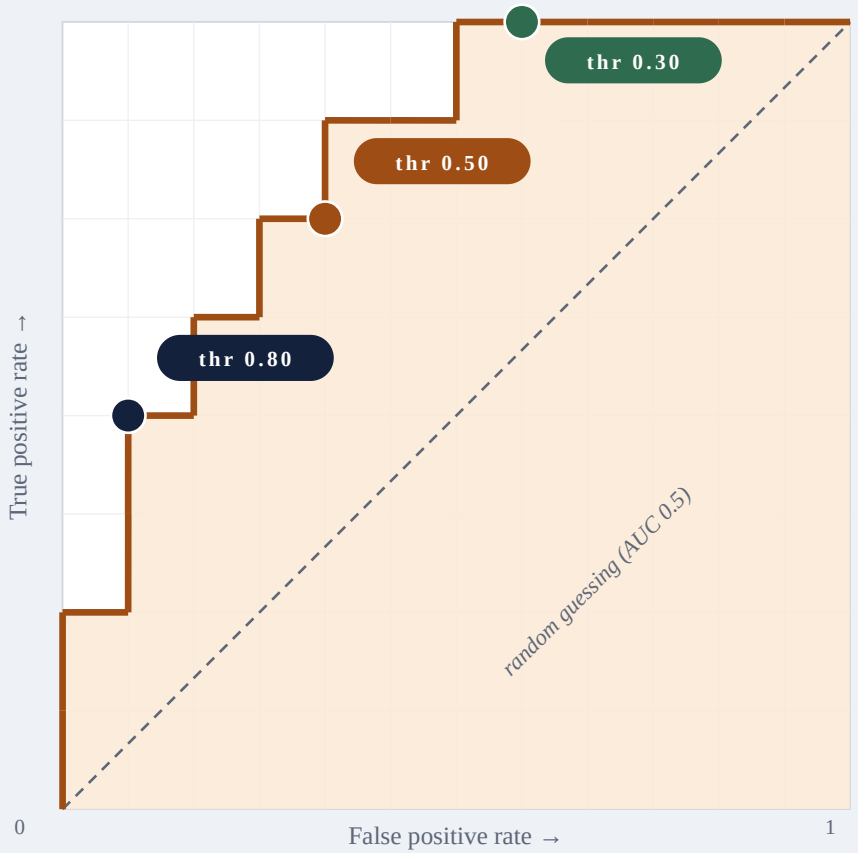
A curve that hugs the top left corner is better. The diagonal means random guessing.

Building an ROC curve by walking down the ranking

WALK DOWN THE RANKING

0.97 + ↑ up ⅛	**0.49** + ↑ up ⅛
0.93 + ↑ up ⅛	**0.44** − → right 1/12
0.90 − → right 1/12	**0.40** − → right 1/12
0.88 + ↑ up ⅛	**0.35** + ↑ up ⅛
0.81 + ↑ up ⅛	**0.31** − → right 1/12
0.76 − → right 1/12	**0.26** − → right 1/12
0.71 + ↑ up ⅛	**0.20** − → right 1/12
0.66 − → right 1/12	**0.15** − → right 1/12
0.60 + ↑ up ⅛	**0.10** − → right 1/12
0.55 − → right 1/12	**0.05** − → right 1/12

Same 20 cases as the threshold slide: 8 positives, 12 negatives.



SHADED AREA = AUC

0.82

79 of 96 positive–negative pairs are ranked in the right order.

READING THE CORNERS

Bottom left: flag nobody.
Top right: flag everybody.
Top left corner: perfect ranking.

REMEMBER

The curve shows every threshold at once. AUC never picks one for you.

SECTION 5 · PROBABILITIES

ROC AUC

DEFINITION

The area under the ROC curve summarises ranking quality as one number: the chance that a randomly chosen positive gets a higher score than a randomly chosen negative.

EXAMPLE

An AUC of 0.85 means that in 85 percent of random positive and negative pairs, the positive case is ranked higher.

REMEMBER

0.5 is random ranking, 1.0 is perfect ranking. AUC says nothing about calibration.

SECTION 5 · PROBABILITIES

ROC curve versus precision recall curve

ROC

True positive rate against false positive rate

- Insensitive to class balance
- Can look optimistic with rare positives
- Good for comparing ranking ability
- Summarised by ROC AUC

PRECISION RECALL

Precision against recall

- Focuses on the positive class
- Reveals false alarm burden with rare positives
- Good for imbalanced, alarm style tasks
- Summarised by average precision

VERDICT When positives are rare and false alarms matter, prefer the precision recall curve. Report both when in doubt.

SECTION 5 · PROBABILITIES

Choosing a decision threshold

1. **Never assume 0.5.** The default threshold is rarely the best one for your costs.
2. **Use validation data.** Choose thresholds on validation data, never the test set.
3. **Apply constraints.** For example, the highest recall with precision of at least 80 percent.
4. **Use costs.** Pick the threshold that minimises total expected cost of errors.
5. **Consider capacity.** Match the number flagged to how many cases staff can review.
6. **Recheck over time.** Thresholds drift as data and base rates change.

SECTION 5 · PROBABILITIES

Cost sensitive evaluation

Assign a realistic cost to each kind of error and benefit to each correct decision, then evaluate models by the total cost or value they would create.

Cost matrix

Write down the cost of a false positive and of a false negative with the business.

Expected value

Multiply counts by costs and add them up for each candidate threshold.

Communicates well

Money or hours saved is easier for stakeholders to understand than AUC.

Two fraud models, two receipts: let the cost matrix choose

AGREED COST MATRIX

	Flag	Pass
Fraud	**£0** fraud stopped	**£500** missed fraud
Genuine	**£20** false alarm call	**£0** smooth payment

A miss costs 25 times a false alarm.

Scenario: 10,000 transactions, 100 of them fraud.

NOTICE

Model B raises eight times as many alarms, yet costs £5,500 less.

Model A · cautious

accuracy 99.1% · precision 55% · recall 60%

Caught	60 frauds	£0
Missed	40 × £500	£20,000
Alarms	50 × £20	£1,000
TOTAL COST		**£21,000**

per 10,000 payments

Model B · eager

accuracy 95.9% · precision 18% · recall 85%

Caught	85 frauds	£0
Missed	15 × £500	£7,500
Alarms	400 × £20	£8,000
TOTAL COST		**£15,500**

per 10,000 payments

WINNER

VERDICT Accuracy and precision pick Model A. The business ledger picks Model B. Report the ledger.

SECTION 5 · PROBABILITIES

Scoring probabilities directly

1 Log loss

Heavily punishes confident wrong predictions. Lower is better.

2 Brier score

Average squared gap between predicted probability and outcome. Lower is better.

3 Why use them

They reward honest probabilities, not just correct rankings.

4 Proper scores

Both are proper scoring rules: the best score comes from true probabilities.

5 Compare to baseline

Always compare with predicting the base rate for everyone.

6 Sensitive to rare cases

A few very confident mistakes can dominate log loss.

SECTION 5 · PROBABILITIES

Calibration

DEFINITION

A model is well calibrated when its predicted probabilities match observed frequencies: among cases given 30 percent, about 30 percent turn out positive.

EXAMPLE

A weather model says 70 percent chance of rain on 100 days. If it rains on about 70 of them, the model is well calibrated.

REMEMBER

Good ranking does not guarantee good calibration. Check both.

SECTION 5 · PROBABILITIES

Reading a reliability diagram

01

Bin

Group predictions into bins by predicted probability.

02

Average

Compute the mean predicted probability in each bin.

03

Observe

Compute the actual share of positives in each bin.

04

Plot

Draw observed against predicted for every bin.

05

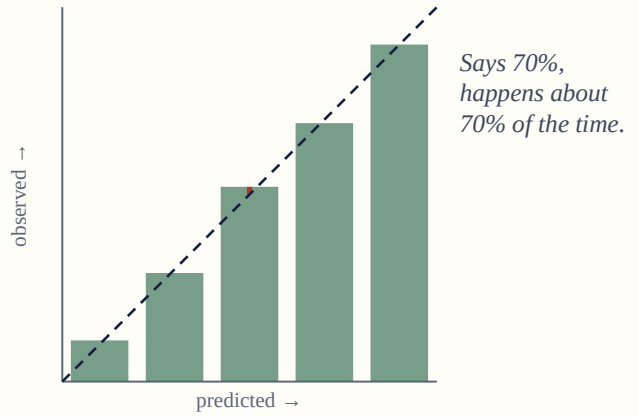
Judge

Points on the diagonal mean good calibration.

Points below the diagonal mean overconfidence. Points above it mean underconfidence. Expected calibration error summarises the average gap.

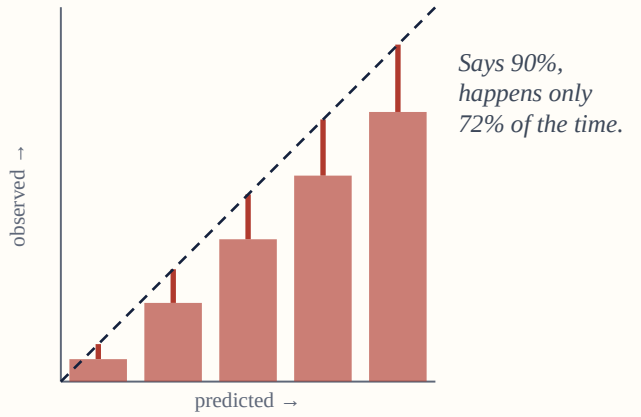
Three reliability diagrams and how to read the gaps

Well calibrated



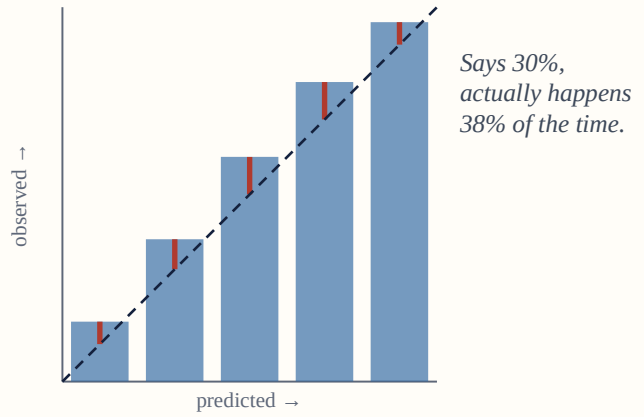
Says 70%, happens about 70% of the time.

Overconfident



Says 90%, happens only 72% of the time.

Underconfident



Says 30%, actually happens 38% of the time.

EXPECTED CALIBRATION ERROR, WORKED FOR THE OVERCONFIDENT MODEL

Predicted (bin mean)	.10	.30	.50	.70	.90
Observed share	.06	.21	.38	.55	.72
Gap	**.04**	**.09**	**.12**	**.15**	**.18**

ECE

0.116

mean gap, equal bins

FIX

Recalibrate on held-out data (Platt, isotonic or temperature scaling), then re-check this diagram.

SECTION 5 · PROBABILITIES

Methods to improve calibration

Method	Idea	Best for
Platt scaling	Fits a simple curve mapping scores to probabilities	Small datasets, smooth errors
Isotonic regression	Fits a flexible stepwise mapping	Larger datasets
Temperature scaling	Softens or sharpens neural network outputs	Deep learning classifiers
Recalibrate regularly	Refit on recent data as base rates shift	Drifting environments



Always fit calibration on data not used to train the model, then confirm the effect on a separate test set.

SECTION 5 · PROBABILITIES

Threshold and probability pitfalls

PITFALL	BETTER PRACTICE
⚠ Using 0.5 because it is the default.	✓ Choose thresholds from costs or constraints.
⚠ Treating scores as true probabilities.	✓ Check calibration before using scores as risks.
⚠ Celebrating high AUC on rare positives.	✓ Check the precision recall curve as well.
⚠ Tuning thresholds on the test set.	✓ Tune on validation and confirm on test once.

SECTION 5 · PROBABILITIES

Section recap

KEY TAKEAWAYS

- Separate ranking, decision and probability quality
- ROC AUC measures ranking across thresholds
- Precision recall curves suit rare positives
- Choose thresholds with costs and capacity
- Log loss and Brier score reward honest probabilities
- Check and fix calibration with reliability diagrams

TRY THIS

Plot ROC, precision recall and reliability diagrams for your model, then choose a threshold using a simple cost matrix.

NEXT UP

Section 6: regression metrics.

SECTION 6

06

Regression metrics

Measuring how far numeric predictions land from reality, and which errors matter most.

IN THIS SECTION

- Absolute and squared errors
- Percentage errors
- Explained variance
- Quantile and interval scores
- Residual analysis
- Choosing a metric

SECTION 6 · REGRESSION

Mean absolute error

DEFINITION

Mean absolute error is the average size of the prediction errors, ignoring direction. It is expressed in the same units as the target.

EXAMPLE

A house price model is off by 10,000, 5,000 and 30,000 pounds on three homes. Its mean absolute error is 15,000 pounds.

REMEMBER

Easy to explain. Every pound of error counts the same.

SECTION 6 · REGRESSION

Root mean squared error

DEFINITION

Root mean squared error squares each error, averages them and takes the square root, so large errors count much more than small ones. It stays in the target’s units.

EXAMPLE

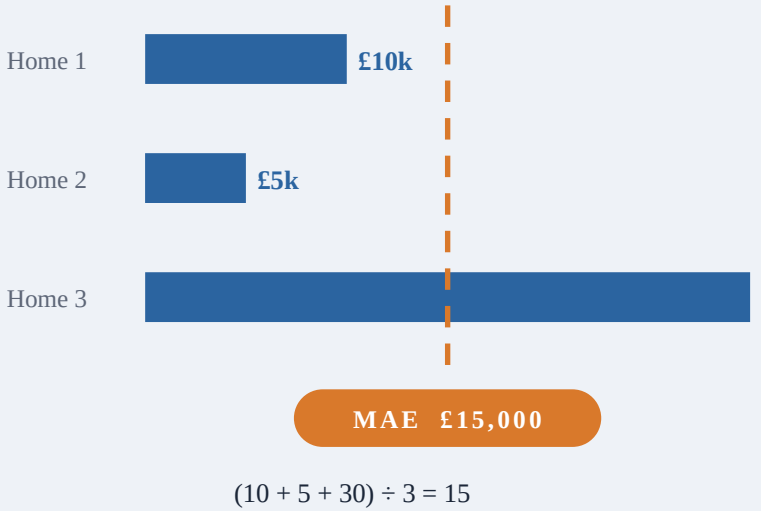
For the same three homes, the 30,000 pound miss dominates, making this error larger than the mean absolute error.

REMEMBER

Use it when large mistakes are disproportionately harmful.

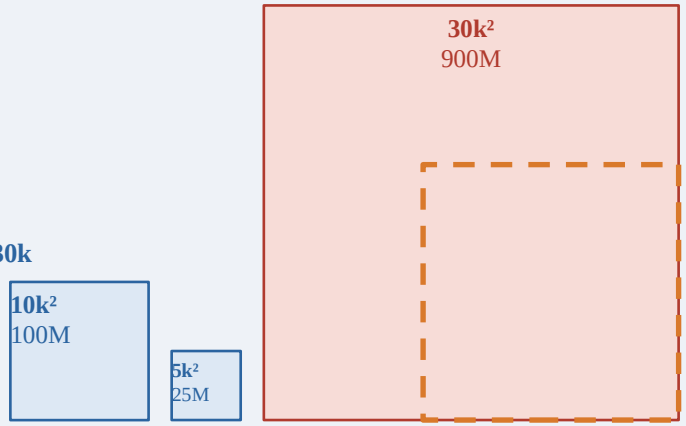
MAE averages lengths; RMSE averages areas

ABSOLUTE ERRORS ARE LENGTHS



(10 + 5 + 30) ÷ 3 = 15

SQUARED ERRORS ARE AREAS



average area 341.7M
side = √341.7M ≈ 18.5k

RMSE £18,484

The dashed square has the average area. Its side is the RMSE.

WHAT IF HOME 3 MISSED BY £90K?

	Now	If 90k
MAE	**15.0k**	**35.0k**
RMSE	**18.5k**	**52.4k**

MAE grows ×2.3
RMSE grows ×2.8
Squaring amplifies the single worst miss.

READ THE GAP RMSE close to MAE: errors are similar sizes. RMSE far above MAE: a few big misses dominate, so go and find them.

SECTION 6 · REGRESSION

Absolute versus squared error

MAE

Mean absolute error

- Treats all errors in proportion
- Robust to a few outliers
- Matches median focused predictions
- Very easy to communicate

RMSE

Root mean squared error

- Penalises big errors heavily
- Sensitive to outliers
- Matches mean focused predictions
- Standard in many benchmarks

VERDICT Report both. A large gap between them signals a few big misses worth investigating.

SECTION 6 · REGRESSION

Percentage errors and their traps

Mean absolute percentage error expresses error relative to the true value, which is intuitive but breaks down in important cases.

Near zero values

Tiny actual values create huge or infinite percentages.

Asymmetry

It punishes over prediction and under prediction differently.

Alternatives

Weighted percentage error, symmetric versions or scaled errors are often safer.

SECTION 6 · REGRESSION

R squared

DEFINITION

R squared describes how much of the variation in the target the model explains, compared with simply predicting the average every time.

EXAMPLE

An R squared of 0.80 means the model removes about 80 percent of the error you would get by always predicting the mean.

REMEMBER

It can be negative on new data and says nothing about error size in real units.

SECTION 6 · REGRESSION

Other useful regression measures

1 Median absolute error

Typical error, unaffected by a few extreme misses.

2 Max error

The worst single miss, vital for safety limits.

3 Mean error, or bias

Shows if predictions are systematically too high or too low.

4 Log based errors

Focus on relative errors for targets spanning many magnitudes.

5 Error within tolerance

Share of predictions within an acceptable range.

6 Business cost

Translate errors into money, stock or time lost.

SECTION 6 · REGRESSION

Evaluating quantiles and intervals

Many decisions need a range rather than a single number. Quantile and interval predictions need their own evaluation methods.

Pinball loss

Scores quantile predictions, penalising misses on the wrong side more heavily.

Coverage

Share of true values falling inside the predicted interval, compared with the promised level.

Sharpness

Narrower intervals are more useful, as long as coverage is honest.

SECTION 6 · REGRESSION

Residual analysis

1. **Plot residuals against predictions.** Look for patterns such as funnels or curves.
2. **Plot against key features.** Find regions where the model is systematically wrong.
3. **Check over time.** Look for drift or seasonal patterns in errors.
4. **Inspect the largest errors.** Read the actual records behind the biggest misses.
5. **Check distribution.** See whether errors are skewed or have heavy tails.
6. **Act.** Add features, transform the target or segment the model.

SECTION 6 · REGRESSION

Regression metric pitfalls

PITFALL	BETTER PRACTICE
⚠ Using percentage error when values can be near zero.	→ ✓ Use scaled or weighted alternatives.
⚠ Reporting R squared alone.	→ ✓ Add error metrics in the target’s own units.
⚠ Ignoring systematic bias.	→ ✓ Report mean error and residual plots.
⚠ Averaging over very different segments.	→ ✓ Report errors per segment and size.

SECTION 6 · REGRESSION

Section recap

KEY TAKEAWAYS

- MAE is robust and easy to explain
- RMSE emphasises large errors
- Percentage errors break near zero
- R squared compares against predicting the mean
- Evaluate intervals with coverage and sharpness
- Residual plots reveal where models fail

TRY THIS

Compute MAE, RMSE, median error and bias for your regression model, then plot residuals against the top three features.

NEXT UP

Section 7: evaluating models on imbalanced data.

SECTION 7

07

Imbalanced data

When the cases that matter most are also the rarest, evaluation needs special care.

IN THIS SECTION

- Why imbalance distorts metrics
- Metrics that cope
- Resampling and evaluation
- Base rates and prevalence
- Capacity based evaluation

SECTION 7 · IMBALANCED DATA

Why imbalance distorts evaluation

When one class is 99 percent of the data, a model can look excellent while never finding the cases you care about.

Accuracy inflates

Predicting the majority class gives very high accuracy for free.

ROC looks rosy

Huge numbers of negatives make false positive rates look small.

Few positives

Metrics on rare classes become noisy and unstable.

Metrics that cope with imbalance

Metric	Why it helps
Precision and recall	Focus directly on the rare positive class
Average precision	Summarises the precision recall curve
F1 or F beta	Balances precision and recall in one number
Balanced accuracy	Gives each class equal weight
Matthews correlation	Uses all four confusion counts fairly
Recall at fixed precision	Matches real operating constraints

SECTION 7 · IMBALANCED DATA

Resampling and honest evaluation

Oversampling, undersampling and synthetic methods such as SMOTE can help training, but they must never change the evaluation data.

Resample training only

Apply resampling inside training folds after splitting.

Keep test realistic

The test set must reflect the true class balance seen in production.

Recalibrate

Resampling distorts probabilities, so recalibrate before using them as risks.

SECTION 7 · IMBALANCED DATA

Prevalence and base rates

DEFINITION

Prevalence is how common the positive class is. Precision depends strongly on it, so the same model can show very different precision in different populations.

EXAMPLE

A test with fixed recall and false positive rate gives high precision in a specialist clinic where the disease is common, but low precision in general screening.

REMEMBER

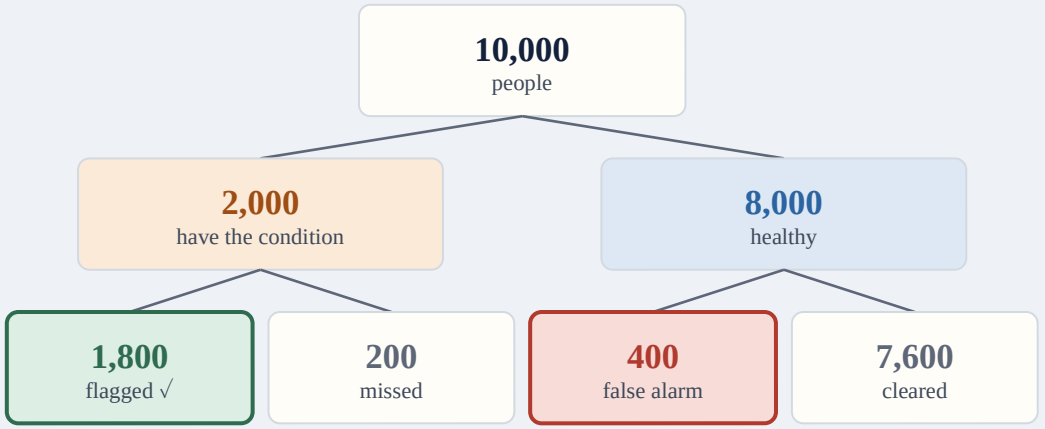
Always state the prevalence when you report precision.

Same model, two populations, very different precision

SAME MODEL IN BOTH: recall 90% · false positive rate 5%

Specialist clinic

20% prevalence

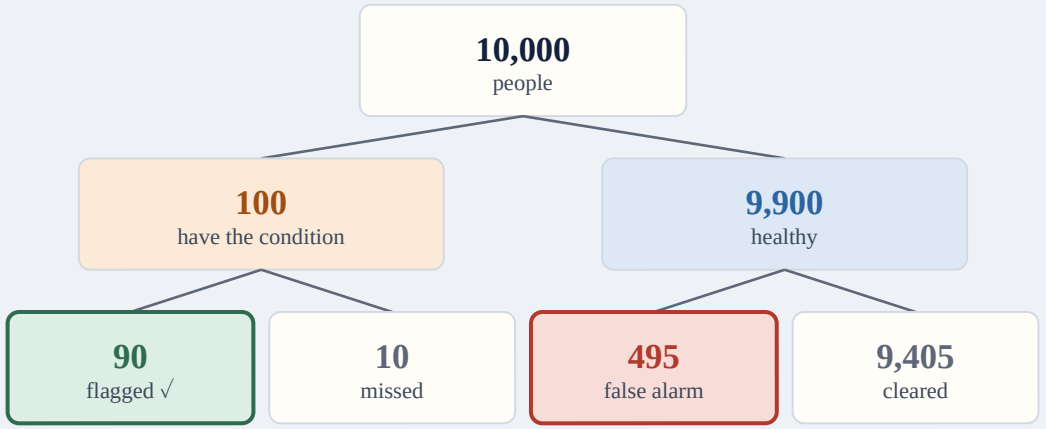


Precision 1,800 ÷ (1,800 + 400)
of flagged people truly have it

82%

General screening

1% prevalence



Precision 90 ÷ (90 + 495)
of flagged people truly have it

15%

! Precision travels badly between populations. Re-estimate it on data with the deployment prevalence.

SECTION 7 · IMBALANCED DATA

Evaluating under limited review capacity

Often staff can only review a fixed number of alerts per day. Evaluate the model the way it will be used: by what it finds within that budget.

Precision at k

Of the top k cases flagged each day, how many are real?

Recall at k

What share of all true cases are found within the top k?

Lift

How many times better than random selection the top flagged cases are.

SECTION 7 · IMBALANCED DATA

Imbalance pitfalls

PITFALL	BETTER PRACTICE
⚠ Evaluating on a rebalanced test set.	✓ Keep the test set at realistic prevalence.
⚠ Quoting precision without prevalence.	✓ State the base rate with every precision figure.
⚠ Too few positives to measure reliably.	✓ Use confidence intervals and gather more positives.
⚠ Using probabilities after resampling.	✓ Recalibrate before treating scores as risks.

SECTION 7 · IMBALANCED DATA

Knowledge check

? Investigators can review 100 alerts a day out of 50,000 transactions. Which metric best reflects the model's usefulness?

MODEL ANSWER

Precision at 100 and recall at 100: how many of the top 100 daily alerts are real fraud, and what share of all fraud they capture. These match the actual workflow far better than accuracy or overall AUC.

WHY IT MATTERS

Evaluating at the real operating point links metrics to work done.

Section recap

KEY TAKEAWAYS

- Imbalance makes accuracy and ROC look misleadingly good
- Use precision, recall, average precision and MCC
- Resample only training data, never test data
- Precision depends on prevalence
- Evaluate at the real review capacity

TRY THIS

For an imbalanced problem, report precision and recall at the top 1, 5 and 10 percent of scores and compare with random selection.

NEXT UP

Section 8: ranking and recommendation metrics.

SECTION 8

08

Ranking and recommendation

When the order of results matters as much as the results themselves.

IN THIS SECTION

- Why ranking is different
- Precision and recall at k
- MRR and MAP
- NDCG
- Beyond accuracy
- Offline versus online gaps

SECTION 8 · RANKING

Why ranking needs its own metrics

Users look at the top few results and rarely scroll. A good ranker puts the most relevant items first, so metrics must reward position.

Position matters

A relevant item in first place is worth more than the same item in twentieth.

Cut offs

Metrics are usually computed at k, the number of results a user actually sees.

Relevance levels

Items can be graded, such as perfect, good, fair or irrelevant.

SECTION 8 · RANKING

Core ranking metrics

1 Precision at k

Share of the top k results that are relevant.

2 Recall at k

Share of all relevant items that appear in the top k.

3 Hit rate at k

Share of users with at least one relevant item in the top k.

4 Mean reciprocal rank

Rewards placing the first relevant result as high as possible.

5 Mean average precision

Rewards placing all relevant results high, averaged over queries.

6 NDCG

Rewards graded relevance with a discount for lower positions.

SECTION 8 · RANKING

Normalised discounted cumulative gain

DEFINITION

NDCG adds up the relevance of results, giving less credit the further down the list each result appears, then divides by the best possible score so results fall between zero and one.

EXAMPLE

Two search engines return the same five relevant pages, but one puts the best page first. That engine earns a higher NDCG.

REMEMBER

NDCG is the standard choice when relevance comes in grades.

NDCG, calculated position by position

	Position 1	Position 2	Position 3	Position 4	Position 5	DCG	NDCG
Discount $1 \div \log_2(\text{position} + 1)$	1.00	0.63	0.50	0.43	0.39	*credit fades with position*	
Ideal order *best possible*	**grade 3** 3 × 1.00 = 3.00	**grade 3** 3 × 0.63 = 1.89	**grade 2** 2 × 0.50 = 1.00	**grade 1** 1 × 0.43 = 0.43	**grade 0** 0 × 0.39 = 0.00	**6.32**	**1.00**
Engine A *strong pages first*	**grade 3** 3 × 1.00 = 3.00	**grade 2** 2 × 0.63 = 1.26	**grade 3** 3 × 0.50 = 1.50	**grade 0** 0 × 0.43 = 0.00	**grade 1** 1 × 0.39 = 0.39	**6.15**	**0.97**
Engine B *same pages, buried*	**grade 0** 0 × 1.00 = 0.00	**grade 1** 1 × 0.63 = 0.63	**grade 3** 3 × 0.50 = 1.50	**grade 2** 2 × 0.43 = 0.86	**grade 3** 3 × 0.39 = 1.16	**4.15**	**0.66**

NDCG = DCG ÷ ideal DCG. Engine A: 6.15 ÷ 6.32 = 0.97. Engine B: 4.15 ÷ 6.32 = 0.66.

! Gain here is the grade itself. Many libraries use 2^grade − 1, which rewards top grades more, so say which one you used.

SECTION 8 · RANKING

Beyond accuracy for recommenders

1 Coverage
Share of the catalogue that ever gets recommended.

2 Diversity
How different the items in one list are from each other.

3 Novelty
How unfamiliar or less popular the recommendations are.

4 Serendipity
Relevant surprises users would not have found themselves.

5 Popularity bias
Whether the system keeps pushing already popular items.

6 Freshness
How quickly new items start being recommended.

SECTION 8 · RANKING

Why offline ranking results often disagree with online

⚠ Historical logs only contain feedback on items users were shown, so offline evaluation is biased towards the old system's choices.

Exposure bias

Items never shown have no clicks, even if they were great.

Position bias

Users click top results partly because they are on top.

Remedies

Counterfactual evaluation, interleaving experiments and A/B tests.

SECTION 8 · RANKING

Interleaving

DEFINITION

Interleaving merges results from two rankers into one list for each user and measures which ranker’s items get more clicks, giving a sensitive comparison with little traffic.

EXAMPLE

A search team blends results from the old and new rankers. Users click new ranker items more often, so it wins with far fewer users than an A/B test would need.

REMEMBER

Interleaving detects preference quickly but does not measure overall business impact.

SECTION 8 · RANKING

Ranking evaluation pitfalls

PITFALL	BETTER PRACTICE
⚠ Evaluating at k far larger than users see.	✓ Choose k from real interface behaviour.
⚠ Treating unclicked items as irrelevant.	✓ Account for items users never saw.
⚠ Optimising only clicks.	✓ Add diversity, coverage and long term engagement.
⚠ Random splits of interaction logs.	✓ Split by time to mimic future recommendations.

SECTION 8 · RANKING

Knowledge check

? A new recommender lifts precision at 10 offline but recommends only the 200 most popular products. Is it ready?

MODEL ANSWER

Probably not. It may be exploiting popularity bias, shrinking catalogue coverage and hurting discovery for niche products and new sellers. Check coverage, diversity and novelty, then confirm with an online test measuring long term engagement and revenue.

WHY IT MATTERS

Accuracy alone can reward a recommender that narrows choice.

Section recap

KEY TAKEAWAYS

- Ranking metrics reward correct order and position
- Use precision, recall and hit rate at a realistic k
- MRR, MAP and NDCG capture position effects
- Measure coverage, diversity and novelty too
- Offline logs carry exposure and position bias
- Interleaving and A/B tests confirm offline gains

TRY THIS

Compute precision at 10, NDCG at 10 and catalogue coverage for a recommender, then explain any trade offs you see.

NEXT UP

Section 9: evaluating forecasts.

SECTION 9

09

Forecasting evaluation

Judging predictions of the future without accidentally peeking at it.

IN THIS SECTION

- Why forecasting is special
- Backtesting
- Naive baselines
- Scaled errors
- Forecast horizons
- Probabilistic forecasts

SECTION 9 · FORECASTING

What makes forecasting different



Time only flows one way. Every evaluation must respect that the future is unknown when the forecast is made.

Order matters

Observations depend on earlier ones, so random shuffling breaks the problem.

Conditions change

Trends, seasons and shocks shift over time.

Many series

Forecasts often cover thousands of products or locations at once.

Rolling origin backtesting

01

Origin

Choose a cut off date and train only on data before it.

02

Forecast

Predict the next period, such as the next four weeks.

03

Score

Compare the forecast with what actually happened.

04

Roll

Move the cut off forward and repeat.

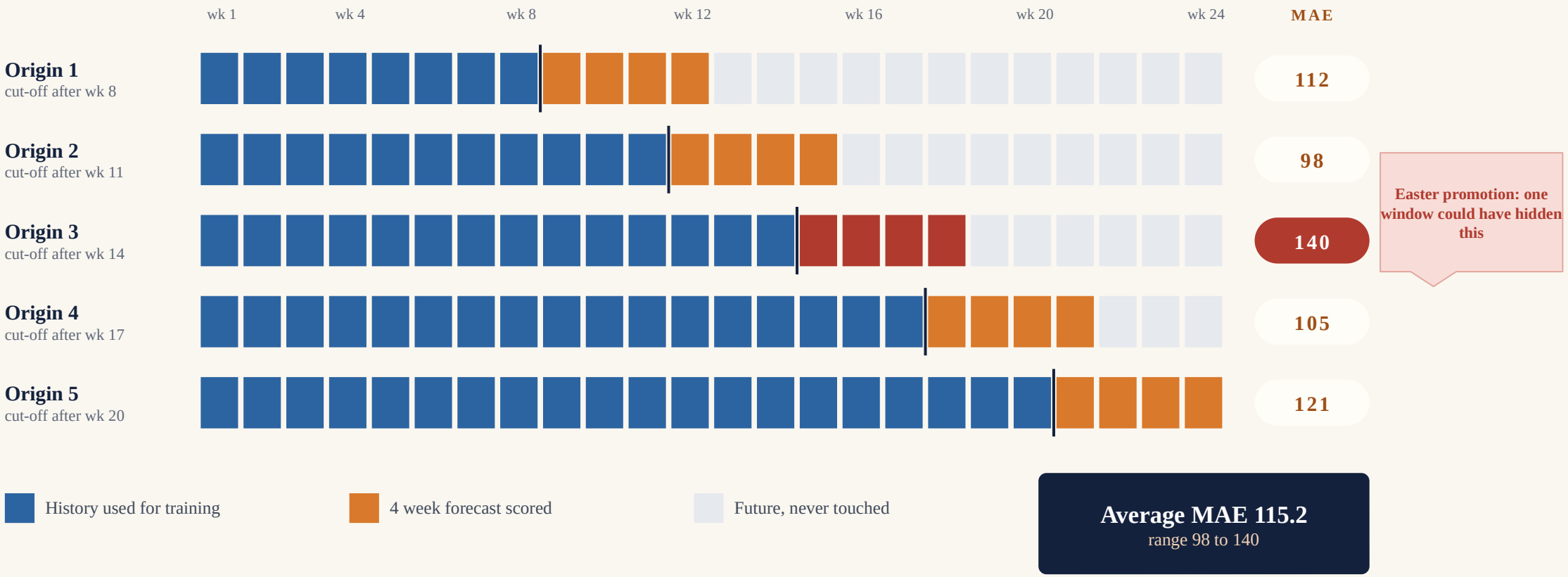
05

Aggregate

Average errors across all origins and horizons.

Backtesting over many origins shows how the model behaves across seasons, promotions and unusual periods, not just one lucky window.

Rolling origin backtesting across 24 weeks



! This is an expanding window. A sliding window drops the oldest weeks so every training history has the same length.

SECTION 9 · FORECASTING

Naive baselines are hard to beat

Simple forecasts are surprisingly strong. Every forecasting model must be compared with them before any claim of skill.

Last value

Tomorrow equals today. Strong for slowly changing series.

Seasonal naive

Next Monday equals last Monday. Strong for weekly and yearly cycles.

Moving average

The average of recent periods. Smooths noise.

Scaled errors

DEFINITION

A scaled error divides a model’s error by the error of a naive baseline on the same series, so values below one mean the model beats the baseline.

EXAMPLE

A product’s forecast scores 0.7, meaning its errors are 30 percent smaller than simply repeating last season’s sales.

REMEMBER

Scaled errors let you compare accuracy fairly across series of very different sizes.

A scaled error of 0.7, worked out week by week

	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Mean
Actual sales	120	135	128	150	142	160	
Naive: repeat last week	110	120	135	128	150	142	
Naive miss	**10**	**15**	**7**	**22**	**8**	**18**	**13.3**
Model forecast	128	125	134	136	136	148	
Model miss	**8**	**10**	**6**	**14**	**6**	**12**	**9.3**

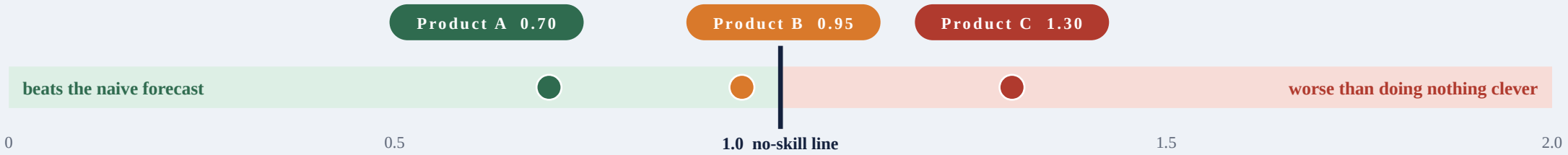
SCALED ERROR

9.3 ÷ 13.3

= 0.70

Errors 30% smaller than repeating last week.

ONE SCALE FOR EVERY PRODUCT



Product B is barely better than naive. Product C should fall back to the naive forecast.

Common forecasting metrics

Metric	What it captures	Watch for
MAE	Average size of misses in units	Hard to compare across series
RMSE	Emphasises big misses	Sensitive to outliers
MAPE	Error as a share of actuals	Breaks with zeros
Weighted MAPE	Total error over total volume	Dominated by big series
MASE	Error scaled by naive baseline	Needs a sensible baseline
Pinball loss	Quality of quantile forecasts	One score per quantile

SECTION 9 · FORECASTING

Forecast horizons

Accuracy usually falls the further ahead you forecast. Evaluate each horizon separately because different decisions depend on different horizons.

Short horizon

Next day staffing needs high accuracy in the very near term.

Medium horizon

Ordering stock weeks ahead depends on the four to eight week view.

Long horizon

Capacity planning months ahead tolerates more error but needs the right trend.

SECTION 9 · FORECASTING

Evaluating probabilistic forecasts

A forecast range, such as 80 percent sure demand will be between 900 and 1,200 units, supports better decisions than a single number.

Coverage

Do 80 percent intervals contain the truth about 80 percent of the time?

Sharpness

Are intervals as narrow as possible while staying honest?

Overall scores

Pinball loss and continuous ranked probability score combine both.

SECTION 9 · FORECASTING

Forecasting pitfalls

PITFALL	BETTER PRACTICE
⚠ Random cross validation on time series.	✓ Use rolling origin backtesting.
⚠ No comparison with naive forecasts.	✓ Always report scaled errors against baselines.
⚠ One average over all horizons.	✓ Report accuracy per horizon that matters.
⚠ Using revised data that was not available then.	✓ Evaluate with data as it was known at each origin.

SECTION 9 · FORECASTING

Section recap

KEY TAKEAWAYS

- Respect time order in every evaluation
- Backtest across many rolling origins
- Beat naive and seasonal naive baselines first
- Use scaled errors to compare across series
- Evaluate each decision horizon separately
- Check coverage and sharpness of forecast ranges

TRY THIS

Backtest your forecast over at least six origins and compare it with a seasonal naive baseline at each horizon.

NEXT UP

Section 10: clustering and anomaly detection.

SECTION 10

10

Clustering and anomaly detection

Evaluating models when there is little or no ground truth to compare against.

IN THIS SECTION

- The challenge without labels
- Internal cluster measures
- External cluster measures
- Evaluating anomaly detection
- Human judgement

SECTION 10 · UNSUPERVISED

The challenge without labels



When there is no right answer to compare against, evaluation must rely on structure, stability and usefulness.

Structure

Are groups compact inside and well separated from each other?

Stability

Do similar clusters appear when the data or random seed changes?

Usefulness

Do the clusters or anomalies help people make better decisions?

SECTION 10 · UNSUPERVISED

Internal cluster measures

1 Silhouette score

How close each point is to its own cluster versus the nearest other cluster.

2 Davies Bouldin index

Average similarity between each cluster and its most similar neighbour. Lower is better.

3 Calinski Harabasz

Ratio of separation between clusters to spread within them. Higher is better.

4 Inertia

Total spread within clusters. Used in the elbow method for choosing cluster counts.

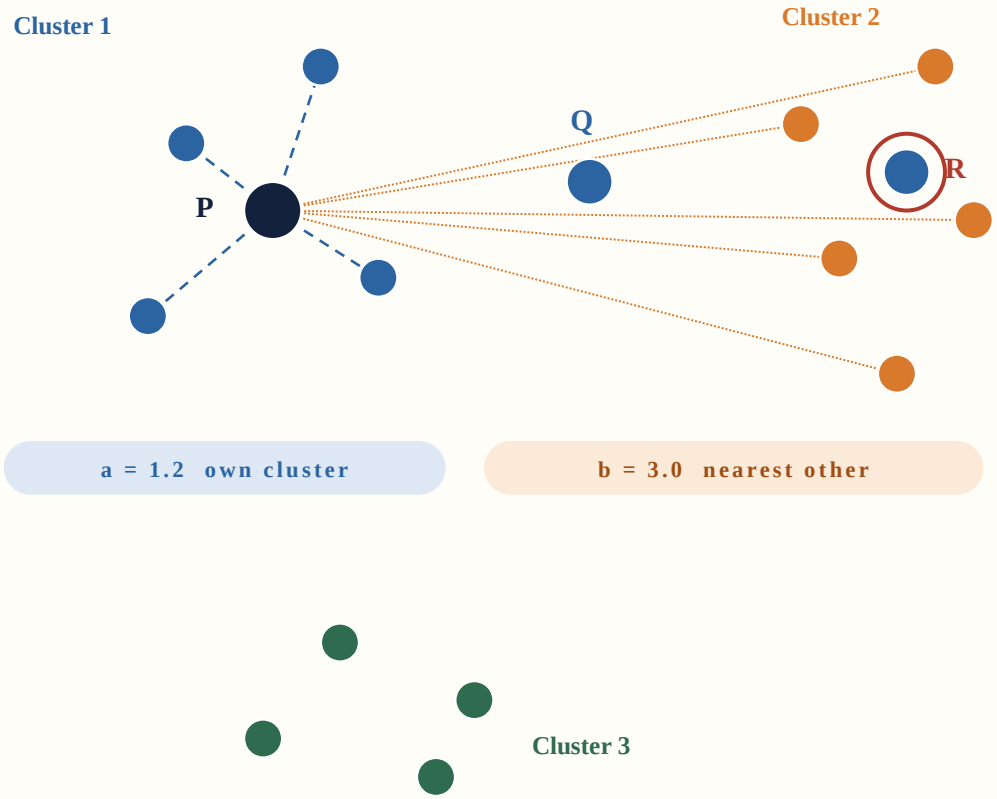
5 Stability

Agreement between clusterings on resampled data.

6 Size balance

Whether clusters are usefully sized rather than one giant group.

The silhouette score, computed for a single point

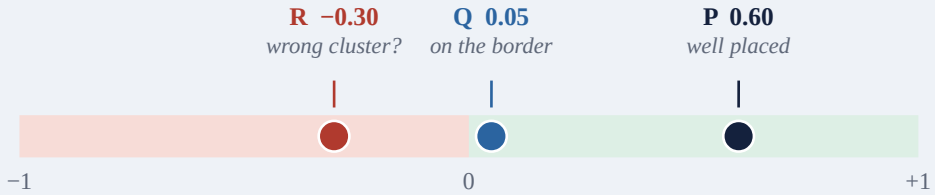


FOR POINT P

s = (b − a) ÷ max(a, b)

= (3.0 − 1.2) ÷ 3.0 = 0.60

WHERE EACH POINT LANDS



FROM POINTS TO A SCORE

The clustering's silhouette score is the average s over every point. Inspect the negative points: they are often mislabelled or sit between groups.

SECTION 10 · UNSUPERVISED

External cluster measures

When some reference labels exist, you can compare clusters against them, while remembering clusters may reveal structure the labels miss.

Adjusted Rand index

Agreement between clustering and labels, corrected for chance.

Mutual information

How much knowing the cluster tells you about the label.

Purity

Share of each cluster made up of its most common label.

SECTION 10 · UNSUPERVISED

Evaluating anomaly detection

Anomalies are rare and often unlabelled, so evaluation combines limited labels, injected examples and expert review.

Labelled incidents

Use known past incidents to measure precision and recall at the top alerts.

Synthetic anomalies

Inject realistic faults into clean data and check how many are caught.

Alert review

Have experts label a sample of alerts to estimate precision over time.

SECTION 10 · UNSUPERVISED

A practical review for unsupervised results

1. **Profile clusters.** Describe each cluster with summary statistics and examples.
2. **Name them.** Ask domain experts whether clusters have a meaningful interpretation.
3. **Test stability.** Rerun with different seeds and samples and compare.
4. **Check actionability.** Confirm each cluster suggests a different action.
5. **Track over time.** See whether clusters persist as new data arrives.

Unsupervised evaluation pitfalls

PITFALL	BETTER PRACTICE
⚠ Choosing cluster count by one internal score.	✓ Combine scores, stability and expert usefulness.
⚠ Treating every anomaly flag as a true problem.	✓ Measure alert precision with expert labels.
⚠ Ignoring feature scaling effects.	✓ Evaluate clusters under consistent preprocessing.
⚠ Clusters nobody can explain or use.	✓ Judge clusters by the decisions they enable.

SECTION 10 · UNSUPERVISED

Section recap

KEY TAKEAWAYS

- Unsupervised evaluation relies on structure, stability and usefulness
- Silhouette and related scores measure compactness and separation
- External measures compare with partial labels
- Anomaly detectors need labelled incidents and expert review
- Usefulness to people is the final test

TRY THIS

Cluster a dataset with three different cluster counts and compare silhouette scores, stability and how easily you can name each cluster.

NEXT UP

Section 11: language, vision and speech tasks.

SECTION 11

11

Language, vision and speech

Task specific metrics for unstructured data, and their important blind spots.

IN THIS SECTION

- Text classification and extraction
- Translation and summarisation
- Object detection
- Segmentation
- Speech recognition
- Human evaluation

Metrics by task

Task	Common metrics	Key caution
Text classification	Precision, recall, macro F1	Label noise in text data
Named entity recognition	Entity level precision and recall	Partial matches matter
Translation	BLEU, chrF, COMET	Overlap is not meaning
Summarisation	ROUGE, factual consistency	Fluent summaries can be false
Image classification	Top 1 and top 5 accuracy	Background shortcuts
Object detection	IoU, mAP	Thresholds change results
Speech recognition	Word error rate	Accents and noise vary

Overlap metrics and their limits

BLEU and ROUGE count words shared with reference texts. They are cheap and repeatable but can reward wrong answers that use the right words.

Strength

Fast, automatic and useful for tracking progress on one benchmark.

Weakness

Correct paraphrases score low and fluent mistakes can score high.

Better options

Learned metrics such as COMET or BERTScore, plus human review.

Intersection over union

DEFINITION

Intersection over union compares a predicted box or mask with the true one by dividing their overlapping area by the area they cover together.

EXAMPLE

A predicted box covering most of a car and a little road has high overlap with the true box and might score 0.8, counting as a correct detection at a 0.5 threshold.

REMEMBER

Detection results depend heavily on the overlap threshold chosen.

Intersection over union, drawn to scale

truth



IoU 0.76

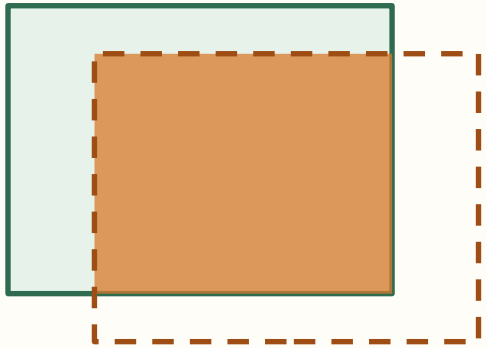
Good localisation

10.36 ÷ 13.64 units²

√ ≥ 0.50

√ ≥ 0.75

truth



IoU 0.48

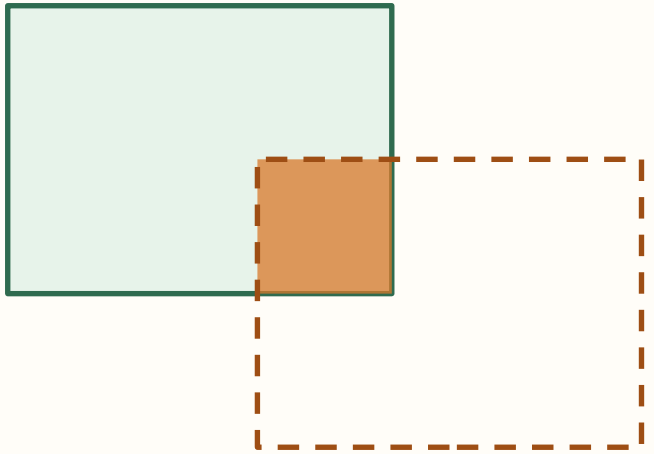
Just misses 0.5

7.75 ÷ 16.25 units²

✗ ≥ 0.50

✗ ≥ 0.75

truth



IoU 0.09

Clear miss

1.96 ÷ 22.04 units²

✗ ≥ 0.50

✗ ≥ 0.75

IoU =  overlap area ÷  area covered by both boxes together

! Averaging over thresholds 0.50 to 0.95 counts the first box at only 6 of 10 thresholds.

Mean average precision for detection

Mean average precision summarises detection quality across classes and confidence levels, and is often averaged over several overlap thresholds.

Per class

Average precision is computed for each object class from its precision recall curve.

Across thresholds

Some benchmarks average results over overlap thresholds from 0.5 to 0.95.

Also check

Small object performance and false detections per image.

Word error rate

DEFINITION

Word error rate counts the words a speech recogniser substituted, deleted or inserted, as a share of the words in the correct transcript.

EXAMPLE

A ten word sentence transcribed with one wrong word and one missing word has a word error rate of 20 percent.

REMEMBER

Report it separately for accents, noise levels and speaking styles.

Counting word errors by lining up the transcripts

Correct transcript	please	book	a	table	for	two	people	at	seven	tonight
Recogniser heard	please	look	a	table	for	two	*(missing)*	at	seven	tonight
Operation	✓	SUB	✓	✓	✓	✓	DEL	✓	✓	✓

S 1 substitution + D 1 deletion + I 0 insertions ÷ N 10 reference words = WER 20% word error rate

SURPRISE 1 · WER CAN EXCEED 100%

Truth “yes”, heard “yes yes okay”: 2 insertions ÷ 1 word = 200%.

SURPRISE 2 · ERRORS ARE NOT EQUAL

“Transfer fifteen pounds” heard as “transfer fifty pounds” is one error, 33% WER, and the wrong payment.

Running good human evaluation

1. **Write clear guidelines.** Define each rating with examples of good and bad outputs.
2. **Train raters.** Calibrate raters on practice items before real scoring.
3. **Blind the comparison.** Hide which system produced each output and randomise order.
4. **Use multiple raters.** Collect several ratings per item and measure agreement.
5. **Sample wisely.** Cover common cases and hard edge cases.
6. **Analyse disagreements.** Disagreements often reveal unclear guidelines or ambiguous tasks.

Pitfalls for unstructured data

PITFALL	BETTER PRACTICE
⚠ Trusting BLEU or ROUGE as quality.	→ ✓ Add learned metrics and human evaluation.
⚠ One overall word error rate.	→ ✓ Break results down by speaker group and condition.
⚠ Images with background shortcuts.	→ ✓ Test on images with changed backgrounds.
⚠ Benchmarks leaked into pretraining data.	→ ✓ Use fresh, private test sets.

Section recap

KEY TAKEAWAYS

- Each task has standard metrics and blind spots
- Overlap metrics measure words, not meaning
- IoU and mAP depend on thresholds
- Word error rate needs breakdowns by group
- Human evaluation needs guidelines, blinding and agreement checks

TRY THIS

Pick one text task and compare an overlap metric with your own human ratings on 30 examples. Where do they disagree?

NEXT UP

Section 12: evaluating large language models and generative systems.

SECTION 12

12

Evaluating LLMs and generative models

Measuring quality when there are many good answers and some dangerous ones.

IN THIS SECTION

- Why generation is hard to score
- Benchmarks and contamination
- Building golden sets
- LLM as a judge
- Evaluating RAG and agents
- Safety and red teaming

Why generative evaluation is hard



There is rarely one correct answer, and a fluent, confident response can still be completely wrong.

Open ended

Many different responses can be equally good.

Multi dimensional

Correctness, helpfulness, tone, safety and format all matter.

Moving target

Prompts, models and user needs change frequently.

Public benchmarks and their limits

Public benchmarks offer a shared yardstick for knowledge, reasoning and coding, but they rarely reflect your exact use case.

Useful for

Shortlisting models and tracking broad capability trends.

Contamination

Test questions may have appeared in training data, inflating scores.

Saturation

Top models cluster near the ceiling, so small differences mean little.

Building a golden evaluation set

1. **Collect real tasks.** Sample real user requests or realistic scenarios from your domain.
2. **Cover the range.** Include common, rare, ambiguous and adversarial cases.
3. **Define success.** Write reference answers or rubrics for each item.
4. **Review with experts.** Have domain experts check references and rubrics.
5. **Version it.** Store the set like code and record changes.
6. **Keep it private.** Protect it from leaking into training or prompt examples.

Ways to score generated output

1 Exact checks
Match against a known answer, number or label.

2 Programmatic tests
Run generated code against unit tests, report pass rates.

3 Format validation
Check JSON or schema compliance automatically.

4 Reference similarity
Semantic similarity to a reference answer.

5 LLM as a judge
A strong model scores outputs against a rubric.

6 Human rating
Experts score samples, the most trusted signal.

Using an LLM as a judge

A capable model can grade outputs at scale against a clear rubric, but its judgements must themselves be evaluated against human ratings.

Make it reliable

Give precise rubrics, examples and ask for reasoning before the score.

Known biases

Judges may favour longer answers, the first option or their own style.

Validate

Check agreement with expert ratings on a sample before trusting it.

Anatomy of a trustworthy LLM judge

```
judge_prompt.txt

ROLE   You grade customer-support answers.

RUBRIC 1. Facts correct ............ 0–2

       2. Answers the real question  0–2

       3. Concise, no padding ...... 0–1

FIRST  Explain your reasoning in 3 lines.

THEN   Output {"score": 0–5} and nothing else.
```

Precise criteria with point values

Reasoning before the score

Structured, parseable output

KNOWN BIASES AND THEIR FIXES

Position bias

Prefers whichever answer comes first.

Fix: Run both orders; count a win only if it survives the swap.

Verbosity bias

Rewards longer answers.

Fix: Credit concision in the rubric; compare at similar lengths.

Self-preference

Favours text in its own style.

Fix: Use a judge from another model family; hide the source.

VALIDATE THE JUDGE BEFORE YOU TRUST IT

1. Experts grade a sample of 200 outputs
2. The judge grades the same 200
3. Compare agreement, e.g. Cohen's kappa
4. Scale up only if it rivals expert-to-expert agreement

Evaluating a RAG system

Retrieval recall	Did the relevant passages appear in the retrieved set?
Retrieval precision	How much retrieved text was actually relevant?
Faithfulness	Is every claim in the answer supported by retrieved text?
Answer relevance	Does the answer address the question asked?
Citation quality	Do cited sources really support the statements?

Diagnose by stage

When a RAG answer is wrong, find out whether retrieval missed the evidence or generation misused it. The fixes are completely different.

Scoring one RAG answer at every stage of the pipeline



Retrieval recall	Retrieval precision	Faithfulness	Answer relevance	Citation quality
relevant passage found?	*how much is relevant?*	*every claim supported?*	*addresses the question?*	*sources back the claims?*

QUESTION **What is the refund window for online orders?**

P1	Refunds are accepted within 30 days of delivery.	*relevant*
P2	Our stores open 9am to 6pm, Monday to Saturday.	*noise*
P3	Gift cards cannot be exchanged for cash.	*noise*
P4	Standard delivery takes 3 to 5 working days.	*noise*
P5	The company was founded in 1998 in Leeds.	*noise*

GENERATED ANSWER

“You can return items within 30 days of delivery [P1], **and return shipping is always free** [P1].”

Claim 1 supported by P1

Claim 2 found in no passage

Diagnosis: search did its job. The generator invented a policy and cited P1 for it. Fix the prompt or the model, not the retriever.

100% Retrieval recall	**20%** Retrieval precision	**50%** Faithfulness	**Yes** Answer relevance	**50%** Citation quality
1 of 1	*1 of 5*	*1 of 2 claims*	*on topic*	*1 of 2 citations*

Evaluating agents

Agents plan and take actions with tools across many steps, so evaluation must look at outcomes, paths, cost and safety together.

Task success

Did the agent achieve the goal in a sandboxed, realistic environment?

Trajectory

Were the steps sensible, efficient and free of harmful actions?

Cost and time

How many steps, tokens and seconds did success require?

Safety evaluation and red teaming

1 Harmful content
Test refusal of dangerous requests and absence of harmful output.

2 Prompt injection
Test whether hidden instructions in documents hijack behaviour.

3 Privacy
Check that personal or confidential data is not leaked.

4 Hallucination
Measure how often unsupported claims appear.

5 Bias
Compare responses across names, groups and dialects.

6 Over refusal
Make sure safe requests are not refused unnecessarily.

Practical LLM quality signals

Signal	What it tells you
Pass rate on golden set	Core task quality against agreed answers
Judge score by rubric	Scalable quality across several dimensions
Faithfulness rate	How often answers stay grounded in sources
Refusal rates	Balance between safety and helpfulness
User feedback	Thumbs, edits and follow up questions in production
Cost and latency	Whether quality is affordable at scale

Variance in generative evaluation

The same prompt can produce different outputs each run, so a single run per item gives noisy, unreliable comparisons.

Repeat runs

Run each item several times and report the average and spread.

Fix settings

Record temperature, model version and prompt version for every run.

Paired comparisons

Compare systems on the same items to reduce noise.

LLM evaluation pitfalls

PITFALL	BETTER PRACTICE
⚠ Deciding by vibes from a few demos.	✓ Use a versioned golden set and rubrics.
⚠ Trusting an unvalidated LLM judge.	✓ Measure judge agreement with experts first.
⚠ Leaderboard scores as proof of fitness.	✓ Test on your own tasks and data.
⚠ Ignoring retrieval when RAG answers fail.	✓ Score retrieval and generation separately.

Section recap

KEY TAKEAWAYS

- Generative outputs need multi dimensional evaluation
- Benchmarks shortlist, golden sets decide
- LLM judges scale but need human validation
- Evaluate RAG retrieval and faithfulness separately
- Agents need outcome, trajectory and safety checks
- Repeat runs to handle output variance

TRY THIS

Create a 25 item golden set with rubrics for an LLM task, score two prompts, and validate an LLM judge against your own ratings.

NEXT UP

Section 13: statistical rigour in evaluation.

SECTION 13

13

Statistical rigour

Knowing whether a difference between models is real or just luck.

IN THIS SECTION

- Every metric is an estimate
- Confidence intervals
- The bootstrap
- Comparing two models
- Multiple comparisons
- Seeds and variance

A metric without an uncertainty estimate is half a result.

☆ Test sets are samples. A different sample would give a slightly different number, and decisions must account for that.

Confidence intervals

DEFINITION

A confidence interval gives a range of plausible values for a metric, reflecting how much it could vary if you evaluated on another sample of the same size.

EXAMPLE

A model’s recall is 82 percent with a 95 percent interval from 78 to 86 percent. A rival at 84 percent is not clearly better.

REMEMBER

Overlapping intervals are a warning that differences may be noise.

SECTION 13 · STATISTICAL RIGOUR

The bootstrap, step by step

01

Resample

Draw a sample of test rows with replacement, the same size as the test set.

02

Score

Compute the metric on that resampled set.

03

Repeat

Do this a thousand times or more.

04

Sort

Line up all the resulting metric values.

05

Read off

The middle 95 percent forms the confidence interval.

The bootstrap works for almost any metric, including F1, AUC and NDCG, without needing special formulas.

The bootstrap, watched resample by resample

ORIGINAL TEST SET: 10 ROWS

	1	2	3	4	5	6	7	8	9	10	
Original	✓	✓	✗	✓	✓	✓	✗	✓	✗	✓	70%

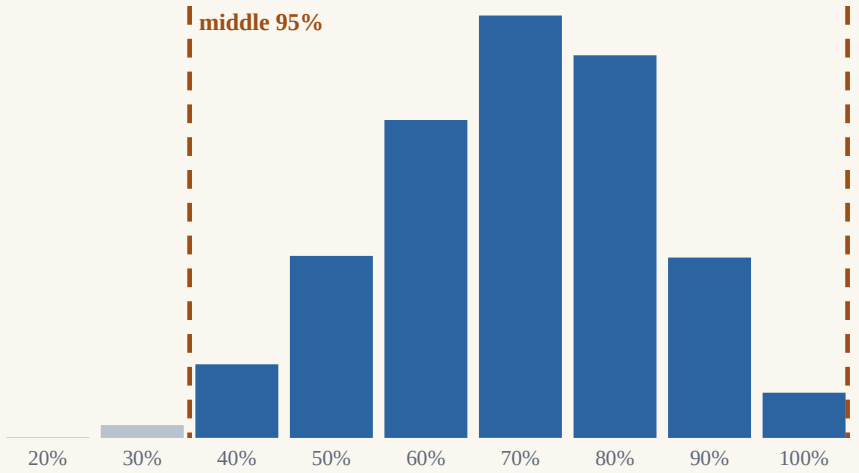
DRAW 10 ROWS WITH REPLACEMENT, AGAIN AND AGAIN

Resample 1	9 ✗	1 ✓	2 ✓	3 ✗	2 ✓	9 ✗	9 ✗	6 ✓	1 ✓	1 ✓	60%
Resample 2	4 ✓	5 ✓	7 ✗	5 ✓	3 ✗	2 ✓	7 ✗	8 ✓	1 ✓	2 ✓	70%
Resample 3	5 ✓	4 ✓	9 ✗	6 ✓	5 ✓	5 ✓	7 ✗	6 ✓	2 ✓	8 ✓	80%

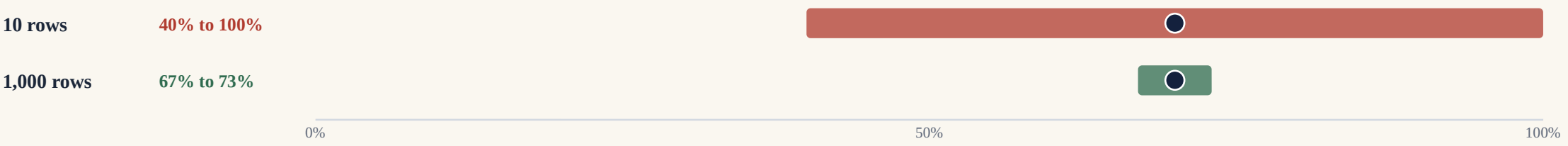
Dark outline = a row drawn more than once.

⋮ *… 2,000 resamples in total*

2,000 RESAMPLED ACCURACIES



SAME 70% ACCURACY, DIFFERENT CERTAINTY



Bootstrap interval for any metric

bootstrap_ci.py

```
import numpy as np
from sklearn.metrics import recall_score

rng = np.random.default_rng(7)
n = len(y_true)
stats = []
for _ in range(2000):
    idx = rng.integers(0, n, n)
    stats.append(recall_score(y_true[idx], y_pred[idx]))
low, high = np.percentile(stats, [2.5, 97.5])
print(f"recall interval: {low:.3f} to {high:.3f}")
```

Any metric

Swap recall for F1, AUC or a custom business metric.

Keep pairs

Resample predictions and labels together, row by row.

Groups

If rows cluster by user, resample whole users instead.

Tests for comparing two models

Test	Compares	Typical use
McNemar’s test	Disagreements on the same items	Two classifiers, one test set
Paired bootstrap	Metric difference on resamples	Almost any metric
Paired t test	Average difference across folds or items	Roughly normal differences
Wilcoxon signed rank	Ranks of paired differences	No normality assumption
DeLong test	Two ROC AUC values	Comparing AUCs on same data

Always compare models on the same test items. Paired comparisons remove much of the noise from item difficulty.

Statistical versus practical significance

STATISTICAL

Is the difference real?

- Asks whether chance could explain it
- Huge test sets make tiny gains significant
- Reported with p values or intervals
- Necessary but not sufficient

PRACTICAL

Does the difference matter?

- Asks whether it changes outcomes
- Judged against costs and effort
- Expressed as an effect size in business terms
- Decides whether to ship

VERDICT Ship only when a difference is both statistically credible and large enough to justify its cost and risk.

SECTION 13 · STATISTICAL RIGOUR

The multiple comparisons trap

Test enough models, slices or metrics and some will look better purely by chance. The more you look, the more false discoveries you make.

Example

Comparing 20 model variants at a 5 percent threshold will often produce one false winner.

Corrections

Bonferroni and Holm methods tighten thresholds when many tests are run.

Better habit

Pre register the main comparison and confirm surprises on fresh data.

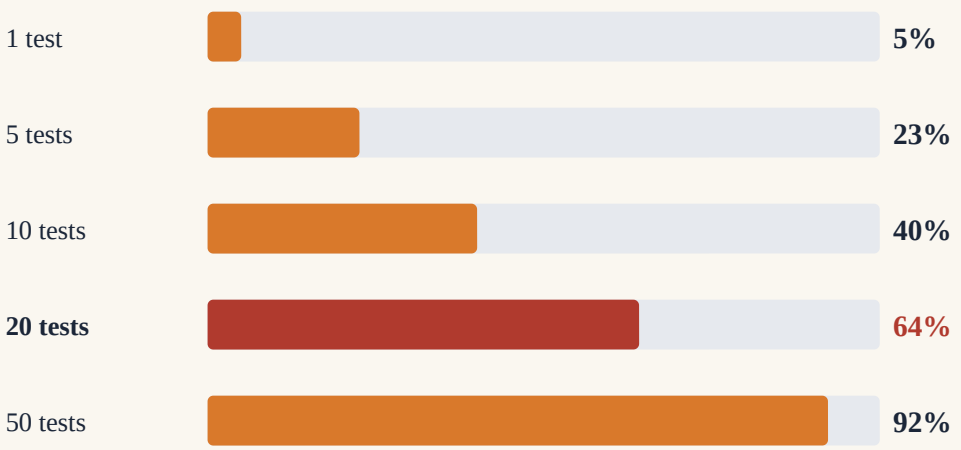
Twenty variants, no real winner, one false discovery

20 VARIANTS VS BASELINE · NONE TRULY BETTER

Variant 1 **p = 0.41**	Variant 2 **p = 0.77**	Variant 3 **p = 0.18**	Variant 4 **p = 0.62**	Variant 5 **p = 0.09**
Variant 6 **p = 0.55**	Variant 7 **p = 0.93**	Variant 8 **p = 0.27**	Variant 9 **p = 0.70**	Variant 10 **p = 0.36**
Variant 11 **p = 0.12**	Variant 12 **p = 0.84**	Variant 13 **p = 0.48**	**Variant 14 p = 0.03**	Variant 15 **p = 0.66**
Variant 16 **p = 0.22**	Variant 17 **p = 0.59**	Variant 18 **p = 0.31**	Variant 19 **p = 0.88**	Variant 20 **p = 0.15**

Variant 14 looks "significant". *It is pure luck: on noise alone, about 1 in 20 tests passes.*

CHANCE OF AT LEAST ONE FALSE WIN AT P < 0.05



$1 - 0.95^n$, assuming independent tests

BONFERRONI CORRECTION

Threshold 0.05 ÷ 20 = 0.0025. Variant 14 (p = 0.03) **no longer passes.**

HABIT Name the one comparison that matters before you look, then confirm any surprise on fresh data.

SECTION 13 · STATISTICAL RIGOUR

Variance from seeds and training

Random initialisation, data order and sampling make results vary between training runs, sometimes as much as the improvement being claimed.

Run several seeds

Train with at least three to five seeds and report the spread.

Compare distributions

Judge whether one method’s range sits clearly above the other’s.

Report honestly

State how many runs were done and how many were discarded.

SECTION 13 · STATISTICAL RIGOUR

How big should a test set be?

1 Rare events dominate

Precision on rare classes needs enough positives, not just rows.

2 Smaller effects need more data

Detecting a one point gain needs far more items than ten points.

3 Use power analysis

Estimate sample size before collecting labels.

4 Check interval width

If intervals are wider than the gain you care about, get more data.

5 Stratify sampling

Oversample important slices for labelling, then reweight.

6 Reuse carefully

Repeated use of one test set slowly overfits it.

Statistical pitfalls

PITFALL	BETTER PRACTICE
⚠ Declaring a winner from a 0.3 point gain.	✓ Report intervals and paired tests.
⚠ Testing many variants, reporting the best.	✓ Correct for multiple comparisons and confirm on fresh data.
⚠ One training run per method.	✓ Run several seeds and show the spread.
⚠ Bootstrapping rows when users repeat.	✓ Resample at the level of independent units.

Section recap

KEY TAKEAWAYS

- Every metric is an estimate with uncertainty
- Confidence intervals show the plausible range
- The bootstrap works for almost any metric
- Use paired tests on the same items
- Separate statistical from practical significance
- Correct for many comparisons and run several seeds

TRY THIS

Compute bootstrap intervals for two models' main metric and a paired bootstrap of their difference. Is the winner really ahead?

NEXT UP

Section 14: error analysis and slicing.

SECTION 14

14

Error analysis and slicing

Moving from how good is the model to where, why and for whom it fails.

IN THIS SECTION

- Why aggregates hide failures
- Slice based evaluation
- Reading the errors
- Error taxonomies
- Behavioural testing
- Prioritising fixes

Averages hide failures



A model can improve on average while getting worse for an important group. Slicing makes those hidden failures visible.

Hidden stratification

Rare but important subgroups vanish in overall metrics.

Simpson’s paradox

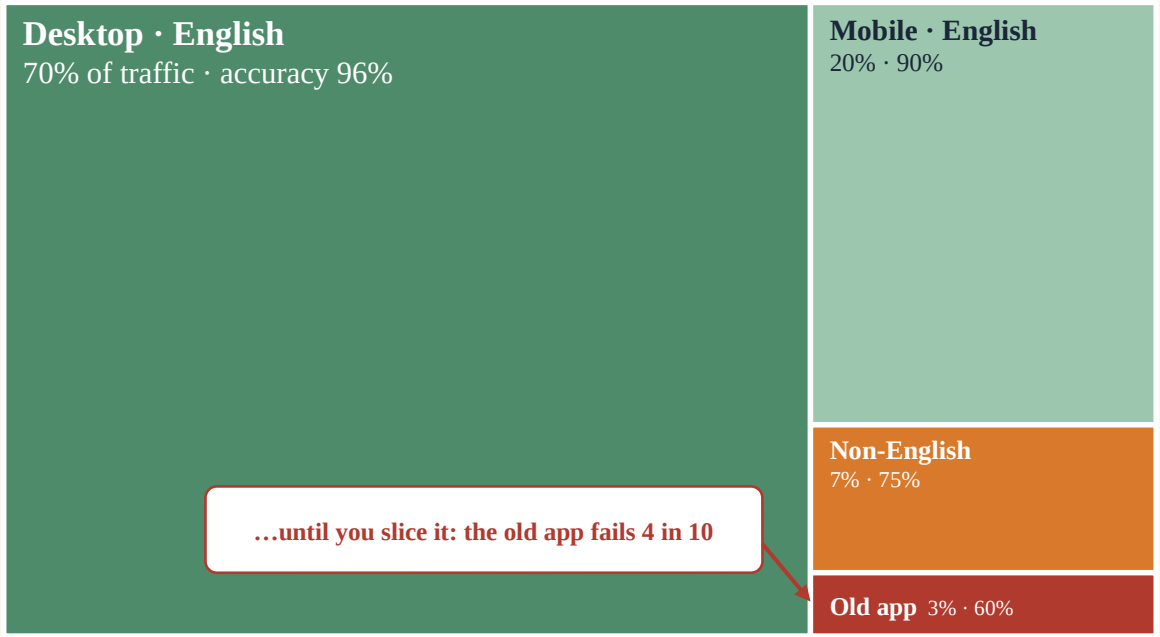
A model can win on every slice yet lose overall, or the reverse.

Business impact

Key customers, products or regions deserve their own metrics.

A 92% average hiding a 60% slice, and Simpson's paradox

Overall accuracy **92%** *looks great…*



Area = share of traffic · colour = accuracy

SIMPSON'S PARADOX IN A MODEL COMPARISON

Slice	Model A	Model B
Easy cases	270 / 300 = 90%	**92 / 100 = 92%** ✓
Hard cases	35 / 100 = 35%	**120 / 300 = 40%** ✓
Overall	**305 / 400 = 76%** ✓	212 / 400 = 53%

WHY IT FLIPS

B wins on both slices yet loses overall, because B was scored on 300 hard cases and A on only 100. The overall figure measures the case mix, not the model.

! Score both models on the same test set, and report key slices beside the average.

Useful ways to slice

1 Demographics

Age bands, regions, languages, where lawful and appropriate.

2 Data source

Channel, device, sensor, partner or upstream system.

3 Time

Hour, weekday, season and recent versus older data.

4 Input traits

Length, image quality, missing fields, rare categories.

5 Confidence

High, medium and low model confidence bands.

6 Business value

High value customers, critical products, priority cases.

SECTION 14 · ERROR ANALYSIS

A systematic error analysis

1. **Collect errors.** Gather a sample of false positives and false negatives.
2. **Read them.** Look at the raw records, text or images, not just numbers.
3. **Tag causes.** Label each error with one or more likely causes.
4. **Count categories.** Find which causes account for most errors.
5. **Estimate gains.** Estimate how much fixing each category would help.
6. **Fix and recheck.** Address the biggest category and evaluate again.

An example error taxonomy

Error category	Example	Typical fix
Label noise	Training label was simply wrong	Relabel and audit labels
Missing feature	Key information not in inputs	Add or engineer features
Rare pattern	Too few examples of this case	Collect or augment data
Ambiguous case	Even experts disagree	Clarify guidelines or abstain
Distribution shift	Inputs differ from training	Retrain on recent data
Model limitation	Pattern too complex for the model	Change architecture

Behavioural testing

Behavioural tests check capabilities directly with targeted examples, like unit tests for model behaviour.

Invariance

Changing a name or irrelevant word should not change the prediction.

Directional

Adding a clearly negative phrase should lower a sentiment score.

Minimum functionality

Simple, obvious cases must always be correct.

SECTION 14 · ERROR ANALYSIS

Prioritising what to fix



Quick wins

Many errors, little effort. Fix these first.

Major projects

Many errors, large effort. Plan and resource them.

Small polish

Few errors, little effort. Batch them together.

Defer

Few errors, large effort. Revisit later if impact grows.

Vertical axis: Share of errors (high at top)

Horizontal axis: Effort to fix (low at left)

SECTION 14 · ERROR ANALYSIS

Error analysis pitfalls

PITFALL	BETTER PRACTICE
⚠ Only looking at aggregate metrics.	✓ Read real errors and slice results.
⚠ Slices with tiny sample sizes.	✓ Show counts and intervals beside slice metrics.
⚠ Fixing the most interesting errors first.	✓ Prioritise by frequency, impact and effort.
⚠ One off analysis that is never repeated.	✓ Automate slice reports for every candidate model.

Section recap

KEY TAKEAWAYS

- Aggregates hide important failures
- Slice by group, source, time, input traits and value
- Read errors and tag their causes
- Use a taxonomy to count and prioritise
- Behavioural tests check specific capabilities
- Automate slice reports in your pipeline

TRY THIS

Read 50 errors from your model, tag their causes and present the top three categories with estimated gains.

NEXT UP

Section 15: fairness evaluation.

SECTION 15

15

Fairness evaluation

Checking that a model's benefits and harms are not unfairly concentrated on some people.

IN THIS SECTION

- Sources of unfairness
- Group fairness definitions
- Measuring disparities
- Impossibility results
- Individual fairness
- Documenting findings

SECTION 15 · FAIRNESS

Where unfairness comes from

1 Historical bias
Past decisions in the data reflect past discrimination.

2 Representation
Some groups have far fewer training examples.

3 Measurement
Labels or features are less accurate for some groups.

4 Proxies
Postcodes or names stand in for protected characteristics.

5 Aggregation
One model fits the majority and fails smaller groups.

6 Deployment
The model is used in ways or places it was not built for.

Common group fairness definitions

Definition	Idea in plain words	Concern it addresses
Demographic parity	Similar positive rates across groups	Unequal access to outcomes
Equal opportunity	Similar recall across groups	Qualified people missed
Equalised odds	Similar recall and false alarm rates	Unequal error burdens
Predictive parity	Similar precision across groups	Unequal meaning of a positive
Calibration by group	Scores mean the same in every group	Misleading risk scores

Which definition fits depends on context, law and stakeholder values. Choose deliberately and document why.

SECTION 15 · FAIRNESS

You cannot satisfy every definition at once

⚠ Research has shown that when base rates differ between groups, several fairness definitions are mathematically incompatible except in special cases.

Implication

Some trade off is unavoidable, so fairness is a decision, not just a calculation.

Involve people

Affected communities, legal and domain experts should help choose priorities.

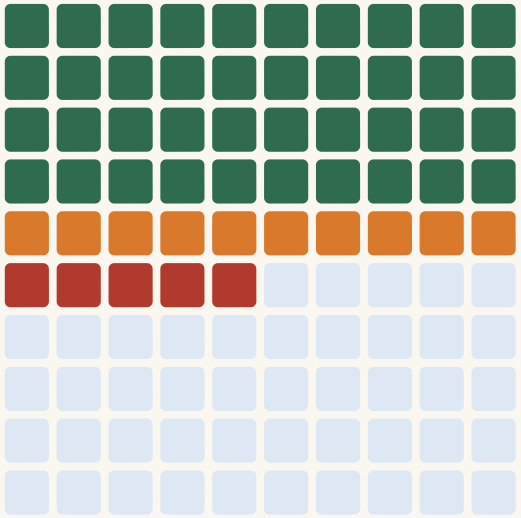
Be transparent

Report which definitions you met and which you did not.

Why fairness definitions collide: one model, two groups

Group A

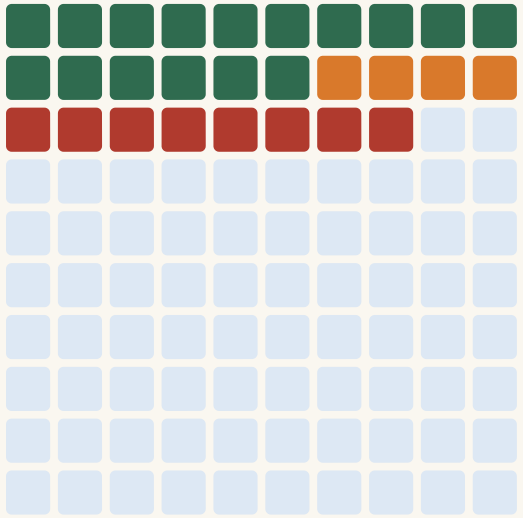
1,000 people · base rate 50%



TP 400 · FN 100 · FP 50 · TN 450

Group B

1,000 people · base rate 20%



TP 160 · FN 40 · FP 80 · TN 720

Caught (TP)

False alarm (FP)

Missed (FN)

Correctly cleared (TN)

Each square = 10 people

SCORECARD

Measure	A	B	Definition
True positive rate	80%	80%	✓ Equal opportunity
False positive rate	10%	10%	✓ Equalised odds
Precision	89%	67%	✗ Predictive parity
Selection rate	45%	24%	✗ Demographic parity

THE COLLISION

Raise B's precision to 89% and you must change its threshold, which moves its error rates away from A's. With different base rates, something has to give.

Choosing which one gives is a decision for people, not a calculation.

SECTION 15 · FAIRNESS

Measuring disparities in practice

1. **Define groups.** Identify relevant groups lawfully and with appropriate safeguards.
2. **Pick metrics.** Choose fairness metrics linked to the harms that matter.
3. **Compute per group.** Calculate metrics for each group with confidence intervals.
4. **Compare gaps.** Report differences or ratios between groups.
5. **Check intersections.** Look at combinations, such as age within gender.
6. **Set tolerances.** Agree acceptable gaps before deciding to ship.

SECTION 15 · FAIRNESS

Individual and counterfactual fairness

Group metrics can miss unfair treatment of individuals. Individual approaches ask whether similar people are treated similarly.

Similar treatment

Two applicants alike in relevant ways should receive similar scores.

Counterfactual tests

Change only a sensitive attribute or its proxy and check whether the decision flips.

Name swap tests

For text models, swap names associated with different groups and compare outputs.

Tools that support fairness evaluation

Tool	What it offers
Fairlearn	Group metrics, disparity dashboards and mitigation methods
AIF360	Large collection of fairness metrics and algorithms
What If Tool	Interactive exploration of model behaviour by slice
Cloud tooling	Bias reports in SageMaker Clarify, Vertex AI and Azure ML

SECTION 15 · FAIRNESS

Fairness pitfalls

PITFALL	BETTER PRACTICE
⚠ Dropping protected attributes and calling it fair.	→ ✓ Test for proxies and measure outcomes by group.
⚠ Reporting one fairness metric without context.	→ ✓ Explain the choice and show several metrics.
⚠ Ignoring small or intersectional groups.	→ ✓ Report counts and intervals for every group.
⚠ Fairness checked once at launch.	→ ✓ Monitor group metrics continuously in production.

SECTION 15 · FAIRNESS

Knowledge check

? A hiring screen has equal precision across two groups but much lower recall for one group. Is it fair?

MODEL ANSWER

It satisfies predictive parity but fails equal opportunity: qualified candidates from one group are missed far more often. Given the harm of being wrongly screened out, many would judge this unacceptable. The team should investigate causes and agree, with legal and stakeholder input, which criterion must hold.

WHY IT MATTERS

Different definitions can tell opposite stories about the same model.

SECTION 15 · FAIRNESS

Section recap

KEY TAKEAWAYS

- Unfairness enters through data, labels, proxies and deployment
- Group definitions measure different kinds of harm
- Several definitions cannot all hold at once
- Measure per group with intervals and intersections
- Test individuals with counterfactual and name swap checks
- Monitor fairness after launch, not only before

TRY THIS

Compute recall, false positive rate and precision per group for one model and write a short paragraph on which gaps matter and why.

NEXT UP

Section 16: robustness and uncertainty.

SECTION 16

16

Robustness and uncertainty

Evaluating how models behave when the world does not look like the test set.

IN THIS SECTION

- Distribution shift
- Perturbation tests
- Adversarial robustness
- Out of distribution detection
- Uncertainty estimation
- Selective prediction

SECTION 16 · ROBUSTNESS

Testing under distribution shift

Real data drifts away from training data. Robustness evaluation deliberately tests on data that differs in realistic ways.

Temporal shift

Evaluate on later periods than training.

Domain shift

Evaluate on new regions, hospitals, devices or customer types.

Report the drop

Show how much performance falls compared with the in distribution test.

Perturbation tests

1 **Noise**
Add realistic sensor, image or audio noise.

2 **Typos and casing**
Introduce spelling errors and capitalisation changes in text.

3 **Missing values**
Remove fields that may be missing in production.

4 **Unit and format changes**
Feed dates, currencies or units in new formats.

5 **Image changes**
Blur, rotate, crop and change lighting.

6 **Paraphrases**
Rephrase prompts or questions while keeping their meaning.

SECTION 16 · ROBUSTNESS

Adversarial robustness

Adversarial evaluation searches for small, deliberate input changes that flip predictions, revealing how easily an attacker could fool the model.

Why it matters

Fraud, spam, security and content moderation face active adversaries.

How to test

Use attack libraries or human red teams to find failing inputs.

Report

Accuracy under attack at a stated attack strength, not only clean accuracy.

Out of distribution detection

DEFINITION

Out of distribution detection evaluates whether a model can recognise inputs unlike anything it was trained on, so it can flag them instead of guessing confidently.

EXAMPLE

A skin lesion classifier receives a photo of a pet. A good system flags it as unfamiliar rather than confidently predicting a diagnosis.

REMEMBER

Evaluate how well the system separates familiar from unfamiliar inputs.

SECTION 16 · ROBUSTNESS

Evaluating uncertainty estimates

Uncertainty is useful only if it is meaningful: high uncertainty should go with more errors, and stated confidence should match reality.

Calibration

Confidence levels should match observed accuracy.

Error correlation

Uncertain predictions should be wrong more often than confident ones.

Methods

Ensembles, Monte Carlo dropout and conformal prediction produce uncertainty.

SECTION 16 · ROBUSTNESS

Conformal prediction

DEFINITION

Conformal prediction wraps any model to output a set or range of answers with a stated coverage guarantee, under the assumption that new data resembles the calibration data.

EXAMPLE

Instead of one diagnosis, a model returns the set pneumonia or bronchitis, built so that about 90 percent of such sets contain the true answer.

REMEMBER

Evaluate both coverage and the average size of the sets.

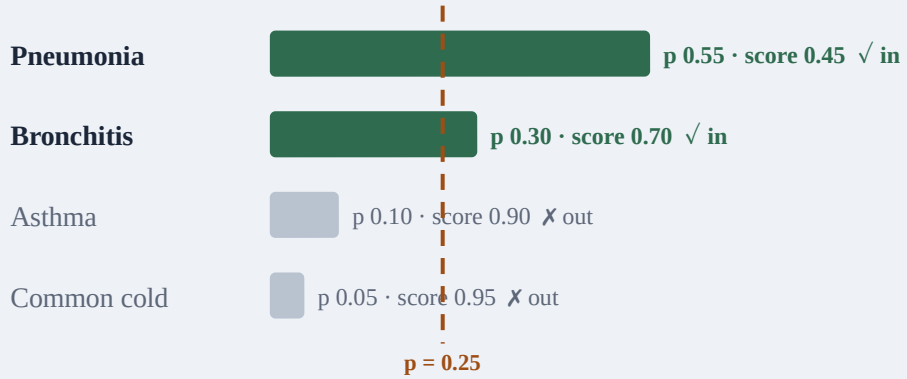
Conformal prediction: from calibration scores to a prediction set

1 **Calibration patients:** score = 1 − probability the model gave the true diagnosis, sorted

0.02	0.04	0.06	0.09	0.11	0.14	0.17	0.20	0.24	0.28	0.31	0.36	0.41	0.47	0.53	0.60	0.68	0.75	0.88
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	**18**	19

2 **Pick the cut-off:** for 90% coverage take rank $\lceil (19 + 1) \times 0.9 \rceil = 18$, so **q = 0.75**

3 **New patient:** keep every diagnosis with score ≤ 0.75, i.e. probability ≥ 0.25



PREDICTION SET

{ pneumonia, bronchitis }

SET SIZE SIGNALS UNCERTAINTY

Clear case → 1 label
Flu 0.93, others under 0.05

Murky case → 3 labels
0.30, 0.28, 0.26 all clear 0.25

! Guarantee: about 90% of sets contain the true diagnosis, provided new patients resemble the calibration patients.

SECTION 16 · ROBUSTNESS

Selective prediction and abstention

Letting a model abstain on uncertain cases and pass them to humans can make the automated portion far more reliable.

Coverage

Share of cases the model decides automatically.

Selective accuracy

Accuracy on the cases it chooses to decide.

Trade off curve

Plot accuracy against coverage to choose an operating point.

Robustness pitfalls

PITFALL	BETTER PRACTICE
⚠ Testing only on clean, curated data.	✓ Add realistic noise, shift and missing data tests.
⚠ Trusting raw confidence scores.	✓ Evaluate calibration and uncertainty quality.
⚠ Ignoring active adversaries.	✓ Red team models used in adversarial settings.
⚠ No path for unfamiliar inputs.	✓ Add detection and human escalation.

Section recap

KEY TAKEAWAYS

- Test on realistic shifted data and report the drop
- Perturbation tests reveal brittleness
- Adversarial tests matter when attackers exist
- Detect unfamiliar inputs rather than guessing
- Uncertainty must be calibrated and correlated with errors
- Abstention trades coverage for reliability

TRY THIS

Apply three realistic perturbations to your test set and report how much each lowers your main metric.

NEXT UP

Section 17: interpretability as part of evaluation.

SECTION 17

17

Interpretability checks

Using explanations to confirm the model is right for the right reasons.

IN THIS SECTION

- Why explanations aid evaluation
- Global importance
- Local explanations
- Catching shortcuts
- Evaluating explanations

Right for the right reasons

Accuracy shows a model is right. Explanations help show whether it is right for reasons that will still hold tomorrow.

Find shortcuts

Reveal reliance on watermarks, IDs or leaked fields.

Build trust

Domain experts can sanity check what drives predictions.

Support decisions

Explain individual outcomes to users and reviewers.

SECTION 17 · INTERPRETABILITY

Global explanation methods

1 Permutation importance

How much performance drops when a feature is shuffled.

2 Mean SHAP values

Average contribution of each feature across many predictions.

3 Partial dependence

How predictions change as one feature varies.

4 Accumulated effects

Similar to partial dependence but safer with correlated features.

5 Surrogate models

A simple model that imitates the complex one for inspection.

6 Coefficients

Direct weights in linear and logistic models.

SECTION 17 · INTERPRETABILITY

Local explanations

Local methods explain one prediction at a time, showing which inputs pushed this specific result up or down.

SHAP

Shares out a prediction among features using ideas from cooperative game theory.

LIME

Fits a simple model around one example to approximate local behaviour.

Counterfactuals

Shows the smallest change that would flip the decision.

SECTION 17 · INTERPRETABILITY

Shortcut learning

DEFINITION

Shortcut learning happens when a model relies on an easy but irrelevant signal that happens to correlate with the label in training data.

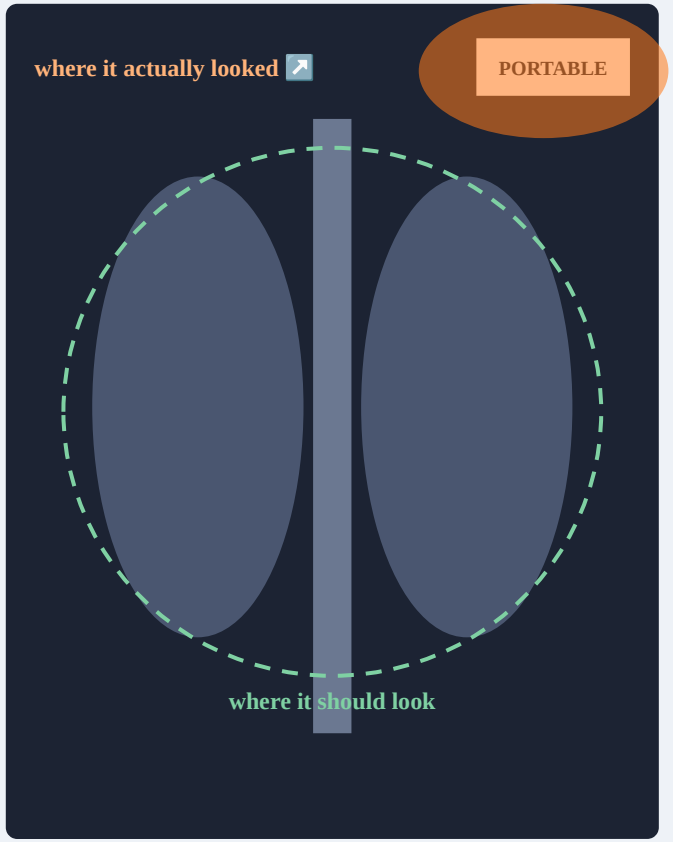
EXAMPLE

A chest scan model learned to detect a hospital's scanner marker, which was more common in sicker patients, instead of signs of disease.

REMEMBER

If explanations point at something an expert finds absurd, investigate before shipping.

Catching a shortcut: the scan model that read the label



Stylised scan with an explanation heat map

HOW THE TEAM PROVED IT

1 Explanation
Heat maps lit up the corner marker on most positive cases.

2 Mask the marker
Cover the corner and re-score the same test set.
AUC 0.93 → 0.78

3 New hospital
Score scans from a site with different scanners.
AUC 0.71

AUC values illustrative

SHORTCUTS SEEN ELSEWHERE

Wolf or husky?
Classifier keyed on snow in the background, not the animal.

Skin lesions
Rulers placed beside worrying lesions became a cue for malignancy.

Text tasks
Single words such as “not” predicted the label regardless of meaning.

TEST FOR IT
Mask, swap or shuffle the suspect signal. If scores collapse, you found a shortcut.

Interpretability pitfalls

PITFALL	BETTER PRACTICE
⚠ Treating explanations as ground truth.	✓ Check explanation stability and faithfulness.
⚠ Misreading importance as causation.	✓ Remember importance shows association, not cause.
⚠ Ignoring correlated features.	✓ Group correlated features or use robust methods.
⚠ Explaining only the averages.	✓ Inspect local explanations for errors and slices.

Section recap

KEY TAKEAWAYS

- Explanations help confirm the model uses sensible signals
- Global methods show overall drivers
- Local methods explain single predictions
- Shortcut learning is caught by expert review of explanations
- Explanations show association, not causation

TRY THIS

Generate permutation importance and five local explanations for your model. Ask a domain expert if anything looks wrong.

NEXT UP

Section 18: online evaluation and A/B testing.

SECTION 18

18

Online evaluation and A/B tests

Proving that a model creates real value for real users.

IN THIS SECTION

- Why online evaluation
- Designing an experiment
- Choosing metrics
- Sample size and duration
- Threats to validity
- Reading results

SECTION 18 · ONLINE EVALUATION

Why offline wins do not always transfer



Offline metrics measure predictions on historical data. Online tests measure what happens when predictions change behaviour.

Feedback effects

Users react to the model, creating data no offline test contained.

Proxy gaps

Better clicks offline may not mean more revenue or satisfaction.

System effects

Latency, interface and integration all shape real outcomes.

Designing an A/B test

1. **Hypothesis.** State what change you expect and why, in plain words.
2. **Unit.** Choose what is randomised, usually users rather than requests.
3. **Metrics.** Fix one primary metric, secondary metrics and guardrails.
4. **Size and duration.** Compute sample size and run full weekly cycles.
5. **Analysis plan.** Decide the statistical method and stopping rules upfront.
6. **Launch criteria.** Agree what result leads to ship, iterate or stop.

A hierarchy of experiment metrics

Primary metric	The single outcome the test is designed to move, such as conversion.
Secondary metrics	Supporting signals that explain how the change worked.
Guardrail metrics	Things that must not get worse, such as latency or complaints.
Model metrics	Online precision, coverage and prediction distributions.
Diagnostic metrics	Sample ratio, exposure counts and data quality checks.

Keep it focused

One primary metric decides the outcome. Guardrails can veto a launch even when the primary metric improves.

SECTION 18 · ONLINE EVALUATION

Sample size and duration

1 **Minimum detectable effect**

The smallest improvement worth detecting drives sample size.

2 **Baseline rate**

Rare outcomes need much larger samples.

3 **Variance**

Noisy metrics like revenue need more users than click rates.

4 **Full cycles**

Run at least one or two full weeks to cover weekday patterns.

5 **Variance reduction**

Methods such as CUPED use pre experiment data to reduce noise.

6 **Power calculators**

Use standard calculators before launch, not after.

Threats to validity

1 Peeking
Stopping as soon as results look good inflates false wins.

2 Sample ratio mismatch
Unequal group sizes signal a broken assignment.

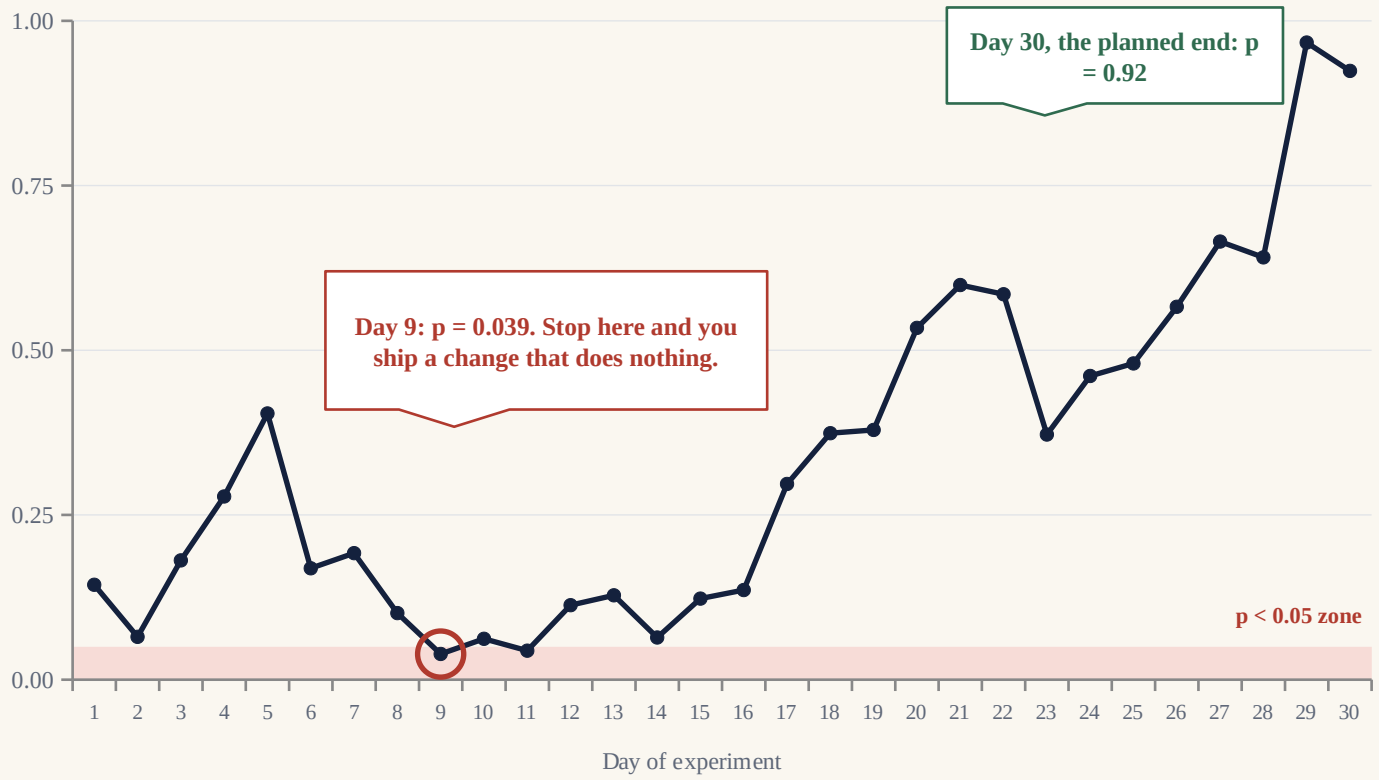
3 Novelty effect
Users react to change itself, then settle down.

4 Interference
Users in different groups affect each other, such as in marketplaces.

5 Multiple metrics
Checking many metrics produces chance wins.

6 Seasonality
Holidays or events during the test distort results.

Peeking: the p-value that wanders below 0.05 and back



Simulated A/A test: identical experience in both groups.

FALSE WIN RATE WITH DAILY PEEKING

≈ 27%

not 5%. From 4,000 simulated A/A tests checked daily for 30 days.

WAYS TO LOOK SAFELY

Fix it in advance

Set sample size and duration before launch; read once.

Use a sequential method

Tests designed for repeated looks keep the error rate honest.

Watch guardrails, not p

Monitor for harm daily; judge the primary metric at the end.

Reading A/B results responsibly

A good readout reports the effect size with its confidence interval, the guardrails and any validity checks, not just a p value.

Effect and interval

Conversion rose 1.2 percent, plausibly between 0.4 and 2.0 percent.

Guardrails

Latency, complaints and refunds stayed within limits.

Segments

Pre planned segment results, labelled exploratory if not planned.

SECTION 18 · ONLINE EVALUATION

A/B tests versus bandits for models

A/B TEST

Fixed allocation

- Clean, unbiased estimate of impact
- Some users get the worse model longer
- Clear yes or no decisions
- Best for major launches

BANDIT

Adaptive allocation

- Moves traffic towards the winner
- Lower cost of exploration
- Effect estimates are harder to interpret
- Best for many short lived options

VERDICT Use A/B tests when you need trustworthy causal evidence. Use bandits when optimising quickly among many options matters more.

Online evaluation pitfalls

PITFALL	BETTER PRACTICE
⚠ Stopping the test the moment it looks significant.	✓ Fix duration or use sequential methods.
⚠ Ignoring a sample ratio mismatch.	✓ Investigate assignment bugs before reading results.
⚠ Shipping on a primary win with broken guardrails.	✓ Let guardrails veto harmful launches.
⚠ Treating unplanned segment wins as proof.	✓ Confirm exploratory findings with a new test.

Knowledge check

? A new model lifts click through rate by 3 percent in week one, but the gain shrinks to zero by week three. What happened?

MODEL ANSWER

Most likely a novelty effect: users explored the changed recommendations at first, then returned to normal behaviour. Judge the model on the stable period, consider longer tests, and look at deeper outcomes such as purchases and retention.

WHY IT MATTERS

Short tests can mistake curiosity for lasting value.

Section recap

KEY TAKEAWAYS

- Online tests measure real behaviour and value
- Randomise users and pre register the analysis
- Use one primary metric plus guardrails
- Plan sample size and run full weekly cycles
- Guard against peeking, ratio mismatch and novelty
- Report effects with intervals, not just p values

TRY THIS

Write a one page experiment plan for your next model launch, including hypothesis, metrics, sample size and decision rules.

NEXT UP

Section 19: evaluation in production.

SECTION 19

19

Evaluation in production

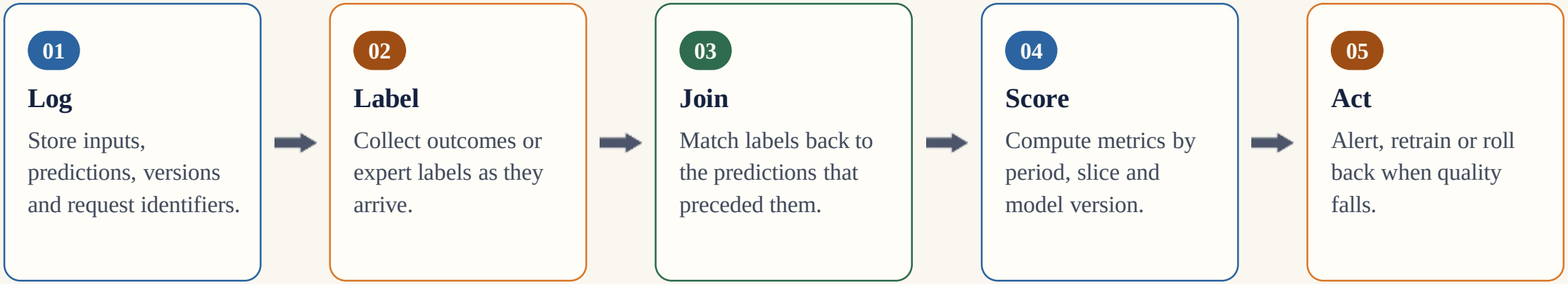
Evaluation does not stop at launch. It becomes a continuous process.

IN THIS SECTION

- Continuous evaluation
- Delayed and partial labels
- Drift as an early warning
- Evaluation gates in pipelines
- Model reports

SECTION 19 · PRODUCTION

The continuous evaluation loop



The same metric code used offline should run in production, so numbers are comparable across the model’s whole life.

SECTION 19 · PRODUCTION

Delayed and partial labels

Outcomes like defaults, churn or diagnoses can take months, and you only see outcomes for cases the model allowed through.

Proxies

Track early signals such as payment delays or user complaints.

Sampled labelling

Have experts label a random sample weekly to estimate accuracy.

Selection bias

Keep a small random holdout that bypasses the model to see unbiased outcomes.

SECTION 19 · PRODUCTION

Drift as an early warning

When labels are slow, shifts in inputs and predictions are the first sign that evaluation results may no longer hold.

Input drift

Changes in feature distributions compared with the training reference.

Prediction drift

Changes in the share of positives or the score distribution.

Treat as a prompt

Drift calls for investigation and fresh evaluation, not automatic panic.

Measuring input drift with the population stability index

Age band	Reference vs current share	Ref	Now	Contribution
18–25		10%	4%	0.055
26–35		20%	12%	0.041
36–45		30%	26%	0.006
46–60		25%	32%	0.017
60+		15%	26%	0.060

Training reference This month

PSI = 0.179

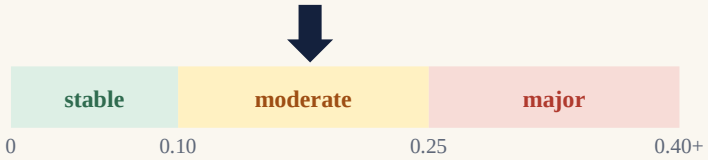
The two biggest contributions come from the extremes: younger customers fell, over-60s grew.

FORMULA

PSI = Σ (now − ref) × ln(now ÷ ref)

60+ band: (0.26 − 0.15) × ln(0.26 ÷ 0.15) = 0.11 × 0.55 = 0.060

COMMON RULE OF THUMB



NEXT STEP

Drift is a prompt, not a verdict. Check accuracy on the newest labelled data, then decide whether to retrain.

Evaluation gates in an ML pipeline

Gate	Check	Example rule
Data	Schema, ranges, freshness	No new nulls in key fields
Quality	Primary metric versus champion	Not worse beyond the interval
Slices	Key segment metrics	No segment drops beyond tolerance
Fairness	Group metric gaps	Within the agreed limit
Calibration	Reliability on recent data	Calibration error under target
Behaviour	Behavioural test suite	All critical tests pass

A standard model evaluation report

Summary	Version, data window and headline results with intervals.
Comparison	Champion versus challenger on all key metrics.
Slices	Per segment results with counts.
Responsible AI	Fairness, calibration and robustness checks.
Errors	Top error categories and example cases.
Decision	Recommendation, approver and date.



Make it routine

Generate the same report automatically for every candidate and every production period, so comparisons are effortless and consistent.

SECTION 19 · PRODUCTION

Production evaluation pitfalls

PITFALL	BETTER PRACTICE
⚠ No logged predictions to evaluate later.	→ ✓ Log predictions with versions and identifiers.
⚠ Only measuring outcomes the model allowed.	→ ✓ Keep a small random holdout for unbiased labels.
⚠ Different metric code offline and online.	→ ✓ Share one evaluation library everywhere.
⚠ Reports produced by hand occasionally.	→ ✓ Automate reports on a fixed schedule.

SECTION 19 · PRODUCTION

Section recap

KEY TAKEAWAYS

- Evaluation continues for the model's whole life
- Join delayed labels back to logged predictions
- Use proxies and sampled labels meanwhile
- Treat drift as an early warning
- Gate releases with automated checks
- Standardise and automate evaluation reports

TRY THIS

Design a weekly evaluation report for a live model: data sources, metrics, slices, owners and the actions triggered.

NEXT UP

Section 20: an end to end case study.

SECTION 20

20

End to end case study

Evaluating a fictional hospital readmission risk model from first question to live use.

IN THIS SECTION

- Framing the problem
- Splits and baselines
- Metrics and thresholds
- Fairness and calibration
- Online evaluation
- Lessons

SECTION 20 · CASE STUDY

The scenario

A fictional hospital wants to identify patients at high risk of readmission within 30 days so nurses can arrange extra follow up.

Decision

Patients above a threshold receive a follow up phone call and care review.

Capacity

Nurses can manage about 40 extra follow ups per day.

Stakes

Missed high risk patients may return sicker. Unneeded calls cost staff time.

SECTION 20 · CASE STUDY

Evaluation decisions at framing

Question	Decision
Split	Train on earlier years, test on the most recent year, grouped by patient
Baselines	Current nurse checklist and a simple logistic regression
Primary metric	Recall among the top 40 daily patients
Secondary	Precision at 40, average precision, calibration
Fairness	Recall and calibration across age, sex and deprivation bands
Online	Stepped rollout comparing wards, with readmission rate as outcome

SECTION 20 · CASE STUDY

Offline evaluation results path

01
Leakage audit
Removed a discharge code entered after readmission.

02
Baselines
Checklist and logistic regression scored first.

03
Candidates
Gradient boosting compared with intervals across folds.

04
Slices
Weaker recall found for patients over 85.

05
Calibration
Isotonic calibration applied on validation data.

The leakage audit removed the single most powerful feature. The headline metric dropped, but the model became honest.

SECTION 20 · CASE STUDY

Choosing the threshold



Rather than a probability cut off, the team ranked patients daily and flagged the top 40, matching nurse capacity exactly.

Capacity first

Evaluation used recall and precision at 40 per day, the real operating point.

Cost view

Estimated nurse hours against avoided readmissions for each candidate model.

Calibrated scores

Calibrated risks let clinicians read scores as meaningful percentages.

One day's top 40 queue, the way the nurses see it

MODEL'S TOP 40 OF 300 DISCHARGES · RANKED BY RISK

#1 readmitted	#2 readmitted	#3 fine	#4 readmitted	#5 readmitted	#6 fine	#7 readmitted	#8 fine
#9 readmitted	#10 fine	#11 readmitted	#12 fine	#13 fine	#14 readmitted	#15 fine	#16 fine
#17 readmitted	#18 fine	#19 fine	#20 readmitted	#21 fine	#22 fine	#23 fine	#24 readmitted
#25 fine	#26 fine	#27 fine	#28 fine	#29 readmitted	#30 fine	#31 fine	#32 fine
#33 readmitted	#34 fine	#35 fine	#36 fine	#37 fine	#38 readmitted	#39 fine	#40 fine

Hits cluster near the top: the ranking works best where nurses start calling.

Readmitted within 30 days (14)

Not readmitted (26)

PRECISION AT 40

35%

14 ÷ 40 calls

RECALL AT 40

39%

14 ÷ 36 readmissions

LIFT

2.9×

35% ÷ 12% base rate

SAME CAPACITY, THREE RANKINGS

	Checklist	Model	Perfect
Readmitted in top 40	9	**14**	36
Precision at 40	23%	**35%**	90%
Recall at 40	25%	**39%**	100%
Lift over random	1.9×	**2.9×**	7.5×

! Report against capacity and against the ceiling: a perfect ranker would reach 90% precision, not 100%, because only 36 of 40 calls can be hits.

SECTION 20 · CASE STUDY

Fairness findings



Slicing showed patients over 85 had lower recall because many lived in care homes, which the data captured poorly.

Root cause

A measurement gap: care home status was missing for many records.

Action

Added a reliable residence feature and checked recall again with intervals.

Governance

Documented remaining gaps and set monitoring on age band recall.

SECTION 20 · CASE STUDY

Online evaluation plan

1. **Shadow.** Scored patients silently for a month and compared with the checklist.
2. **Clinician review.** Nurses reviewed a sample of flags to check clinical sense.
3. **Stepped rollout.** Wards adopted the model in stages for comparison.
4. **Outcomes.** Tracked 30 day readmissions, call completion and workload.
5. **Guardrails.** Watched missed high risk patients and nurse overtime.
6. **Monthly report.** Automated slices, calibration and drift for governance review.

SECTION 20 · CASE STUDY

Lessons from the case study

The most valuable evaluation work was not choosing the algorithm. It was removing leakage, matching capacity and finding who was underserved.

Honesty first

A lower but honest score beats a higher leaked one.

Operating point

Metrics at real capacity made results meaningful to nurses.

Slices matter

Averages hid the weakest group, the oldest patients.

Section recap

KEY TAKEAWAYS

- Frame the decision, capacity and stakes first
- Audit leakage before trusting any metric
- Evaluate at the real operating point
- Calibrate scores people will read as risks
- Slice for fairness and fix measurement gaps
- Confirm value with careful online evaluation

TRY THIS

Rewrite the framing table for a model in your own domain, including split, baselines, metrics, fairness and online plan.

NEXT UP

Section 21: checklists, glossary and final quiz.

SECTION 21

21

Checklists, glossary and next steps

Reusable tools for every evaluation you run from now on.

IN THIS SECTION

- Offline evaluation checklist
- Responsible AI checklist
- Launch and online checklist
- Anti patterns
- Glossary
- Final knowledge check

SECTION 21 · TOOLKIT

Offline evaluation checklist

- [ ] Business goal and error costs written down
- [ ] Split respects time, groups and duplicates
- [ ] Leakage audit completed
- [ ] Simple baselines evaluated
- [ ] Metrics chosen before experiments
- [ ] Threshold chosen on validation data
- [ ] Confidence intervals reported
- [ ] Several seeds or folds compared
- [ ] Test set opened only once
- [ ] Results reproducible from code

SECTION 21 · TOOLKIT

Responsible AI checklist

- [ ] Slices by key segments reported
- [ ] Fairness metrics chosen and justified
- [ ] Intersectional groups examined
- [ ] Calibration checked overall and by group
- [ ] Robustness to realistic shifts tested
- [ ] Out of distribution handling defined
- [ ] Explanations reviewed by experts
- [ ] Limitations written in the model card
- [ ] Sign off from risk or legal where needed
- [ ] Monitoring plan covers group metrics

SECTION 21 · TOOLKIT

Launch and online checklist

- [ ] Shadow or offline replay completed
- [ ] Experiment hypothesis written
- [ ] Primary, secondary and guardrail metrics set
- [ ] Sample size and duration calculated
- [ ] Randomisation unit chosen
- [ ] Sample ratio checks automated
- [ ] Stopping rules agreed in advance
- [ ] Predictions logged with versions
- [ ] Label collection and joining planned
- [ ] Evaluation report scheduled

SECTION 21 · TOOLKIT

Evaluation anti patterns

PITFALL	BETTER PRACTICE
⚠ The leaderboard chaser: tuning until the test set agrees.	→ ✓ Keep a locked test set and fresh confirmation data.
⚠ The single number: one metric, no interval, no baseline.	→ ✓ Report several metrics, baselines and uncertainty.
⚠ The average illusion: no slices, no fairness checks.	→ ✓ Slice results and measure group gaps.
⚠ The launch and forget: no evaluation after release.	→ ✓ Evaluate continuously with logged predictions.

The whole evaluation journey, with its four traps



SECTION 21 · TOOLKIT

Glossary, part one

Term	Meaning
Baseline	A simple reference model or rule the model must beat
Bootstrap	Resampling the test set to estimate uncertainty
Calibration	Agreement between predicted probabilities and observed frequencies
Confusion matrix	Table of true and false positives and negatives
Cross validation	Rotating validation folds for stable estimates
Data leakage	Information unavailable at prediction time used in evaluation
F1 score	A balance of precision and recall in one number
Generalisation	Performance on new, unseen data

SECTION 21 · TOOLKIT

Glossary, part two

Term	Meaning
Guardrail metric	A metric that must not worsen during an experiment
NDCG	Ranking metric rewarding relevant items placed near the top
Precision	Share of flagged items that are truly positive
Recall	Share of true positives the model found
ROC AUC	Probability a positive is ranked above a negative
Slice	A subgroup of data evaluated separately
Stratification	Keeping class or group proportions consistent across splits
Test set	Data held back for one final, honest estimate

SECTION 21 · TOOLKIT

Final knowledge check 1

? Your model beats the champion by 0.4 points of AUC on one test run. What must you check before claiming success?

MODEL ANSWER

Compute confidence intervals and a paired comparison on the same items, run several seeds, confirm no tuning happened on the test set, and check that slices and calibration did not worsen. Then judge whether the gain is large enough to matter for the business.

WHY IT MATTERS

Small differences are often noise until proven otherwise.

SECTION 21 · TOOLKIT

Final knowledge check 2

? A fraud model is evaluated with a random split and scores far better than in production. What is the likely problem?

MODEL ANSWER

The random split probably mixed time periods and possibly the same customers across training and test, leaking future patterns and identities. Evaluate again with a time based, group aware split and audit features for target and temporal leakage.

WHY IT MATTERS

Split design often matters more than model choice.

SECTION 21 · TOOLKIT

Final knowledge check 3

? An LLM assistant scores well with an LLM judge but users complain answers are wrong. What should you do?

MODEL ANSWER

Validate the judge against expert human ratings, add factual correctness and faithfulness checks to the rubric, expand the golden set with real user failures, and score retrieval separately if the system uses RAG.

WHY IT MATTERS

An unvalidated judge can confidently measure the wrong thing.

SECTION 21 · TOOLKIT

Your learning path from here

01 — Practise
Run the offline checklist on one existing model.

02 — Quantify
Add intervals, seeds and paired tests.

03 — Deepen
Add slices, fairness and robustness checks.

04 — Go live
Design and run a careful A/B test.

05 — Automate
Build evaluation reports into your pipeline.

Recommended reading: Trustworthy Online Controlled Experiments by Kohavi, Tang and Xu, and the Fairlearn and sklearn model evaluation guides.

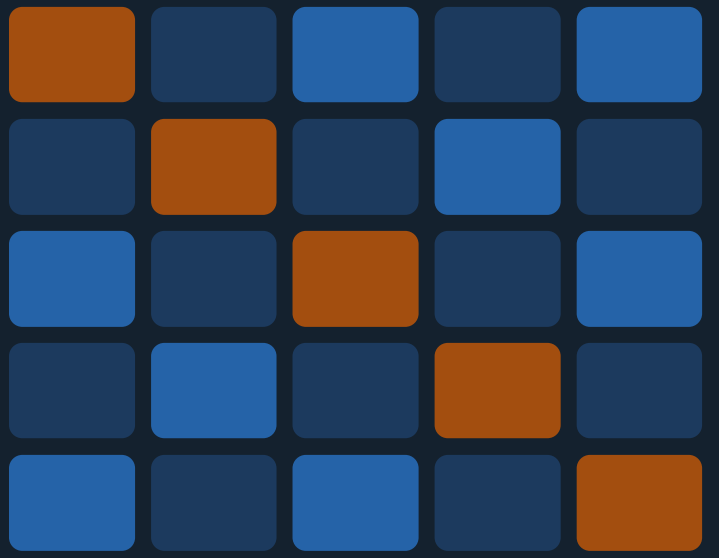
Good evaluation turns a promising model into a trustworthy one.

☆ Thank you for completing this tutorial. Measure honestly, question surprising results, and let evidence decide what ships.

A COMPLETE PRACTITIONER HANDBOOK

Validating Machine Learning Models

From the first data split to monitoring in production: how to know, with evidence, that a model will work on data it has never seen.



BEFORE WE BEGIN

ML engineers

You train models and need a repeatable, defensible way to decide which ones ship.

Data scientists

You report results to stakeholders and want numbers that survive scrutiny.

Leads and reviewers

You sign off on models and need to know which questions to ask.

Students

You want the practical habits that textbooks mention only in passing.

KEY IDEA

The single sentence to remember

Validation is the discipline of producing **honest evidence** about how a model will behave on data it has not seen, under the conditions it will actually face.

BEFORE WE BEGIN

What you will be able to do by the end

- Choose the right data split for random, grouped, and time-ordered data
- Run and interpret cross-validation, including nested cross-validation
- Pick metrics that match the business cost of each kind of error
- Detect and prevent the many forms of data leakage
- Quantify uncertainty so a result is a range, not a single lucky number
- Find the slices where a model quietly fails
- Test fairness, robustness and calibration before launch
- Evaluate LLMs and generative systems with rubrics and judges
- Validate safely in production with shadow, canary and A/B tests
- Write a validation report a sceptical reviewer will accept

HOW TO READ THIS DECK

Every text box has a job

Throughout the tutorial, coloured boxes signal what kind of information you are looking at. Learn the code once and you can skim any slide in seconds.

KEY IDEA

The core concept of the slide. If you read nothing else, read these.

PITFALL

A mistake that teams make in real projects, often without noticing.

PRO TIP

Practical advice from experience that saves time or avoids rework.

ANALOGY

An everyday comparison that makes an abstract idea concrete.

DEFINITION

A precise meaning for a term, written in plain language.

IN THE WILD

A realistic scenario showing the idea at work in an actual project.

HOW TO READ THIS DECK

More box types you will meet

MYTH

A popular belief that sounds right but leads you astray.

REALITY

What actually holds true, and why.

RULE OF THUMB

A quick heuristic to use when you have no time to think deeply.

CHECK YOURSELF

A short question to test your understanding before moving on.

INTERVIEW ANGLE

How this topic tends to come up in technical interviews.

LESSON

The transferable takeaway from a case study.

ROADMAP

Nineteen sections, one journey

01 Why validation matters	02 Foundations of generalisation	03 Splitting your data	04 Cross-validation in depth
05 Classification metrics	06 Regression metrics	07 Metrics for other tasks	08 Imbalance, thresholds, calibration
09 Data leakage	10 Tuning and model selection	11 Uncertainty and statistics	12 Error analysis and robustness
13 Fairness and responsibility	14 Explainability as validation	15 LLMs and generative models	16 Validation in production
17 Governance and MLOps	18 End-to-end case studies	19 Toolkit and cheat sheets	

THE PREMISE

A model is only as trustworthy as the evidence you gathered about it.

Training produces a model. Validation produces the evidence. Without the second, the first is a guess with good manners.

THE MINDSET

Six principles that run through everything

1

Simulate the future

Every test should mimic how the model will meet new data in real life.

2

Protect the test set

Look at it as rarely as possible. Every peek spends some of its honesty.

3

Measure what matters

Metrics should reflect the real costs of errors, not convenience.

4

Report ranges

One number hides luck. Show the spread you would expect on a rerun.

5

Look beneath averages

Strong overall scores can hide groups where the model fails badly.

6

Keep validating

The world drifts. Validation continues after launch, not just before.

SECTION 01

01

Why validation matters

What goes wrong when models are not validated properly, and what good validation buys you.

- Cost of failure
- Training vs real performance
- Stakeholders
- What validation is not

THE CORE PROBLEM

Training scores flatter every model

A model is optimised to do well on the examples it was trained on. So its performance on those examples tells you how well it memorised, not how well it will cope with tomorrow's data.

ANALOGY

A student who has seen the exam paper in advance will score highly. The score says nothing about whether they understand the subject. Training accuracy is that pre-seen exam.

KEY IDEA

We need a separate, untouched set of examples that plays the role of the future. Everything in validation is about building and protecting that stand-in.

THE COST OF GETTING IT WRONG

When validation fails, someone pays

Financial loss

A credit model that looked excellent offline approves risky loans because the test set leaked future information.

Harm to people

A triage model under-detects illness in a group that was rare in the validation data.

Lost confidence

Stakeholders see the live numbers fall short of the promise and stop trusting the team’s results.

Wasted time

Months of engineering go into deploying a model that never had a real edge over the simple baseline.

REMEMBER

Most of these failures are not caused by bad algorithms. They come from evidence that looked solid but answered the wrong question.

A FAMILIAR STORY

The offline–online gap

1. **Great offline score**
   The team reports a big improvement on the held-out set.
2. **Launch**
   The model goes live with an announcement and a dashboard.
3. **Disappointing live results**
   Business metrics barely move, or get worse.
4. **Investigation**
   Leakage, a split that ignored time, or a metric that did not match the goal is found.
5. **Rework**
   The fix takes longer than doing validation properly in the first place.

IN THE WILD

A churn model scored superbly in testing. Later it emerged that one feature was set by the retention team **after** a customer had already announced they were leaving. Offline, it was a near-perfect signal. Live, it simply did not exist yet at prediction time.

WHAT VALIDATION GIVES YOU

Four things good validation buys

01

An honest estimate

How well the model will do on new data, with a realistic range around it.

02

A fair comparison

Confidence that model B is genuinely better than model A, not just luckier.

03

Known weaknesses

A map of where the model fails, so you can fix it, guard it or warn users.

04

A defensible decision

Evidence you can show to a reviewer, regulator or customer.

CLEARING UP TERMS

Validation, evaluation, verification, testing

Term	What it usually means	Question it answers
Evaluation	Measuring a model’s performance with chosen metrics	How good is it?
Validation	Gathering evidence the model is fit for its intended use	Should we trust it for this job?
Verification	Checking the system was built as specified	Did we build it right?
Testing	Software checks on code, data and pipelines	Does it behave as designed?
Validation set	Data held out for tuning and model choice	Which version do we pick?
Test set	Data held out for the final, one-off estimate	How will the chosen model do?

⚠ PITFALL

Some fields swap the names “validation set” and “test set”. Always state which set was used for what in any report, so nobody has to guess.

CLEARING UP MISCONCEPTIONS

What validation is not

MYTH

Validation is a single step at the end of a project.

REALITY

It shapes the project from the first split to post-launch monitoring.

MYTH

A high accuracy number means the model is ready.

REALITY

One number can hide imbalance, leakage or failing groups.

MYTH

If cross-validation looks good, the model is safe.

REALITY

Cross-validation is only as good as the way folds mirror real use.

THE BIGGER PICTURE

Validation sits across the whole lifecycle

1 → 2 → 3 → 4 → 5 → 6

Frame

Define the decision, users and cost of errors.

Split

Design held-out data that mimics the future.

Develop

Train and tune using validation data only.

Confirm

Final test, slices, fairness and robustness.

Deploy

Shadow, canary and online experiments.

Monitor

Track drift and live performance continuously.

PRO TIP

Write a one-page **validation plan** before you train anything: the split, the metrics, the baselines and the pass criteria. It stops you moving the goalposts once results come in.

WHO CARES ABOUT THE RESULT

Different stakeholders, different questions

Stakeholder	Their real question	Evidence they need
Product owner	Will this improve the outcome we care about?	Business-aligned metric, online test
Engineering	Will it be reliable and fast enough?	Latency, failure modes, robustness tests
Risk and compliance	Can we defend it if challenged?	Documentation, fairness checks, audit trail
End users	Will it treat me fairly and correctly?	Slice performance, explanations
Leadership	Is it worth the cost and risk?	Gain over baseline with uncertainty

KEY IDEA

A validation report should answer every row, in the language of each audience.

SECTION 1 RECAP

Why validation matters

REMEMBER

Takeaways

- Training scores measure memory, not skill
- Failures usually come from flawed evidence, not flawed algorithms
- Validation spans the whole lifecycle

CHECK YOURSELF

Your team reports 98% accuracy on the data the model was trained on. What is the first question you should ask?

ANSWER

“What is the score on data the model never saw during training or tuning, and how was that data chosen?”

SECTION 02

02

Foundations of generalisation

The ideas that explain why models fail on new data: overfitting, underfitting, bias, variance and the assumptions behind every test.

Generalisation | Overfitting | Underfitting | Bias and variance | Learning curves | Baselines

THE GOAL

Generalisation is the whole point

DEFINITION

Generalisation

Generalisation is a model's ability to perform well on new examples drawn from the same situation it will be used in, not just on the examples it learned from.

IN PLAIN WORDS

We want a model that has learned the underlying pattern, not one that has memorised the particular examples it happened to see.

ANALOGY

A good cook understands why recipes work and can improvise with new ingredients. A poor one can only reproduce the dishes they practised.

FAILURE MODE ONE

Overfitting: learning the noise

An overfit model has captured quirks, coincidences and noise in the training data as if they were real patterns. It looks brilliant on familiar data and stumbles on anything new.

WARNING SIGNS

- Training score far better than validation score
- Performance jumps around between folds or reruns
- Very complex model on a small dataset
- Predictions hinge on oddly specific features

ANALOGY

Memorising that every photo of a wolf in your training set has snow in it, then calling any dog in the snow a wolf.

THE FIX

- More or better data
- Simpler model or stronger regularisation
- Early stopping
- Fewer, more meaningful features

FAILURE MODE TWO

Underfitting: missing the pattern

An underfit model is too simple, too constrained, or too poorly fed to capture the real structure. It does badly everywhere, including on its own training data.

WARNING SIGNS

- Training and validation scores are both poor
- Scores barely improve with more data
- Errors follow a clear, systematic pattern
- Model barely beats a trivial baseline

ANALOGY

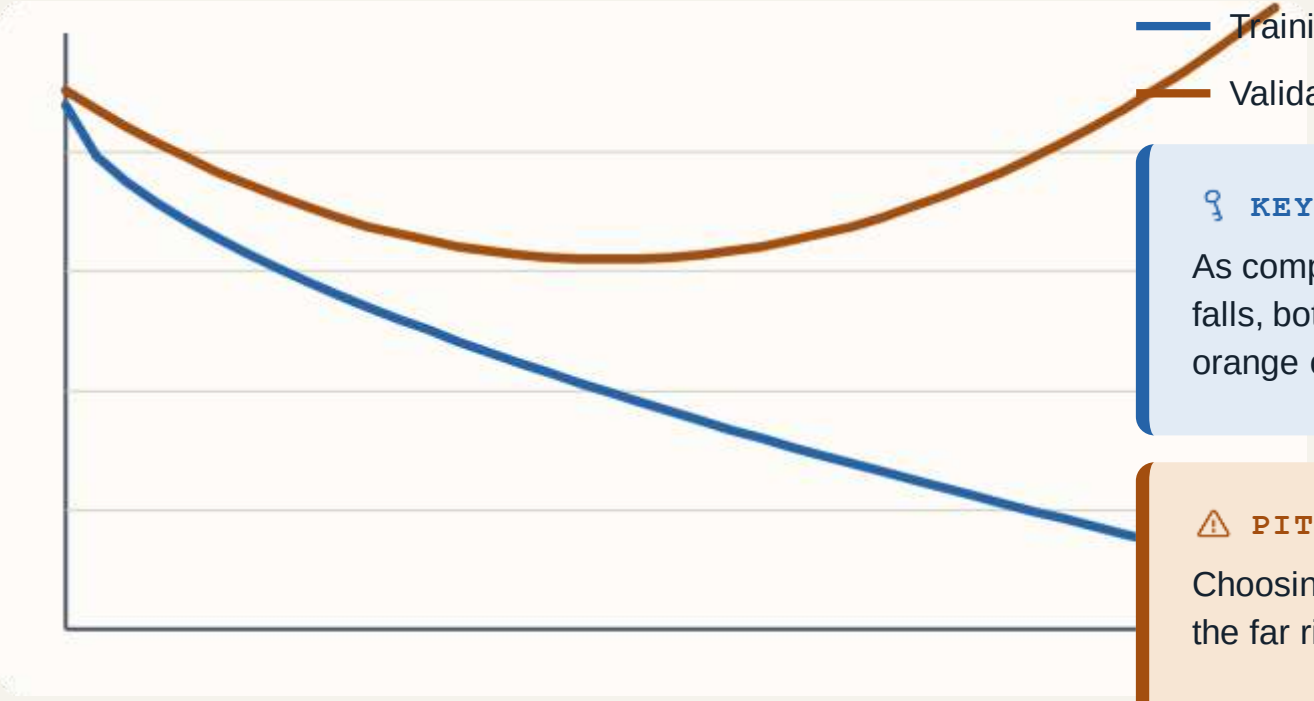
Trying to describe a winding road using only a straight ruler. No amount of extra practice will make the ruler bend.

THE FIX

- Richer features or representations
- More flexible model
- Less regularisation
- Train longer or tune better

THE CLASSIC PICTURE

Complexity versus error



Across: model complexity · Up: error

KEY IDEA

As complexity grows, training error keeps falling. Validation error falls, bottoms out, then rises. The sweet spot is the bottom of the orange curve.

PITFALL

Choosing the model with the lowest training error always picks the far right of this chart: the most overfit option.

TWO SOURCES OF ERROR

Bias and variance, without the maths

Bias

Error from wrong assumptions. A high-bias model is consistently off in the same direction, no matter which sample it learns from. It underfits.

Variance

Error from sensitivity to the particular sample. A high-variance model changes a lot when trained on a slightly different dataset. It overfits.

Irreducible noise

Randomness in the world that no model can predict. It sets a floor on achievable error.

ANALOGY

Think of archers. A biased archer hits a tight cluster in the wrong place. A high-variance archer scatters arrows all around the target. A good archer is both accurate and consistent.

TWO SOURCES OF ERROR

Reading bias and variance from your results

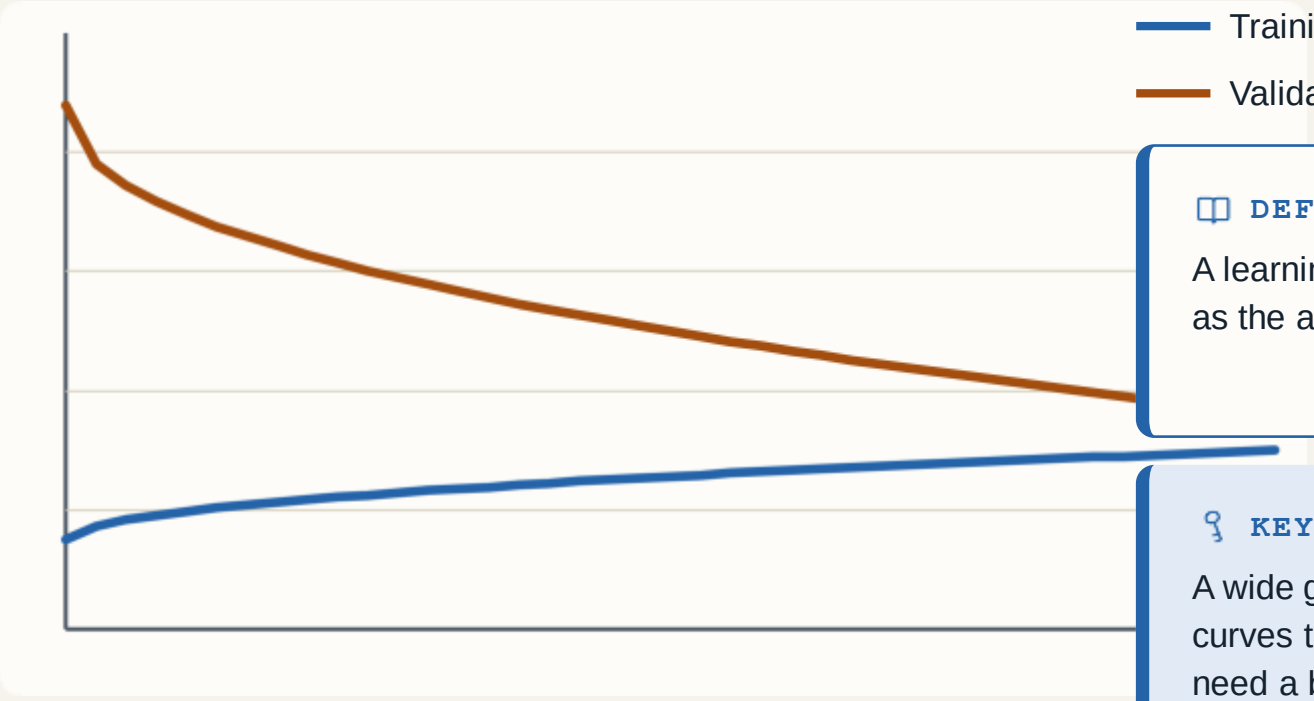
What you see	Likely problem	What to try
Poor training score, poor validation score	High bias (underfitting)	More capacity, better features
Great training score, poor validation score	High variance (overfitting)	More data, regularisation, simpler model
Good training score, good validation score	Healthy fit	Confirm on test and slices
Validation score varies wildly across folds	High variance or small data	More data, repeated CV, simpler model
Both scores fine, live performance poor	Not bias or variance: a mismatch	Check leakage and data drift

PRO TIP

The last row matters most in practice. When offline and online disagree, suspect the **validation design**, not the model.

DIAGNOSTIC TOOL

Learning curves



Across: training set size · Up: error

DEFINITION

A learning curve plots performance on training and validation data as the amount of training data increases.

KEY IDEA

A wide gap that is still closing means more data will help. Two curves that have met at a poor level mean more data will not; you need a better model or features.

DIAGNOSTIC TOOL

Reading learning curves

Gap still closing

Variance problem. Collecting more data is likely to pay off.

Converged high

Bias problem. More data will not help much; change the model or features.

Converged low

Healthy. Further gains will need new information, not more of the same.

Erratic

Too little data per point or unstable training. Repeat with several seeds.

PRO TIP

Learning curves are cheap insurance before a costly data-collection project. They tell you whether buying more labels is likely to be worth it.

DIAGNOSTIC TOOL

Validation curves for one setting

DEFINITION

A validation curve shows training and validation scores as you vary a single hyperparameter, such as tree depth or regularisation strength.

WORKED EXAMPLE

For tree depth: shallow trees score poorly on both sets; very deep trees score perfectly on training but worse on validation. The depth where validation peaks is your candidate.

PITFALL

Picking the peak of a validation curve is itself a form of fitting. Once you choose based on it, that validation score becomes optimistic. You still need an untouched test set or nested cross-validation.

THE HIDDEN ASSUMPTION

Same conditions in, same conditions out

Every standard validation method assumes that the data you test on comes from the same process as the data the model will see in use. When that assumption breaks, even perfect validation gives the wrong answer.

DEFINITION

Covariate shift: the inputs change, but the relationship to the outcome stays the same.

DEFINITION

Label shift: how common each outcome is changes, for example fraud rises sharply.

DEFINITION

Concept drift: the relationship itself changes, for example customer behaviour after a price rise.

KEY IDEA

Validation design is mostly about making the held-out data resemble the deployment world as closely as possible.

ALWAYS COMPARE

Baselines: the most underrated validation tool

A score means nothing on its own. Is 85% good? Only if the simplest sensible approach gets much less.

RULE OF THUMB

No model ships until it beats a trivial baseline **and** the best simple baseline, by a margin larger than the uncertainty in the scores.

Baseline	Example
Majority class	Always predict the most common label
Mean or median	Always predict the average value
Last value	Tomorrow equals today in forecasting
Seasonal naive	Same as this day last week
Simple rule	The heuristic the business already uses
Linear model	A plain regression on key features

ALWAYS COMPARE

Why baselines catch so many problems

IN THE WILD

A model scores 99% accuracy detecting a rare defect. The majority-class baseline also scores 99%, because 99% of parts have no defect. The model learned nothing useful.

IN THE WILD

A demand forecast looks sophisticated, but “same as last week” is only slightly worse. The complex pipeline adds cost and fragility for a tiny gain.

IN THE WILD

A model beats the baseline by a wide margin. Suspiciously wide. Investigation finds a leaked identifier. Baselines expose too-good-to-be-true results, too.

SECTION 2 RECAP

Foundations of generalisation

REMEMBER

Takeaways

- Overfitting learns noise; underfitting misses signal
- Bias is consistent error; variance is instability
- Learning curves tell you whether more data will help
- Every test assumes the future looks like the held-out data
- Always beat sensible baselines

CHECK YOURSELF

Training error is very low and validation error is high, but the gap shrinks as you add data. What is happening and what should you do?

ANSWER

Overfitting driven by variance. More data should help; meanwhile try regularisation or a simpler model.

SECTION 03

03

Splitting your data

How to carve data into training, validation and test sets so that the held-out parts genuinely stand in for the future.

Hold-out | Three-way split | Stratification | Group splits | Time splits | Split sizes

THE BASIC MOVE

The hold-out split



DEFINITION

Set aside a portion of the data before any modelling. Train on the rest. Score the model once on the held-out portion to estimate performance on unseen data.

USE IT WHEN

You have plenty of data, compute is limited, or you need a quick first estimate.

AVOID IT WHEN

Data is small: one random split can be lucky or unlucky, and the estimate will be noisy.

THE STANDARD DESIGN

Train, validation and test



DEFINITION

Training set: the model learns its internal settings from these examples.

DEFINITION

Validation set: used to compare models, tune hyperparameters and pick thresholds.

DEFINITION

Test set: touched once, at the end, to estimate how the chosen model will perform.

RULE OF THUMB

If a decision was made using a dataset, that dataset can no longer give an unbiased estimate of the result of that decision.

WHY THREE SETS

Every look at data spends its honesty

Each time you choose something based on a dataset, the model’s score on that dataset becomes a little optimistic, because you selected the option that happened to do well there.

ANALOGY

If you roll twenty dice and keep the highest, the score you kept overstates how good a typical roll is. Tuning on validation data is choosing the best roll.

KEY IDEA

The validation score of your final model is biased upwards. The test set exists to give one fresh, unbiased reading after all choices are frozen.

PITFALL

Teams that “just check” the test set after each experiment have silently turned it into a second validation set. Its estimate is then no longer trustworthy.

PROTECTING THE TEST SET

House rules for the test set

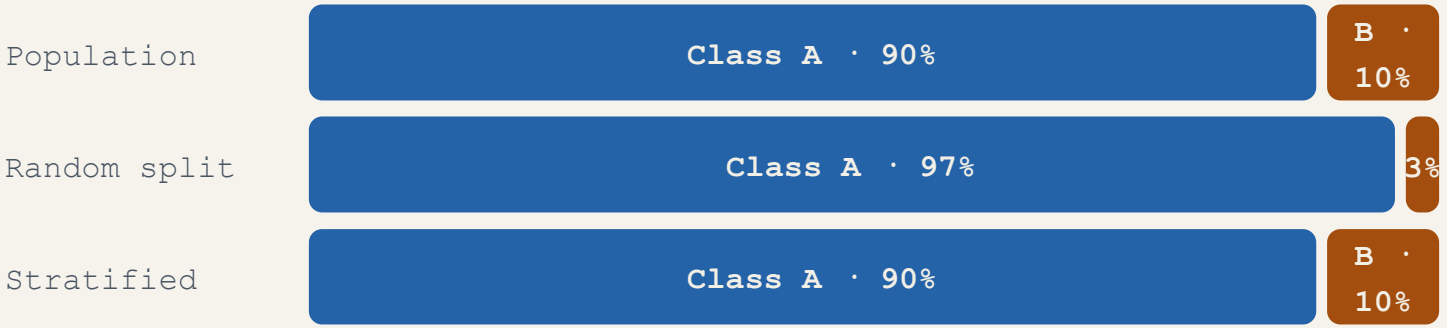
- Create it first, before exploring the data in depth
- Store it separately, with restricted access if possible
- Never use it for feature selection, scaling or tuning
- Log every time it is evaluated, and by whom
- Evaluate only frozen, final candidates
- If it has been overused, retire it and collect a fresh one

PRO TIP

Some teams hash the test-set IDs and add a check to their pipeline that refuses to train if any test ID appears in the training data. Cheap, automatic and surprisingly effective.

KEEPING PROPORTIONS

Stratified splitting



A random split of imbalanced data can leave the held-out set with too few rare cases. Stratification keeps class proportions the same in every part.

USE IT WHEN

Classification with imbalanced classes; small datasets; multi-class problems with rare labels.

PRO TIP

You can stratify on more than the label: for example on region or customer type, to keep important subgroups represented in every split.

HIDDEN DUPLICATES

Group splits: keep related rows together

When several rows come from the same source, a random split puts some in training and some in testing. The model then recognises the source rather than learning the task.

Same person

Multiple visits, scans or transactions from one patient or customer.

Same object

Several photos of the same product, or readings from the same machine.

Same place

Neighbouring map tiles or houses on the same street.

Same document

Sentences or chunks cut from one article or contract.

THE FIX

Split by a **group identifier** so every row from one group lands in the same part. Then the test genuinely asks: does this work for a group we have never seen?

HIDDEN DUPLICATES

What a group leak looks like

IN THE WILD

Medical imaging

A skin-lesion classifier scores superbly. Patients often had several photos taken in one visit, and the split was random. Many test images had a near-twin in training. When re-split by patient, performance dropped sharply.

IN THE WILD

Voice

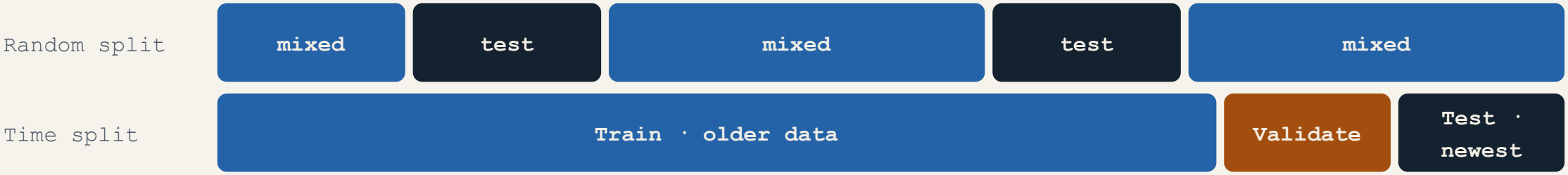
A speaker-recognition model is tested on new recordings of the same speakers it trained on. It is then deployed for new customers, where it struggles. The test asked the wrong question.

ASK YOURSELF

Ask: “In deployment, will the model meet entities it has already seen, or brand-new ones?” Your split must answer the same question.

TIME MATTERS

Time-based splits



KEY IDEA

If the model will predict the future from the past, the test must also be in the future relative to the training data. A random split lets the model peek at tomorrow.

PITFALL

Random splits on time-ordered data are one of the most common causes of models that shine offline and fail in production.

TIME MATTERS

Gaps and embargoes



DEFINITION

A **gap** (or embargo) is a stretch of time left unused between training and test periods.

WHY IT MATTERS

Labels often take time to mature, and features computed over rolling windows can overlap the boundary. The gap stops information from bleeding across.

WORKED EXAMPLE

Predicting whether a loan defaults within 12 months? Training labels near the cut-off are not known yet at prediction time. Leave a gap at least as long as the outcome window.

CHOOSING SIZES

How big should each part be?

Situation	Typical approach	Reasoning
Small data (hundreds of rows)	Cross-validation, small or no fixed test set	Every row is precious; reuse it carefully
Medium (thousands)	Around 60/20/20 or 70/15/15	Enough for stable estimates in each part
Large (millions)	98/1/1 can be plenty	A small fraction is still a big sample
Rare outcomes	Size test set by positive count	Hundreds of positives matter more than total rows
Many subgroups	Ensure each group has enough test rows	Slice results need their own sample size

RULE OF THUMB

Size the test set by how precisely you need to know the answer, not by a fixed percentage. A wider range of uncertainty is acceptable for a prototype, not for a medical device.

BEFORE YOU SPLIT

Questions to answer first

ASK YOURSELF

Will the model see new **people**, **products**, or **places** in deployment? Then split by that group.

ASK YOURSELF

Will the model predict **forward in time**? Then split by time, with a gap.

ASK YOURSELF

Are some outcomes **rare**? Then stratify and check positive counts.

ASK YOURSELF

Could there be **duplicates** or near-duplicates? Then deduplicate before splitting.

ASK YOURSELF

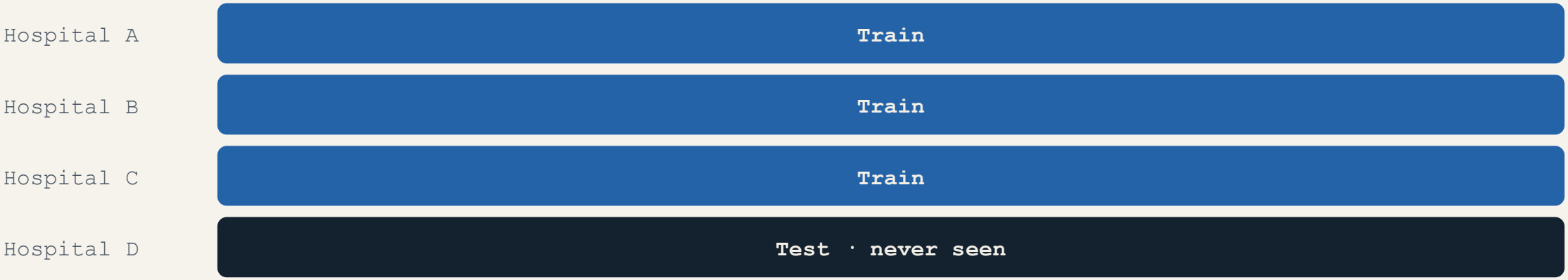
Will deployment data come from a **new site** or source? Then hold out a whole site.

ASK YOURSELF

Are labels **delayed**? Then leave a gap and use only matured labels.

REAL-WORLD SPLITS

Holding out whole sources



KEY IDEA

If a model will be rolled out to new sites, stores, countries or devices, hold out entire sources. This is sometimes called **out-of-distribution** or **external** validation.

IN THE WILD

Imaging models trained at one hospital have learned scanner-specific markings that do not exist elsewhere. External validation at a different hospital is the only way to discover this before deployment.

SECTION 3 RECAP

Splitting your data

REMEMBER

Takeaways

- Train to learn, validate to choose, test once to confirm
- Stratify imbalanced labels
- Split by group when rows are related
- Split by time when predicting the future, with a gap
- Hold out whole sites for external validation

CHECK YOURSELF

You are predicting which customers will buy next month. Each customer has many monthly rows. How should you split?

ANSWER

By time first, so the test months come after training months; consider grouping by customer too if new customers matter.

SECTION 04

04 Cross-validation in depth

Reusing data efficiently to get stable estimates, and the many variants for grouped, imbalanced and time-ordered data.

K-fold · Stratified · Repeated · Leave-one-out · Group · Time series · Nested

THE IDEA

Why cross-validation exists

A single hold-out split gives one estimate from one particular slice of data. With small or medium datasets, that estimate can swing a lot depending on which rows happened to land in the test set.

KEY IDEA

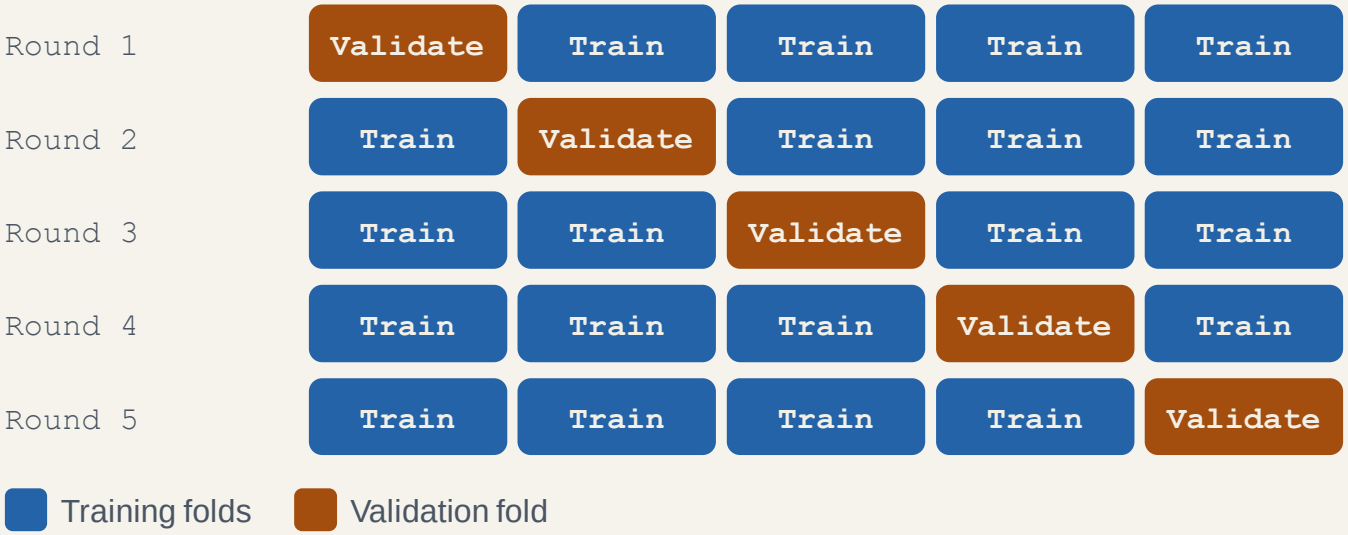
Cross-validation rotates the role of held-out data, so every row is used for checking exactly once. You get several estimates and can see how much they vary.

ANALOGY

Instead of judging a chef on one dish, you ask them to cook five different meals and look at the average and the spread. One bad night no longer decides the verdict.

THE WORKHORSE

K-fold cross-validation



1. **Shuffle and split**
   Divide data into K equal folds.

2. **Rotate**
   Train on K−1 folds, score the one left out.

3. **Repeat K times**
   Each fold is held out once.

4. **Summarise**
   Report the average and the spread.

THE WORKHORSE

Choosing K

K	Training data per round	Character	Good for
3	About two thirds	Fast, a little pessimistic	Big data, slow models
5	Four fifths	Balanced default	Most problems
10	Nine tenths	Less pessimistic, more compute	Small to medium data
N (one per row)	All but one row	Very slow, can be unstable	Tiny datasets only

RULE OF THUMB

Start with 5 or 10 folds. Larger K means each model trains on more data, so estimates are less pessimistic, but you pay with more training runs.

PITFALL

The fold scores are not independent: the training sets overlap heavily. So the spread across folds understates the true uncertainty somewhat.

VARIANT

Stratified k-fold

DEFINITION

Each fold keeps the same proportion of each class as the whole dataset.

USE IT WHEN

Any classification task, and especially when one class is rare.

PITFALL

With very rare classes, some folds may contain only a handful of positives. Check counts per fold, and reduce K if needed.

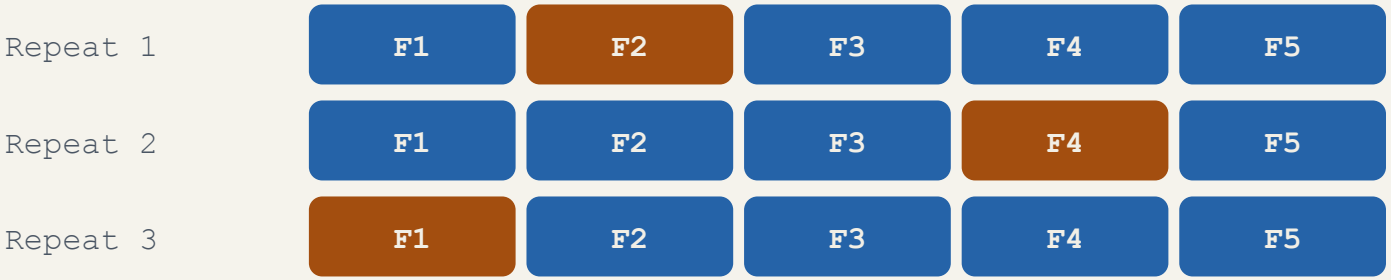
PYTHON · SCIKIT-LEARN

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring="average_precision")
print(scores.mean(), scores.std())
```

VARIANT

Repeated k-fold



Each repeat reshuffles the data before folding, so different rows end up together.

KEY IDEA

Running k-fold several times with different shuffles averages away the luck of one particular partition. Estimates become more stable.

USE IT WHEN

Small datasets, close comparisons between models, or results that move a lot with the random seed.

VARIANT

Leave-one-out and its cousins

Leave-one-out

Each row is the test set once. Uses almost all data for training, but needs as many fits as rows and can give jumpy estimates.

Leave-one-group-out

Each whole group (patient, store, site) is held out once. Ideal when groups are few and important.

Leave-p-out

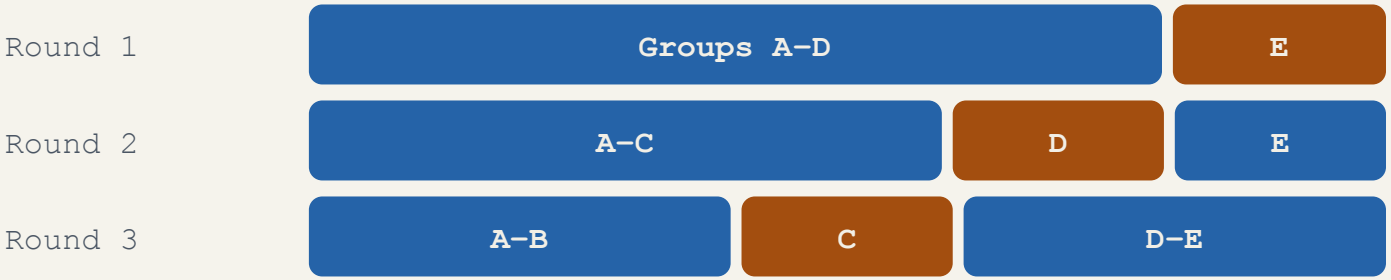
Every possible set of p rows is held out. Rarely practical beyond tiny data; mainly of theoretical interest.

PRO TIP

Leave-one-group-out is the go-to for multi-site studies: train on all hospitals but one, test on the excluded hospital, rotate. It directly measures transfer to an unseen site.

VARIANT

Group k-fold



Folds are made of whole groups, so no group is split between training and validation.

USE IT WHEN

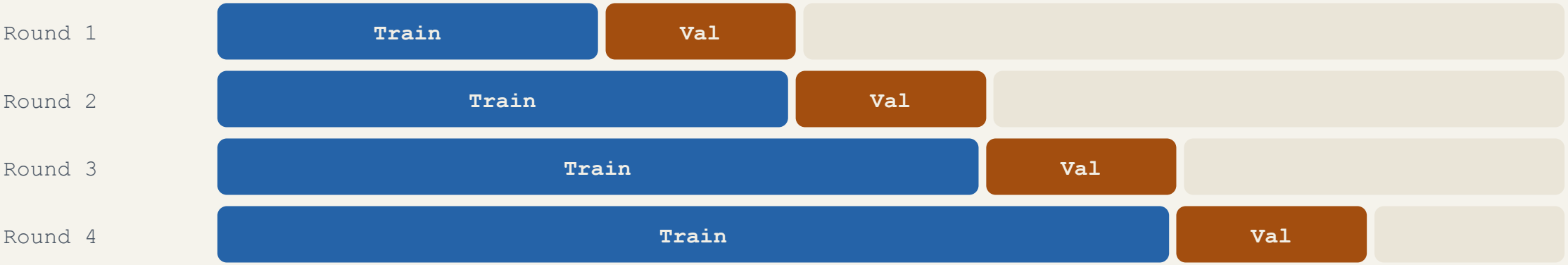
Rows from the same patient, user, device, session or document.

PRO TIP

Use **stratified group k-fold** when you need both: whole groups per fold and similar class balance across folds.

VARIANT

Time-series cross-validation



DEFINITION

Expanding window: training always starts at the beginning and grows; validation is always the next period.

DEFINITION

Sliding window: training is a fixed-length recent window that moves forward. Better when old data is no longer relevant.

KEY IDEA

Validation always comes after training. The model never sees the future it is asked to predict.

VARIANT

Purging and embargo for overlapping labels



DEFINITION

When each label spans a period (for example, “price rises over the next 10 days”), training rows near the validation window share information with it. **Purging** removes those overlapping rows; an **embargo** adds a buffer after validation.

USE IT WHEN

Finance, sensor data, any problem where outcomes are defined over windows that can overlap.

THE ADVANCED MUST-KNOW

Nested cross-validation

If you tune hyperparameters with cross-validation and then report the best cross-validated score, that score is optimistic: you picked the winner on the same folds you measured it with.

DEFINITION

Nested CV wraps tuning inside an outer loop. The inner loop chooses settings; the outer loop scores the whole tuning procedure on data it never touched.

1. **Outer split**
   Hold out one outer fold.
2. **Inner tuning**
   Run CV on the remaining folds to pick settings.
3. **Refit**
   Train with the chosen settings on all inner data.
4. **Score**
   Evaluate on the outer fold. Repeat for each.

THE ADVANCED MUST-KNOW

Nested cross-validation in code

PYTHON · SCIKIT-LEARN

```
from sklearn.model_selection import GridSearchCV, cross_val_score,
KFold

inner = KFold(n_splits=3, shuffle=True, random_state=1)
outer = KFold(n_splits=5, shuffle=True, random_state=2)

search = GridSearchCV(pipeline, param_grid, cv=inner,
                      scoring="roc_auc")

# each outer fold re-runs the whole search
scores = cross_val_score(search, X, y, cv=outer,
                         scoring="roc_auc")
```

KEY IDEA

The outer score estimates how well **your whole procedure** (including tuning) performs, not one particular setting.

PRO TIP

Nested CV tells you what to expect. For the final model, re-run the inner search on all available training data.

CHOOSING A SCHEME

Which cross-validation should I use?

Your data looks like…	Use	Why
Independent rows, balanced labels	K-fold (5 or 10)	Simple and reliable
Independent rows, imbalanced labels	Stratified k-fold	Every fold has rare cases
Rows grouped by entity	Group or stratified group k-fold	No entity on both sides
Time-ordered	Expanding or sliding window	Never train on the future
Overlapping time labels	Purged CV with embargo	Removes boundary leakage
Small data and tuning	Nested, repeated CV	Honest estimate of the procedure
Few sites, deploy to new ones	Leave-one-group-out	Measures transfer to unseen sites

COMMON MISTAKES

Cross-validation traps

⚠ PITFALL

Scaling, imputing or selecting features on the full dataset before splitting into folds.

⚠ PITFALL

Oversampling minority cases before CV, so copies of the same row land in training and validation.

⚠ PITFALL

Reporting the best of many tuned configurations as if it were an unbiased estimate.

⚠ PITFALL

Shuffling time-series data into random folds.

⚠ PITFALL

Forgetting groups, so the same patient appears in different folds.

⚠ PITFALL

Reporting only the mean and hiding a large fold-to-fold spread.

🔧 THE FIX

Put every data-dependent step inside a **pipeline** that is fitted separately within each training fold.

PRACTICAL CODE

The pipeline pattern

PYTHON · SCIKIT-LEARN

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000)),
])
# imputer and scaler are re-fitted inside every fold
cross_val_score(pipe, X, y, cv=cv)
```

KEY IDEA

A pipeline guarantees that preprocessing learns only from the training part of each fold.

PRO TIP

Treat anything with a fit step (encoders, imputers, selectors, samplers, text vectorisers) as part of the model.

REPORTING

How to report cross-validation results

DO

- Report the mean **and** the spread across folds
- State the scheme: type, K, repeats, grouping
- Show the per-fold scores in an appendix
- Mention whether tuning was nested

AVOID

- Quote one rounded number with no context
- Hide unstable folds
- Compare models evaluated on different folds

WORKED EXAMPLE

A good one-liner

“Mean ROC AUC 0.87 across 5×5 repeated stratified group k-fold (grouped by customer), fold range 0.84 to 0.90. Hyperparameters tuned with nested CV.”

PRO TIP

When comparing models, use **the same folds** for all of them, so differences reflect the models, not the splits.

SECTION 4 RECAP

Cross-validation in depth

REMEMBER

Takeaways

- Rotate held-out data to get stable estimates
- Match the scheme to the data: stratified, group, time
- Nest tuning inside an outer loop for honest estimates
- Put every fitted step inside a pipeline
- Report spread, not just the mean

CHECK YOURSELF

You tune 200 configurations with 5-fold CV and report the best mean score. Why is that score optimistic?

ANSWER

You selected the configuration that did best on those exact folds, partly by luck. Nested CV or a fresh test set gives the honest number.

SECTION 05

05 Classification metrics

Every major way to score a classifier, what each one rewards, and when each one misleads.

- Confusion matrix
- Precision and recall
- F-scores
- ROC and PR curves
- Log loss
- MCC
- Multi-class

BEFORE ANY FORMULA

Choosing a metric is choosing which mistakes you care about.

There is no neutral metric. Each one weighs false alarms and missed cases differently. Pick the one whose weighting matches the real-world cost.

THE FOUNDATION

The confusion matrix

	Predicted: positive	Predicted: negative
Actual: positive	**True positive** Correctly flagged a real case	**False negative** Missed a real case
Actual: negative	**False positive** Raised a false alarm	**True negative** Correctly left alone

KEY IDEA

Every classification metric is a different summary of these four counts.

PRO TIP

Always look at the raw matrix before any single score. It shows you exactly where the errors are.

TWO ERROR TYPES

False positives versus false negatives

Application	A false positive means	A false negative means	Usually worse
Cancer screening	Unneeded follow-up test	Missed cancer	False negative
Spam filter	Real email hidden	Spam in inbox	False positive
Fraud detection	Blocked honest customer	Fraud gets through	It depends on amounts
Content moderation	Legitimate post removed	Harmful post stays up	Context dependent
Predictive maintenance	Unneeded inspection	Unexpected breakdown	False negative

ASK YOURSELF

Before choosing any metric, write down: what does each error cost, and who bears that cost?

THE TEMPTING ONE

Accuracy

DEFINITION

The share of all predictions that were correct.

USE IT WHEN

Classes are reasonably balanced and both error types cost about the same.

AVOID IT WHEN

Classes are imbalanced. With 1% positives, predicting “negative” for everyone scores 99%.

IN THE WILD

A fraud model reported 99.8% accuracy. Fraud was about 0.2% of transactions. The model had learned to say “not fraud” every time. Accuracy was the wrong lens entirely.

QUALITY OF ALARMS

Precision

DEFINITION

Of all the cases the model flagged as positive, the share that really were positive.

IN PLAIN WORDS

“When the model raises the alarm, how often is it right?”

USE IT WHEN

False alarms are costly: manual review queues, customer friction, wasted interventions.

ANALOGY

A smoke alarm with high precision rarely goes off for burnt toast. When it sounds, you believe it.

PITFALL

A model can get very high precision by flagging only the few most obvious cases and missing everything else.

COVERAGE OF CASES

Recall (sensitivity)

DEFINITION

Of all the cases that really were positive, the share the model caught.

IN PLAIN WORDS

“Of everything we should have caught, how much did we catch?”

USE IT WHEN

Missing a case is costly: disease, safety faults, serious fraud.

ANALOGY

A fishing net with high recall catches nearly every fish in the area, along with plenty of weeds.

PITFALL

Flagging everything gives perfect recall. Recall must always be read alongside precision or false-alarm rate.

THE OTHER SIDE

Specificity and the false-alarm rate

DEFINITION

Specificity: of all truly negative cases, the share the model correctly left alone.

DEFINITION

False-positive rate: of all truly negative cases, the share wrongly flagged. It is the flip side of specificity.

DEFINITION

Negative predictive value: when the model says “no”, how often it is right.

KEY IDEA

Medicine traditionally pairs **sensitivity and specificity**. Information retrieval and most ML teams pair **precision and recall**. They answer different questions; both are valid.

THE TRADE-OFF

Precision and recall pull against each other

Most classifiers output a score. A threshold turns the score into a yes or no. Lower the threshold and you catch more cases (recall up) but raise more false alarms (precision down).

Threshold	Flags	Precision	Recall
Very low	Almost everything	Low	Very high
Medium	Moderate number	Medium	Medium
Very high	Only the surest cases	Very high	Low

RULE OF THUMB

You do not have “a precision” or “a recall”. You have a precision and recall **at a chosen threshold**. Always state it.

PRO TIP

Ask the business for a constraint, such as “the review team can handle 200 cases a day”. That fixes the threshold, and you then maximise recall within it.

COMBINING TWO NUMBERS

F1 and the F-beta family

DEFINITION

F1 combines precision and recall into one score that is only high when both are high. A weakness in either pulls it down sharply.

DEFINITION

F-beta lets you lean one way: settings that favour recall suit screening; settings that favour precision suit costly interventions.

USE IT WHEN

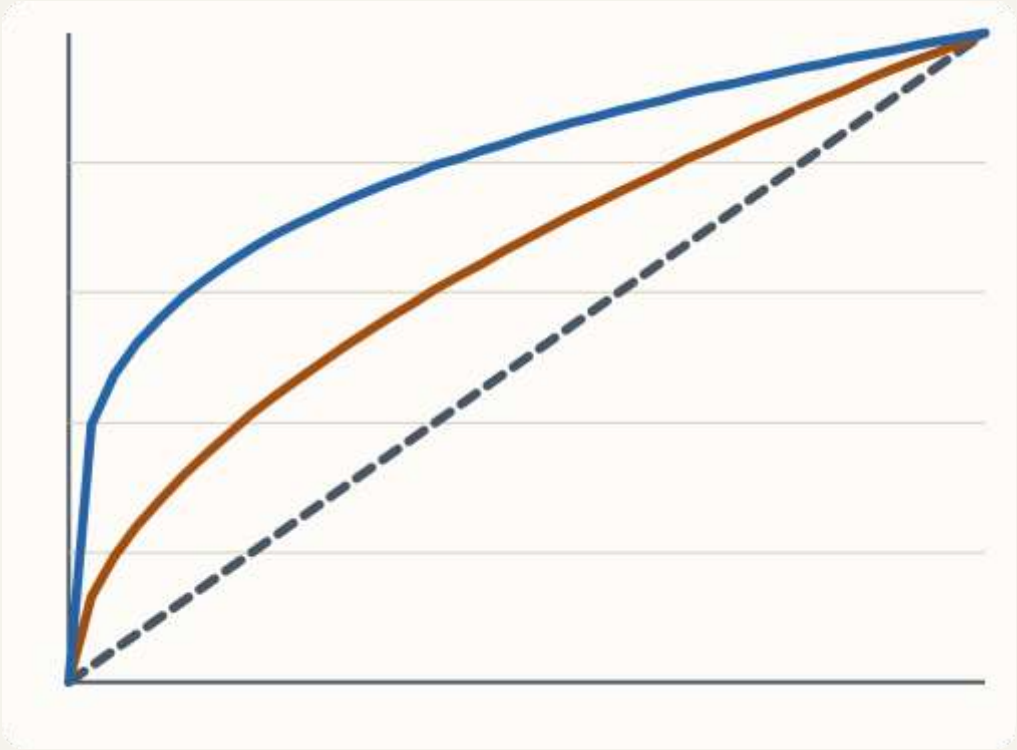
You need one number to rank models and both error types matter.

PITFALL

F1 ignores true negatives completely, depends on the threshold, and treats both errors as equally costly. That last assumption is rarely true.

THRESHOLD-FREE VIEW

The ROC curve and ROC AUC



Across: false-alarm rate · Up: recall

Strong model

Weaker model

Random guessing

DEFINITION

The ROC curve traces recall against false-alarm rate across every threshold. **ROC AUC** is the area under it.

IN PLAIN WORDS

ROC AUC is the chance that a randomly chosen positive gets a higher score than a randomly chosen negative.

THRESHOLD-FREE VIEW

When ROC AUC misleads

KEY IDEA

ROC AUC measures **ranking** quality only. It is unaffected by class balance and says nothing about whether scores are well calibrated.

PITFALL

With rare positives, a model can have a high ROC AUC yet produce mostly false alarms at any useful threshold, because a tiny false-alarm rate on millions of negatives is still a flood.

USE IT WHEN

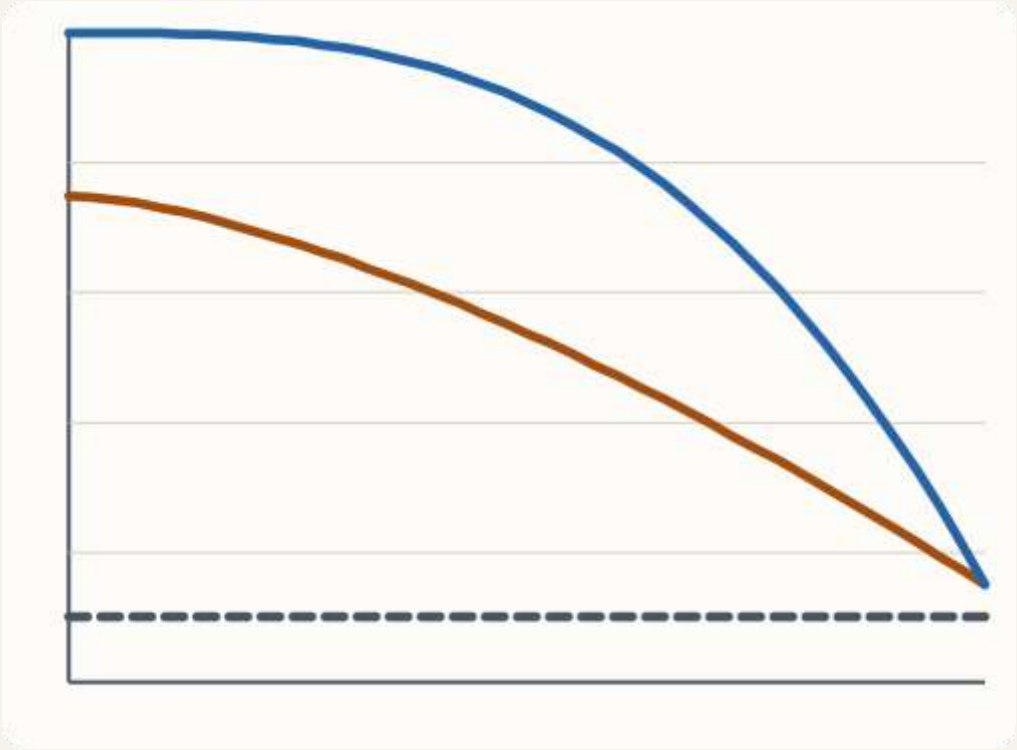
Classes are reasonably balanced, or you care about ranking across the whole range.

RULE OF THUMB

If positives are rare and you care about the flagged cases, prefer the precision–recall curve.

THRESHOLD-FREE VIEW

Precision–recall curve and average precision



Across: recall · Up: precision

Strong model

Weaker model

Baseline = positive rate

DEFINITION

The PR curve shows precision at every level of recall. **Average precision** (PR AUC) summarises it.

KEY IDEA

Its baseline is the positive rate, not one half. With 1% positives, 0.30 can be a strong result.

SCORING PROBABILITIES

Log loss and the Brier score

DEFINITION

Log loss punishes confident wrong answers very heavily. Saying 99% sure and being wrong costs far more than saying 60% sure and being wrong.

DEFINITION

Brier score is the average squared gap between predicted probability and what happened. Gentler on confident mistakes; easy to explain.

USE IT WHEN

Probabilities themselves are used: pricing, risk scores, expected-value decisions, combining models.

PRO TIP

These are **proper scoring rules**: a model scores best by reporting its honest belief. That makes them ideal for training and comparing probabilistic models.

BALANCED SUMMARIES

Matthews correlation and Cohen’s kappa

DEFINITION

MCC uses all four cells of the confusion matrix. It is high only when the model does well on both classes, so it resists being fooled by imbalance.

DEFINITION

Cohen’s kappa measures agreement beyond what chance alone would produce. Often used to compare models with human annotators.

WORKED EXAMPLE

A model that always predicts the majority class can have accuracy of 99% but an MCC of zero. MCC immediately reveals that it has learned nothing.

RULE OF THUMB

If you must report a single threshold-dependent number for an imbalanced binary problem, MCC is a robust choice.

MORE THAN TWO CLASSES

Averaging across classes

Averaging	How it works	Effect	Use when
Macro	Score each class, then simple average	Every class counts equally	Rare classes matter
Weighted	Average weighted by class size	Big classes dominate	Overall picture matters
Micro	Pool all decisions, then score	Equals accuracy for single-label	Every decision counts equally
Per class	Report each class separately	Nothing hidden	Always, in an appendix

⚠ PITFALL

Weighted and micro averages can look excellent while a small but important class is almost never predicted correctly. Always inspect the per-class table.

MORE THAN TWO CLASSES

Top-k accuracy and multi-label tasks

DEFINITION

Top-k accuracy: the right answer appears among the model's k highest-ranked guesses. Useful when a human chooses from a shortlist.

DEFINITION

Multi-label tasks allow several correct labels per example. Use per-label metrics, subset accuracy (all labels exactly right), or Hamming loss (share of label decisions wrong).

IN THE WILD

A support-ticket router suggests three possible teams to an agent. Top-1 accuracy is modest, but top-3 accuracy is high, and that matches how the tool is actually used.

METRIC CHOOSER

Which classification metric?

Situation	Primary metric	Also report
Balanced, equal costs	Accuracy or F1	Confusion matrix
Rare positives, alerts reviewed	Average precision; precision at fixed workload	Recall at that threshold
Missing cases is dangerous	Recall at a minimum precision	False-alarm rate
Ranking quality overall	ROC AUC	PR curve if imbalanced
Probabilities drive decisions	Log loss or Brier score	Calibration plot
Single robust number, imbalanced	MCC	Per-class results
Many classes, rare ones matter	Macro F1	Per-class table

SECTION 5 RECAP

Classification metrics

REMEMBER

Takeaways

- Start from the confusion matrix and real error costs
- Accuracy fails under imbalance
- Precision and recall depend on the threshold
- ROC AUC ranks; PR AUC focuses on positives
- Proper scoring rules judge probabilities

CHECK YOURSELF

Positives are 0.5% of data. Model A has ROC AUC 0.95 and average precision 0.12. Model B has ROC AUC 0.93 and average precision 0.31. Which is better for an alert queue?

ANSWER

Usually B. Its flagged cases are far more often real, which is what an alert queue cares about.

SECTION 06

06 Regression metrics

Measuring numeric predictions: error sizes, percentage errors, explained variation and the residual plots that reveal what numbers hide.

MAE | RMSE | Percentage errors | R-squared | Quantile loss | Residuals

THE BASICS

Average size of the error

DEFINITION

MAE

MAE (mean absolute error): the typical size of a miss, in the same units as the target. Every error counts in proportion to its size.

DEFINITION

RMSE

RMSE (root mean squared error): also in target units, but large errors count disproportionately. One huge miss inflates it a lot.

USE IT WHEN

MAE: when a miss of 20 is simply twice as bad as a miss of 10, and outliers should not dominate.

USE IT WHEN

RMSE: when big misses are especially harmful, such as under-stocking critical supplies.

PRO TIP

Report both. If RMSE is much larger than MAE, a few large errors are driving the result; go and find them.

ROBUST CHOICES

Median error and Huber-style losses

DEFINITION

Median absolute error: the middle miss. Almost unaffected by a handful of extreme errors.

DEFINITION

Huber-style losses: behave like squared error for small misses and like absolute error for big ones. A compromise between MAE and RMSE.

IN THE WILD

House-price predictions include a few mansions worth many times the typical home. RMSE is dominated by them. Median error describes the typical buyer’s experience, while a separate high-value slice reports the mansions.

RELATIVE ERRORS

Percentage errors and their traps

DEFINITION

MAPE: the average error as a percentage of the true value. Easy to explain to business users.

PITFALL

Explodes when true values are near zero; punishes over-forecasts and under-forecasts unequally; meaningless for targets that can be negative.

THE FIX

Symmetric MAPE reduces the asymmetry. **Weighted percentage error** (total absolute error divided by total actual) avoids division by tiny values and suits sales and demand data.

RULE OF THUMB

Never use MAPE on intermittent demand with many zero days. Use a scaled error instead.

EXPLAINED VARIATION

R-squared, carefully

DEFINITION

R-squared: how much of the variation in the target the model explains, compared with always predicting the average. One is perfect; zero matches the average; it can go negative on new data.

IN PLAIN WORDS

“How much better than just guessing the mean are we?”

PITFALL

A high R-squared does not mean predictions are accurate enough to be useful, and a low one does not mean the model is useless. It depends on the scale of the errors that matter.

PITFALL

R-squared on training data always rises as you add features. Only out-of-sample values mean anything.

SCALE-FREE COMPARISON

Scaled errors for forecasting

DEFINITION

MASE compares the model's error with the error of a simple naive forecast on the same series. Below one beats the naive method; above one loses to it.

WHY IT MATTERS

It works across series of very different sizes and never divides by zero, so you can average it over thousands of products.

WORKED EXAMPLE

A retailer forecasts 5,000 products. Raw errors are dominated by the best-sellers. A scaled error tells you, product by product, whether the model beats "same as last season".

PREDICTING RANGES

Quantile (pinball) loss and interval coverage

DEFINITION

Quantile loss scores a prediction like “90% of the time demand will be below this”. It penalises under-predictions and over-predictions with different weights.

DEFINITION

Interval coverage checks whether, say, 80% prediction intervals actually contain the truth about 80% of the time.

KEY IDEA

When decisions depend on risk (stock levels, staffing, capacity), validate the **range** a model predicts, not only its central estimate.

PITFALL

Intervals that are far too wide always cover the truth. Check sharpness (how narrow they are) as well as coverage.

THE MOST IMPORTANT PLOT

Residual analysis

Residuals vs prediction

A funnel shape means errors grow with the size of the prediction. Consider modelling on a log scale.

Residuals vs each feature

A curve or pattern means the model is missing structure in that feature.

Residuals over time

A trend or jump suggests drift or a change in the process.

Distribution of residuals

Heavy tails or skew show where averages hide extreme misses.

KEY IDEA

Metrics summarise. Residual plots diagnose. A good regression validation always includes at least the first and third plots.

BUSINESS FRAMING

Asymmetric costs in regression

IN THE WILD

Hospital capacity

Under-forecasting hospital bed demand means patients wait; over-forecasting means idle staff. The costs are unequal, so a symmetric metric like RMSE misrepresents what the hospital cares about.

THE FIX

Use a cost-weighted error, or quantile loss at the level that matches the cost ratio. Report over- and under-prediction separately.

PRO TIP

Ask stakeholders: “Would you rather be over by 10 or under by 10?” If they hesitate, errors are asymmetric and your metric should be too.

METRIC CHOOSER

Which regression metric?

Situation	Primary metric	Also report
Typical error matters, outliers are noise	MAE or median error	Residual distribution
Large misses are very costly	RMSE	Worst-case errors
Stakeholders think in percentages	Weighted percentage error	MAE by segment
Many series of different scales	MASE or scaled error	Per-series breakdown
Risk-based decisions	Quantile loss, interval coverage	Interval width
Unequal over/under costs	Cost-weighted error	Bias (average signed error)

SECTION 6 RECAP

Regression metrics

REMEMBER

Takeaways

- MAE for typical error; RMSE when big misses matter
- Percentage errors break near zero
- R-squared is relative, not absolute
- Validate prediction ranges, not just points
- Always inspect residual plots

CHECK YOURSELF

RMSE is three times larger than MAE on your test set. What does that suggest?

ANSWER

A small number of very large errors. Find them: they may be outliers, data errors or a segment the model handles badly.

SECTION 07

07 Metrics for other tasks

Validation beyond plain classification and regression: ranking, recommendation, clustering, vision, language, forecasting and anomaly detection.

Ranking · Recommenders · Clustering · Vision · NLP · Forecasting · Anomalies

RANKING AND SEARCH

Order matters

DEFINITION

Precision at k: of the top k results, how many are relevant.

DEFINITION

Recall at k: of all relevant items, how many appear in the top k.

DEFINITION

MRR: how high, on average, the first relevant result appears.

DEFINITION

NDCG: rewards placing the most relevant items nearest the top, with credit fading further down.

DEFINITION

MAP: average precision across many queries, rewarding relevant items ranked early.

DEFINITION

Hit rate: share of users or queries with at least one relevant item in the top k.

RULE OF THUMB

Choose k to match the screen: if users see five results, evaluate at five.

RECOMMENDERS

Beyond accuracy in recommendation

A recommender can score well on offline ranking metrics and still produce a dull, repetitive or unfair experience.

> ⚠ **PITFALL**
>
> Offline data only contains feedback on items the old system chose to show. A new model is judged on a biased record of the past.

Quality	Question
Coverage	How much of the catalogue ever gets recommended?
Diversity	Are the suggestions varied within a list?
Novelty	Does it surface less obvious items?
Serendipity	Pleasant surprises users would not find alone?
Popularity bias	Is it just recommending best-sellers?

Offline evaluation of logged decisions

DEFINITION

Off-policy evaluation estimates how a new policy would have performed using logs from the old one, by re-weighting outcomes according to how likely each choice was under each policy.

PITFALL

These estimates can be very noisy when the new policy chooses actions the old one rarely took. They also need the old system’s choice probabilities to have been logged.

PRO TIP

Log the probability with which every recommendation was shown, and keep a small amount of randomised exploration in production. Both make future offline validation far more reliable.

CLUSTERING

Validating without labels

DEFINITION

Internal measures (silhouette and similar) check whether clusters are tight and well separated, using only the data itself.

DEFINITION

External measures (adjusted Rand index, mutual information) compare clusters with known labels, when you have some.

KEY IDEA

The real test is **usefulness and stability**: do clusters stay similar when you resample the data, and do domain experts find them meaningful and actionable?

PITFALL

Internal scores favour certain shapes of cluster. A high silhouette score on the wrong representation is still the wrong answer.

COMPUTER VISION

Detection and segmentation

IoU

How much a predicted box or mask overlaps the true one, relative to their combined area.

mAP

Average precision across classes, often across several overlap thresholds. The standard for object detection.

Dice score

Overlap measure common in medical segmentation, focusing on the region of interest.

Small objects

Report performance by object size; small items are often where models fail.

PITFALL

A detection model’s mAP depends heavily on the overlap threshold chosen. Always state it, and compare models at the same setting.

LANGUAGE TASKS

Classic NLP metrics

Task	Common metrics	Watch out for
Text classification	F1, macro F1, MCC	Label imbalance, annotator disagreement
Named entities	Entity-level precision, recall, F1	Partial matches counted inconsistently
Translation	BLEU, chrF, COMET-style learned metrics	Surface overlap ignores meaning
Summarisation	ROUGE, learned metrics, human ratings	Fluent but unfaithful summaries
Question answering	Exact match, token F1	Correct answers phrased differently

KEY IDEA

Overlap metrics reward matching words, not being right. For generative output, pair them with human or model-based judgement, covered in section 15.

FORECASTING

Validating forecasts properly

KEY IDEA

Use **rolling-origin evaluation**: repeatedly forecast from successive cut-off points and score each forecast against what actually happened next.

RULE OF THUMB

Evaluate at the forecast horizons that matter. A model great at next-day forecasts may be poor four weeks ahead.

- Compare against naive and seasonal naive
- Score each horizon separately
- Check promotions, holidays and shocks
- Validate intervals, not just points
- Use scaled errors across series

ANOMALY DETECTION

Validating rare-event detectors

⚠ PITFALL

True anomalies are rare and often unlabelled. The few labelled ones may not resemble the next attack or failure.

THE FIX

Combine sources of evidence: precision at the top of the ranked list, injected synthetic anomalies, expert review of flagged cases, and back-testing on known past incidents.

KEY IDEA

Report the alert volume at the chosen threshold. A detector that flags 5,000 events a day is unusable, whatever its recall.

IN THE WILD

A machine-monitoring model was tuned on past breakdowns. Experts reviewing its top 50 alerts each week found faults no label had recorded. Those reviews became the most valuable validation data.

SECTION 7 RECAP

Metrics for other tasks

REMEMBER

Takeaways

- Ranking: evaluate the top of the list users actually see
- Recommenders: also check diversity, coverage and logged bias
- Clustering: stability and usefulness beat internal scores
- Generative text: overlap metrics are not correctness
- Rare events: report alert volume and expert precision

CHECK YOURSELF

Your search results page shows ten links. Which metric family fits best?

ANSWER

Ranking metrics at ten: precision at 10, NDCG at 10, or MRR if the first good result matters most.

SECTION 08

08

Imbalance, thresholds and calibration

Turning scores into decisions you can trust: handling rare classes, choosing thresholds and checking that probabilities mean what they say.

Imbalanced data | Resampling | Threshold tuning | Cost curves | Calibration | Reliability plots

RARE OUTCOMES

What imbalance does to validation

⚠ PITFALL

Accuracy becomes almost meaningless.

⚠ PITFALL

Test sets may hold too few positives to estimate recall reliably.

⚠ PITFALL

Random folds can contain zero positives.

⚠ PITFALL

Small changes in a few cases swing metrics wildly.

⚠ PITFALL

Default thresholds of one half are almost always wrong.

⚠ PITFALL

Resampling done carelessly leaks into validation.

RULE OF THUMB

Count your positives, not your rows. A test set with 40 positive cases cannot pin down recall to within a few percentage points, however many negatives it has.

RARE OUTCOMES

Resampling without fooling yourself

DEFINITION

Oversampling repeats or synthesises minority cases; **undersampling** drops majority cases; **class weights** make rare mistakes cost more during training.

KEY IDEA

Resampling changes training. It must **never** be applied to validation or test data, which must keep the real-world class balance.

PYTHON · IMBALANCED-LEARN

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

pipe = Pipeline([
    ("smote", SMOTE(random_state=0)),
    ("model", clf),
])
# SMOTE runs only on each training fold
cross_val_score(pipe, X, y, cv=cv,
    scoring="average_precision")
```

FROM SCORES TO DECISIONS

Threshold selection is part of the model

1. **Agree the objective**
   Maximise expected value, hit a minimum recall, or fit a review capacity.
2. **Tune on validation data**
   Sweep thresholds and compute the objective at each one.
3. **Freeze it**
   Record the chosen threshold with the model version.
4. **Confirm on the test set**
   Report all metrics at the frozen threshold.
5. **Monitor it live**
   Base rates drift; the best threshold drifts with them.

⚠ PITFALL

Choosing the threshold on the test set inflates every threshold-based metric you then report from it.

FROM SCORES TO DECISIONS

Cost-based thresholds

WORKED EXAMPLE

Fraud review

Each missed fraud costs about £400. Each false alarm costs about £8 in review time and customer friction. A false alarm is roughly fifty times cheaper than a miss, so the best threshold flags many more cases than a default would.

KEY IDEA

Put a cost on each cell of the confusion matrix, then choose the threshold that minimises total expected cost on validation data. This aligns the model directly with the business.

PRO TIP

Show stakeholders a simple table: at each candidate threshold, how many alerts per day, how many real cases caught, and the expected net saving. Decisions become concrete.

PROBABILITIES YOU CAN TRUST

What calibration means

DEFINITION

A model is **calibrated** when its probabilities match reality: of all cases it scores at 70%, about 70% turn out positive.

KEY IDEA

Good ranking and good calibration are different properties. A model can rank perfectly yet claim 99% when the truth is 60%.

ANALOGY

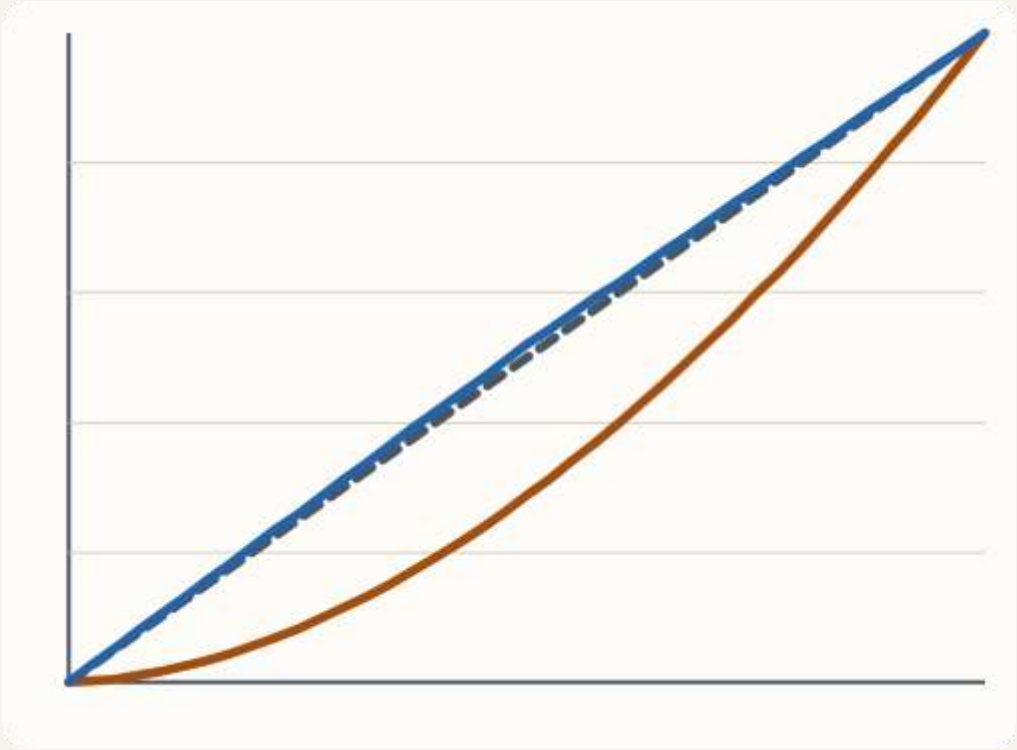
A well-calibrated weather forecaster says 30% chance of rain on many days, and it rains on about three in ten of them.

USE IT WHEN

Calibration matters when probabilities feed decisions: pricing, triage, expected-value maths, or being shown to users.

PROBABILITIES YOU CAN TRUST

Reliability diagrams



Across: predicted probability · Up: observed rate

Well calibrated

Overconfident

Perfect calibration

DEFINITION

Group predictions into bins by predicted probability and plot each bin’s average prediction against the share of real positives.

KEY IDEA

Points below the diagonal: the model is overconfident. Above it: underconfident.

PROBABILITIES YOU CAN TRUST

Measuring and fixing calibration

DEFINITION

Expected calibration error summarises the average gap between predicted and observed rates across bins. Simple, but sensitive to how bins are chosen.

THE FIX

Platt scaling fits a simple curve on top of scores; **isotonic regression** fits a flexible stepwise curve; **temperature scaling** is common for neural networks.

PITFALL

Fit calibration on separate data (a calibration split or cross-validation). Calibrating on the training data simply re-learns the original overconfidence.

PRO TIP

Recheck calibration after any resampling or class weighting: those techniques deliberately distort probabilities.

PROBABILITIES YOU CAN TRUST

Calibration under changing base rates

IN THE WILD

A model was calibrated when 2% of applications were fraudulent. During a fraud wave the rate tripled. Its 10% scores now corresponded to much higher real risk, and the fixed threshold let more fraud through.

THE FIX

Monitor predicted versus observed rates over time, and recalibrate on recent data when they diverge. Thresholds tied to costs may need to move too.

KEY IDEA

Calibration is a property of a model **and** a population. When the population changes, the calibration must be rechecked.

SECTION 8 RECAP

Imbalance, thresholds and calibration

REMEMBER

Takeaways

- Count positives; small counts mean wide uncertainty
- Resample training folds only, never test data
- Choose and freeze thresholds on validation data
- Link thresholds to real costs or capacity
- Check calibration with reliability diagrams

CHECK YOURSELF

You oversampled positives to a 50/50 balance before training. What happens to the model’s raw probabilities on real data?

ANSWER

They will be too high. Recalibrate on data with the real class balance before using them as probabilities.

SECTION 09

09 Data leakage

The silent killer of model validity: how information from the future or from the answer sneaks into training, and how to stop it.

- Target leakage
- Train-test contamination
- Temporal leakage
- Preprocessing leaks
- Detection
- Prevention

THE GOLDEN RULE

If a result looks too good to be true, assume leakage until proven otherwise.

Leakage gives models information they will not have at prediction time. It produces superb offline scores and disappointing live performance.

DEFINITIONS

Two families of leakage

Target leakage

A feature contains information about the outcome that would not be available when the prediction is actually made. The model cheats by reading the answer.

Train–test contamination

Information from validation or test data influences training, through shared rows, shared preprocessing, or repeated peeking.

ASK YOURSELF

For every feature, ask: “At the exact moment the model makes this prediction in production, would this value already be known, and would it have this exact value?”

TARGET LEAKAGE

Features that know the answer

Prediction task	Leaky feature	Why it leaks
Will patient develop sepsis?	Antibiotics prescribed	Often given because sepsis was suspected
Will customer churn?	Account closure reason	Only filled in after they leave
Will loan default?	Collections contact count	Collections start after missed payments
Is this claim fraudulent?	Investigation outcome code	Produced by the fraud team after review
Will the part fail?	Repair ticket opened	Opened when failure is already happening

KEY IDEA

Many leaky features are **consequences** of the outcome, not causes. Map the timeline of when each field is created.

TEMPORAL LEAKAGE

Information from the future

⚠ PITFALL

Random splits of time-ordered data.

⚠ PITFALL

Rolling averages that include the day being predicted.

⚠ PITFALL

Features computed from the whole dataset, such as an average over all years.

⚠ PITFALL

Data that was later corrected or backfilled used as if known at the time.

⚠ PITFALL

Labels defined over windows that overlap the training cut-off.

⚠ PITFALL

Using today's customer attributes for a prediction made last year.

THE FIX

Build features **as of** each prediction timestamp, using only data recorded before that moment. Feature stores with point-in-time joins exist to do exactly this.

PREPROCESSING LEAKAGE

Fitting on everything before splitting

PITFALL

Scaling, normalising or imputing using statistics from the full dataset lets test data shape the training inputs.

PITFALL

Selecting the “best” features using all labels, then cross-validating, gives famously inflated scores, especially with many features and few rows.

PITFALL

Target encoding categories using all rows puts the answer into the feature.

THE FIX

Split first. Fit every transformer on training data only, then apply it to validation and test data. Use pipelines so this happens automatically inside each fold.

WORKED EXAMPLE

With thousands of random, meaningless features and a few hundred rows, selecting features on all data before CV can produce impressive-looking accuracy on pure noise.

CONTAMINATION

Duplicates, near-duplicates and shared sources

⚠ PITFALL

The same record appears twice, once in each split. Common after data merges or augmentation.

⚠ PITFALL

Near-duplicates: the same photo resized, the same text lightly edited, consecutive video frames.

⚠ PITFALL

Augmented copies of one image placed in different splits.

THE FIX

Deduplicate **before** splitting, using exact hashes plus similarity search for near-duplicates. Augment only after splitting, and only the training set.

IN THE WILD

Benchmark datasets in vision and language have repeatedly been found to contain test items that also appear in training sets, sometimes with small edits. Published scores were inflated as a result.

CONTAMINATION

Leakage for large language models

KEY IDEA

Pre-trained models have seen vast amounts of web text. Public benchmark questions, and their answers, may have been in that text.

PITFALL

A strong benchmark score may reflect memorisation rather than ability, and you cannot fully inspect the training data.

THE FIX

Build private, freshly written evaluation sets; include items created after the model's training cut-off; paraphrase or perturb items and check if scores hold.

PRO TIP

If performance collapses when questions are lightly reworded, suspect memorisation.

LEAKAGE VIA PEOPLE

Human leakage: overfitting to the test set

Even with perfect pipelines, a team that runs hundreds of experiments and keeps whatever improves the test score will slowly fit the test set by hand.

ANALOGY

Like a student who retakes the same practice exam until they pass. The final score reflects familiarity with that paper, not mastery.

THE FIX

- Use validation data for day-to-day decisions
- Ration test-set evaluations and log them
- Refresh the test set periodically
- Keep a final “vault” set nobody sees until sign-off

DETECTION

How to spot leakage

Too-good scores

Performance far above baselines, published results or domain intuition.

One dominant feature

A single feature carries most of the model's importance.

Timeline review

For each top feature, when is it created relative to the prediction?

Adversarial validation

Train a model to tell training rows from test rows. If it succeeds easily, the sets differ in a suspicious way.

PRO TIP

Try training a model on each top feature alone. If one feature by itself nearly matches the full model, investigate it carefully before celebrating.

PREVENTION

A leakage-prevention checklist

- [ ] Write down the exact prediction moment for the use case
- [ ] Document when every feature becomes available
- [ ] Deduplicate before splitting, including near-duplicates
- [ ] Split by group and time where relevant
- [ ] Fit all preprocessing inside pipelines, per fold
- [ ] Augment and resample only training data
- [ ] Build features with point-in-time joins
- [ ] Run adversarial validation on train versus test
- [ ] Inspect the top features for plausibility
- [ ] Ration and log test-set evaluations

SECTION 9 RECAP

Data leakage

REMEMBER

Takeaways

- Leakage gives the model information it won’t have live
- Check feature timing against the prediction moment
- Fit preprocessing per fold, inside pipelines
- Deduplicate before splitting
- Human peeking at the test set is leakage too

CHECK YOURSELF

A hospital readmission model’s best feature is “discharge summary length”. Should you worry?

ANSWER

Check timing. If the summary is written at or after discharge and the prediction must be made earlier, it leaks. Even if available, it may proxy for complexity in ways worth understanding.

SECTION 10

10

Tuning and model selection

Choosing hyperparameters and models without overfitting the validation process itself.

Search strategies · Selection bias · Early stopping · Simplicity · Ensembles · Final refit

SEARCH STRATEGIES

How to search for good settings

Method	How it works	Strength	Weakness
Grid search	Tries every combination on a grid	Simple, exhaustive	Wasteful in many dimensions
Random search	Samples combinations at random	Efficient when few settings matter	No learning from results
Bayesian optimisation	Models the score and picks promising settings	Sample-efficient	More complex, sequential
Successive halving	Gives more budget to the best performers	Fast for big searches	Can drop slow starters
Evolutionary	Mutates and recombines good settings	Handles odd search spaces	Many evaluations

PRO TIP

Random search usually beats grid search for the same budget, because it explores more distinct values of the settings that actually matter.

A SUBTLE DANGER

Overfitting the validation set

The more configurations you try, the more likely one looks good by chance on your particular validation data. The winning score is biased upwards, and the bias grows with the number of trials.

ANALOGY

Flip a thousand coins ten times each. One will probably land heads nine or ten times. It is not a magic coin.

THE FIX

- Use nested CV or a separate untouched test set
- Prefer bigger validation sets for big searches
- Repeat CV to reduce noise
- Don’t chase tiny improvements

RULE OF THUMB

Any gain smaller than the fold-to-fold spread is probably noise.

Early stopping as validation

DEFINITION

Stop training when validation performance stops improving, and keep the best checkpoint.

PITFALL

The epoch you pick is chosen using validation data, so the final validation score is optimistic. Confirm on the test set.

PRO TIP

Use a patience setting so you don’t stop on a single noisy dip, and log the full curve for review.

WORKED EXAMPLE

Validation loss falls for 30 epochs, wobbles until epoch 45, then climbs steadily. With patience of 10, training stops at about epoch 55 and restores the checkpoint from the lowest point.

CHOOSING BETWEEN MODELS

Prefer simpler when it is close

RULE OF THUMB

The one-standard-error rule, in words

If a simpler model performs within the uncertainty of the best one, choose the simpler model. It is usually easier to explain, cheaper to run and more robust to change.

KEY IDEA

Model selection is not just about the top score. Weigh latency, cost, maintainability, interpretability, fairness and robustness alongside accuracy.

Criterion	Model A (gradient boosting)	Model B (logistic regression)
Validation score	Slightly higher	Within uncertainty of A
Latency and cost	Higher	Very low
Explainability	Needs extra tooling	Direct from coefficients
Likely choice		B, unless the gain is material

Validating combined models

PITFALL

Stacking trains a second model on the first models' predictions. If those predictions were made on their own training data, the stacker learns from overconfident inputs.

THE FIX

Generate first-level predictions **out-of-fold**: each row's prediction comes from a model that did not train on it. Then validate the whole stack with an outer loop.

PRO TIP

Ensembles often improve average scores. Check that they do not harm calibration or slow inference beyond what the product can accept.

AFTER SELECTION

The final refit

1. **Freeze the recipe**
   Features, model type, hyperparameters, threshold and preprocessing.
2. **Confirm once**
   Score the frozen recipe on the untouched test set.
3. **Decide on refitting**
   Optionally retrain on training plus validation data, or all data, to use every example.
4. **Sanity-check**
   Make sure the refit model behaves like the validated one on a holdout or recent sample.
5. **Record everything**
   Data version, code version, seeds and the test result.

⚠ PITFALL

Refitting on all data leaves nothing to test the final artefact. Keep a small recent holdout, or rely on shadow deployment to confirm it.

SECTION 10 RECAP

Tuning and model selection

REMEMBER

Takeaways

- Random or Bayesian search beats naive grids
- More trials means more optimistic best scores
- Early stopping uses validation data too
- Prefer simpler models when results are close
- Stack with out-of-fold predictions

CHECK YOURSELF

After 500 tuning runs, the best model beats the second best by a tiny margin. What should you do?

ANSWER

Treat them as tied. Check uncertainty, then decide on simplicity, cost and robustness rather than the tiny score gap.

SECTION 11

11

Uncertainty and statistical rigour

Turning single scores into honest ranges, and telling real improvements apart from noise.

- Confidence intervals
- Bootstrap
- Comparing models
- Significance
- Multiple comparisons
- Sample size

UNCERTAINTY

A score without a range is a rumour.

Rerun your evaluation on a different sample and the number will change. Validation must tell you by how much.

WHY IT MATTERS

Every metric is an estimate

Your test set is one sample of the world. A different sample of the same size would give a somewhat different score. The smaller the test set, or the rarer the event, the bigger that wobble.



ANALOGY

Polling 100 voters gives a rough answer. Polling 10,000 gives a much tighter one. Test sets work the same way.

KEY IDEA

Report a **confidence interval**: a range likely to contain the true performance. Narrow ranges support confident decisions; wide ranges call for more data.

THE PRACTICAL TOOL

Bootstrap confidence intervals

1. **Resample**
   Draw rows from the test set with replacement, same size as the original.
2. **Score**
   Compute the metric on the resampled set.
3. **Repeat**
   Do this a thousand or more times.
4. **Read the range**
   The middle 95% of the scores forms the interval.

KEY IDEA

The bootstrap works for almost any metric, including awkward ones like F1 or NDCG, without special formulas.

PITFALL

Resample at the right level. If rows are grouped by patient, resample **patients**, not rows, or the interval will be too narrow.

THE PRACTICAL TOOL

Bootstrap in code

PYTHON

```
import numpy as np
from sklearn.metrics import average_precision_score

rng = np.random.default_rng(0)
stats = []
for _ in range(2000):
    idx = rng.integers(0, len(y_test), len(y_test))
    if y_test[idx].sum() == 0:
        continue
    stats.append(average_precision_score(
        y_test[idx], scores[idx]))
low, high = np.percentile(stats, [2.5, 97.5])
```

PRO TIP

Use the same resampled indices for two models to get an interval on their **difference**. That is what actually decides between them.

WORKED EXAMPLE

“Average precision 0.41 (95% bootstrap interval 0.36 to 0.46, 2,000 resamples of customers).”

COMPARING MODELS

Is model B really better than A?

KEY IDEA

Compare models on **the same examples** (paired comparison). Shared easy and hard cases cancel out, so real differences show up much more clearly.

DEFINITION

McNemar's test compares two classifiers by looking only at the cases where they disagree. **Paired bootstrap** gives an interval on the score difference.

Question	Approach
Two classifiers, one test set	McNemar's test or paired bootstrap of the difference
Two models across CV folds	Corrected paired tests that account for overlapping training sets
Many models across many datasets	Rank-based tests with post-hoc comparisons
Two forecasts	Tests that compare forecast error series over time

SIGNIFICANCE IN PRACTICE

Statistical versus practical significance

DEFINITION

Statistically significant: the difference is unlikely to be pure chance.

DEFINITION

Practically significant: the difference is big enough to matter for users or the business.

WORKED EXAMPLE

With millions of test rows, a tiny difference can be statistically significant yet worthless. With a few hundred rows, a large and important difference may not reach significance.

RULE OF THUMB

Agree in advance on the **smallest improvement worth shipping**. Then ask whether the interval on the difference clears it.

HIDDEN TRAPS

Multiple comparisons and forking paths

⚠ PITFALL

Test twenty slices, metrics or variants and, by chance alone, about one will look significantly different.

⚠ PITFALL

Choosing which analysis to report after seeing results (the “garden of forking paths”) inflates confidence even without formal tests.

THE FIX

- Pre-register the main metric and comparison
- Adjust for multiple tests when hunting across many slices
- Treat unexpected findings as hypotheses to confirm on new data
- Report how many things you tried

PLANNING

How much test data is enough?

Goal	Rough guide
Estimate accuracy within a few points	Several hundred to a thousand cases
Estimate recall for a rare class	Hundreds of positive cases, not total rows
Detect a small difference between models	Many thousands of paired cases
Reliable results for each subgroup	Enough cases in each group, not just overall

PRO TIP

Run a quick bootstrap on a pilot sample. If the interval is too wide for the decision at hand, you need more test data before you can conclude anything.

VARIABILITY FROM TRAINING

Seeds and training noise

KEY IDEA

Uncertainty does not only come from the test sample. Retraining with a different random seed, data order or initialisation can change results, especially for neural networks.

THE FIX

Train several seeds and report the spread. Compare models across seeds, not from a single lucky run.

IN THE WILD

A new architecture appeared to beat the baseline by a clear margin. Across five seeds each, the ranges overlapped almost entirely. The “improvement” was a single fortunate initialisation.

SECTION 11 RECAP

Uncertainty and statistical rigour

REMEMBER

Takeaways

- Always report an interval, not just a point
- Bootstrap at the level of independent units
- Compare models on paired data
- Separate statistical from practical significance
- Account for multiple comparisons and seeds

CHECK YOURSELF

Model B beats A by 0.5 points; the 95% interval on the difference runs from −0.4 to +1.4. What do you conclude?

ANSWER

The evidence does not show B is better. The interval includes zero; you need more data or should treat them as equivalent.

SECTION 12

12

Error analysis and robustness

Looking beneath the average: finding failing slices, stress-testing inputs and checking behaviour under change.

Error analysis | Slicing | Stress tests | Behavioural tests | Adversarial | Shift

ERROR ANALYSIS

Read your mistakes

1. **Sample errors**
   Pull 50–100 misclassified or high-error cases.

2. **Tag them**
   Note why each went wrong: label error, ambiguity, missing feature, rare case.

3. **Count the tags**
   Rank causes by frequency and impact.

4. **Act on the biggest**
   Fix data, add features or change the approach.

KEY IDEA

An afternoon reading errors often teaches more than a week of tuning. Patterns appear that no metric reveals.

PRO TIP

Include the highest-confidence errors. Confident mistakes often expose label errors or leakage.

SLICING

Averages hide failing groups

Slice	Share of data	Recall	Note
Overall	100%	High	Looks ready to ship
New customers	8%	Much lower	Little history to learn from
Mobile app users	35%	Similar to overall	Fine
Non-English text	4%	Very low	Hidden by the average
Night-time transactions	12%	Lower	Different behaviour patterns

KEY IDEA

Decide the important slices **before** testing: by user type, region, device, language, time, data source and protected characteristics. Report each with its sample size and interval.

SLICING

Finding slices you did not think of

DEFINITION

Automated slice discovery searches combinations of features to find groups where error is unusually high, such as “older devices in rural areas on weekends”.

PITFALL

With many candidate slices, some will look bad by chance. Confirm discovered slices on fresh data before acting.

PRO TIP

Cluster the errors themselves, for example by embedding misclassified texts or images. Groups of similar failures often point to a missing concept in the training data.

STRESS TESTING

Robustness to messy inputs

Missing values

Blank fields, nulls in new places, missing sensors.

Noise

Typos, blur, compression, background noise in audio.

Format changes

New units, date formats, encodings, schema tweaks.

Extremes

Values far outside the training range.

KEY IDEA

Deliberately corrupt validation inputs in realistic ways and measure how gracefully performance degrades. A sudden collapse is a risk to report.

BEHAVIOURAL TESTING

Unit tests for model behaviour

Test type	What it checks	Example
Minimum functionality	Simple cases the model must get right	“This was great” is positive
Invariance	Irrelevant changes don’t change the output	Changing a name doesn’t change sentiment
Directional	Relevant changes move output the right way	Adding “not” flips sentiment
Monotonicity	More of a risk factor never lowers risk	Higher debt never reduces default risk

PRO TIP

Keep these tests in your CI pipeline. They catch regressions that aggregate metrics miss, and they document expected behaviour for reviewers.

ADVERSARIAL ROBUSTNESS

Inputs designed to fool the model

DEFINITION

Adversarial examples are inputs changed in small, deliberate ways that cause wrong predictions, often imperceptible to humans.

USE IT WHEN

Fraud, spam, content moderation, security, or any setting where people have a motive to game the model.

PITFALL

Validating only on natural data overstates robustness in adversarial settings. Attackers adapt to whatever the model does.

THE FIX

Red-team the model; test known evasion tactics; monitor for shifts in attack patterns after launch.

DISTRIBUTION SHIFT

Testing under change

KEY IDEA

Build validation sets that represent **plausible futures**, not just the past: new regions, new devices, seasonal peaks, economic shocks.

IN THE WILD

A demand model validated on two calm years failed during a sudden shock, when buying patterns changed overnight. A stress test using a past disruption would have exposed its fragility.

DEFINITION

Adversarial validation trains a classifier to separate training data from new data. Features that make this easy are where the shift lives.

PRO TIP

Hold out the most recent period, the most unusual period, and a new segment. Report all three alongside the standard test.

SECTION 12 RECAP

Error analysis and robustness

REMEMBER

Takeaways

- Read and categorise real errors
- Pre-define important slices and report each
- Corrupt inputs to test graceful degradation
- Write behavioural tests for expected logic
- Validate against plausible future shifts

CHECK YOURSELF

Overall recall is strong, but one region contributes 3% of data and shows very low recall. What do you do?

ANSWER

Report it explicitly with its interval, investigate why, and decide with stakeholders whether to fix, guard or limit use in that region.

SECTION 13

13

Fairness and responsible validation

Checking that a model’s errors and benefits are distributed acceptably across groups of people.

Group metrics · Trade-offs · Proxies · Intersections · Process

WHY IT BELONGS HERE

Fairness is a validation question

KEY IDEA

A model can be accurate overall and still make far more mistakes for one group than another. That is a validation failure: the evidence did not cover everyone the model affects.

WHY IT MATTERS

Unequal error rates can cause real harm, damage trust, and breach equality and data-protection law in many jurisdictions.

IN THE WILD

A recruitment screening tool learned from historic hiring decisions. Validation on overall accuracy looked fine. Slice analysis showed it systematically down-ranked applicants from groups that had been under-hired in the past.

MEASURING IT

Common group fairness measures

Measure	Asks whether…	Typical use
Demographic parity	Groups receive positive decisions at similar rates	Allocation of opportunities
Equal opportunity	Groups have similar recall among true positives	Benefits that should reach qualified people
Equalised odds	Groups have similar recall and false-alarm rates	High-stakes screening
Predictive parity	A flag means the same probability across groups	Risk scores used by humans
Calibration by group	Probabilities are accurate within each group	Any probability shown or acted on

HARD TRUTHS

You cannot satisfy every definition at once

KEY IDEA

When groups have different underlying rates of the outcome, several fairness measures mathematically cannot all be equal at the same time. Choosing which matters is a values decision, not a technical one.

THE FIX

Agree the fairness criterion with domain experts, affected communities, legal and ethics advisers **before** validation. Document the reasoning.

PRO TIP

Show decision-makers a table of each candidate criterion and what satisfying it would cost elsewhere. Make the trade-off visible rather than buried in a metric choice.

HIDDEN ROUTES

Proxies and intersections

⚠ PITFALL

Removing a protected attribute does not remove bias. Postcode, school, name or shopping habits can act as **proxies** for it.

⚠ PITFALL

A model can look fair for each group separately yet fail for **intersections**, such as older women or young men in one region.

THE FIX

Keep protected attributes available for **evaluation** (with appropriate safeguards), even if they are not used as model inputs. You cannot measure what you cannot see.

RULE OF THUMB

Check intersections that have enough data to be meaningful, and flag those that are too small to assess.

PROCESS

A fairness validation workflow

1. **Identify groups**
   Who is affected, and which groups could be harmed?
2. **Choose criteria**
   Agree which measures matter and acceptable gaps.
3. **Measure**
   Report each measure by group, with intervals.
4. **Investigate**
   Find causes: data, labels, features, thresholds.
5. **Mitigate and document**
   Fix, guard or limit use; record the decision.

KEY IDEA

Fairness validation is never finished at launch. Monitor group metrics in production, because populations and behaviour change.

SECTION 13 RECAP

Fairness and responsible validation

REMEMBER

Takeaways

- Overall accuracy can hide unequal errors
- Different fairness measures can conflict
- Choose criteria with stakeholders and document why
- Watch for proxies and intersections
- Keep monitoring after launch

CHECK YOURSELF

You removed gender from the inputs. Is the model now guaranteed to be gender-fair?

ANSWER

No. Other features can act as proxies. You still need to measure outcomes by gender to know.

14

SECTION 14

Explainability as a validation tool

Using explanations to check that a model is right for the right reasons.

Global importance | Local explanations | Sanity checks | Shortcut learning

RIGHT FOR THE RIGHT REASONS

Why explanations help validation

KEY IDEA

Metrics tell you **whether** a model is right. Explanations help you see **why**, and whether the reasons would hold up in the real world.

IN THE WILD

Shortcut learning

An image classifier for a disease scored very well. Explanations showed it focused on hospital labels and rulers in the images, not the anatomy. It had learned a shortcut.

ANALOGY

A student who gets the right answer by copying the person next to them scores well today. Asking them to show their working reveals the problem before the real exam.

TOOLS

Global and local explanations

Technique	Scope	What it shows	Caution
Permutation importance	Global	Score drop when a feature is shuffled	Misleading with correlated features
SHAP-style values	Global and local	Each feature’s contribution to a prediction	Explains the model, not the world
Partial dependence	Global	Average effect of changing one feature	Hides interactions
Counterfactuals	Local	Smallest change that flips the decision	May suggest unrealistic changes
Saliency maps	Local, images	Which pixels drove the output	Can be noisy or unstable

PRO TIP

Compute importance on **validation** data, not training data. Training-data importance can reward features the model merely memorised.

USING THEM WELL

Validation checks with explanations

ASK YOURSELF

Do the most important features make sense to a domain expert?

ASK YOURSELF

Is any feature suspiciously dominant, hinting at leakage?

ASK YOURSELF

Do effects point in sensible directions, such as more risk factors meaning more risk?

ASK YOURSELF

Do explanations look similar across groups, or does the model rely on proxies for some?

ASK YOURSELF

Are local explanations stable for near-identical inputs?

ASK YOURSELF

Would the model's reasoning survive a change in context?

PITFALL

Explanations are approximations. A plausible explanation does not prove the model is correct; it is one more piece of evidence.

SECTION 14 RECAP

Explainability as a validation tool

REMEMBER

Takeaways

- Use explanations to test for right reasons
- Compute importance on held-out data
- Dominant odd features often mean leakage
- Ask domain experts to review drivers
- Treat explanations as evidence, not proof

CHECK YOURSELF

Your top feature is a database row ID. What does that tell you?

ANSWER

Almost certainly leakage or a data-ordering artefact. IDs should carry no real-world meaning; investigate before anything else.

SECTION 15

15

Evaluating LLMs and generative models

Validating systems whose outputs are open-ended text, code or images, where there is rarely one correct answer.

Why it's different | Eval sets | Human rating | LLM-as-judge | RAG evaluation | Safety

WHAT CHANGES

Why generative evaluation is different

Many right answers

A good summary can be worded in countless ways, so exact matching fails.

Fluent but wrong

Outputs can sound confident and be factually incorrect.

Non-deterministic

The same prompt can give different outputs; evaluate several samples.

Contamination

Public benchmarks may sit in the training data.

KEY IDEA

Generative evaluation combines **task-specific test sets**, **clear rubrics**, **human judgement** and **automated judges**, each validating the others.

BUILDING BLOCKS

A good evaluation set for an LLM application

- Real user queries, sampled and anonymised
- Edge cases and tricky phrasings
- Questions with no answer in the sources
- Adversarial and unsafe prompts
- Coverage of every major intent
- Reference answers or grading notes

RULE OF THUMB

Start with 50–100 carefully curated examples that represent real use. A small, high-quality set beats a big, generic benchmark for product decisions.

PRO TIP

Version the evaluation set like code. Add every production failure you find as a new test case.

DEFINING GOOD

Rubrics turn opinions into measurements

Criterion	What a high score looks like
Correctness	Facts are accurate and consistent with the sources
Faithfulness	Every claim is supported by the provided context
Completeness	Covers all parts of the question
Relevance	Stays on topic, no padding
Safety and policy	Refuses harmful requests; no private data leaked
Tone and format	Matches the brand voice and required structure

PRO TIP

Use simple scales (pass/fail or 1–3) with written examples of each level. Fine-grained 1–10 scales produce noisy, inconsistent ratings

GOLD STANDARD

Human evaluation done properly

✓ DO

- Write clear guidelines with examples
- Blind raters to which system produced each output
- Randomise order in side-by-side comparisons
- Use several raters per item for a sample
- Measure agreement between raters

⚠ AVOID

- Let the developers be the only judges
- Rate on vague criteria like “quality”
- Ignore disagreement between raters
- Use too few items to see real differences

KEY IDEA

If trained raters disagree often, your rubric is ambiguous. Fix the rubric before trusting any scores, human or automated.

SCALING UP

LLM-as-judge

DEFINITION

A strong language model grades outputs against a rubric, either scoring one answer or choosing the better of two.

PITFALL

- May prefer longer answers
- May favour the first option shown
- May prefer its own style of writing
- Can be fooled by confident wording

USE IT WHEN

Fast iteration, regression testing, and large evaluation sets where human rating would be too slow.

THE FIX

Validate the judge: compare its ratings with human ratings on a sample, swap answer order, and recheck when the judge model changes.

Evaluating retrieval-augmented generation

1

Retrieval

Did we fetch the right passages?

2

Generation

Did the answer use them faithfully?

3

End to end

Did the user get a correct, helpful answer?

DEFINITION

Context recall and precision: relevant passages found, and little irrelevant noise.

DEFINITION

Faithfulness or groundedness: claims supported by the retrieved context.

DEFINITION

Answer correctness: matches the reference or rubric.

PRO TIP

Evaluate retrieval and generation separately. When the final answer is wrong, you need to know which stage failed.

SAFETY

Red-teaming and safety evaluation

KEY IDEA

Generative systems must be validated for what they should **not** do: harmful content, leaking private data, following injected instructions, or making up citations.

DEFINITION

Red-teaming means people (and automated tools) deliberately trying to make the system misbehave, then turning each success into a regression test.

- Prompt-injection attempts in documents and inputs
- Requests for personal or confidential data
- Questions outside the allowed scope
- Jailbreak and role-play style attacks

ONGOING

Evaluating LLM systems after launch

KEY IDEA

Model providers update models, prompts change, and user behaviour shifts. Rerun the evaluation suite on every change, just like software tests.

PRO TIP

Collect lightweight live signals: thumbs up or down, edits users make to drafts, escalations to humans, and a regular sample of conversations reviewed against the rubric.

IN THE WILD

A support assistant passed its evaluation suite at launch. A silent upstream model update changed its tone and made it more verbose. Scheduled re-evaluation caught the change within a day.

SECTION 15 RECAP

Evaluating LLMs and generative models

REMEMBER

Takeaways

- Build a small, real, versioned evaluation set
- Write rubrics with examples for each level
- Validate any LLM judge against humans
- Evaluate retrieval and generation separately
- Red-team and re-evaluate on every change

CHECK YOURSELF

Your LLM judge rates system B much higher than A, but B's answers are twice as long. What should you check?

ANSWER

Length bias. Compare against human ratings, control for length, and check whether the extra content is actually correct and useful.

SECTION 16

16 Validation in production

The final and longest stage: proving the model works with real users, and catching the moment it stops working.

Shadow mode · Canary · A/B tests · Monitoring · Drift · Retraining

PRODUCTION

Offline validation is a forecast. Production is the result.

No held-out set fully captures real users, real systems and a changing world. Plan to keep validating after launch.

SAFE ROLLOUT

Shadow deployment

The new model receives real production traffic and makes predictions, but those predictions are only logged, not acted on. The old system keeps making the actual decisions.

KEY IDEA

Shadow mode checks that the model works on real data, through the real pipeline, at real speed, with zero risk to users.

USE IT WHEN

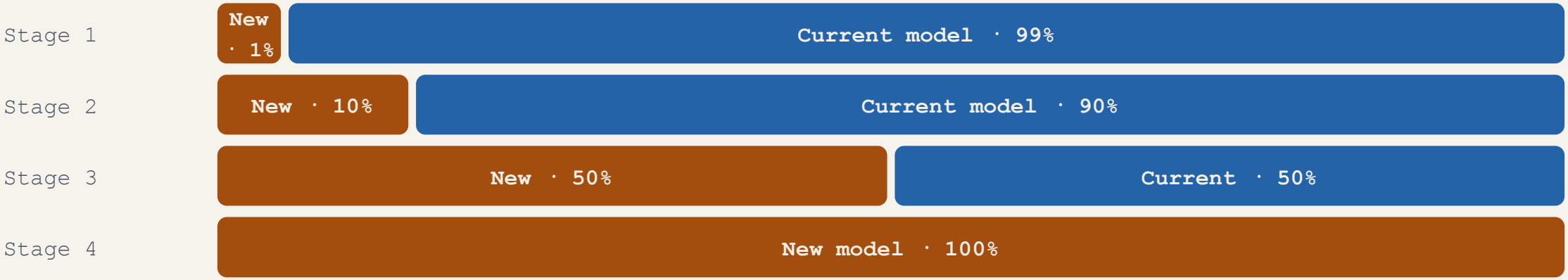
Before any user-facing launch; whenever the feature pipeline changes; when offline data may not match live data.

PITFALL

Shadow mode cannot measure effects on behaviour, because nobody acts on the predictions. You still need an online experiment for that.

SAFE ROLLOUT

Canary releases



DEFINITION

Release to a small share of traffic first, watch technical and business health checks, then expand step by step.

PRO TIP

Define automatic rollback triggers in advance: error rates, latency, or a key metric falling past an agreed limit.

CAUSAL EVIDENCE

A/B testing models

DEFINITION

Randomly assign users to the current model (control) or new model (treatment), then compare business outcomes over a pre-agreed period.

KEY IDEA

Randomisation is what lets you say the model **caused** the difference, rather than seasonality or a marketing campaign.

DO

- Pick one primary metric in advance
- Calculate the sample size needed
- Run for full weekly cycles
- Check guardrail metrics too

AVOID

- Stop the moment results look good
- Change the metric after seeing data
- Ignore novelty effects that fade

CAUSAL EVIDENCE

When A/B tests are hard

Challenge	Alternative
Network effects between users	Randomise by cluster, region or marketplace
Ranking comparisons	Interleaving: mix both models' results in one list
Many options to explore	Multi-armed bandits that shift traffic to winners
Cannot randomise for ethical reasons	Careful quasi-experiments or staged rollouts by region
Very long-term outcomes	Validated short-term proxy metrics, plus long holdouts

PRO TIP

Keep a small long-term holdout group that never gets the new model. It measures the cumulative effect of many model changes over months.

MONITORING

What to monitor after launch

Data quality

Missing values, schema changes, broken upstream feeds, out-of-range values.

Input drift

Changes in the distribution of each feature compared with training.

Prediction drift

Shifts in the distribution of scores and positive rates.

Live performance

Metrics computed once true labels arrive.

System health

Latency, error rates, throughput, cost.

Business impact

The outcome the model was built to improve.

MONITORING

Kinds of drift

DEFINITION

Data drift: the inputs change, such as a new customer segment or a sensor recalibration.

DEFINITION

Label drift: outcome rates change, such as a rise in fraud or seasonal demand.

DEFINITION

Concept drift: the relationship changes, so the same inputs now mean something different.

PRO TIP

Drift measures such as population stability or distribution distance tests are **alarms, not verdicts**. Drift in an unimportant feature may not matter; investigate before retraining.

PITFALL

Labels often arrive weeks or months later. Use input and prediction drift as early warnings until true performance can be measured.

MONITORING

Closing the loop on labels

⚠ PITFALL

When the model's decisions change what you observe (a declined loan never shows whether it would have been repaid), new labels become biased. This is a **feedback loop**.

THE FIX

Keep a small random exploration or holdout, log decision probabilities, and use human review samples to get unbiased labels over time.

IN THE WILD

A fraud model blocked most suspicious transactions, so they were never confirmed as fraud. Over time, the training data contained fewer examples of the patterns it blocked, and retrained versions grew weaker at catching them.

KEEPING IT FRESH

Retraining as a validated process

1. **Trigger**
   Scheduled, or driven by drift and performance alerts.

2. **Retrain**
   Same pipeline, newer data, versioned inputs.

3. **Validate automatically**
   Time-based test, baselines, slices, fairness and behavioural tests.

4. **Compare to champion**
   The challenger must beat the current model by an agreed margin.

5. **Roll out safely**
   Shadow or canary before full release, with rollback ready.

SECTION 16 RECAP

Validation in production

REMEMBER

Takeaways

- Shadow first: real data, zero risk
- Canary with automatic rollback triggers
- A/B tests give causal evidence of impact
- Monitor data, predictions, performance and business
- Treat retraining as a validated pipeline

CHECK YOURSELF

Prediction drift alarms fire, but labels arrive in 60 days. What can you do now?

ANSWER

Investigate which features drifted and why, review a sample manually, check business proxies, and consider a safe fallback if the shift is severe.

SECTION 17

17

Governance, documentation and MLOps

Making validation repeatable, reviewable and auditable, from model cards to automated checks in the pipeline.

Validation plan · Model cards · Independent review · Regulation · CI for ML · Reproducibility

START RIGHT

The validation plan

KEY IDEA

Write the plan **before** modelling, agree it with stakeholders, and keep it under version control. It is your protection against moving the goalposts.

PRO TIP

Treat any later change to the plan as a decision that must be justified and recorded, not a quiet edit.

- Intended use and users
- Prediction moment and inputs
- Split strategy and test set
- Primary and guardrail metrics
- Baselines to beat
- Slices and fairness criteria
- Pass and fail thresholds
- Rollout and monitoring plan

DOCUMENTATION

Model cards and datasheets

DEFINITION

Model card

A **model card** is a short, structured document describing what a model is for, how it was validated, how it performs across groups, and its known limitations.

DEFINITION

Datasheet

A **datasheet for a dataset** records how data was collected, labelled and cleaned, what it covers, and what it should not be used for.

Model card section	Contents
Intended use	Users, decisions supported, out-of-scope uses
Evaluation data	Source, time period, split method, size
Results	Metrics with intervals, overall and by slice
Limitations	Known failure modes, conditions not tested

INDEPENDENT CHALLENGE

Model risk management and review

KEY IDEA

In regulated industries such as banking, models are reviewed by an independent validation team that did not build them. They challenge assumptions, rerun tests and can block deployment.

WHY IT MATTERS

Builders are invested in their models. An independent reviewer brings fresh scepticism and catches errors the team has become blind to.

PRO TIP

Even without a formal function, ask a colleague outside the project to try to break your validation. Pay them in coffee.

WORKED EXAMPLE

Banking supervisors on both sides of the Atlantic publish model risk management expectations, covering independent validation, documentation, ongoing monitoring and model inventories.

REGULATION

The regulatory direction of travel

European Union

The EU AI Act sets requirements for high-risk AI systems, including data governance, accuracy and robustness, technical documentation, human oversight and post-market monitoring.

United Kingdom

A principles-based approach led by existing regulators, such as those for data protection, financial services and medical devices, applying principles like safety, transparency and fairness.

Sector rules

Medical devices, credit and insurance already carry specific validation and evidence expectations in many countries.

REMEMBER

This is a summary, not legal advice. Requirements and timelines evolve; check current guidance with your legal and compliance teams for your sector and region.

AUTOMATION

Continuous validation in the ML pipeline

1 → 2 → 3 → 4 → 5

Data checks

Schema, ranges, missing values, duplicates, leakage tests.

Training

Versioned data, code and configuration.

Model checks

Metrics vs baseline and champion, slices, fairness.

Behaviour tests

Invariance, direction, safety suites.

Gate

Only promote models that pass every check.

KEY IDEA

Validation should be **code**, not a notebook someone remembers to run. Every model version passes through the same automated gates.

AUTOMATION

A model quality gate in code

PYTHON · ILLUSTRATIVE

```python
def quality_gate(cand, champ, slices):
    checks = {
        "beats_baseline": cand.ap > baseline.ap + 0.02,
        "not_worse": cand.ap_low >= champ.ap - 0.01,
        "calibrated": cand.ece <= 0.03,
        "slices_ok": all(s.recall >= 0.6
                         for s in slices),
        "latency_ok": cand.p95_ms <= 120,
    }
    failed = [k for k, ok in checks.items() if not ok]
    return len(failed) == 0, failed
```

PRO TIP

Thresholds in the gate come from the validation plan, not from whatever the latest model happens to achieve.

PITFALL

The numbers here are placeholders. Set yours from business needs and measured uncertainty.

REPRODUCIBILITY

Can someone else get the same result?

- Data versioned and snapshotted, with split IDs saved
- Code and environment pinned (packages, versions)
- Random seeds recorded for splits and training
- Experiment tracking with parameters and metrics
- Evaluation scripts stored with the model
- Every test-set evaluation logged with date and author

RULE OF THUMB

If you cannot reproduce a validation result, you cannot defend it. Reproducibility is the foundation of audit and trust.

SECTION 17 RECAP

Governance, documentation and MLOps

REMEMBER

Takeaways

- Write and freeze a validation plan first
- Publish a model card with slice results and limits
- Seek independent challenge
- Automate validation as pipeline gates
- Make every result reproducible

CHECK YOURSELF

A model passed validation six months ago. A new version is ready. Which checks should run?

ANSWER

All of them, automatically: data checks, metrics against baseline and champion, slices, fairness, behaviour tests, then shadow or canary rollout.

SECTION 18

18 End-to-end case studies

Three realistic projects walked through from framing to production, showing how the pieces fit together.

Credit risk | Medical imaging | Retail demand | LLM support assistant

CASE 1 · CREDIT RISK

Framing and splitting

IN THE WILD

The brief

A lender wants to predict whether a new applicant will miss payments within 12 months, to support approval decisions.

ASK YOURSELF

When is the prediction made? At application. What is known then? Only application data and credit history up to that date.



KEY IDEA

Split by application date with a 12-month gap so every label has fully matured. Customers who reapply stay on one side of the split.

CASE 1 · CREDIT RISK

Metrics, leakage and fairness

DEFINITION

Metrics: ROC AUC for ranking, calibration for pricing, expected loss at the approval threshold for the business.

PITFALL

Leakage found: a “collections flag” field was updated after defaults. Removed after a timeline review.

KEY IDEA

Fairness: approval rates and calibration compared across age bands and other monitored groups.

WARNING SIGNS

Reject inference problem: past declined applicants have no outcome labels, so training data under-represents riskier profiles.

THE FIX

Mitigation: a small randomised approval band for borderline cases to collect unbiased labels over time.

LESSON

Independent model risk review rebuilt the test set from raw data and confirmed the results before sign-off.

CASE 2 · MEDICAL IMAGING

Validation that matches clinical use

IN THE WILD

The brief

A model flags chest X-rays likely to show a serious finding, so radiologists can review them first.

PITFALL

The first validation used random image splits. Patients with several scans appeared on both sides, and scores were inflated.

THE FIX

- Split by patient
- External test at a different hospital
- Reference labels from a panel of radiologists
- Report recall at a fixed false-alarm rate
- Slice by scanner type, age and sex

CASE 2 · MEDICAL IMAGING

What external validation revealed

WARNING SIGNS

Shortcut found

Performance dropped at the external hospital. Explanations showed attention on text markers and portable-scanner artefacts typical of very sick inpatients.

THE FIX

Response

Masked text markers, added more varied training data, and ran a prospective silent trial where predictions were logged but not shown to clinicians.

LESSON

Internal test sets share hidden quirks with training data. For high-stakes models, external and prospective validation are not optional extras; they are the real test.

CASE 3 · RETAIL DEMAND

Forecasting thousands of products

IN THE WILD

The brief

A grocer forecasts daily demand for 8,000 products across 300 stores to plan orders.

KEY IDEA

Rolling-origin evaluation over the past year, forecasting 1 to 14 days ahead, scored with a scaled error against seasonal naive.

Check	Finding
Overall vs naive	Clear improvement
New products	Worse than naive
Promotions	Big under-forecasts
Holidays	Missed peaks
Fresh produce	Strong gains

CASE 3 · RETAIL DEMAND

Turning findings into decisions

THE FIX

New products: fall back to a similar-product model until enough history exists.

THE FIX

Promotions: add planned-promotion features, built only from information known when orders are placed.

THE FIX

Business metric: simulate stock-outs and waste using the forecasts, not just error scores.

KEY IDEA

Rollout: run in shadow alongside planners for six weeks, comparing order recommendations.

PRO TIP

Monitoring: weekly scaled-error dashboards by category and store, with drift alerts.

LESSON

The overall score hid both the biggest wins and the biggest risks. Slicing drove every key decision.

CASE 4 · LLM SUPPORT ASSISTANT

Evaluating a retrieval-based assistant

IN THE WILD

The brief

A company launches an assistant that answers customer questions using its help-centre articles.

KEY IDEA

An evaluation set of 300 real, anonymised questions, including 40 with no answer in the help centre and 30 adversarial prompts.

Layer	Measure
Retrieval	Right article in top 5
Faithfulness	Claims supported by sources
Correctness	Rubric, human plus judge
Safety	Refusals, no data leaks
Live	Escalation rate, ratings

CASE 4 · LLM SUPPORT ASSISTANT

What the evaluation caught

WARNING SIGNS

Problem

When no article answered the question, the assistant often invented a plausible-sounding policy instead of saying it did not know.

THE FIX

Fix

Prompt and retrieval changes to encourage “I don’t know” with a handover, plus a dedicated test category for unanswerable questions.

LESSON

Include the questions your system **should not** answer. They are where generative systems fail most dangerously, and they are often missing from evaluation sets.

19

SECTION 19

Toolkit and cheat sheets

Checklists, quick-reference tables and a final self-test to use on every project.

Master checklist | Red flags | Report template | Tools | Glossary | Final quiz

MASTER CHECKLIST

Before training

- [ ] Use case, users and decision clearly defined
- [ ] Prediction moment and available inputs documented
- [ ] Cost of each error type agreed
- [ ] Primary metric and guardrails chosen
- [ ] Baselines defined
- [ ] Data deduplicated
- [ ] Split strategy matches deployment (group, time, site)
- [ ] Test set created and locked away
- [ ] Important slices and fairness criteria listed
- [ ] Validation plan written and versioned

MASTER CHECKLIST

Before launch

- [ ] All preprocessing fitted inside pipelines
- [ ] Tuning nested or kept off the test set
- [ ] Test set evaluated once, on the frozen model
- [ ] Results reported with intervals
- [ ] Beats baselines by a meaningful margin
- [ ] Slices, fairness and calibration checked
- [ ] Robustness and behaviour tests passed
- [ ] Top features reviewed for leakage
- [ ] Model card and validation report complete
- [ ] Shadow, canary and rollback plan ready

QUICK REFERENCE

Red flags that should stop you

⚠ PITFALL

A score far above published results or intuition.

⚠ PITFALL

One feature dominates importance.

⚠ PITFALL

Random splits on time-ordered or grouped data.

⚠ PITFALL

No baseline in the results.

⚠ PITFALL

A single number with no range.

⚠ PITFALL

Test set used many times during development.

⚠ PITFALL

Accuracy reported on heavily imbalanced data.

⚠ PITFALL

No results broken down by slice.

⚠ PITFALL

No plan for monitoring after launch.

QUICK REFERENCE

A validation report template

Section	What to include
1. Summary	Decision supported, headline result with interval, recommendation
2. Data	Sources, dates, size, labelling, known gaps
3. Method	Split design, CV scheme, tuning, threshold selection
4. Results	Primary and secondary metrics vs baselines, with intervals
5. Deep dives	Slices, fairness, calibration, robustness, error analysis
6. Limitations	Untested conditions, risks, mitigations
7. Deployment	Rollout plan, monitoring, retraining triggers, owners

QUICK REFERENCE

Useful tools

Need	Examples of tools
Splits, CV, metrics, pipelines	scikit-learn, imbalanced-learn
Experiment tracking	MLflow, Weights & Biases
Data validation	Great Expectations, Pandera, TensorFlow Data Validation
Fairness	Fairlearn, AIF360
Explainability	SHAP, scikit-learn inspection tools, Captum
Monitoring and drift	Evidently, NannyML, cloud monitoring services
LLM evaluation	Evaluation harnesses, RAG evaluation libraries, custom rubric scripts

PRO TIP

Tools change quickly. The principles in this deck do not. Pick tools that make the principles easy to follow, and check each project's current documentation.

GLOSSARY

Key terms, part 1

Term	Plain meaning
Generalisation	Performing well on new, unseen data
Overfitting	Learning noise and quirks of the training data
Hold-out set	Data kept aside for checking, not training
Cross-validation	Rotating which part of the data is held out
Stratification	Keeping class proportions equal across splits
Data leakage	Information unavailable at prediction time used in training
Nested CV	Tuning inside an outer loop for honest estimates

GLOSSARY

Key terms, part 2

Term	Plain meaning
Precision	When the model says yes, how often it is right
Recall	Of all real cases, the share the model finds
Calibration	Predicted probabilities match observed rates
Bootstrap	Resampling the test set to estimate uncertainty
Drift	Data or relationships changing over time
Shadow mode	Running a model live without acting on its output
Model card	Short document of a model’s use, results and limits

FINAL SELF-TEST

Five questions to check your understanding

1. **Splitting**
   Why is a random split wrong for predicting next month’s sales?

2. **Leakage**
   Name two ways preprocessing can leak test information into training.

3. **Metrics**
   Why can a model with 99% accuracy be useless?

4. **Uncertainty**
   What does it mean if the interval on a model difference includes zero?

5. **Production**
   What can shadow mode tell you, and what can it not?

FINAL SELF-TEST

Answers

ANSWER

1 · Splitting

Random splits let the model learn from future months. Split by time, with a gap if labels take time to mature.

ANSWER

2 · Leakage

Scaling, imputation, feature selection or target encoding fitted on the full dataset before splitting.

ANSWER

3 · Metrics

With heavy imbalance, always predicting the majority class achieves it while catching nothing.

ANSWER

4 · Uncertainty

The data does not show a real difference. Treat the models as equivalent or gather more evidence.

ANSWER

5 · Production

Shadow mode shows real-data behaviour and pipeline health. It cannot show effects on user behaviour or outcomes.

PRO TIP

Score yourself honestly. Any wrong answer points to the section to revisit.

PULLING IT TOGETHER

The ten commandments of model validation

1

Simulate deployment

Splits mirror real use.

2

Guard the test set

Look once, at the end.

3

Beat baselines

Simple first, always.

4

Match metric to cost

Errors are not equal.

5

Hunt leakage

Too good means suspicious.

6

Report ranges

Intervals, not points.

7

Slice everything

Averages hide failures.

8

Check calibration

Probabilities must mean it.

9

Document it

Plans, cards, reports.

10

Keep validating

Production is the real test.

FINAL THOUGHT

Validation is not the step that slows you down. It is the step that lets you move fast without breaking trust.

Every hour invested in honest evidence saves days of rework, rollbacks and difficult conversations later.

THANK YOU

Questions and discussion

Bring a model you are working on. We will walk through its split, metrics, leakage risks and production plan together, using the checklists in section 19.

Anthony Chinedu Echem

Machine Learning Engineer · Phexara Ltd

Deploying Machine Learning Models

From a trained notebook model to a reliable, observable, secure production service

Patterns · MLOps · Monitoring · LLMs

BUILD
Package, serve, containerise

SHIP
Automate, release, scale

RUN
Monitor, retrain, govern



Training a good model is only the beginning. Value appears when predictions reach real users, reliably, every day.

The gap

Many teams build models that never leave a notebook because the path to production is unclear.

The goal

Give you a complete, practical map of every decision between a trained model and a trusted service.

The promise

No heavy mathematics. Plain language, patterns, checklists and worked examples you can reuse.

GETTING STARTED

1 Data scientists

Who train strong models and now need to ship them without depending on a separate team.

2 ML engineers

Who own serving, pipelines and infrastructure and want a structured reference.

3 Software engineers

Who are integrating models into products and need to understand their quirks.

4 Platform and DevOps

Who run the clusters, registries and pipelines that models depend on.

5 Technical leads

Who must choose architectures, tools and team responsibilities with confidence.

6 Students and career changers

Who want an end to end view that bridges coursework and industry practice.

GETTING STARTED

What you should already know

- Basic Python and working with packages
- How a model is trained, validated and saved
- Common metrics such as accuracy, precision and recall
- Using a terminal and running scripts
- The idea of an API that receives and returns data
- Version control with Git at a basic level
- Helpful but optional: Docker and the cloud
- Helpful but optional: SQL and data pipelines

GETTING STARTED

Learning outcomes

1 Choose a pattern

Pick batch, online, streaming or edge deployment for a given business problem and defend the choice.

2 Package and serve

Turn a trained model into a versioned artifact behind a tested, documented API.

3 Containerise and scale

Build lean container images and run them on Kubernetes or managed cloud services.

4 Automate delivery

Design CI and CD pipelines with quality gates, registries and safe release strategies.

5 Observe and improve

Monitor systems, data and predictions, detect drift and trigger retraining.

6 Govern responsibly

Apply security, privacy, fairness and audit practices that stand up to scrutiny.

GETTING STARTED

Roadmap of the twenty sections

No.	Section	No.	Section
1	Foundations of deployment	11	Release strategies
2	Deployment patterns	12	Performance optimisation
3	Preparing the model	13	Monitoring and observability
4	Serving through APIs	14	Retraining and the model lifecycle
5	Containerisation	15	Edge and mobile deployment
6	Serving frameworks	16	Deploying large language models
7	Kubernetes and orchestration	17	Security, privacy and governance
8	Cloud and managed platforms	18	Reliability and cost
9	CI and CD for machine learning	19	End to end case study
10	Testing models for production	20	Checklists, glossary and next steps



Each section opens with a divider, teaches through concept, comparison and practice slides, and closes with a recap and an exercise.

GETTING STARTED

How the slides are designed to be read

Concept slides	Introduce one idea with a lead statement and three supporting cards.
Comparison slides	Place two options side by side and end with a clear verdict.
Flow slides	Show a process as numbered boxes connected by arrows.
Practice slides	Code, checklists, pitfalls and knowledge checks you can act on.
Recap slides	Summarise the section, set an exercise and preview what comes next.

Reading guide

Blue boxes carry core ideas. Orange boxes warn about traps. Yellow sticky notes hold tips worth remembering. Dark boxes give verdicts and next steps.

SECTION 1

01

Foundations of deployment

What it really means to put a model into production, and why it is harder than it looks.

IN THIS SECTION

- What deployment means
- Training versus serving
- Why projects stall
- The ML system landscape
- The deployment lifecycle
- Roles and maturity levels

SECTION 1 · FOUNDATIONS

What is model deployment?

DEFINITION

Deployment is the process of making a trained model available so that other systems or people can use its predictions reliably, securely and repeatedly in real conditions.

EXAMPLE

A churn model trained in a notebook becomes a nightly job that scores every customer and writes risk scores into the CRM, where account managers act on them each morning.

REMEMBER

A model is not deployed until someone outside the data team depends on its output.

SECTION 1 · FOUNDATIONS

Training world versus serving world

TRAINING

Experiment focused

- Static historical data loaded in bulk
- Runs for minutes or hours, then stops
- Success means a better validation score
- Failures are cheap and private
- One person controls the environment

SERVING

Product focused

- Live, messy data arriving continuously
- Runs indefinitely and must stay available
- Success means value, speed and reliability
- Failures are visible and costly
- Many teams and systems depend on it

VERDICT Deployment is the bridge between these worlds. Most of the work is engineering the serving side so it behaves as safely as the training side was measured.

SECTION 1 · FOUNDATIONS

Why so many models never reach production

1 No clear owner

Nobody is accountable for the model after training, so handover stalls.

2 Notebook only code

Logic lives in cells with hidden state that cannot be rerun reliably.

3 Training serving skew

Features are computed differently in production than in training.

4 Missing infrastructure

There is no registry, pipeline or serving platform to deploy onto.

5 Unclear value

The business metric the model should move was never agreed.

6 Risk and compliance

Legal, privacy or fairness questions surface late and block release.

“

The model is a small box in the middle of a very large system.

☆ Industry experience, popularised by the paper on hidden technical debt in machine learning systems, shows that model code is a minor share of a production ML system.

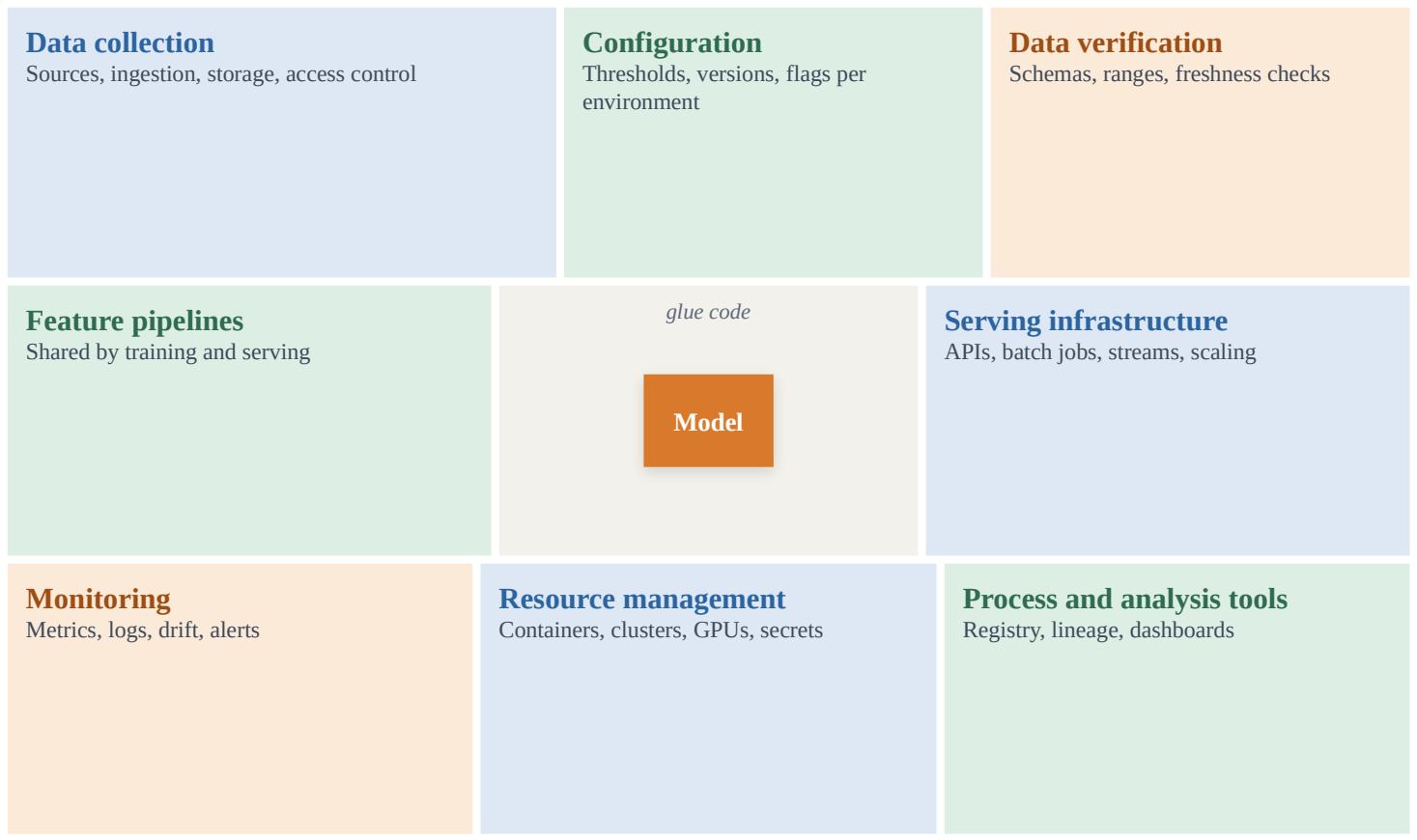
Structure of a production ML system

Data collection	Sources, ingestion, storage and access control for raw data.
Feature pipeline	Validation, transformation and feature storage shared by training and serving.
Model	The trained artifact plus its metadata, version and evaluation results.
Serving layer	APIs, batch jobs or streams that deliver predictions to consumers.
Infrastructure	Containers, clusters, networking, secrets and compute resources.
Monitoring	Metrics, logs, drift checks, alerts and feedback for retraining.

Where effort goes

Expect to spend far more time on data, configuration, serving and monitoring than on the model itself. Plan your team and schedule around that reality.

The model really is the small box in the middle



Tile sizes indicate typical engineering effort, after Sculley et al., Hidden Technical Debt in ML Systems (2015).

FAILURES THAT START OUTSIDE THE MODEL

Feature pipeline
An upstream team changed a field from pounds to pence. Predictions went wrong; the service stayed green.

Monitoring
Nobody watched input drift, so a broken sensor fed zeros for six weeks.

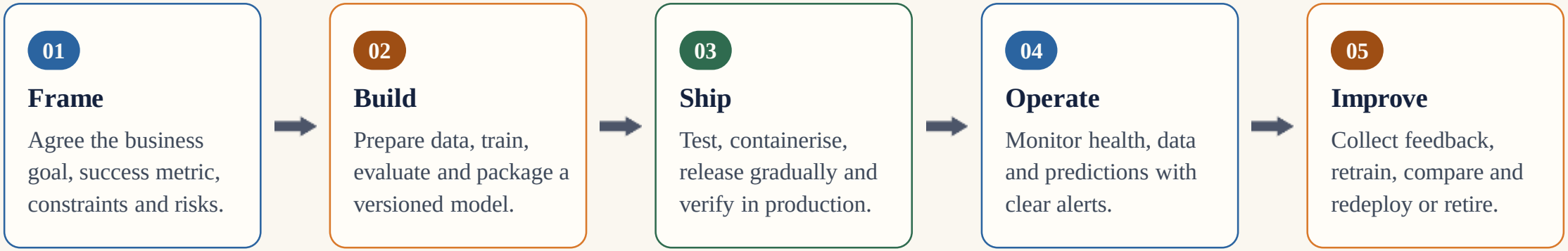
Configuration
A typo pointed production at last year's model version.

REMEMBER

Plan the boxes around the model before polishing the model.

SECTION 1 · FOUNDATIONS

The end to end deployment lifecycle



This is a loop, not a line. Every production model eventually returns to the Build stage as data and the world change.

SECTION 1 · FOUNDATIONS

Deployment is a product decision

Before writing serving code, decide who uses the prediction, how fast they need it, and what happens when it is wrong.

Consumer

A human dashboard, another service, a mobile app or a batch report each demand a different design.

Latency need

Milliseconds for a checkout fraud check, hours for a weekly demand forecast.

Cost of error

A wrong film recommendation is harmless. A wrong medical triage score is not.

Who does what in a deployment

Role	Typical responsibilities
Data scientist	Problem framing, features, training, evaluation, model documentation
ML engineer	Packaging, serving, pipelines, performance, model registry
Data engineer	Ingestion, data quality, feature pipelines, storage
Platform or DevOps	Clusters, CI and CD tooling, networking, secrets, cost controls
Product owner	Business metric, acceptance criteria, release decisions
Risk and compliance	Privacy, fairness, regulatory and audit sign off



In small teams one person may hold several roles. What matters is that every responsibility has a named owner.

MLOps maturity levels

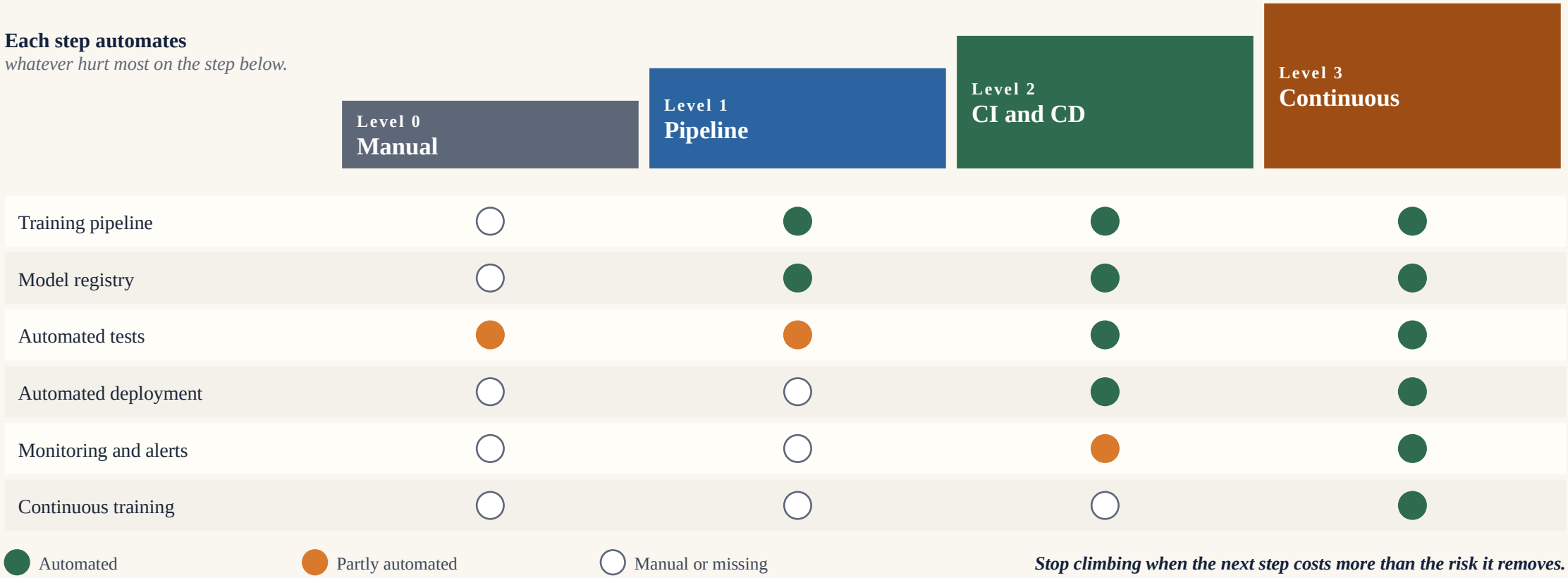
Level	What it looks like	Main risk
Level 0	Manual training, manual handover, scripts run by hand	Slow, fragile, unrepeatable releases
Level 1	Automated training pipeline and model registry	Deployment still manual and error prone
Level 2	CI and CD for code, data and models with tests	Monitoring may lag behind delivery
Level 3	Continuous training, monitoring and automatic gates	Complexity and cost of the platform



These levels echo the widely cited Google Cloud MLOps maturity description. Aim for the lowest level that safely meets your needs, then climb.

Climbing the MLOps maturity staircase, one capability at a time

Each step automates
whatever hurt most on the step below.



Stop climbing when the next step costs more than the risk it removes.

SECTION 1 · FOUNDATIONS

Foundational mistakes to avoid

PITFALL	BETTER PRACTICE
⚠ Treating deployment as a final step after the model is finished.	✓ Design for deployment from day one: inputs, latency, owners and monitoring.
⚠ Optimising only for offline accuracy.	✓ Track the business metric and operational metrics alongside model quality.
⚠ Building custom infrastructure for the first model.	✓ Start with managed or proven tools and add complexity only when needed.
⚠ Assuming the model will stay accurate forever.	✓ Plan monitoring and retraining before the first release.

Knowledge check

? A model scores 95 percent accuracy offline but is never used by the sales team. Has it been deployed?

MODEL ANSWER

No. Deployment means the predictions are available and relied upon in a real workflow. A high offline score with no consumer delivers no value. The team should identify the consumer, the delivery channel and the action the prediction triggers.

WHY IT MATTERS

It reframes success from model quality to delivered business value.

Section recap

KEY TAKEAWAYS

- Deployment makes predictions reliably usable by real consumers
- Serving conditions differ sharply from training conditions
- Most project failures are organisational and infrastructural
- The model is a small part of a larger system
- Deployment is a continuous lifecycle, not a single event

TRY THIS

Pick one model you have built. Write down its consumer, required latency, cost of a wrong prediction and who would own it in production.

NEXT UP

Section 2: the main deployment patterns and how to choose between them.

SECTION 2

02

Deployment patterns

The handful of shapes that almost every production model takes, and how to pick the right one.

IN THIS SECTION

- Batch prediction
- Online real time serving
- Streaming inference
- Edge and on device
- Embedded and serverless
- Choosing and combining

SECTION 2 · PATTERNS

Batch prediction

Batch prediction scores a large set of records on a schedule and stores the results for later use. It is the simplest and often the most robust pattern.

How it works

A scheduled job loads data, runs the model over every row and writes predictions to a table or file.

Great for

Churn scores, demand forecasts, lead scoring, monthly risk ratings and recommendations refreshed nightly.

Trade off

Predictions can be stale and you score records nobody may ever look at.

SECTION 2 · PATTERNS

Online real time serving

Online serving answers one request at a time through an API, returning a prediction in milliseconds while the user or system waits.

How it works

A client sends features in a request, the service runs the model and returns a response immediately.

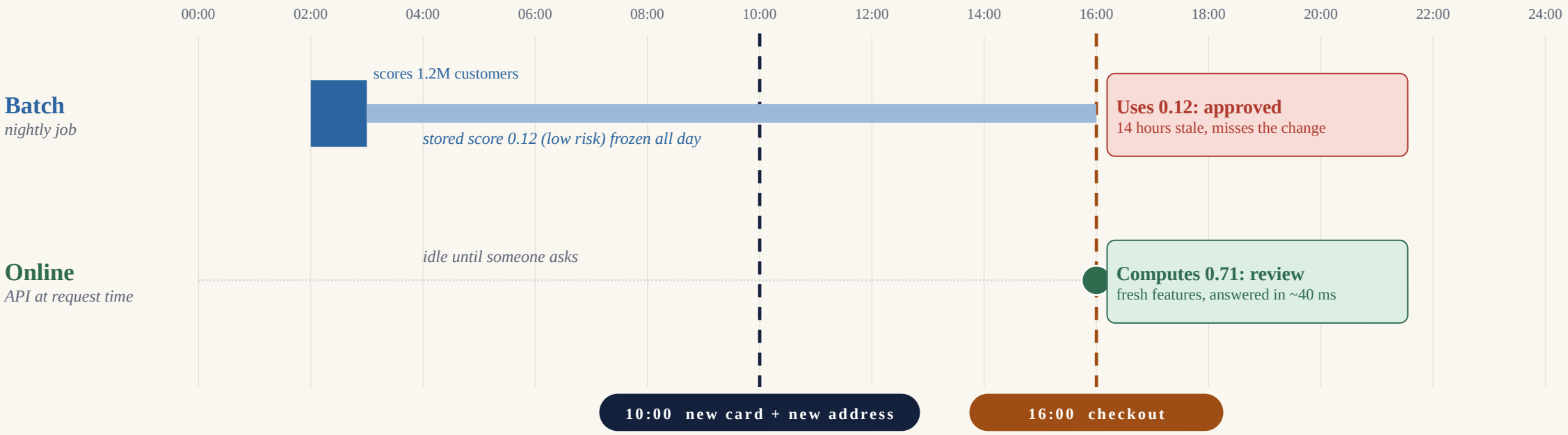
Great for

Fraud checks at payment, search ranking, dynamic pricing, chat assistants and personalised content.

Trade off

Needs high availability, tight latency budgets, scaling and careful feature freshness.

Batch versus online, told through one customer's day



BATCH PAYS OFF WHEN

Inputs change slower than the schedule, and a few hours of staleness costs nothing. Cheap, simple, easy to rerun.

ONLINE PAYS OFF WHEN

The decision depends on what just happened. You pay for uptime, a latency budget and fresh features.

SECTION 2 · PATTERNS

Streaming inference

Streaming inference consumes events from a message stream, scores them continuously and publishes results back to another stream.

How it works

Events arrive on a platform such as Kafka, a consumer applies the model and emits predictions downstream.

Great for

Sensor anomaly detection, clickstream personalisation, transaction monitoring and log analysis.

Trade off

Adds stream processing complexity, ordering concerns and stateful feature windows.

SECTION 2 · PATTERNS

Edge and on device deployment

Edge deployment runs the model on the device where data is created, such as a phone, camera, vehicle or factory gateway.

How it works

A compressed model ships inside an app or firmware and runs locally without a network round trip.

Great for

Offline use, privacy sensitive data, very low latency vision and speech, and bandwidth savings.

Trade off

Limited memory and compute, harder updates and many hardware variations to support.

SECTION 2 · PATTERNS

Embedded model versus model as a service

EMBEDDED

Model inside the application

- Loaded directly in the app process
- No network hop, lowest latency
- Model updates require app redeploys
- Couples app and model dependencies

SERVICE

Model behind its own API

- Runs as a separate service
- Independent scaling and versioning
- Adds network latency and a dependency
- Reusable by many applications

VERDICT Embed for small, stable models owned by one team. Use a separate service when models change often or serve several consumers.

SECTION 2 · PATTERNS

Serverless inference

Serverless platforms run your prediction code on demand and scale to zero when idle, so you pay only for requests actually served.

How it works

You upload a function or container and the provider starts instances automatically as requests arrive.

Great for

Spiky or low traffic workloads, prototypes, internal tools and lightweight models.

Trade off

Cold starts add delay, memory and runtime limits apply, and GPU support varies by provider.

SECTION 2 · PATTERNS

Pattern comparison at a glance

Pattern	Latency	Freshness	Complexity	Typical example
Batch	Hours	Scheduled	Low	Nightly churn scores
Online	Milliseconds	Per request	Medium	Card fraud check
Streaming	Seconds	Continuous	High	Sensor anomaly alerts
Edge	Milliseconds	On device	High	Phone camera filter
Serverless	Variable	Per request	Low to medium	Internal text classifier



Latency here means how long until a prediction is available to the consumer, not only model compute time.

SECTION 2 · PATTERNS

A decision path for choosing a pattern

01

Need it now?

If the consumer waits for the answer, start from online or edge.

02

Offline use?

If devices lack connectivity or data is private, prefer edge.

03

Events flow?

If data arrives as continuous events, consider streaming.

04

Known inputs?

If all inputs exist in advance, batch is usually cheapest.

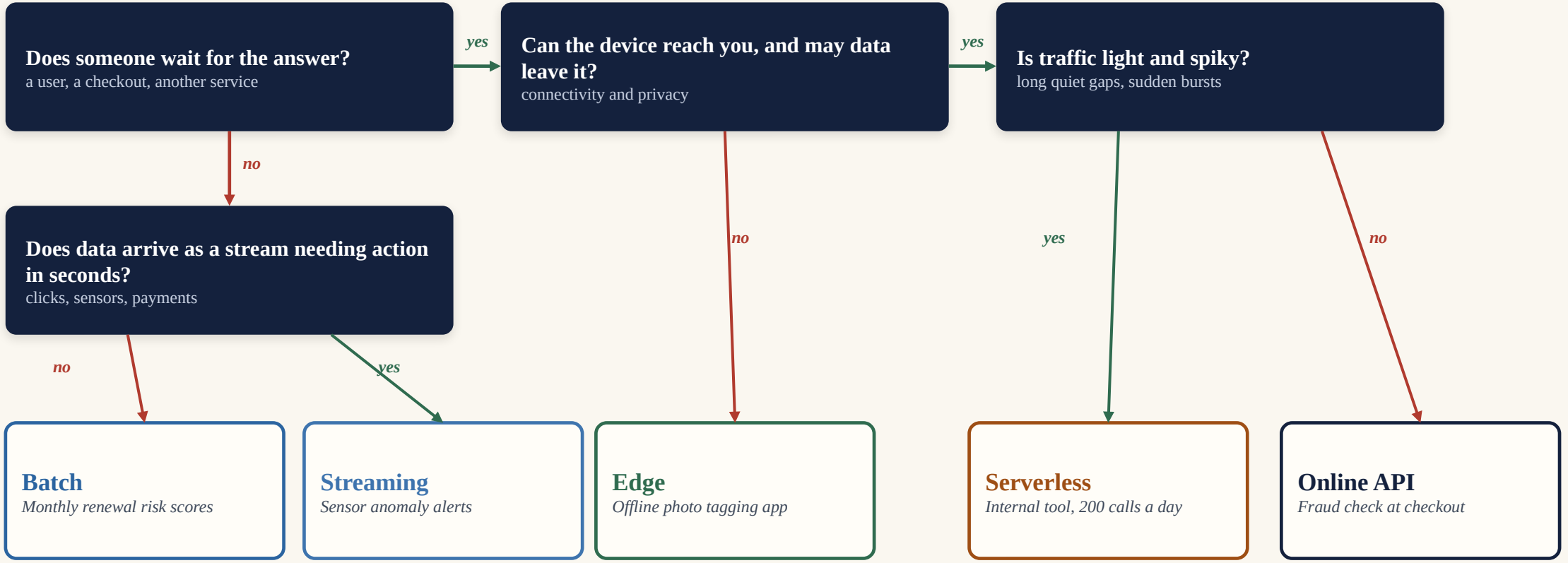
05

Traffic shape

Spiky and light traffic suits serverless.

When in doubt, begin with batch. It is easier to move from batch to online than to retreat from an overbuilt real time system.

The pattern decision path, drawn as a flowchart



! Each 'yes' along the top row adds operating cost. Answer for today's needs, not for the scale you hope to reach.

SECTION 2 · PATTERNS

Hybrid patterns are common



Mature systems often combine patterns: heavy work in batch, fresh adjustments online.

Precompute plus lookup

Batch jobs compute recommendations nightly and an API serves them from a fast key value store.

Two stage ranking

A cheap model retrieves candidates in batch and a richer model reranks them online.

Edge plus cloud

A small device model filters events and sends only uncertain cases to a larger cloud model.

SECTION 2 · PATTERNS

Pattern selection pitfalls

PITFALL	BETTER PRACTICE
⚠ Building real time serving because it sounds modern.	✓ Match the pattern to how quickly the business acts on predictions.
⚠ Ignoring how features will be available at request time.	✓ Confirm every feature can be computed within the latency budget.
⚠ Choosing edge without an update strategy.	✓ Plan over the air model updates and version checks before launch.
⚠ Using serverless for a large GPU model with steady load.	✓ Use dedicated instances when traffic is constant and models are heavy.

Knowledge check

? An insurer wants risk scores for every policy before monthly renewal letters are printed. Which pattern fits best?

MODEL ANSWER

Batch prediction. All policies are known in advance, the decision happens on a monthly schedule and nobody waits on a live response. A scheduled job can score every policy and write results to the renewals system at low cost.

WHY IT MATTERS

It shows that the business process, not technology fashion, should drive the pattern.

SECTION 2 · PATTERNS

Section recap

KEY TAKEAWAYS

- Batch is simple, cheap and robust for scheduled decisions
- Online serving answers live requests within milliseconds
- Streaming handles continuous events with low delay
- Edge runs locally for privacy, offline use and speed
- Serverless suits spiky, light workloads
- Hybrid designs mix patterns for cost and freshness

TRY THIS

List three products you use daily. For each, guess which deployment pattern powers its main prediction and explain why.

NEXT UP

Section 3: preparing the model artifact so it is fit for production.

SECTION 3

03

Preparing the model

Turning a trained model into a reproducible, portable and documented production artifact.

IN THIS SECTION

- From notebook to package
- Serialisation formats
- Bundling preprocessing
- Dependencies and reproducibility
- Validation and model cards
- Artifact checklist

SECTION 3 · PREPARING THE MODEL

From notebook to production package

01 Refactor

Move logic out of cells into functions and modules with clear inputs and outputs.

02 Pipeline

Wrap preprocessing and model into one object that is trained and saved together.

03 Serialise

Save the artifact in a stable format with a unique version identifier.

04 Document

Record data, metrics, limits and intended use in a model card.

05 Register

Store artifact and metadata in a model registry for controlled release.

A useful test: could a colleague rebuild and load your model from the repository alone, without asking you a single question?

Common model serialisation formats

Format	Typical use	Strength	Caution
Pickle	Python objects, sklearn	Simple, universal in Python	Unsafe to load untrusted files
Joblib	Large sklearn models	Efficient with big arrays	Same security risk as pickle
ONNX	Cross framework export	Portable, fast runtimes	Not every operator converts
SavedModel	TensorFlow and Keras	Includes graph and signatures	TensorFlow ecosystem only
TorchScript or export	PyTorch production	Runs without Python source	Tracing can miss dynamic logic
Safetensors	Model weights, LLMs	Safe and fast to load	Stores tensors, not full pipelines
Native booster	XGBoost, LightGBM	Stable across versions	Framework specific



Never unpickle a file from an untrusted source. Loading a pickle can execute arbitrary code on your server.

SECTION 3 · PREPARING THE MODEL

ONNX as a portability layer

ONNX, the Open Neural Network Exchange, is an open format that lets a model trained in one framework run in another runtime or on different hardware.

Why use it

Decouple training framework from serving runtime and unlock optimised engines such as ONNX Runtime.

Where it shines

Neural networks, many classical models through converters, CPU inference and mixed hardware fleets.

Watch out

Validate that converted outputs match the original within a small tolerance on real samples.

Bundle preprocessing with the model

train_pipeline.py

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import GradientBoostingClassifier
import joblib

prep = ColumnTransformer([
    ("num", StandardScaler(), ["age", "tenure"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["plan"]),
])
model = Pipeline([("prep", prep), ("clf", GradientBoostingClassifier())])
model.fit(X_train, y_train)
joblib.dump(model, "churn_model_v3.joblib")
```

One object

Scaling and encoding travel with the model, so serving cannot apply them differently.

Unknown categories

Ignoring unseen plan values prevents crashes when a new product launches.

Versioned name

The file name carries a version, but the registry remains the source of truth.

SECTION 3 · PREPARING THE MODEL

Training serving skew

DEFINITION

Training serving skew is any difference between how data is prepared during training and how it is prepared at prediction time, causing the model to see inputs it never learned from.

EXAMPLE

Training computed average spend over 30 days using complete data, but the live API computes it over whatever days are available, so new customers look very different in production.

REMEMBER

Share one feature implementation between training and serving whenever possible.

Training serving skew: one customer, two answers

Training pipeline

Offline SQL over complete history



30 day window: 25 empty days + 5 days with spend

£150 ÷ 30 days **£5 / day**

Same customer: joined 5 days ago, spent £150

Live API

Python code over the days available



only the 5 days that exist

£150 ÷ 5 days **£30 / day**

A 6× difference. The model learned that £30 a day means a loyal big spender. This brand-new customer is misjudged, and nothing errors.

One definition

Compute features with one shared library or a feature store, used by both training and serving.

Log what was served

Store the exact features each prediction used and compare their distribution with training data.

Test both paths

For a sample of customers, compute features both ways in CI and assert they match.

SECTION 3 · PREPARING THE MODEL

Reproducibility essentials

1 Pin dependencies

Lock exact library versions in a lock file so the same code always installs the same packages.

2 Version the data

Record the dataset snapshot or query and its hash used for each training run.

3 Fix random seeds

Set seeds for libraries that use randomness to make runs comparable.

4 Track experiments

Log parameters, metrics and artifacts with a tool such as MLflow.

5 Capture environment

Record Python version, operating system and hardware alongside the run.

6 Store the code commit

Tag every model with the Git commit that produced it.

Input validation and schemas

Every production model needs an explicit contract describing the fields it accepts, their types, ranges and allowed values.

Define the schema

Write it once with a library such as Pydantic, Pandera or Great Expectations and store it with the model.

Reject early

Return a clear error for missing fields, wrong types or impossible values instead of a silent bad prediction.

Log violations

Count and log rejected inputs. Rising rejections are often the first sign of an upstream change.

What belongs in a model card

Overview	Purpose, owner, version, date and intended users.
Data	Training sources, time range, preprocessing and known gaps.
Performance	Metrics overall and across important groups and segments.
Limitations	Situations where the model should not be trusted or used.
Ethics	Fairness checks, sensitive features and potential harms.
Operations	Inputs, outputs, latency, monitoring and retraining plan.



Why bother

Model cards, proposed by researchers at Google in 2019, make limitations visible to the people deploying and relying on a model. They also support audits and regulation.

Packaging pitfalls

PITFALL	BETTER PRACTICE
⚠ Saving only the estimator and rewriting preprocessing in the API.	✓ Save the full pipeline so transforms are identical everywhere.
⚠ Using loose requirements without versions.	✓ Pin every dependency and rebuild from a lock file.
⚠ Loading model files from shared drives at runtime.	✓ Pull versioned artifacts from a registry with checksums.
⚠ Leaving the model as an undocumented binary.	✓ Ship a model card and schema next to every artifact.

SECTION 3 · PREPARING THE MODEL

Production artifact checklist

- [ ] Preprocessing and model saved as one pipeline
- [ ] Stable serialisation format chosen deliberately
- [ ] Unique version and Git commit recorded
- [ ] Training data snapshot referenced
- [ ] Dependencies pinned in a lock file
- [ ] Input and output schema defined
- [ ] Evaluation metrics stored with the artifact
- [ ] Model card written and reviewed
- [ ] Load test: artifact loads in a clean environment
- [ ] Registered in the model registry

Section recap

KEY TAKEAWAYS

- Refactor notebook code into reusable modules
- Choose serialisation formats with security and portability in mind
- Bundle preprocessing to prevent training serving skew
- Pin dependencies and version data for reproducibility
- Validate inputs against an explicit schema
- Document every model with a model card

TRY THIS

Take an existing notebook model and save it as a single pipeline with a schema and a one page model card.

NEXT UP

Section 4: exposing the model through a reliable API.

SECTION 4

04

Serving through APIs

Designing prediction endpoints that are fast, predictable, documented and safe to depend on.

IN THIS SECTION

- REST and gRPC
- Request and response design
- A FastAPI service
- Health checks and versioning
- Concurrency and batching
- Errors and documentation

REST versus gRPC for model serving

REST

JSON over HTTP

- Human readable and easy to debug
- Works in every language and browser
- Larger payloads and slower parsing
- Ideal for public and general APIs

GRPC

Protocol Buffers over HTTP/2

- Compact binary messages
- Strong typed contracts from schema files
- Supports streaming in both directions
- Ideal for high volume internal traffic

VERDICT Start with REST for simplicity and reach. Move hot internal paths to gRPC when payload size or latency becomes a measured problem.

Designing the request and response

A good prediction API is boring: explicit fields, clear types, stable versions and useful metadata in every response.

Request

Named features with types, an optional request identifier and no hidden defaults.

Response

Prediction, probability or score, model version and the request identifier.

Metadata

Timestamps and latency help debugging, audits and joining predictions to outcomes later.

A boring, excellent prediction API, annotated line by line

```
POST /v1/predict

{
  "request_id": "a1f3-9c2e",                    (1)
  "features": {
    "tenure_months": 14,                        (2)
    "plan": "premium",
    "monthly_spend_gbp": 42.50
  }
}
```

→

```
200 OK

{
  "request_id": "a1f3-9c2e",                    (3)
  "prediction": "churn",
  "probability": 0.73,                          (4)
  "model_version": "churn-2.4.1",               (5)
  "latency_ms": 18,                             (6)
  "timestamp": "2026-11-01T18:20:05Z"           (6)
}
```

1. **Request ID.** Created by the caller so the prediction can be joined to the real outcome weeks later.
2. **Named, typed features.** Units in the names and no hidden defaults: a missing field is an error, not a zero.
3. **Echoed ID.** Lets the caller's logs and yours line up during incidents.
4. **Probability, not just a label.** Callers can apply their own threshold without a new model.
5. **Model version.** Answers 'which model said this?' for debugging, audits and rollback.
6. **Latency and timestamp.** Feed latency objectives and time-based joins to outcomes.

SECTION 4 · SERVING APIS

A minimal FastAPI prediction service

app.py

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
import joblib

app = FastAPI(title="Churn API", version="3.0")
model = joblib.load("churn_model_v3.joblib")

class Customer(BaseModel):
    age: int = Field(ge=18, le=110)
    tenure: float = Field(ge=0)
    plan: str

@app.post("/v3/predict")
def predict(c: Customer):
    p = model.predict_proba([c.model_dump()])[0][1]
    return {"churn_probability": round(p, 4), "model_version": "3.0"}
```

Load once

The model loads at startup, not on every request, which saves seconds per call.

Validated input

Pydantic rejects ages outside a sensible range with a clear error message.

Versioned path

The version in the URL lets old clients keep working during upgrades.

Health, readiness and liveness

Orchestrators and load balancers need simple endpoints to know whether your service is alive and ready to receive traffic.

Liveness

Answers whether the process is running. If it fails repeatedly, the platform restarts the container.

Readiness

Answers whether the model is loaded and dependencies reachable. Traffic waits until it passes.

Startup

Gives large models extra time to load before liveness checks begin to count failures.

SECTION 4 · SERVING APIS

Versioning prediction APIs

1 Version in the path

Use paths such as v1 and v2 so clients choose explicitly when to upgrade.

2 Echo model version

Return the exact model version in every response for traceability.

3 Separate API and model

API contract versions change rarely. Model versions can change often behind them.

4 Deprecate politely

Announce end dates, log callers of old versions and give migration notes.

5 Never break silently

Changing a field meaning without a version bump is a breaking change.

6 Keep a changelog

Record what changed, why and which clients are affected.

SECTION 4 · SERVING APIS

Life of a prediction request

01 Receive
Gateway authenticates the caller and applies rate limits.

02 Validate
Schema checks reject malformed or impossible inputs.

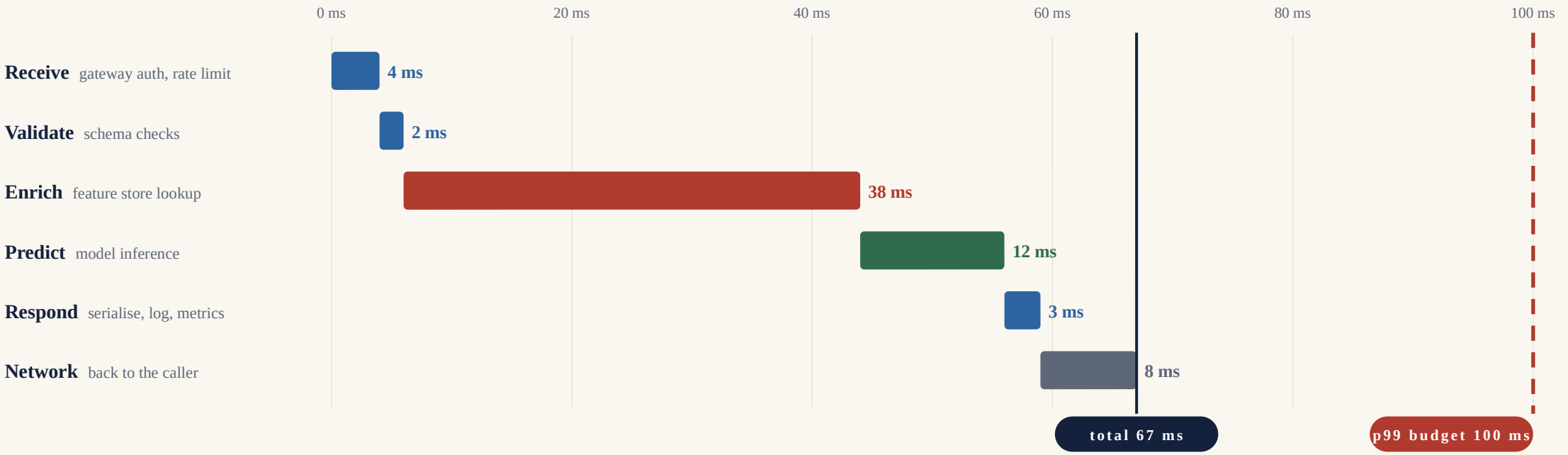
03 Enrich
Service fetches extra features from a feature store or cache.

04 Predict
Model runs, ideally within a strict time budget.

05 Respond
Return result, log inputs and outputs, emit metrics.

Measure the latency of each stage separately. Feature lookup is often slower than the model itself.

Where the milliseconds go inside one prediction request



57% of the time is the feature lookup

The model itself is only 18%.

WHAT IF FEATURES ARE CACHED?



Lookup 38 → 3 ms

Illustrative p95 timings

Concurrency, workers and async

Model inference is usually CPU or GPU bound, so concurrency settings matter more than clever code for throughput.

Workers

Run several worker processes so multiple requests use multiple CPU cores in parallel.

Async input and output

Use async handlers for network calls such as feature lookups, not for heavy model compute.

Thread pools

Offload blocking model calls to a thread or process pool so the event loop stays responsive.

SECTION 4 · SERVING APIS

Request batching

DEFINITION

Request batching groups several incoming requests into one model call, so hardware such as a GPU processes many inputs at once and throughput rises sharply.

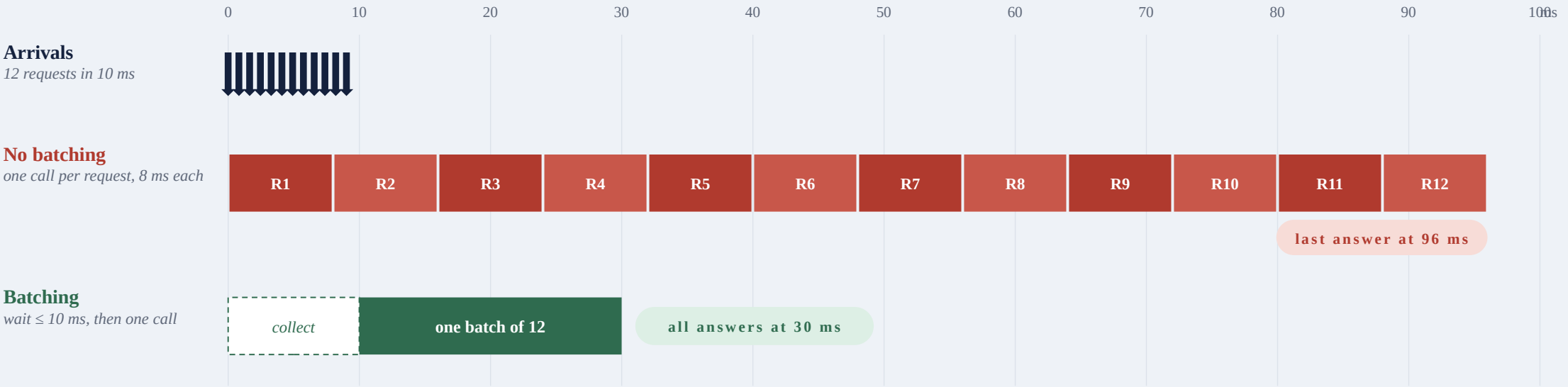
EXAMPLE

A server waits up to 10 milliseconds to collect as many as 32 image requests, runs them as one batch on the GPU, then returns each result to its caller.

REMEMBER

Batching trades a little latency per request for much higher total throughput.

Request batching on a timeline: twelve requests, two strategies



THROUGHPUT ON ONE GPU

Batch size	Time per call	Requests per second
1	8 ms	**125**
32	20 ms	**1,600**

Illustrative timings

TWO KNOBS TO TUNE

Max batch size · max wait

Raise them for throughput; cap the wait so p99 latency stays inside the budget.

Error handling that callers can act on

Status	Meaning	Model service example
400	Bad request	Field has the wrong type or is missing
401 or 403	Not authorised	Missing or invalid API key or token
422	Unprocessable input	Age of 250 fails the schema range check
429	Too many requests	Caller exceeded its rate limit
500	Server error	Unexpected exception inside the model code
503	Unavailable	Model still loading or overloaded
504	Timeout	Feature lookup exceeded its time budget



Return a short error code and message in the body. Never return stack traces or internal paths to callers.

SECTION 4 · SERVING APIS

API pitfalls

PITFALL	BETTER PRACTICE
⚠ Loading the model inside the request handler.	✓ Load once at startup and reuse it for every request.
⚠ Returning only a bare number.	✓ Return score, label, model version and request identifier.
⚠ No timeouts on feature lookups.	✓ Set strict timeouts and a fallback when dependencies are slow.
⚠ Undocumented endpoints.	✓ Publish OpenAPI documentation and example requests.

Section recap

KEY TAKEAWAYS

- Choose REST for reach and gRPC for heavy internal traffic
- Design explicit, versioned requests and responses
- Load models once and validate every input
- Expose liveness, readiness and startup checks
- Use workers and batching to raise throughput
- Return clear, safe error codes

TRY THIS

Wrap your packaged model in a FastAPI service with a schema, a health endpoint, a versioned path and three example requests.

NEXT UP

Section 5: packaging the service into a container.

SECTION 5

05

Containerisation

Packaging your model service so it runs the same way on a laptop, a test server and a production cluster.

IN THIS SECTION

- Why containers
- Images and containers
- Anatomy of a Dockerfile
- Multi stage and lean images
- GPU images
- Registries, tags and scanning

SECTION 5 · CONTAINERS

Why containers matter for ML

It works on my machine stops being an excuse when the machine itself is shipped.

Consistency

Code, libraries, system packages and the model run identically in every environment.

Isolation

Each service has its own dependencies, so library conflicts between models disappear.

Portability

The same image runs on Kubernetes, managed cloud services or a single virtual machine.

SECTION 5 · CONTAINERS

Images versus containers

IMAGE

The blueprint

- Read only template built from a Dockerfile
- Made of stacked, cached layers
- Stored and shared through a registry
- Identified by a tag and a digest

CONTAINER

The running instance

- Created from an image at runtime
- Has its own process, network and filesystem
- Disposable and replaceable at any time
- Many containers can share one image

VERDICT Build an image once, test it, then promote that exact image through every environment. Never rebuild for production.

Structure of a model service Dockerfile

1. **Base image.** Start from a small, official Python image with a fixed version tag.
2. **Working directory.** Set a clear folder inside the image where the application lives.
3. **Dependencies first.** Copy the lock file and install packages before copying code, to reuse cached layers.
4. **Application and model.** Copy source code and either the model file or a script that downloads it by version.
5. **Non root user.** Create and switch to an unprivileged user for safety.
6. **Start command.** Expose the port and launch the web server with a fixed number of workers.

Why 'dependencies first' makes rebuilds take seconds, not minutes

Dependencies first

after editing app.py

`CMD ["uvicorn", "src.app:app"]`	*rebuilt*
`USER app`	*rebuilt*
`COPY src/ ./src`	*rebuilt*
`RUN pip install -r requirements.txt`	*cached*
`COPY requirements.txt .`	*cached*
`WORKDIR /app`	*cached*
`FROM python:3.11-slim`	*cached*

Rebuild ≈ 10 seconds

Build order: bottom to top

Everything copied first

after editing app.py

`CMD ["uvicorn", "src.app:app"]`	*rebuilt*
`USER app`	*rebuilt*
`RUN pip install -r requirements.txt`	*rebuilt*
`COPY . .` ← code change breaks the cache here	*rebuilt*
`WORKDIR /app`	*cached*
`FROM python:3.11-slim`	*cached*

Rebuild ≈ 4 minutes

Rule: order layers from least to most often changed.

SECTION 5 · CONTAINERS

Multi stage builds

A multi stage build uses one stage with compilers and build tools, then copies only the finished results into a clean, small runtime stage.

Build stage

Installs compilers, builds wheels and prepares assets that need heavy tooling.

Runtime stage

Starts from a slim base and receives only the installed packages and app code.

Result

Smaller images download faster, start sooner and expose fewer components to attackers.

A multi-stage build ships only what runs

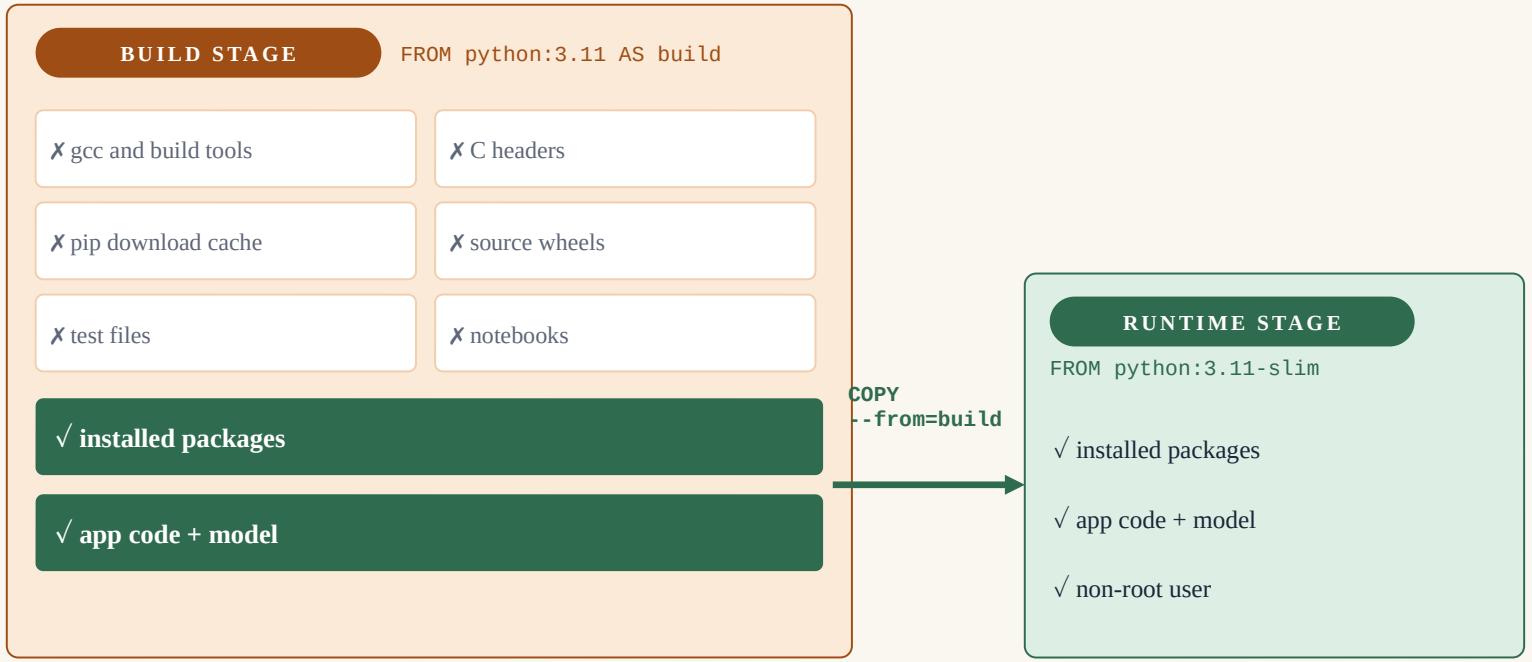


IMAGE SIZE AND PULL TIME

Single stage

2.1 GB · pull 21 s

Multi-stage

480 MB · pull 4.8 s

Pull time at 100 MB/s. Sizes illustrative.

```
FROM python:3.11 AS build
RUN pip install --prefix=/install -r requirements.txt
FROM python:3.11-slim    COPY --from=build /install /usr/local    COPY src/ /app/src
```

SECTION 5 · CONTAINERS

Keeping images lean

1 Slim bases

Prefer slim or distroless images over full operating system images.

2 Ignore files

Use a dockerignore file to exclude data, notebooks, caches and secrets.

3 No cache leftovers

Disable package manager caches during installation.

4 Combine steps

Group related install commands so temporary files do not persist in layers.

5 Only runtime libraries

Keep training and plotting libraries out of serving images.

6 Externalise big models

Pull large weights at startup from a registry or object storage when sensible.

GPU enabled images

GPU inference needs a compatible stack: the host driver, the container toolkit, and CUDA libraries inside the image that match your framework build.

Base choice

Start from vendor CUDA runtime images or framework images that already bundle matching libraries.

Version alignment

Framework, CUDA and driver versions must be compatible. Check support tables before upgrading.

Size awareness

GPU images are often several gigabytes, so cache them on nodes to avoid slow scaling.

Registries, tags and digests

Registry	Central store for images, such as ECR, Artifact Registry, ACR or Docker Hub.
Repository	One named collection of images for a service, for example churn api.
Tag	Human friendly label such as 3.0.1 or a Git commit identifier.
Digest	Content hash that uniquely and permanently identifies one exact image.

Golden rule

Deploy by digest or by an immutable tag. A tag like latest can point at different images over time, which makes incidents hard to reproduce.

Container security essentials

- Run as a non root user
- Use minimal, regularly updated base images
- Scan images for known vulnerabilities in CI
- Never bake secrets into image layers
- Inject secrets at runtime from a secret store
- Set read only filesystems where possible
- Limit CPU and memory for every container
- Sign images and verify signatures on deploy

SECTION 5 · CONTAINERS

Container pitfalls

PITFALL	BETTER PRACTICE
⚠ Copying the whole project folder, including datasets.	✓ Use a dockerignore file and copy only what serving needs.
⚠ Rebuilding images separately for each environment.	✓ Build once and promote the same digest everywhere.
⚠ Storing API keys in environment lines of the Dockerfile.	✓ Load secrets at runtime from a managed secret store.
⚠ Using the latest tag in production manifests.	✓ Pin immutable version tags or digests.

SECTION 5 · CONTAINERS

Section recap

KEY TAKEAWAYS

- Containers make model services consistent and portable
- Images are blueprints, containers are running instances
- Order Dockerfile steps to maximise cache reuse
- Multi stage builds produce small, safer images
- Align driver, CUDA and framework versions for GPUs
- Deploy immutable tags or digests and scan every image

TRY THIS

Containerise your FastAPI service, measure the image size, then apply three techniques from this section and measure again.

NEXT UP

Section 6: dedicated model serving frameworks.

SECTION 6

06

Serving frameworks

When a hand written API is not enough, specialised model servers bring batching, multi model hosting and speed.

IN THIS SECTION

- Why a model server
- Triton and framework servers
- KServe and Seldon
- BentoML and Ray Serve
- MLflow model serving
- A selection guide

Why use a dedicated model server?



Model servers solve the same problems every team eventually hits, so you do not have to reinvent them.

Built in batching

Dynamic batching groups requests automatically to use hardware efficiently.

Many models, one server

Load, unload and version several models with consistent endpoints.

Standard protocols

Common inference protocols make clients and tools interchangeable.

One model server, many models: from folder to endpoints

```
model_repository/
├─ fraud/
│  ├─ config.pbtxt
│  ├─ 1/model.onnx
│  └─ 2/model.onnx
├─ churn/
│  ├─ config.pbtxt
│  └─ 3/model.pt
└─ ranker/
   ├─ config.pbtxt
   └─ 1/model.plan
```

one folder per model, one number per version

MODEL SERVER

loads every model at start, watches for new versions

- Dynamic batching
- Several models share one GPU
- Version policy: serve latest 1 or N
- Health and metrics endpoints

! Add a model by adding a folder.

ENDPOINTS CLIENTS CALL

`POST /v2/models/fraud/infer`
latest fraud version

`POST /v2/models/fraud/versions/1/infer`
pin an older version

`POST /v2/models/churn/infer`
same shape, different model

`GET /v2/health/ready`
for the load balancer

`GET /metrics`
Prometheus scrape

Paths follow the open inference protocol (v2).

SECTION 6 · SERVING FRAMEWORKS

NVIDIA Triton Inference Server

Triton is an open source inference server that hosts models from many frameworks and is heavily optimised for GPUs as well as CPUs.

Multi framework

Serves TensorRT, ONNX, PyTorch, TensorFlow, Python and other backends side by side.

Performance

Dynamic batching, concurrent model execution and model ensembles out of the box.

Best for

High throughput GPU workloads, vision, speech and mixed model fleets.

Framework native servers

Some deep learning frameworks ship their own serving systems that understand their model formats natively.

TensorFlow Serving

Mature, high performance server for SavedModel files with versioned model directories.

TorchServe

PyTorch serving tool with handlers and management APIs. It moved to limited maintenance, so check its status first.

Trade off

Excellent for a single framework, less helpful when your team uses several.

SECTION 6 · SERVING FRAMEWORKS

KServe and Seldon Core on Kubernetes

These platforms add a model serving layer on top of Kubernetes, turning a short manifest into a scalable, observable prediction service.

KServe

Open source project offering an InferenceService resource, autoscaling and scale to zero, plus canary rollouts.

Seldon Core

Kubernetes serving with inference graphs, explainers and monitoring. Review its current licence before adopting.

Best for

Teams already running Kubernetes who want standardised serving for many models.

BentoML and Ray Serve

These Python first frameworks let you define services in code, then package and scale them with little infrastructure knowledge.

BentoML

Defines services in Python, builds deployable bundles and container images, with adaptive batching.

Ray Serve

Scales Python model code across a Ray cluster and composes multiple models into pipelines.

Best for

Python teams, complex multi model logic and fast iteration from prototype to production.

SECTION 6 · SERVING FRAMEWORKS

MLflow model serving

MLflow packages models in a standard format with a signature, and can serve them locally or build container images directly from the registry.

Model flavours

One format wraps sklearn, XGBoost, PyTorch and custom Python models consistently.

Registry link

Serve exactly the version promoted in the registry, keeping lineage intact.

Best for

Teams already tracking experiments in MLflow who want a short path to serving.

Serving framework comparison

Framework	Main strength	Best fit	Consider
Triton	GPU speed, many backends	High throughput inference	Configuration learning curve
TF Serving	Mature TensorFlow serving	TensorFlow only teams	Single framework
KServe	Kubernetes native, autoscaling	Platform teams on Kubernetes	Needs cluster expertise
Seldon Core	Inference graphs, explainers	Complex Kubernetes setups	Licence terms
BentoML	Python first packaging	Fast moving product teams	Less control of low level tuning
Ray Serve	Distributed composition	Multi model pipelines	Operating a Ray cluster
MLflow	Registry integrated	MLflow centred workflows	Fewer advanced serving features

Tool landscapes change quickly. Confirm project activity, licence and support before committing to any framework.

SECTION 6 · SERVING FRAMEWORKS

Choosing a serving framework

01 Workload

GPU heavy and high volume points towards Triton or similar engines.

02 Platform

Existing Kubernetes favours KServe. None favours managed services.

03 Team skills

Python only teams move fastest with BentoML or Ray Serve.

04 Ecosystem

MLflow users can start with its built in serving.

05 Prove it

Benchmark two finalists on your real model and traffic.

A plain FastAPI service remains a perfectly valid choice for a few light models. Adopt a framework when it removes real pain.

SECTION 6 · SERVING FRAMEWORKS

Framework selection pitfalls

PITFALL	BETTER PRACTICE
⚠ Choosing the most powerful tool for one small model.	✓ Match tool complexity to the size and number of models.
⚠ Ignoring licence and maintenance status.	✓ Review licences, release activity and support options first.
⚠ Trusting vendor benchmarks alone.	✓ Benchmark with your own model, payloads and hardware.
⚠ Locking business logic into framework specific code.	✓ Keep core prediction logic in plain, testable modules.

Section recap

KEY TAKEAWAYS

- Model servers provide batching, multi model hosting and standard protocols
- Triton excels at GPU throughput across frameworks
- KServe and Seldon bring serving to Kubernetes
- BentoML and Ray Serve suit Python first teams
- MLflow links registry and serving closely
- Always benchmark candidates on real workloads

TRY THIS

Serve the same model with FastAPI and with one framework from this section. Compare latency, throughput and effort.

NEXT UP

Section 7: orchestrating model services with Kubernetes.

SECTION 7

07

Kubernetes and orchestration

Running many model services reliably across a fleet of machines without babysitting each one.

IN THIS SECTION

- Why orchestration
- Core Kubernetes objects
- Resources and GPUs
- Autoscaling
- Configuration and secrets
- Packaging with Helm

SECTION 7 · ORCHESTRATION

Why orchestration?



One container is easy. Fifty containers across twenty machines, restarting and scaling on their own, needs an orchestrator.

Self healing

Failed containers restart automatically and move away from unhealthy machines.

Scaling

Replicas grow and shrink with traffic, and nodes can be added as demand rises.

Declarative

You describe the desired state and the system works continuously to match it.

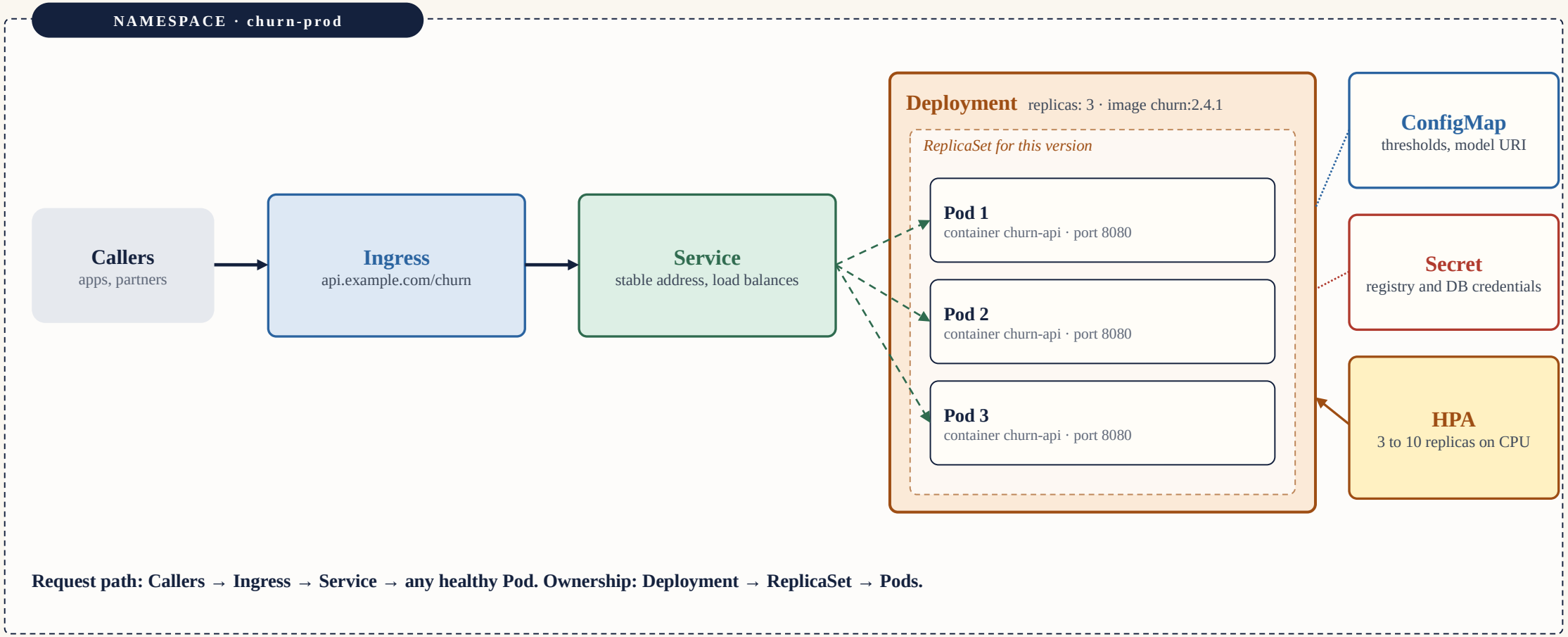
Core Kubernetes objects for model serving

Pod	Smallest unit. One or more containers sharing network and storage.
Deployment	Manages replicas of a Pod and rolls out new versions safely.
Service	Stable internal address that load balances across matching Pods.
Ingress or Gateway	Routes external HTTP traffic to Services by host and path.
ConfigMap and Secret	Inject configuration and credentials without rebuilding images.
Namespace	Groups and isolates resources for a team, model or environment.

Mental model

A Deployment keeps the right number of Pods alive. A Service gives them one stable address. An Ingress lets the outside world reach that address.

How the Kubernetes objects fit together for one model service



Request path: Callers → Ingress → Service → any healthy Pod. Ownership: Deployment → ReplicaSet → Pods.

A Deployment for a model service

churn_deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: churn
spec:
  replicas: 3
  template:
    spec:
      containers:
        name: api
        image: registry.example.com/churn:3.0.1
        resources:
          requests: {cpu: 500m, memory: 1Gi}
          limits: {memory: 2Gi}
        readinessProbe: {httpGet: {path: /ready, port: 8080}}
```

Simplified

Selectors, labels and the list marker before name are trimmed so the structure stays readable.

Pinned image

A precise version tag means every replica runs the same code and model.

Readiness

Traffic only arrives once the model is loaded and the ready check passes.

Requests and limits

Requests tell the scheduler how much CPU and memory a Pod needs. Limits cap what it may consume. Both are essential for stable model services.

Requests

Guarantee capacity and drive placement. Set them from measured usage under realistic load.

Memory limits

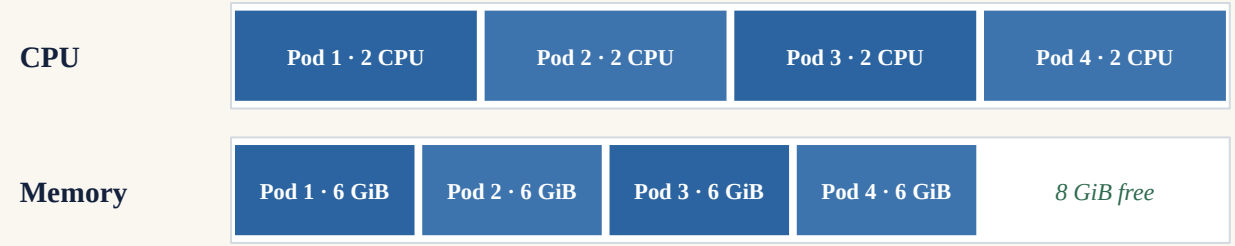
A container exceeding its memory limit is killed. Leave headroom for model loading peaks.

CPU limits

Exceeding CPU limits causes throttling and latency spikes, so many teams set requests only.

Requests decide where Pods land; limits decide how they fail

ONE NODE · 8 CPU · 32 GIB ALLOCATABLE



Pod 5 · Pending

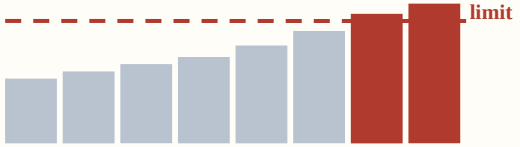
```
0/1 nodes are available: 1 Insufficient cpu.
```

Each Pod requests 2 CPU and 6 GiB. CPU runs out first, so memory sits idle. Right-size requests from measured usage.

CROSSING A LIMIT AT RUNTIME

Memory over limit

model + traffic peak



Container is killed: OOMKilled, exit code 137. Kubernetes restarts it and requests in flight fail. Leave headroom for model loading.

CPU over limit

inference burst



Container keeps running but is throttled. Latency spikes rather than errors, so many teams set CPU requests without limits.

! Set requests from measured p95 usage under realistic load; set the memory limit above the loading peak.

SECTION 7 · ORCHESTRATION

Scheduling GPUs

GPUs are exposed to Kubernetes through device plugins, then requested by Pods like any other resource. They are expensive, so placement deserves care.

Request GPUs

Pods ask for a whole number of GPUs. A device plugin makes them visible on each node.

Dedicated pools

Taints and tolerations keep ordinary workloads off costly GPU nodes.

Sharing

Techniques such as multi instance GPU or time slicing let small models share one card.

SECTION 7 · ORCHESTRATION

Scaling Pods versus scaling nodes

PODS

Horizontal Pod Autoscaler

- Adds or removes replicas of a Deployment
- Reacts to CPU, memory or custom metrics
- Fast, typically within a minute
- Limited by free capacity on nodes

NODES

Cluster or node autoscaler

- Adds or removes whole machines
- Reacts to Pods that cannot be scheduled
- Slower, often several minutes
- Tools include Cluster Autoscaler and Karpenter

VERDICT Use both together. Pods scale for quick bursts, nodes scale to supply capacity, and event based tools such as KEDA scale on queue length.

SECTION 7 · ORCHESTRATION

Choosing the right autoscaling signal

1 CPU utilisation

Simple default for CPU bound models with steady request sizes.

2 Requests per second

Better when request cost is predictable and traffic is spiky.

3 Queue length

Ideal for asynchronous and batch style workers reading from a queue.

4 Latency percentile

Scale when slow responses appear, with care to avoid oscillation.

5 GPU utilisation

Useful for GPU servers, exported through vendor monitoring tools.

6 Scheduled scaling

Pre scale before known peaks such as morning logins or match days.

The autoscaler's arithmetic, and a day of replicas

HPA FORMULA

desired = ⌈ current × observed ÷ target ⌉

Morning surge · target CPU 60%
4 replicas at 90%: 4 × 90 ÷ 60 = 6

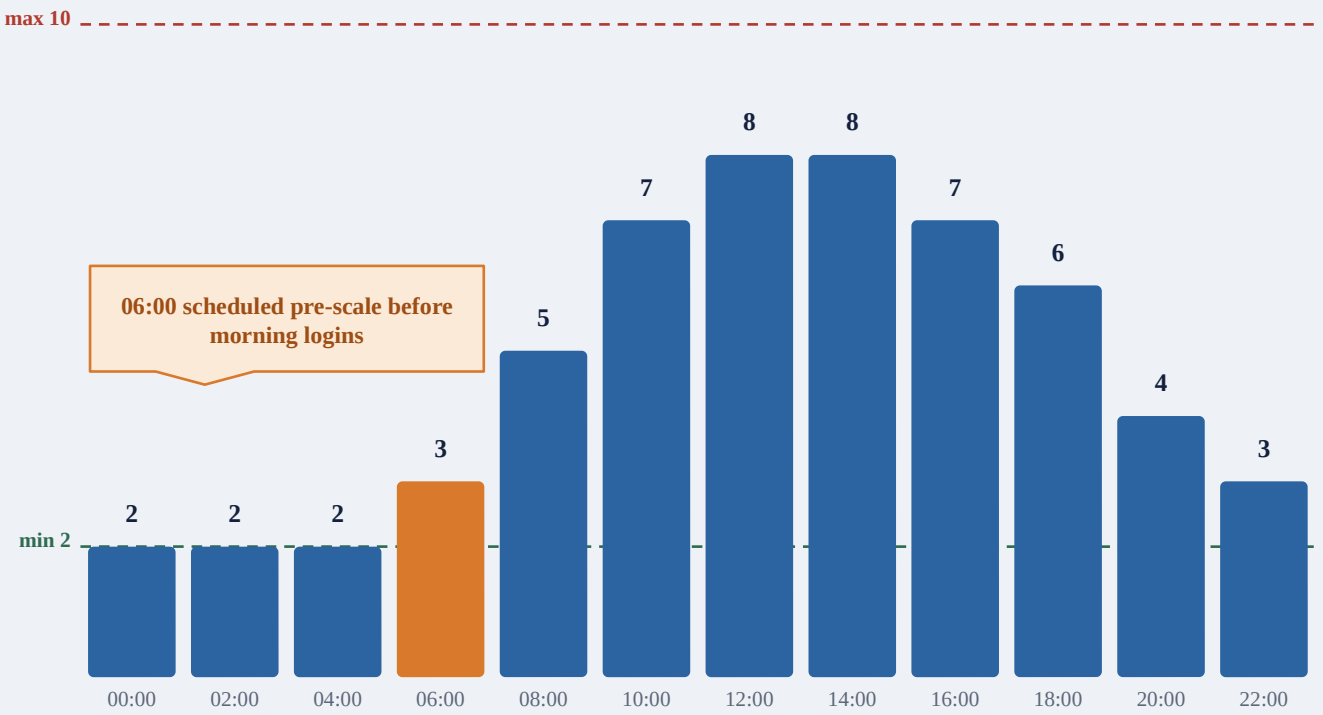
6 Pods

Evening lull
6 replicas at 45%: 6 × 45 ÷ 60 = 4.5

5 Pods

Always rounded up. Scale-down waits a 5 minute stabilisation window by default.

REPLICAS THROUGH ONE DAY · MIN 2, MAX 10



! Pick the signal that tracks user pain: CPU for steady requests, queue length for workers, requests per second for spiky APIs.

SECTION 7 · ORCHESTRATION

Configuration and secrets

Keep images identical across environments and inject everything that differs, such as model version, endpoints and credentials, at deploy time.

ConfigMaps

Hold non sensitive settings like thresholds, feature flags and the active model version.

Secrets

Hold credentials. Enable encryption at rest and restrict who can read them.

External stores

Sync from Vault or a cloud secret manager so rotation happens in one place.

SECTION 7 · ORCHESTRATION

Packaging with Helm

Helm bundles Kubernetes manifests into reusable, parameterised charts, so every model service can share one well tested template.

Charts

A chart is a folder of templates plus default values describing a full application.

Values

Each model or environment overrides a few values such as image tag and replica count.

Releases

Helm tracks installed versions and can roll back to a previous release quickly.

SECTION 7 · ORCHESTRATION

Kubernetes pitfalls for ML

PITFALL	BETTER PRACTICE
⚠ No resource requests, so Pods land on overloaded nodes.	→ ✓ Set requests from load tests and review them regularly.
⚠ Readiness that passes before the model is loaded.	→ ✓ Make readiness depend on a successful warm up prediction.
⚠ One giant namespace for everything.	→ ✓ Separate namespaces per team or environment with quotas.
⚠ Hand edited manifests drifting between clusters.	→ ✓ Template with Helm and deploy through GitOps.

Section recap

KEY TAKEAWAYS

- Orchestrators restart, place and scale containers for you
- Deployments, Services and Ingress form the core serving trio
- Requests and limits keep model services stable
- Scale Pods quickly and nodes for capacity
- Inject configuration and secrets at deploy time
- Template services with Helm charts

TRY THIS

Deploy your container to a local cluster such as kind or minikube with two replicas, a readiness probe and an autoscaler.

NEXT UP

Section 8: managed cloud platforms for machine learning.

SECTION 8

08

Cloud and managed platforms

Letting a provider run the undifferentiated heavy lifting so your team can focus on models and outcomes.

IN THIS SECTION

- The managed spectrum
- Amazon SageMaker
- Google Vertex AI
- Azure Machine Learning
- Serverless containers
- Managed versus self hosted

The spectrum of hosting choices

Fully managed endpoints	Upload a model, receive an endpoint. Minimal operations work.
Serverless containers	Bring a container, the platform scales it and handles networking.
Managed Kubernetes	Provider runs the control plane, you run workloads and nodes.
Virtual machines	You manage the operating system, runtime and scaling.
On premises hardware	Full control and full responsibility, including hardware and power.

Rule of thumb

Move down the stack only when you have a specific need the higher option cannot meet, such as cost at scale, special hardware or strict control.

Who runs what: the hosting spectrum as a responsibility map

	Fully managed endpoint	Serverless containers	Managed Kubernetes	Virtual machines	On premises
Model and inference code	You	You	You	You	You
Container and serving runtime	Provider	You	You	You	You
Scaling and load balancing	Provider	Provider	You	You	You
Operating system and patching	Provider	Provider	Shared	You	You
Cluster control plane	Provider	Provider	Provider	You	You
Hardware, power, network	Provider	Provider	Provider	Provider	You

Count the orange cells: each one needs an owner, an on-call rota and a patching plan.



Amazon SageMaker

SageMaker is the AWS platform for building, training and deploying models, with several inference options for different traffic shapes.

Real time endpoints

Persistent, autoscaling endpoints for low latency requests, including multi model hosting.

Serverless and async

Serverless inference for spiky traffic, asynchronous inference for large or slow requests.

Batch transform

Runs a model over datasets in storage without keeping an endpoint running.

Google Vertex AI

Vertex AI unifies the Google Cloud machine learning workflow, from data and training through a model registry to online and batch prediction.

Online prediction

Deploy registered models to autoscaling endpoints with traffic splitting between versions.

Batch prediction

Score data from BigQuery or Cloud Storage on demand or on a schedule.

Pipelines

Orchestrate training and deployment steps with managed pipeline runs.

Azure Machine Learning

Azure Machine Learning provides workspaces, registries and managed endpoints, integrating with the wider Microsoft data and security ecosystem.

Managed online endpoints

Host models with blue green deployments and traffic allocation across versions.

Batch endpoints

Run large scoring jobs over data in storage using compute clusters.

Governance

Ties into Azure identity, networking and policy controls for enterprises.

SECTION 8 · CLOUD PLATFORMS

Serverless containers and functions

Services such as Google Cloud Run, AWS Lambda, AWS App Runner and Azure Container Apps run containers or functions without managing servers.

Strengths

Scale to zero, pay per use, simple deployment and automatic HTTPS endpoints.

Limits

Cold starts, request time limits, memory ceilings and varying GPU availability.

Sweet spot

Light models, internal tools, prototypes and unpredictable traffic patterns.

SECTION 8 · CLOUD PLATFORMS

Managed versus self hosted

MANAGED

Provider operates the platform

- Fast start with little infrastructure skill
- Built in scaling, security and monitoring
- Higher unit cost at large steady volume
- Some lock in to provider interfaces

SELF HOSTED

Your team operates the platform

- Full control of hardware and software
- Potentially cheaper at large, steady scale
- Needs dedicated platform engineers
- You own uptime, patches and upgrades

VERDICT Default to managed until cost, control or compliance give a measured reason to self host. Revisit the decision yearly.

SECTION 8 · CLOUD PLATFORMS

Controlling cloud inference cost

1 Right size instances

Measure real utilisation and pick the smallest instance that meets latency targets.

2 Scale to demand

Use autoscaling and scale to zero for quiet endpoints.

3 Spot capacity

Run batch scoring on discounted spare capacity that may be interrupted.

4 Commit for baseline

Use savings plans or reservations for predictable steady load.

5 Share endpoints

Host several small models on one endpoint where supported.

6 Tag everything

Label resources by team and model so costs are attributable.

SECTION 8 · CLOUD PLATFORMS

Three ways to host one model, priced for a month

Always on, sized for peak

`3 replicas × 730 h × £1.50`

£3,285

per month

Simple, but pays for idle GPUs every night.

£0.33 per 1,000 predictions

Autoscaled 1 to 3

`avg 1.6 replicas × 730 h × £1.50`

£1,752

per month

47% cheaper; needs warm-up and good signals.

£0.18 per 1,000 predictions

Nightly batch on spot

`2 h × 30 nights × £0.60`

£36

per month

Only if predictions can be a day old.

£0.004 per 1,000 predictions

Assumes 10 million predictions a month, an illustrative £1.50 per GPU hour and a 60% spot discount.

LEVER ORDER Right-size first, then autoscale, then move anything that tolerates delay to batch on spot, then commit to a baseline.

Tag every endpoint by team and model, or none of these numbers can be measured.

Cloud pitfalls

PITFALL	BETTER PRACTICE
Leaving test endpoints running for months.	Set budgets, alerts and automatic clean up for idle endpoints.
Moving large datasets across regions repeatedly.	Keep compute close to data and watch egress charges.
Clicking resources together in the console.	Define infrastructure as code with Terraform or similar tools.
Ignoring provider quotas until launch day.	Request GPU and endpoint quota increases well in advance.

SECTION 8 · CLOUD PLATFORMS

Section recap

KEY TAKEAWAYS

- Hosting options range from fully managed to on premises
- SageMaker, Vertex AI and Azure ML cover online, batch and pipelines
- Serverless containers suit light and spiky workloads
- Default to managed, self host for measured reasons
- Cost control is a design activity, not an afterthought

TRY THIS

Price the same model on one managed endpoint and one serverless container for 10,000 and 1,000,000 requests a day using the provider calculators.

NEXT UP

Section 9: automating delivery with CI and CD for machine learning.

SECTION 9

09

CI and CD for machine learning

Automating the journey from a commit or new data to a safely released model.

IN THIS SECTION

- What changes in ML pipelines
- Pipeline stages
- Model registry
- Quality gates
- Infrastructure as code and GitOps
- Environments and approvals

What makes ML delivery different?

Software changes when code changes. ML systems change when code, data or the model change, so all three need pipelines.

Code

Serving code, feature logic and training scripts go through normal software CI.

Data

New data needs validation for schema, quality and distribution before training.

Model

Each new model needs evaluation, comparison and approval before release.

A complete ML delivery pipeline

01 — Commit

Code or config change triggers CI with linting and unit tests.

02 — Train

Pipeline validates data, trains and evaluates the candidate model.

03 — Register

Passing models are stored with metrics and lineage in the registry.

04 — Build

Service image is built, scanned and pushed with the model version.

05 — Release

Staged rollout to test, then production, with automated checks.

Separate continuous integration of code, continuous training of models and continuous delivery of services, but connect them through the registry.

SECTION 9 · CI AND CD

The model registry

A model registry is the single source of truth for which model versions exist, how they performed and which one each environment should serve.

Stores

Artifacts, metrics, parameters, data references, code commit and model card for every version.

Stages

Marks versions as candidate, staging, production or archived, often through aliases.

Examples

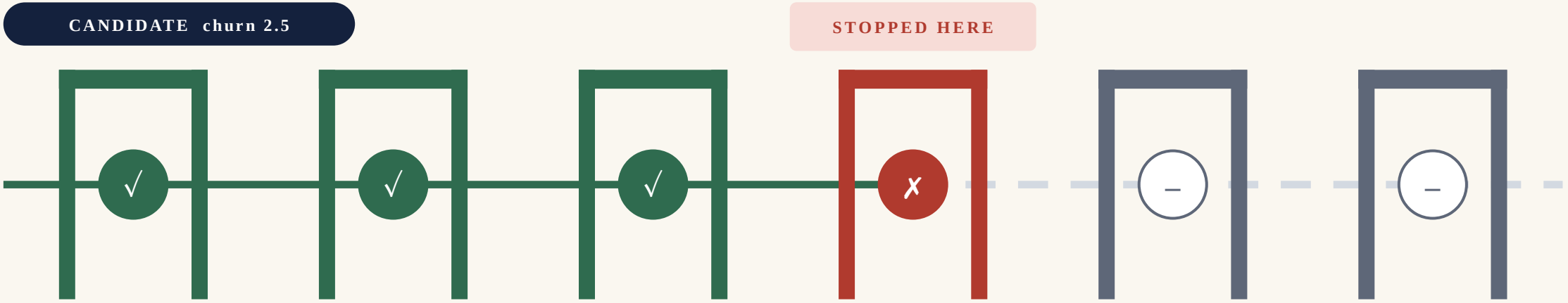
MLflow Model Registry, SageMaker Model Registry, Vertex AI Model Registry and Azure ML registries.

Quality gates before release

Gate	What it checks	Example rule
Unit tests	Feature and serving code correctness	All tests pass
Data validation	Schema, missing values, ranges	No new nulls in key fields
Model quality	Metric on a fixed holdout set	Not worse than production model
Fairness	Metric gaps across groups	Gap within agreed tolerance
Performance	Latency and memory under load	p95 latency under budget
Security	Image and dependency scans	No critical vulnerabilities

Gates must be automated and visible. A gate that a person can quietly skip is a suggestion, not a gate.

A candidate model meets the quality gates, and is stopped at one



WHAT HAPPENS NEXT

The pipeline fails, the owner gets the slice report, and nothing reaches the registry. Overriding needs a recorded approval, not a quiet skip.

NOTICE

Better overall AUC did not save it. Gates check what averages hide.

Infrastructure as code

1 Declare it

Describe clusters, endpoints, buckets and permissions in versioned files.

2 Review it

Infrastructure changes go through pull requests like application code.

3 Reproduce it

Recreate entire environments for testing or disaster recovery.

4 Tools

Terraform, OpenTofu, Pulumi, AWS CDK and cloud native templates.

5 Plan before apply

Preview changes and require approval for production.

6 Detect drift

Regularly compare real infrastructure against the declared state.

GitOps

DEFINITION

GitOps is a delivery approach where a Git repository holds the desired state of every environment, and an automated agent continuously makes the live system match it.

EXAMPLE

To release model 3.1, an engineer merges a change that updates the image tag in the production values file. Argo CD notices and rolls out the new version automatically.

REMEMBER

If it is not in Git, it is not deployed. Rollback is simply reverting a commit.

Environments and promotion

1. **Development.** Fast feedback for engineers with small samples and mocked dependencies.
2. **Integration.** Services talk to real dependencies and contract tests run automatically.
3. **Staging.** Production like data volumes, load tests and end to end checks.
4. **Production canary.** A small share of real traffic validates the release.
5. **Production.** Full rollout once canary metrics remain healthy.
6. **Promotion rule.** The same image and model version move forward unchanged at every stage.

SECTION 9 · CI AND CD

Human approvals where they matter

Automation handles the routine. Humans should approve the decisions where context, risk or accountability matter most.

When to require

High impact decisions, regulated domains, large metric changes or first releases of new models.

What to show

Metric comparison, fairness results, data changes and a link to the model card.

Record it

Log who approved, when and why, so audits can reconstruct every release.

CI and CD pitfalls

PITFALL	BETTER PRACTICE
Training models on a laptop and uploading files manually.	Train in a pipeline that records data, code and parameters.
Comparing a new model on a different test set each time.	Keep a versioned evaluation set and a fixed comparison procedure.
Releasing straight to all users.	Promote through staging and a canary with automatic checks.
Pipelines nobody can read.	Keep pipeline definitions in Git with clear stage names and owners.

Section recap

KEY TAKEAWAYS

- ML pipelines must handle code, data and model changes
- The registry connects training and delivery
- Automated gates protect quality, fairness and security
- Infrastructure as code and GitOps make releases reproducible
- Promote the same artifact through every environment
- Use human approval for high risk decisions

TRY THIS

Draw your ideal pipeline for one model. Mark every gate, the tool that enforces it and who is alerted when it fails.

NEXT UP

Section 10: testing models for production.

SECTION 10

10

Testing models for production

Building confidence in code, data and model behaviour before real users ever see a prediction.

IN THIS SECTION

- The ML testing pyramid
- Data tests
- Model behaviour tests
- Integration and contract tests
- Load and soak tests
- Testing in production safely

The ML testing pyramid

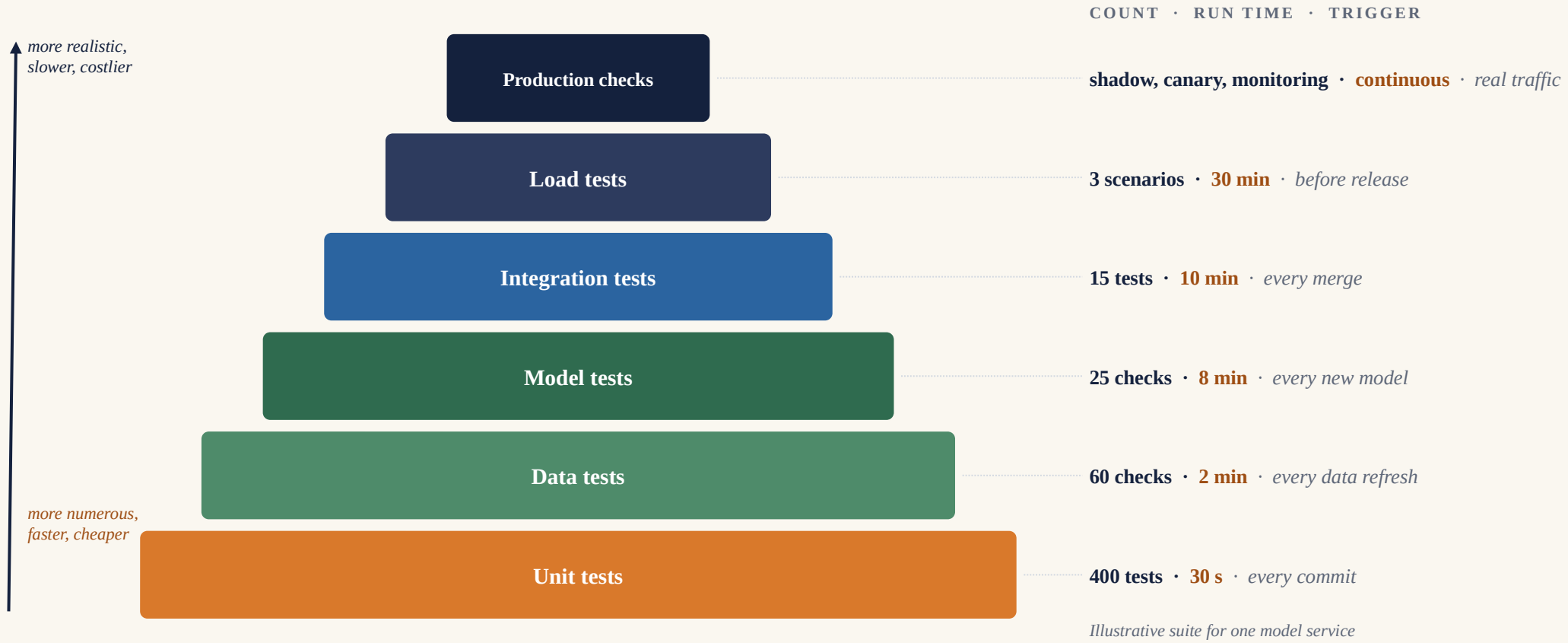
Unit tests	Feature functions, parsers and utilities tested in isolation.
Data tests	Schemas, ranges, nulls and distributions of training and serving data.
Model tests	Quality thresholds, behaviour checks and fairness on fixed datasets.
Integration tests	API, model and dependencies working together in a test environment.
Load tests	Latency, throughput and stability under realistic and peak traffic.
Production checks	Shadow, canary and monitoring validate behaviour with real data.



Balance

Many fast, cheap tests at the bottom, fewer slow and expensive tests at the top. Every layer catches problems the others miss.

The ML testing pyramid with counts, run times and triggers



! Push each check as low as it will go: a bug caught by a 30 second unit test never reaches a 30 minute load test.

Data tests worth automating

1 Schema
Expected columns and types are present, with no unexpected additions.

2 Completeness
Missing value rates stay below agreed limits for key fields.

3 Ranges
Values sit within plausible bounds, such as ages and prices.

4 Uniqueness
Identifiers that should be unique really are unique.

5 Freshness
Data is recent enough for the decision it supports.

6 Distribution
Feature distributions have not shifted sharply from the reference.

SECTION 10 · TESTING

Behavioural tests for models

Beyond a single accuracy number, behavioural tests check that the model reacts sensibly to deliberate, understandable changes in its inputs.

Invariance

Changing an irrelevant detail, such as a customer name, should not change the prediction.

Directional

Raising missed payments should never lower a credit risk score.

Minimum functionality

Simple, obvious cases must be predicted correctly every time.

SECTION 10 · TESTING

Behavioural tests, shown as real inputs and outputs

Test type	Input change		Risk score	Rule	Result
Invariance	Name: Sarah Jones	**Name: Mohammed Ali**	0.21 → 0.21	*must not change*	✓ PASS
Directional	Missed payments: 0	**Missed payments: 3**	0.21 → 0.48	*must not go down*	✓ PASS
Minimum functionality	Obvious high-risk case	**6 missed, debt 3× income**	0.91	*must be ≥ 0.80*	✓ PASS
Directional	Income: £30,000	**Income: £60,000**	0.30 → 0.36	*must not go up*	✗ FAIL

CAUGHT Higher income should never raise credit risk. The model had learned a quirk of the data, and CI blocked it before any customer saw it.

A behavioural test with pytest

test_model_behaviour.py

```
import joblib, pandas as pd

model = joblib.load("churn_model_v3.joblib")

def score(**fields):
    return model.predict_proba(pd.DataFrame([fields]))[0][1]

def test_longer_tenure_does_not_raise_churn():
    short = score(age=40, tenure=2, plan="basic")
    long = score(age=40, tenure=60, plan="basic")
    assert long <= short

def test_unknown_plan_is_handled():
    assert 0 <= score(age=40, tenure=10, plan="new") <= 1
```

Directional

Encodes domain knowledge: loyal customers should not look riskier.

Robustness

A plan never seen in training must still produce a valid probability.

Run in CI

These tests run on every candidate model before registration.

Integration and contract tests

Integration tests run the real service with its dependencies. Contract tests make sure the API still honours what every client expects.

Integration

Start the container, call the endpoint with sample payloads and check status codes and response shape.

Contract

Store example requests and responses agreed with consumers and test them on every build.

Golden records

Keep a small set of inputs with known expected outputs to detect silent changes.

Performance test types

Test	Purpose	Typical question answered
Load	Expected traffic levels	Do we meet latency targets at normal peak?
Stress	Beyond expected limits	Where and how does the service break?
Spike	Sudden bursts	Does autoscaling react fast enough?
Soak	Long duration	Do memory leaks or slowdowns appear over hours?
Cold start	Fresh instances	How long until a new replica serves traffic?

Tools such as Locust, k6 and JMeter generate traffic. Always test with realistic payload sizes and feature values.

SECTION 10 · TESTING

Testing in production, safely

Some behaviour only appears with real traffic. Safe techniques let you learn from production without exposing every user to risk.

Shadow traffic

Copy real requests to the new model and compare outputs without returning them to users.

Canary

Send a small share of traffic to the new version and watch error and quality signals.

Synthetic probes

Call endpoints continuously with known inputs to detect outages and drift in outputs.

Testing pitfalls

PITFALL	BETTER PRACTICE
⚠ Only checking overall accuracy.	✓ Add behavioural, slice and fairness tests to every evaluation.
⚠ Load testing with tiny dummy payloads.	✓ Use realistic payloads and feature distributions.
⚠ Tests that depend on random training runs.	✓ Fix seeds and test against stored artifacts.
⚠ No tests for the preprocessing code.	✓ Unit test every feature transformation and edge case.

SECTION 10 · TESTING

Knowledge check

? A new model improves overall AUC but performs much worse for customers in one region. Should it ship?

MODEL ANSWER

Not automatically. Slice tests have revealed a regression for an important group. The team should investigate the cause, discuss business and fairness impact with stakeholders, and either fix the model, add a fallback for that region or make an explicit, documented decision.

WHY IT MATTERS

Averages hide harm. Slice metrics protect both users and the business.

Section recap

KEY TAKEAWAYS

- Test code, data, model behaviour and the running service
- Automate data tests for schema, ranges and freshness
- Behavioural tests encode domain knowledge
- Contract tests protect API consumers
- Load, spike and soak tests reveal operational limits
- Shadow and canary release test safely in production

TRY THIS

Write three behavioural tests and one contract test for your model service and add them to your CI pipeline.

NEXT UP

Section 11: release strategies for getting new models live safely.

SECTION 11

11

Release strategies

How to move users from one model version to the next while keeping risk small and reversible.

IN THIS SECTION

- Recreate and rolling
- Blue green
- Canary releases
- Shadow deployment
- A/B tests and bandits
- Feature flags and rollback

Recreate versus rolling update

RECREATE

Stop everything, then start

- All old replicas stop before new ones start
- Simple and avoids mixed versions
- Causes downtime during the switch
- Acceptable for internal batch services

ROLLING

Replace replicas gradually

- New replicas start as old ones drain
- No downtime when readiness is correct
- Two versions serve traffic briefly
- Default strategy for Kubernetes Deployments

VERDICT Rolling updates suit most online services. Make sure both versions are compatible with the same API contract during the overlap.

Recreate, rolling and blue green, drawn on the same clock



Downtime: recreate only. Mixed versions: rolling. Double capacity: blue green.

Blue green deployment

Blue green runs two complete environments. Blue serves users while green receives the new version, then traffic switches in one step.

How it works

Deploy and test green fully, then point the router or load balancer from blue to green.

Strength

Instant cutover and instant rollback by switching back to blue.

Cost

Requires double capacity during the release window.

SECTION 11 · RELEASE STRATEGIES

Canary release in practice

01 — Deploy
Start the new version beside the old one with no traffic.

02 — One percent
Route a small slice of real requests to the canary.

03 — Observe
Compare errors, latency and prediction quality signals.

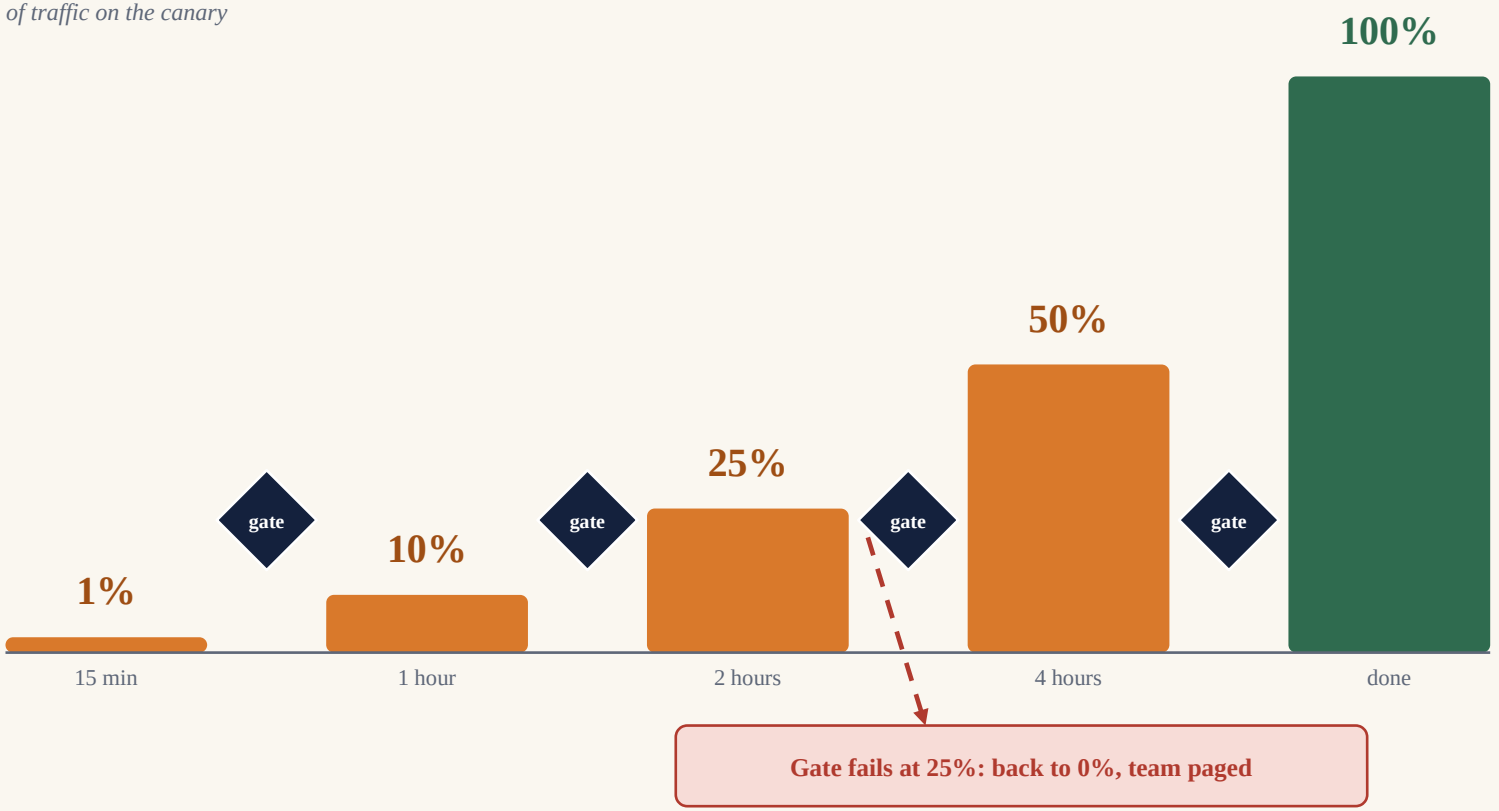
04 — Expand
Increase gradually, for example to 10, 25, 50 percent.

05 — Complete
Shift all traffic, or roll back automatically on bad signals.

Tools such as Argo Rollouts, Flagger, service meshes and managed endpoint traffic splitting automate these steps and the rollback decision.

A canary climbing its staircase, with an automatic way down

share of traffic on the canary



EACH GATE CHECKS

Error rate ≤ stable + 0.1 pp

p99 latency ≤ stable + 10%

Prediction mix close to stable

Business signal not worse

Example thresholds

! Judge model canaries on prediction quality too: a model can be fast and error-free while being wrong.

Shadow deployment

DEFINITION

In a shadow deployment the new model receives a copy of real production requests, but its predictions are only logged and compared, never returned to users.

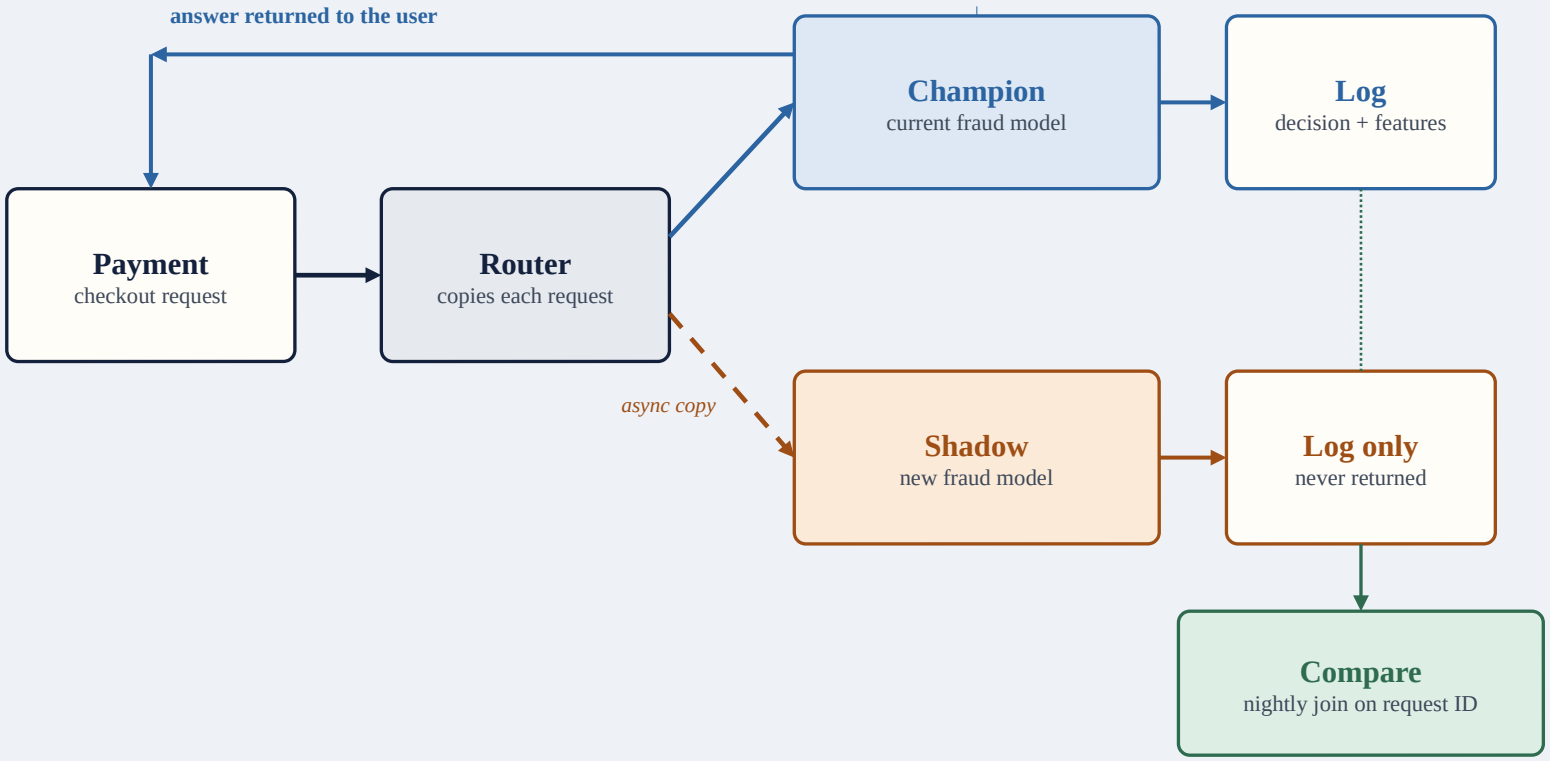
EXAMPLE

A new fraud model shadows the current one for two weeks. Analysts compare flagged transactions and confirm it catches more fraud without many extra false alarms before any customer is affected.

REMEMBER

Shadow tests behaviour on real data with zero user risk, but doubles inference cost.

Shadow deployment: real traffic, zero user risk



TWO-WEEK SHADOW REPORT

Payments compared	**1.2 million**
Same decision	**98.4%**
Flagged by champion	**1,980**
Flagged by shadow	**2,310**
Extra fraud confirmed	**+190 cases**
Shadow p99 latency	**72 ms (budget 100)**

Illustrative figures

! The shadow must never touch the user path: separate resources, async copies, strict timeouts.

A/B testing models

An A/B test randomly assigns users to the current or new model and measures the effect on a business outcome, not just model metrics.

Randomise users

Assign by user, not by request, so each person has a consistent experience.

Decide upfront

Fix the primary metric, guardrail metrics and test duration before starting.

Mind the stats

Small effects need large samples. Work with an analyst on sample size and significance.

Multi armed bandits

DEFINITION

A bandit approach shifts traffic automatically towards the model that is performing better while the test runs, balancing learning with earning.

EXAMPLE

An ecommerce site tests four recommendation models. Traffic gradually concentrates on the model with the highest click through rate, reducing the cost of showing weaker models.

REMEMBER

Bandits optimise quickly but make clean causal conclusions harder than A/B tests.

Feature flags for models

Feature flags separate deploying code from releasing behaviour, letting you switch models on or off for chosen users without redeploying.

Targeting

Enable a new model for staff, a region or a percentage of users first.

Kill switch

Turn a misbehaving model off in seconds and fall back to a safe default.

Hygiene

Remove old flags once a release is complete to avoid confusing logic.

A rollback plan you can trust

1. **Keep the previous version warm.** Retain the last good image and model so switching back is immediate.
2. **Define triggers.** Write down the metric thresholds that force a rollback without debate.
3. **Automate the switch.** Let the rollout tool reverse traffic automatically when triggers fire.
4. **Mind the data.** Ensure schemas and stored predictions remain compatible with the old version.
5. **Rehearse.** Practise rollback in staging so it works under pressure.
6. **Review.** After any rollback, hold a blameless review and record lessons.

Choosing a release strategy

Strategy	User risk	Extra cost	Best when
Recreate	High	None	Downtime is acceptable, internal tools
Rolling	Medium	Low	Standard updates with compatible APIs
Blue green	Low	High	Instant switch and rollback needed
Canary	Low	Low	Gradual validation with real traffic
Shadow	None	High	High stakes models need real data proof
A/B test	Medium	Medium	Business impact must be measured

Release pitfalls

PITFALL	BETTER PRACTICE
⚠ Judging a canary only on error rates.	✓ Also compare prediction distributions and business signals.
⚠ Canary windows too short to see real behaviour.	✓ Run long enough to include peak hours and varied users.
⚠ Stopping an A/B test as soon as it looks good.	✓ Fix duration and sample size before starting.
⚠ No tested path back to the old model.	✓ Keep the previous version deployable and rehearse rollback.

Section recap

KEY TAKEAWAYS

- Rolling updates avoid downtime for compatible changes
- Blue green gives instant switch and rollback at higher cost
- Canaries limit blast radius with gradual traffic
- Shadow deployments test on real data with no user exposure
- A/B tests and bandits measure business impact
- Rollback must be automatic, rehearsed and fast

TRY THIS

Design a canary plan for your model: traffic steps, duration, metrics, thresholds and who is notified.

NEXT UP

Section 12: making models faster and cheaper to serve.

SECTION 12

12

Performance optimisation

Making predictions faster and cheaper without sacrificing the quality users depend on.

IN THIS SECTION

- Latency and throughput
- Profiling first
- Quantisation, pruning, distillation
- Compilers and runtimes
- Hardware choices
- Caching and cold starts

SECTION 12 · PERFORMANCE

Latency versus throughput

LATENCY

Time for one request

- How long a single user waits
- Reported as percentiles such as p50, p95, p99
- Critical for interactive products
- Batching usually makes it slightly worse

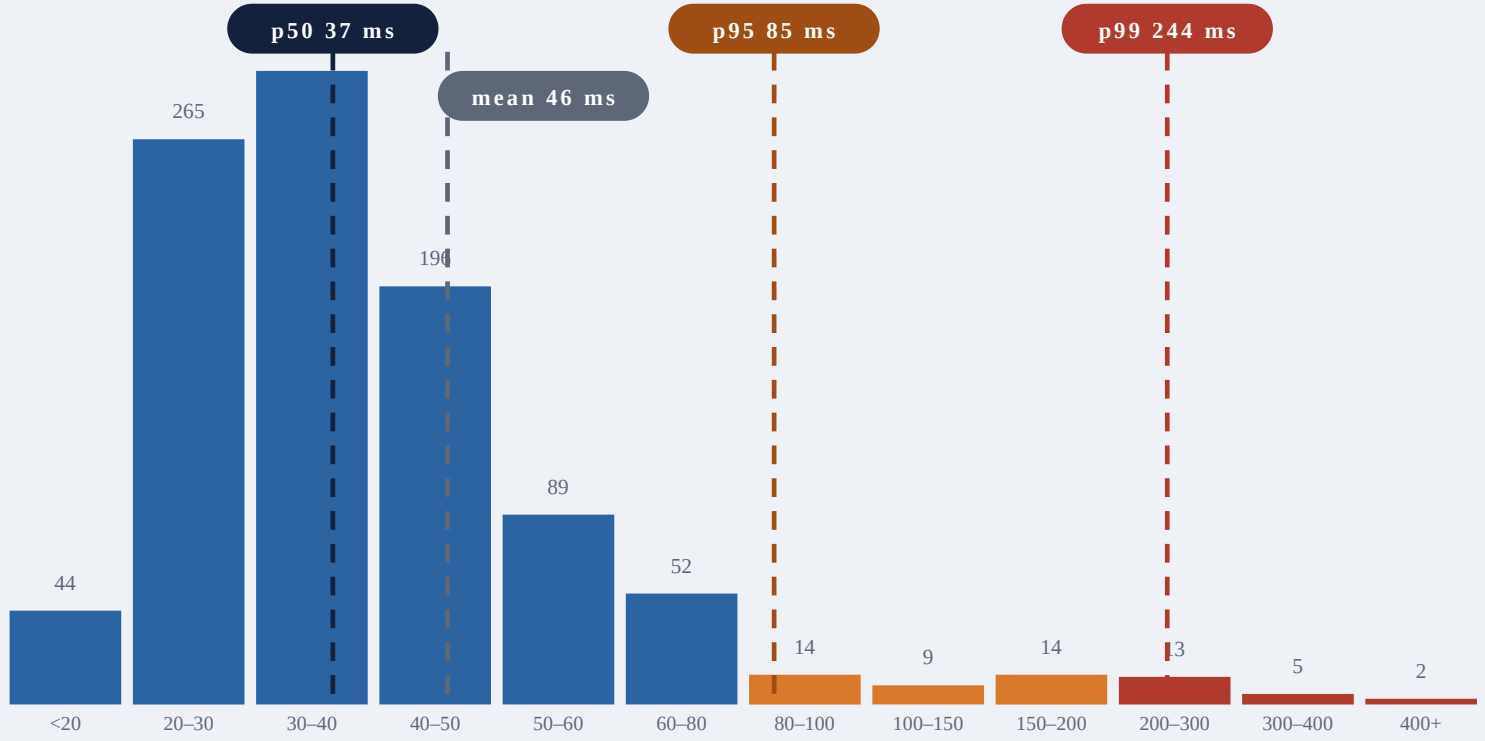
THROUGHPUT

Requests completed per second

- How much work the system handles
- Drives infrastructure cost per prediction
- Critical for batch and high volume APIs
- Batching usually improves it greatly

VERDICT Agree a latency budget at a high percentile first, then maximise throughput within it. Averages hide the slow requests users remember.

Why averages hide the slow requests users remember



Simulated latencies

THE TAIL

6.6×

p99 versus the median

FAN-OUT MAKES IT WORSE

$1 - 0.99^{20} \approx 18\%$

A page that makes 20 model calls hits at least one p99-slow call in nearly one view in five.

! Set budgets at p95 or p99, and plot the histogram before trusting the mean.

SECTION 12 · PERFORMANCE

Profile before you optimise



Guessing where time goes is usually wrong. Measure every stage of a request before changing anything.

Break it down

Time network, validation, feature lookup, preprocessing, model and serialisation separately.

Use tools

Python profilers, framework profilers and tracing systems show where time is spent.

Fix the biggest

Optimise the slowest stage first. Often it is data access, not the model.

SECTION 12 · PERFORMANCE

Quantisation

DEFINITION

Quantisation stores and computes model weights with fewer bits, for example 8 bit integers instead of 32 bit floats, reducing memory and often speeding up inference.

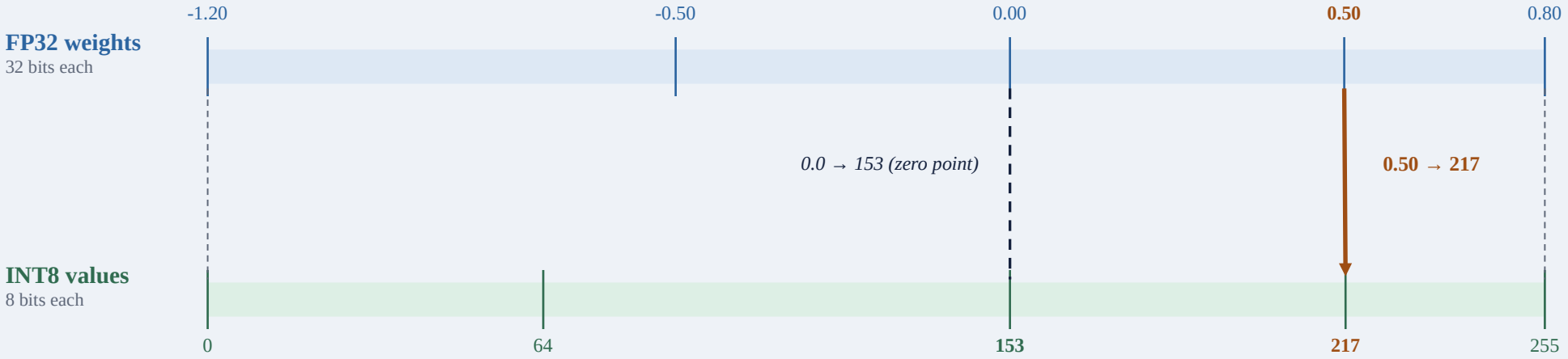
EXAMPLE

A text classifier quantised to 8 bit integers becomes about four times smaller and runs noticeably faster on CPU, with only a small, measured drop in accuracy.

REMEMBER

Always compare quality before and after. Calibrate with representative data.

Quantisation, worked out for a single weight



Scale
`(0.80 − (−1.20)) ÷ 255`

0.00784

Zero point
`round(1.20 ÷ 0.00784)`

153

Quantise 0.50
`round(0.50 ÷ 0.00784) + 153`

217

Read back
`(217 − 153) × 0.00784`

0.502 *error 0.002*

100M parameters: **400 MB** at 4 bytes → **100 MB** at 1 byte. Then re-measure accuracy on the test set.

SECTION 12 · PERFORMANCE

Model compression techniques

1 Quantisation

Fewer bits per weight and activation. Dynamic, static and quantisation aware training options exist.

2 Pruning

Removes weights or whole structures that contribute little to predictions.

3 Distillation

Trains a small student model to imitate a large teacher model.

4 Low rank methods

Approximate large weight matrices with smaller factors.

5 Smaller architectures

Choose efficient model families designed for speed from the start.

6 Feature reduction

Fewer, cheaper features speed both lookup and prediction.

Compilers and optimised runtimes

Inference engines rewrite the model graph for specific hardware, fusing operations and choosing fast kernels automatically.

ONNX Runtime

Cross platform engine with execution providers for CPUs, GPUs and accelerators.

TensorRT

NVIDIA engine that optimises models aggressively for its GPUs.

Others

OpenVINO for Intel hardware, Apache TVM and PyTorch compilation features.

Choosing inference hardware

Hardware	Strengths	Best for	Watch
CPU	Cheap, available, simple	Tree models, small networks	Slow for large deep models
GPU	Massive parallel compute	Vision, speech, LLMs, batching	Cost and idle time
Cloud accelerators	Custom silicon value	Large scale, specific frameworks	Portability limits
Edge NPUs	Low power on device	Mobile and embedded models	Operator support

Many tabular models run perfectly well on CPUs. Reach for GPUs when the model and volume justify the cost.

Caching predictions and features

If the same input appears often, the fastest prediction is the one you do not recompute. Caches also protect slow downstream systems.

Prediction cache

Store results keyed by a hash of the input with a sensible expiry time.

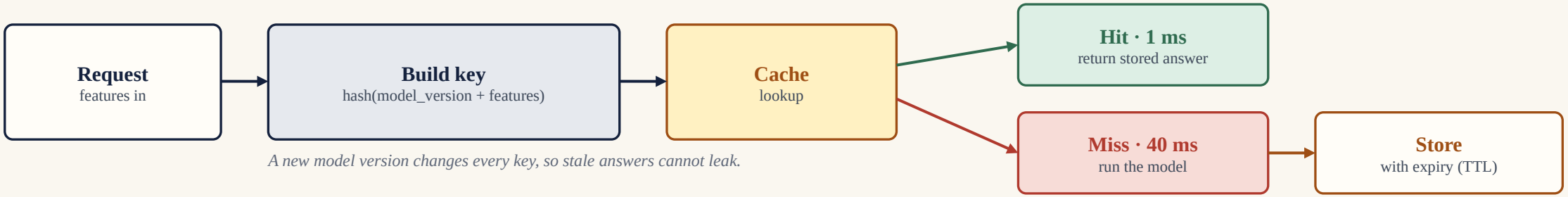
Feature cache

Keep frequently used features in memory or a fast key value store.

Invalidation

Clear caches when the model version changes so stale answers never leak.

What a prediction cache buys you, in milliseconds and calls



A new model version changes every key, so stale answers cannot leak.

AVERAGE LATENCY

$$h \times 1\ \text{ms} + (1 - h) \times 40\ \text{ms}$$

Fewer model calls also protect slow downstream systems such as the feature store.

Hit rate h	Average latency	Model calls per 1,000
0%	40 ms	1,000
50%	20.5 ms	500
80%	8.8 ms	200
95%	2.95 ms	50

! Caching only helps when inputs repeat. Measure the real hit rate before building one, and set an expiry that matches how fast features change.

SECTION 12 · PERFORMANCE

Cold starts and warm up

A new replica must load the model, initialise libraries and sometimes compile kernels before it can answer quickly.

Warm up requests

Run a few dummy predictions during startup before reporting ready.

Keep minimum replicas

Avoid scaling all the way to zero for latency sensitive services.

Faster loading

Use smaller artifacts, memory mapped weights and images cached on nodes.

SECTION 12 · PERFORMANCE

A performance tuning workflow

1. **Set budgets.** Define latency percentile targets and cost per thousand predictions.
2. **Baseline.** Load test the current service and record results.
3. **Profile.** Find the slowest stages under realistic traffic.
4. **Change one thing.** Apply one optimisation at a time to understand its effect.
5. **Check quality.** Confirm accuracy and slice metrics remain acceptable.
6. **Keep or revert.** Adopt only changes that improve the target without harm.

SECTION 12 · PERFORMANCE

Performance pitfalls

PITFALL	BETTER PRACTICE
Optimising the model while a slow database dominates latency.	Profile end to end and fix the largest bottleneck first.
Reporting only average latency.	Track p95 and p99 latency against an agreed budget.
Quantising without measuring accuracy loss.	Evaluate compressed models on the full test suite.
Benchmarking on a powerful laptop.	Benchmark on the same hardware used in production.

Section recap

KEY TAKEAWAYS

- Set latency budgets at high percentiles
- Profile every stage before optimising
- Quantisation, pruning and distillation shrink models
- Compilers and runtimes accelerate inference
- Choose hardware by model type and volume
- Cache wisely and warm up new replicas

TRY THIS

Export your model to ONNX, run it with ONNX Runtime, and compare latency and outputs with the original.

NEXT UP

Section 13: monitoring and observability.

SECTION 13

13

Monitoring and observability

Knowing, at any moment, whether your model is healthy, accurate and still delivering value.

IN THIS SECTION

- Four layers of monitoring
- Logs, metrics and traces
- Data and concept drift
- Delayed ground truth
- Alerts and objectives
- Tools and dashboards

“

Models fail silently. The service stays up while the predictions quietly become wrong.

☆ Traditional software monitoring catches crashes. ML monitoring must also catch a healthy looking service that has stopped being useful.

Four layers of ML monitoring

Infrastructure	CPU, memory, GPU, disk, network and container restarts.
Service	Request rate, error rate, latency percentiles and saturation.
Data	Input schema violations, missing values and feature distributions.
Model	Prediction distribution, confidence, and accuracy once labels arrive.
Business	Conversions, losses avoided, complaints and other outcome metrics.

Cover all four

Each layer answers a different question. Teams that watch only infrastructure discover model problems from angry customers instead of dashboards.

SECTION 13 · MONITORING

Logs, metrics and traces

1 Metrics

Numeric time series such as latency and error counts, cheap to store and ideal for alerts.

2 Logs

Detailed event records such as each request, prediction and error with context.

3 Traces

Follow one request across services to see where time is spent.

4 Prediction logs

Inputs, outputs, model version and request identifier, kept for analysis.

5 Standards

OpenTelemetry offers a vendor neutral way to emit all three signals.

6 Privacy

Mask or exclude personal data before logging, and set retention limits.

Data drift

DEFINITION

Data drift, also called covariate shift, happens when the distribution of input features in production moves away from the distribution seen during training.

EXAMPLE

A loan model trained before a recession now sees many more applicants with irregular income. The inputs look different, even though the rules linking them to default may be unchanged.

REMEMBER

Drift in inputs is an early warning, not proof that accuracy has fallen.

Concept drift

DEFINITION

Concept drift happens when the relationship between inputs and the correct answer changes, so patterns the model learned are no longer true.

EXAMPLE

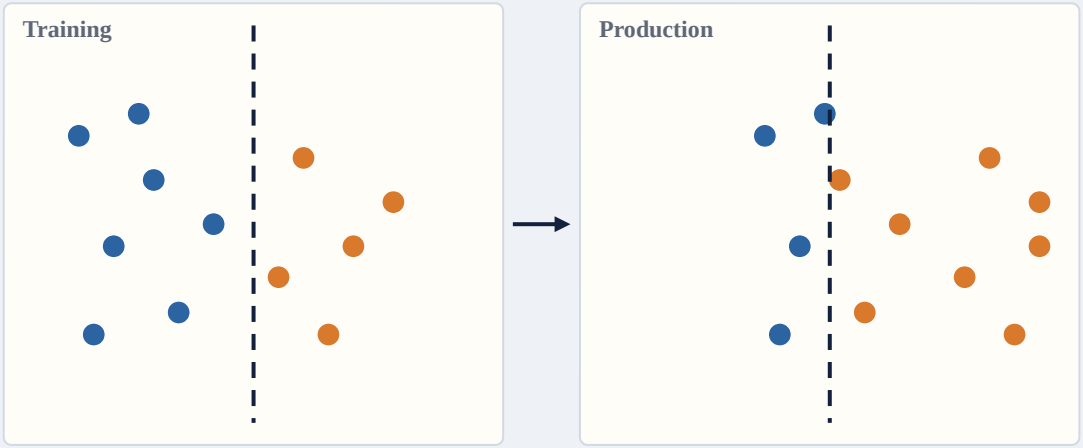
Fraudsters change tactics after a new security feature launches. Transactions look similar to before, but which ones are fraudulent has changed.

REMEMBER

Concept drift usually needs fresh labels to detect and new training data to fix.

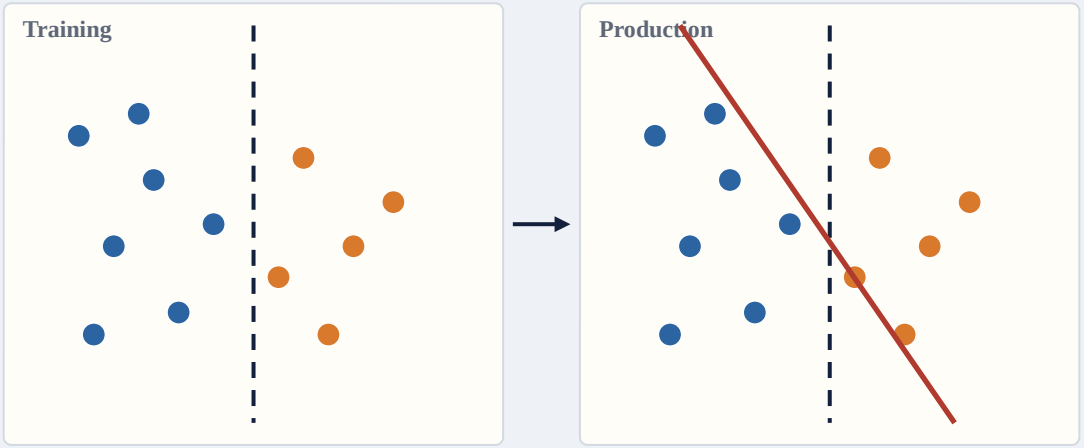
Data drift moves the points; concept drift moves the line

DATA DRIFT · COVARIATE SHIFT



Inputs move right; the rule is unchanged.

CONCEPT DRIFT



Same inputs; the true boundary (red) has rotated.

	Data drift	Concept drift
What changes	Distribution of inputs	Link between inputs and the answer
Detect without labels?	Yes: PSI, KS test, feature stats	No: needs labels or proxies
Typical response	Investigate, maybe retrain	Retrain on recent labelled data

Common drift detection approaches

Approach	Idea in plain words	Good for
Population stability index	Compares how values spread across bins	Scorecards, tabular features
Kolmogorov Smirnov test	Checks if two numeric samples differ	Continuous features
Chi squared test	Compares category frequencies	Categorical features
Distance measures	Scores how far apart distributions are	Many feature types
Domain classifier	Trains a model to tell old data from new	High dimensional data
Embedding drift	Tracks shifts in embedding space	Text, image and LLM inputs

Statistical tests become oversensitive with huge samples. Combine them with practical thresholds and human judgement.

SECTION 13 · MONITORING

When ground truth arrives late

Many outcomes are known only weeks or months later, such as loan defaults or customer churn, so accuracy cannot be measured in real time.

Proxy signals

Watch prediction distributions, confidence and early business indicators meanwhile.

Join later

Store request identifiers so outcomes can be matched to predictions when they arrive.

Label sampling

Have experts label a small random sample quickly to estimate current accuracy.

Service level objectives

A service level objective is a measurable target for reliability that the team commits to, such as 99.9 percent of requests answered under 200 milliseconds.

Indicators

Choose the signals users feel: availability, latency and freshness of predictions.

Error budget

The allowed amount of failure. Spending it fast pauses risky releases.

Model objectives

Add targets for quality, such as weekly precision staying above an agreed level.

An error budget, worked out and burned down

OBJECTIVE

99.9%

of requests succeed over 30 days

BUDGET IN TIME

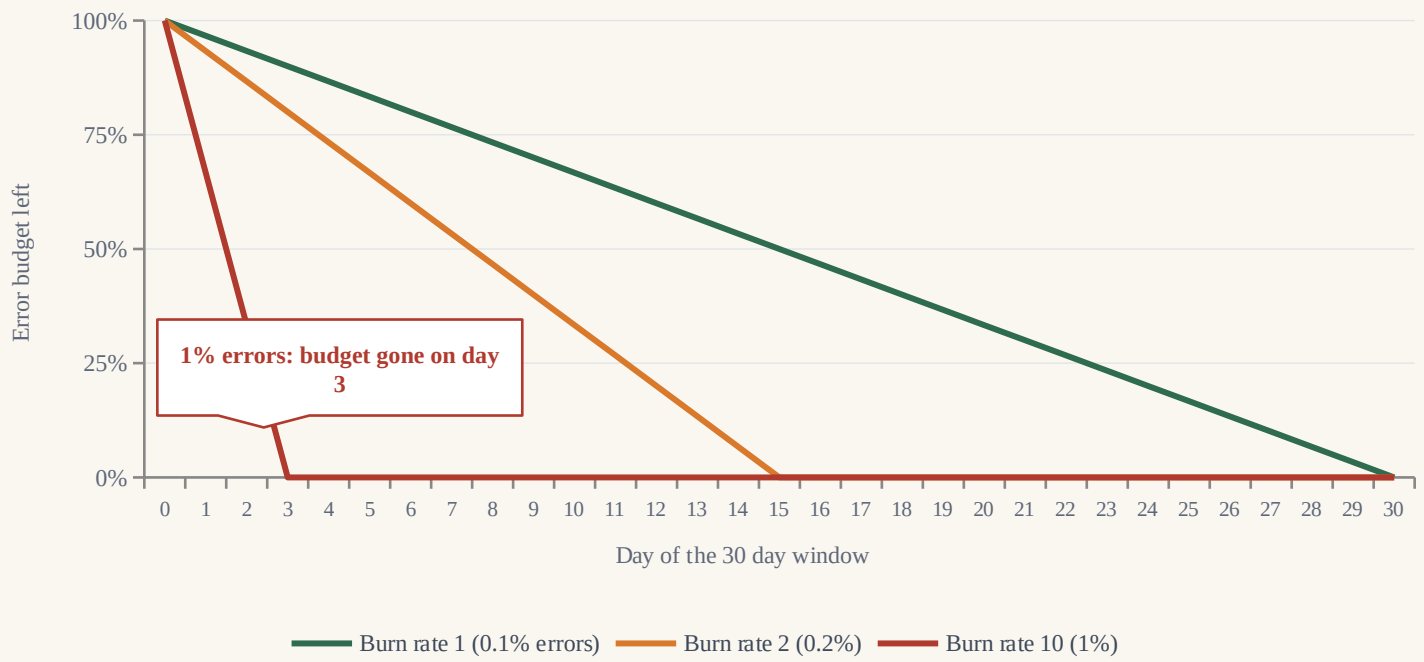
43.2 min

30 × 24 × 60 × 0.001

BUDGET IN REQUESTS

10,000

failures allowed out of 10 million



More than 50% left

Ship features and experiments freely.

Below 25% left

Only fixes and low-risk changes.

Exhausted

Freeze risky releases; reliability work first.

SECTION 13 · MONITORING

Alerting without alarm fatigue

1 **Alert on symptoms**

Page people for user impact such as high errors, not every internal blip.

2 **Severity levels**

Distinguish urgent pages from tickets that can wait until morning.

3 **Runbooks**

Every alert links to clear steps for diagnosis and response.

4 **Owners**

Route each alert to a named team with on call coverage.

5 **Tune thresholds**

Review noisy alerts regularly and remove or adjust them.

6 **Test alerts**

Trigger alerts deliberately to prove they reach the right people.

Monitoring tools landscape

Category	Examples
Metrics and dashboards	Prometheus, Grafana, cloud native monitoring services
Logs and traces	OpenTelemetry, Elasticsearch, Loki, Jaeger, Datadog
ML monitoring	Evidently, NannyML, Arize, Fiddler, cloud model monitors
Data quality	Great Expectations, Soda, Pandera, dbt tests
Alerting	Alertmanager, PagerDuty, Opsgenie, Slack integrations

Tools change often. Prioritise open standards so you can switch vendors without rewriting instrumentation.

A useful model dashboard

Health summary	Availability, error rate and p95 latency against objectives.
Traffic	Requests per minute by client, version and region.
Inputs	Drift scores and missing value rates for the top features.
Outputs	Prediction distribution and average confidence over time.
Quality	Accuracy or precision as labels arrive, with slice breakdowns.
Business	The outcome metric the model was built to improve.

Design tip

Put the answer to is it working at the top. Details live lower down for investigation. Always show the model version beside every chart.

SECTION 13 · MONITORING

Monitoring pitfalls

PITFALL	BETTER PRACTICE
⚠ Monitoring only CPU and uptime.	✓ Add data, prediction and business layers.
⚠ Alerting on every statistically significant drift.	✓ Combine drift with impact thresholds and review weekly.
⚠ Logging raw personal data for debugging.	✓ Mask sensitive fields and enforce retention rules.
⚠ Dashboards nobody owns or reads.	✓ Assign owners and review dashboards in regular meetings.

Section recap

KEY TAKEAWAYS

- Models can fail silently while services look healthy
- Monitor infrastructure, service, data, model and business layers
- Data drift warns early, concept drift needs labels
- Plan for delayed ground truth with proxies and joins
- Use objectives and error budgets to guide releases
- Alert on symptoms with runbooks and owners

TRY THIS

Sketch a monitoring dashboard for your model with one chart per layer and define two alerts with thresholds and owners.

NEXT UP

Section 14: retraining and managing the model lifecycle.

SECTION 14

14

Retraining and the model lifecycle

Keeping models useful as the world changes, and retiring them gracefully when it is time.

IN THIS SECTION

- Retraining triggers
- Continuous training pipelines
- Feedback loops
- Champion and challenger
- Data and model lineage
- Deprecation and retirement

SECTION 14 · LIFECYCLE

When should a model be retrained?

1 On a schedule

Weekly or monthly retraining suits steadily changing domains.

2 On drift

Retrain when monitored drift passes an agreed threshold.

3 On performance drop

Retrain when measured accuracy falls below target.

4 On new data

Retrain when enough fresh labelled data has accumulated.

5 On business change

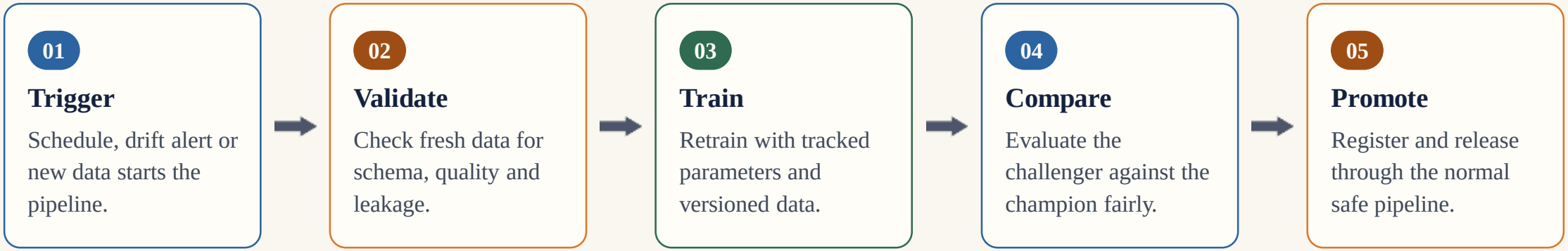
New products, regions or policies often demand retraining.

6 On request

Allow controlled manual triggers for investigations and fixes.

SECTION 14 · LIFECYCLE

A continuous training pipeline



Continuous training should never bypass the release gates. A retrained model is still a new model that can fail in new ways.

SECTION 14 · LIFECYCLE

Champion and challenger

DEFINITION

The champion is the model currently serving production. A challenger is a candidate that must prove itself better, on agreed criteria, before replacing the champion.

EXAMPLE

Each month a retrained churn model challenges the champion on the same holdout set and in a shadow test. It is promoted only if precision improves and no customer segment gets worse.

REMEMBER

A newer model is not automatically a better model.

SECTION 14 · LIFECYCLE

Feedback loops and their dangers

When predictions influence the data later used for training, the model can reinforce its own mistakes or biases over time.

Example

A model approves only some applicants, so outcomes are learned only for those approved.

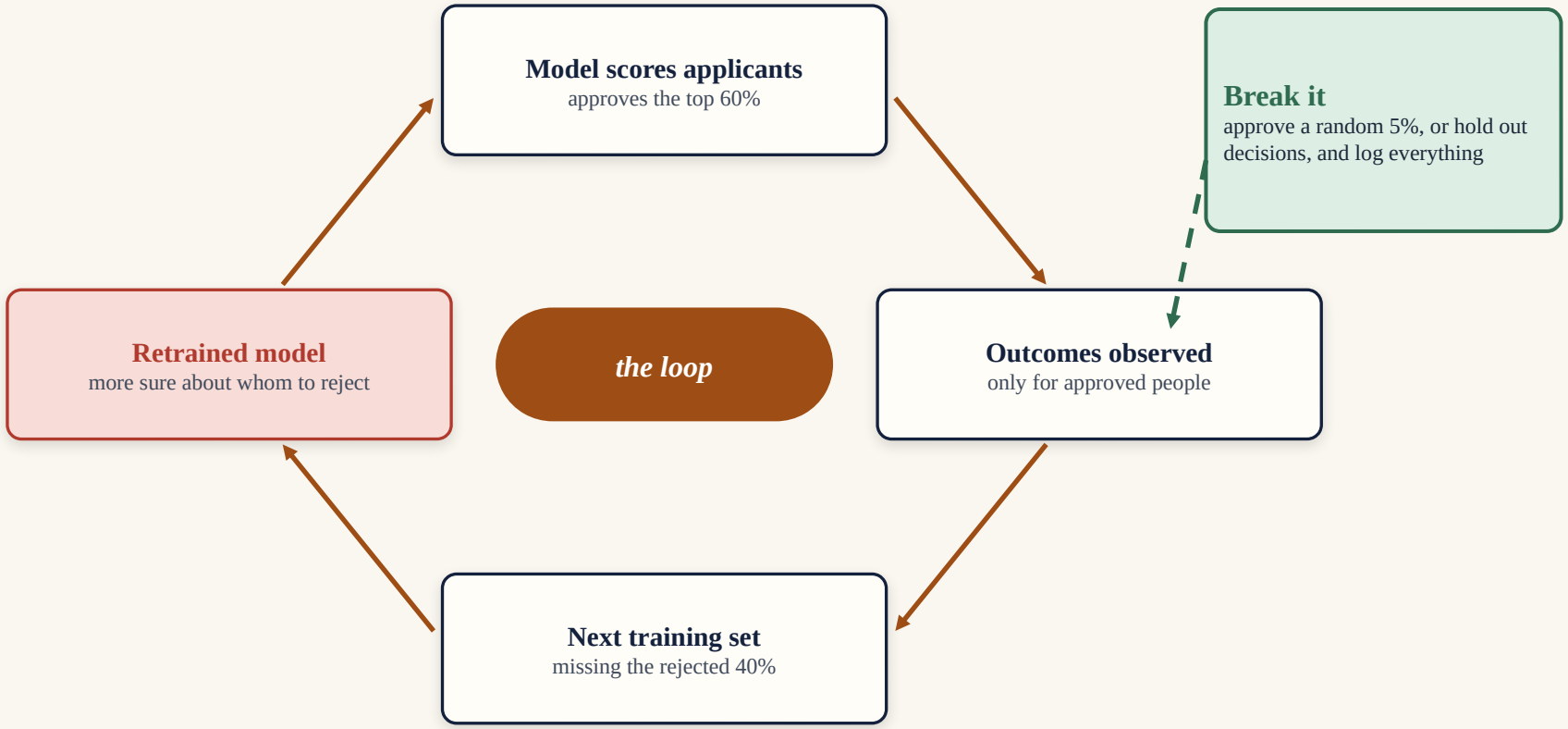
Consequences

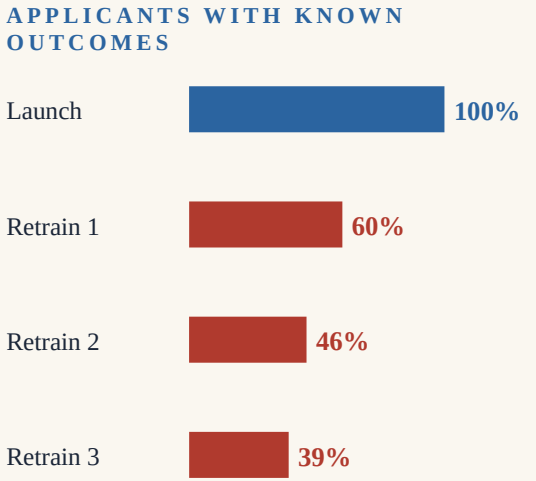
Blind spots grow, fairness can degrade and performance estimates become optimistic.

Mitigations

Hold out random exploration traffic, log decisions and use careful evaluation methods.

How a lending model teaches itself a blind spot





Illustrative. Offline metrics look better each round while the blind spot grows.

! Any model whose decisions shape its own future training data needs exploration traffic and decision logs from day one.

SECTION 14 · LIFECYCLE

Lineage: knowing where every model came from

Data version	Snapshot or query plus hash of the training data.
Code version	Git commit of training and feature code.
Parameters	Hyperparameters, seeds and configuration used.
Artifact	Model file, checksum and registry version.
Deployment	Which environments served it, when, and with what traffic share.

Why it matters

When a regulator, customer or engineer asks why a prediction was made, lineage lets you reconstruct the exact model, data and code involved.

SECTION 14 · LIFECYCLE

Retiring a model gracefully

1. **Decide.** Agree retirement because of replacement, low value or new regulation.
2. **Announce.** Tell consumers the date and the replacement or alternative.
3. **Monitor callers.** Log remaining traffic and contact clients still using the old version.
4. **Switch off.** Remove traffic, then decommission infrastructure and alerts.
5. **Archive.** Keep artifacts, model card and lineage for audits and comparisons.
6. **Clean up.** Delete unneeded data in line with retention and privacy rules.

SECTION 14 · LIFECYCLE

Lifecycle pitfalls

PITFALL	BETTER PRACTICE
⚠ Retraining automatically and deploying without review.	✓ Route every retrained model through the same gates and canary.
⚠ Retraining on data shaped by the model itself.	✓ Keep exploration data and watch for feedback loops.
⚠ Losing track of which data trained which model.	✓ Record data versions and lineage in the registry.
⚠ Zombie models that run forever unnoticed.	✓ Review model inventory and retire those with no owner or value.

Section recap

KEY TAKEAWAYS

- Retrain on schedules, drift, performance or new data
- Continuous training still passes every release gate
- Challengers must beat champions on agreed criteria
- Watch for feedback loops created by predictions
- Lineage makes every prediction explainable after the fact
- Retire models deliberately and archive their history

TRY THIS

Write a one page retraining policy for your model: triggers, comparison rules, approvers and retirement criteria.

NEXT UP

Section 15: deploying models to edge and mobile devices.

SECTION 15

15

Edge and mobile deployment

Running models where the data is born: phones, browsers, cameras, vehicles and machines.

IN THIS SECTION

- Why deploy at the edge
- Device constraints
- Mobile runtimes
- Browser inference
- Updating models over the air
- Privacy at the edge

Why deploy at the edge?

1 Latency
No network round trip, so responses feel instant.

2 Offline use
Works in tunnels, fields, aircraft and remote sites.

3 Privacy
Sensitive data such as faces or voices can stay on the device.

4 Bandwidth
Send only results or rare events instead of raw video.

5 Cost
Inference runs on hardware users already own.

6 Resilience
Keeps working when cloud services are unavailable.

SECTION 15 · EDGE AND MOBILE

Constraints you must design for

Edge devices are small, varied and battery powered. A model that runs comfortably on a server may be far too large or slow on a phone.

Memory and storage

Models often need to be a few megabytes to tens of megabytes.

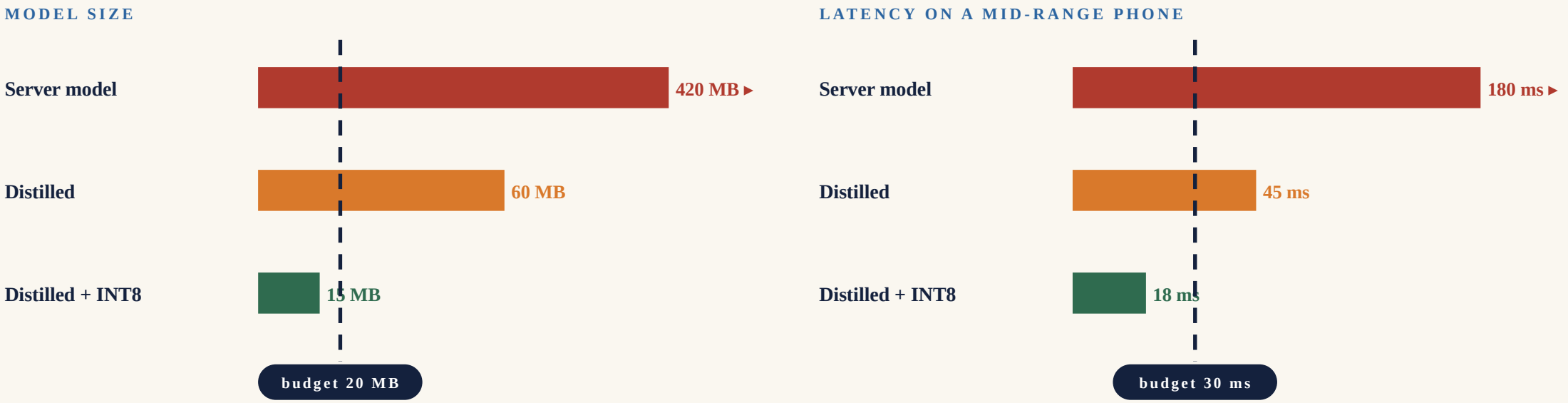
Compute and power

Heavy inference drains batteries and heats devices, causing throttling.

Fragmentation

Different chips, operating systems and accelerators behave differently.

Shrinking a server model to fit a phone's budget



Bars capped at 100; ▸ means off the scale. Illustrative figures.

ACCURACY ON THE SAME TEST SET

92.4% **Server model**
FP32 · original · 21× too big

91.8% **Distilled**
smaller student model · still 3× too big

91.5% **Distilled + INT8**
quantised for the NPU · meets every budget

On device runtimes

Runtime	Platforms	Notes
LiteRT (TensorFlow Lite)	Android, iOS, embedded Linux, microcontrollers	Google renamed TensorFlow Lite as LiteRT
Core ML	Apple devices	Uses the Apple Neural Engine where available
ONNX Runtime Mobile	Android, iOS	Shares the ONNX model format with servers
ExecuTorch	Mobile and edge	PyTorch solution for on device inference
OpenVINO	Intel edge hardware	Optimised for Intel CPUs, GPUs and NPUs

Choose the runtime that matches your device fleet and training framework, then test on the slowest device you support.

Inference in the browser

Libraries such as TensorFlow.js, ONNX Runtime Web and Transformers.js run models directly in the browser using WebAssembly, WebGL or WebGPU.

Benefits

No server cost per prediction and user data never leaves the page.

Limits

Download size matters, so models must be small and cached sensibly.

Good fits

Image filters, text classification, autocomplete and accessibility features.

Over the air model updates

01 — Publish
Upload a signed model file with version and compatibility metadata.

02 — Check
Devices ask for updates on a schedule or at app start.

03 — Download
Fetch in the background on suitable network and battery.

04 — Verify
Check signature and checksum before activation.

05 — Activate
Swap atomically and keep the old model for rollback.

Roll updates out to a small share of devices first and collect anonymous health metrics before a full release.

Privacy and federated learning

Edge deployment keeps raw data local. Federated learning goes further by training across many devices and sharing only model updates.

On device personalisation

Adapt a model to one user locally without uploading their data.

Federated learning

Devices compute updates, a server aggregates them into a better global model.

Extra protection

Secure aggregation and differential privacy reduce what updates can reveal.

SECTION 15 · EDGE AND MOBILE

Edge pitfalls

PITFALL	BETTER PRACTICE
⚠ Testing only on the latest flagship phone.	✓ Benchmark on low end and older devices you support.
⚠ Shipping models inside app releases only.	✓ Add over the air updates with version checks.
⚠ Unprotected model files inside apps.	✓ Assume extraction is possible and avoid embedding secrets.
⚠ No telemetry from devices.	✓ Collect privacy respecting metrics on latency and failures.

Section recap

KEY TAKEAWAYS

- Edge gives speed, offline use, privacy and lower cost
- Design for memory, power and device variety
- Pick runtimes that match devices and frameworks
- Browsers can run small models with no server
- Update models over the air with signing and rollback
- Federated learning trains without centralising raw data

TRY THIS

Convert a small image model to an on device format and measure size and latency on a phone or in a browser.

NEXT UP

Section 16: deploying large language models.

SECTION 16

16

Deploying large language models

What changes when the model has billions of parameters and answers in free text.

IN THIS SECTION

- How LLM serving differs
- APIs versus self hosting
- Serving engines
- Memory and batching
- RAG in production
- Evaluation, guardrails and cost

SECTION 16 · LLMS

Classic ML versus LLM serving

CLASSIC ML

Small, fast, structured

- Megabytes to a few gigabytes
- Single fast forward pass per request
- Structured inputs and outputs
- Accuracy measured against labels

LLMS

Huge, generative, open ended

- Gigabytes to hundreds of gigabytes
- Generates output token by token
- Free text in and free text out
- Quality needs nuanced evaluation

VERDICT LLM deployment shares the same foundations, but memory, generation latency, evaluation and safety all need extra attention.

Hosted API versus self hosted LLM

HOSTED API

Call a provider

- Fastest path to production
- Access to frontier capability
- Pay per token, little operations work
- Data handling depends on provider terms

SELF HOSTED

Run open weight models

- Full control of data and versions
- Can be cheaper at high steady volume
- Needs GPUs and serving expertise
- You own scaling, updates and safety

VERDICT Many teams start with hosted APIs, then self host specific workloads when volume, privacy or customisation justify the effort.

LLM serving engines

Engine	Known for	Consider
vLLM	Efficient memory paging, high throughput	Popular open source default
Hugging Face TGI	Production server for open models	Check current maintenance status
TensorRT LLM	Peak performance on NVIDIA GPUs	More build and tuning effort
SGLang	Fast structured generation and caching	Rapidly evolving project
llama.cpp and Ollama	CPU and local machine inference	Best for small or local use

This space moves monthly. Benchmark engines again on your own model and prompts before each major decision.

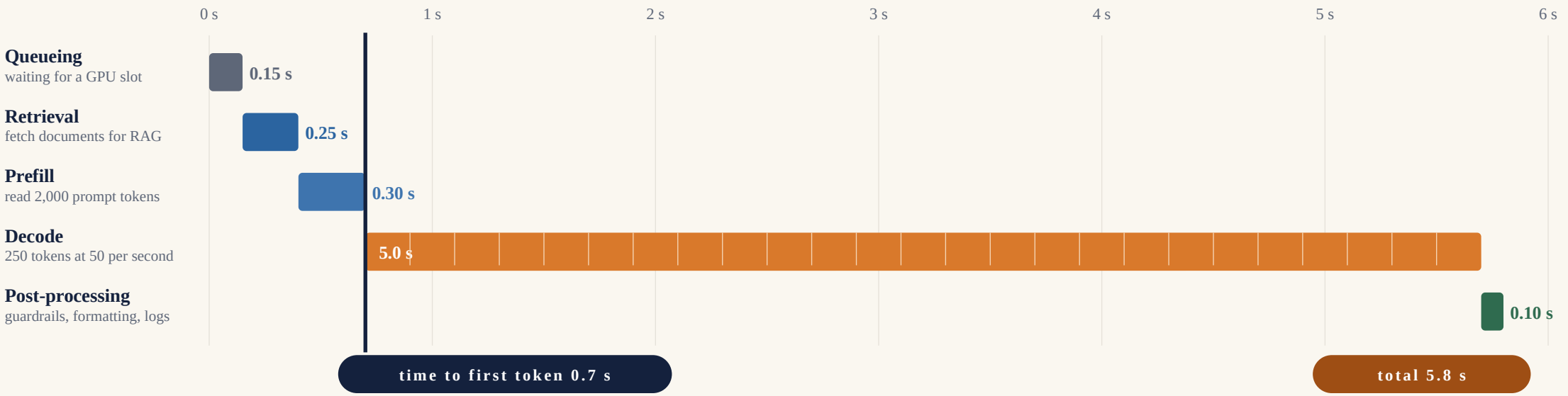
Where LLM latency comes from

Queueing	Waiting for a free slot on a busy GPU.
Prefill	Reading the whole prompt. Long prompts cost more.
Decode	Generating each new token one after another.
Retrieval	Fetching documents for RAG before generation starts.
Post processing	Guardrails, formatting and logging after generation.

Two numbers

Track time to first token, which users feel as responsiveness, and tokens per second, which decides how long full answers take. Stream tokens to the user.

Anatomy of one 5.8 second LLM answer



THE LEVER FOR EACH STAGE

Queueing	Retrieval	Prefill	Decode	Post
continuous batching, more replicas	cache embeddings, faster index	shorter prompts, prefix caching	cap output length, smaller model, speculative decoding	log asynchronously

SECTION 16 · LLMS

Memory and batching techniques

1 KV cache

Stores attention state so earlier tokens are not recomputed for each new token.

2 Paged attention

Manages cache memory in pages to waste less GPU memory.

3 Continuous batching

Adds and removes requests from a running batch at every step.

4 Quantised weights

Four or eight bit weights fit bigger models on fewer GPUs.

5 Prefix caching

Reuses computation for shared prompt beginnings such as system prompts.

6 Speculative decoding

A small draft model proposes tokens that the large model verifies.

Why the KV cache, not the weights, limits how many users fit

BYTES PER TOKEN = 2 (K and V) × layers × KV heads × head size × bytes per value

Full multi-head attention

older 7B design · 32 KV heads

`2 × 32 × 32 × 128 × 2`

0.5 MiB	2 GiB	≈ 4
per token	per 4,096-token chat	chats that fit

Grouped-query attention

8B design · 8 KV heads

`2 × 32 × 8 × 128 × 2`

128 KiB	512 MiB	≈ 18
per token	per 4,096-token chat	chats that fit

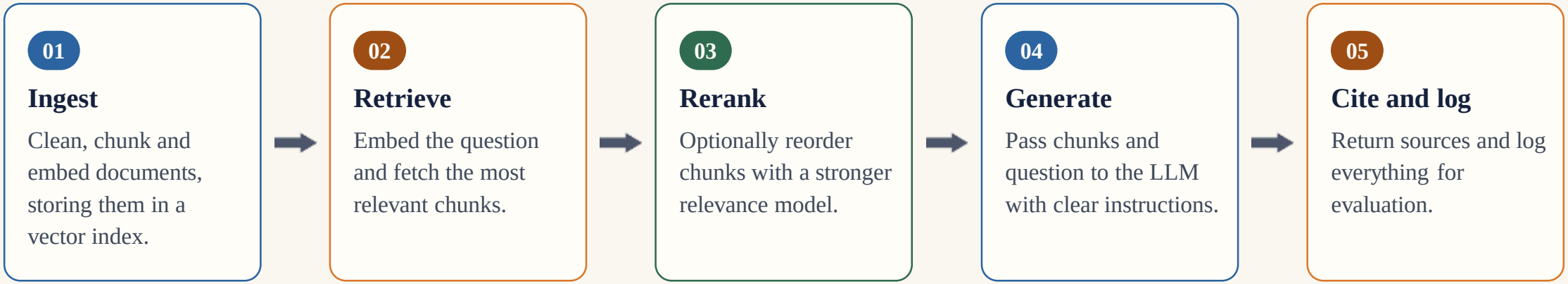
ONE 24 GB GPU



Paged attention stops each chat reserving its maximum length up front, so real capacity is often higher. Ignores activations and overhead.

SECTION 16 · LLMS

Retrieval augmented generation in production



Most RAG failures are retrieval failures. Monitor retrieval quality separately from answer quality.

Evaluating LLM applications

There is rarely one correct answer, so LLM evaluation combines automated checks, model based judging and human review on curated test sets.

Golden datasets

Curated questions with reference answers or rubrics, versioned like code.

Automated scoring

Exact checks, similarity measures and LLM as a judge with validated rubrics.

Human review

Expert sampling of real traffic catches failures that metrics miss.

Guardrails for LLM systems

1 Input filtering
Detect abuse, personal data and prompt injection attempts.

2 Output filtering
Check for unsafe content, data leakage and policy breaches.

3 Grounding checks
Verify answers are supported by retrieved sources.

4 Structured output
Validate JSON or schema outputs before passing them to other systems.

5 Least privilege tools
Agents and tools get only the permissions they truly need.

6 Human in the loop
Require approval for high impact actions such as payments.

Managing LLM cost

LLM costs scale with tokens processed, so prompt design, routing and caching often matter more than hardware discounts.

Shorter prompts

Trim instructions and retrieved context to what actually improves answers.

Model routing

Send simple requests to small models and hard ones to large models.

Caching

Cache answers to repeated questions and reuse shared prompt prefixes.

SECTION 16 · LLMS

LLMOps essentials

- Version prompts like code, with reviews
- Pin model versions, not moving aliases
- Keep a golden evaluation set per use case
- Run evaluations on every prompt or model change
- Track time to first token and tokens per second
- Monitor cost per request and per user
- Log prompts and responses with privacy controls
- Test for prompt injection regularly
- Provide user feedback buttons and review them
- Plan fallbacks when a provider is unavailable

LLM deployment pitfalls

PITFALL	BETTER PRACTICE
⚠ Judging quality from a handful of demo prompts.	✓ Evaluate on a broad, versioned test set before release.
⚠ Letting model output trigger actions directly.	✓ Validate outputs and require approval for risky actions.
⚠ Unbounded context windows that explode cost.	✓ Budget tokens per request and monitor spend daily.
⚠ Upgrading the underlying model silently.	✓ Pin versions and rerun evaluations before switching.

Section recap

KEY TAKEAWAYS

- LLMs share deployment foundations but stretch memory, latency and evaluation
- Choose hosted APIs or self hosting based on volume, privacy and control
- Serving engines use caching and continuous batching for efficiency
- RAG quality depends on retrieval quality
- Guardrails and evaluation are part of the product
- Manage cost through prompts, routing and caching

TRY THIS

Build a tiny RAG service over ten documents and create a 20 question golden set to evaluate it after every change.

NEXT UP

Section 17: security, privacy and governance.

SECTION 17

17

Security, privacy and governance

Protecting models, data and people, and proving it to customers and regulators.

IN THIS SECTION

- Threats unique to ML
- Securing the pipeline
- Access and secrets
- Privacy and data protection
- Regulation and the EU AI Act
- Fairness, explainability, audit

Threats unique to ML systems

1 Adversarial inputs
Carefully crafted inputs that push a model into wrong predictions.

2 Data poisoning
Malicious records inserted into training data to corrupt behaviour.

3 Model extraction
Repeated queries used to copy a model or steal its behaviour.

4 Membership inference
Attacks that reveal whether a person was in the training data.

5 Prompt injection
Hidden instructions that hijack LLM applications and agents.

6 Supply chain
Compromised libraries, base images or downloaded model files.

Securing the ML supply chain

Data sources	Verify provenance, access controls and integrity checks.
Dependencies	Pin versions, scan for vulnerabilities and use trusted mirrors.
Pretrained models	Prefer safe formats, verify checksums and review licences.
Build system	Isolated, reproducible builds with signed outputs.
Registry	Signed images and models, verified again at deploy time.

Trust nothing by default

Treat every external dependency, dataset and pretrained model as untrusted until verified. Frameworks such as SLSA describe levels of build integrity.

Access control and secrets

- Authenticate every caller of the model API
- Use short lived tokens over static keys
- Grant least privilege to services and people
- Store secrets in a dedicated secret manager
- Rotate credentials automatically
- Encrypt data in transit and at rest
- Rate limit to slow extraction attempts
- Log access and review it regularly
- Separate production from development access
- Remove access promptly when people leave

Privacy and data protection

In the UK and EU, the GDPR and UK GDPR govern personal data used to train and run models, including rights around automated decisions.

Data minimisation

Use only the personal data you genuinely need, and pseudonymise where possible.

Lawful basis

Document why you may process the data and tell people clearly.

Automated decisions

Significant solely automated decisions carry extra rights, including human review.

The EU AI Act in brief

The EU AI Act entered into force in 2024 and applies obligations in phases over several years, based on the risk level of each AI system.

Risk tiers

Some practices are prohibited, high risk systems face strict duties, and others carry transparency rules.

High risk duties

Risk management, data governance, documentation, logging, human oversight and accuracy.

Check dates

Timelines and guidance continue to evolve, so confirm current obligations with legal experts.

The EU AI Act's four risk tiers and its key dates



Classify every model in your inventory against these tiers.

KEY DATES

1 Aug 2024
Act enters into force

2 Feb 2025
Prohibitions and AI literacy apply

2 Aug 2025
General-purpose AI model duties

2 Aug 2026
Transparency duties apply

2 Dec 2027
Stand-alone high-risk systems*

2 Aug 2028
High-risk AI in regulated products*

** As moved by the 2026 AI Omnibus amendment. Check current guidance with legal experts.*

Fairness in production

A model can be accurate on average and still treat some groups worse. Fairness must be measured before release and monitored after it.

Measure slices

Compare error rates and outcomes across relevant groups and regions.

Choose definitions

Different fairness definitions conflict, so agree which matters for your use case.

Monitor over time

Group performance can drift even when overall metrics look stable.

Global versus local explanations

GLOBAL

How the model behaves overall

- Which features matter most in general
- Useful for validation and governance
- Supports model cards and reviews
- Example: overall feature importance

LOCAL

Why one prediction was made

- Which features drove this specific result
- Useful for customers and support teams
- Supports appeals and adverse action notices
- Example: SHAP values for one applicant

VERDICT Production systems often need both. Compute local explanations on demand or store them with predictions for later review.

Audit readiness

1 Model inventory
A list of every model in production with owner and purpose.

2 Decision logs
Inputs, outputs, version and explanations for important decisions.

3 Approvals
Who approved each release, when and based on what evidence.

4 Documentation
Model cards, data sheets and risk assessments kept current.

5 Incident records
What went wrong, impact and corrective actions taken.

6 Retention
Logs kept long enough for audits but no longer than allowed.

Security and governance pitfalls

PITFALL	BETTER PRACTICE
⚠ Loading pretrained models from unknown sources.	✓ Use trusted sources, safe formats and checksum verification.
⚠ Open prediction endpoints without authentication.	✓ Authenticate, authorise and rate limit every endpoint.
⚠ Treating compliance as a final sign off.	✓ Involve privacy and risk teams from project framing.
⚠ No record of why a decision was made.	✓ Log versions, inputs and explanations for key decisions.

Section recap

KEY TAKEAWAYS

- ML adds threats such as poisoning, extraction and prompt injection
- Secure data, dependencies, models, builds and registries
- Apply least privilege and managed secrets everywhere
- Respect GDPR rights including those on automated decisions
- Know where your systems sit under the EU AI Act
- Measure fairness, provide explanations and stay audit ready

TRY THIS

Run a threat modelling session for your model service. List assets, attackers, attack paths and one mitigation for each.

NEXT UP

Section 18: reliability and cost management.

SECTION 18

18

Reliability and cost

Designing model services that keep working when things break, at a price the business can sustain.

IN THIS SECTION

- Failure is normal
- Timeouts, retries and circuit breakers
- Graceful degradation
- Redundancy and recovery
- Incident response
- FinOps for ML

> *Everything fails eventually. Reliable systems are designed to fail gracefully and recover quickly.*

☆ Plan for slow dependencies, lost nodes, bad deployments and corrupted data, not only for the happy path.

Resilience patterns

1 Timeouts

Never wait forever on a dependency. Set budgets for each call.

2 Retries with backoff

Retry transient failures with growing delays and random jitter.

3 Circuit breakers

Stop calling a failing dependency for a while to let it recover.

4 Bulkheads

Isolate resources so one overloaded model cannot starve others.

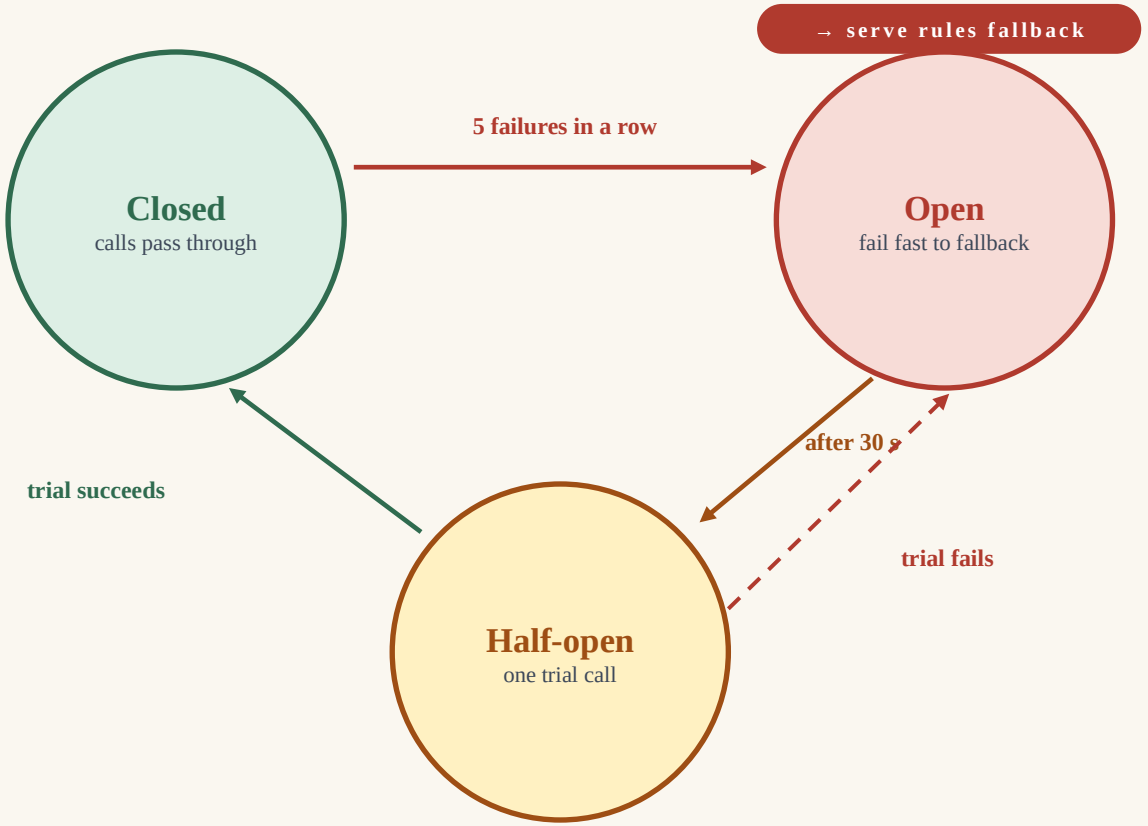
5 Load shedding

Reject low priority requests early when the system is saturated.

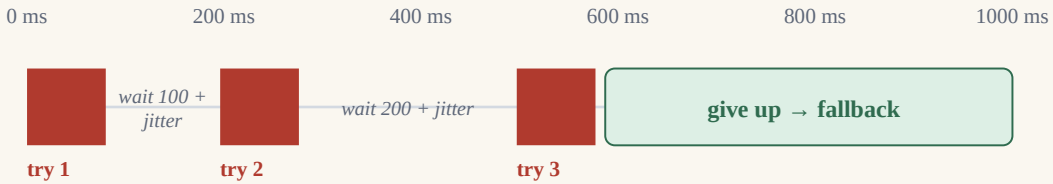
6 Idempotency

Make repeated requests safe so retries cannot cause duplicate effects.

Retries with backoff and a circuit breaker, working together



ONE CALL, THREE ATTEMPTS



Timeout 80 ms per attempt: never wait forever.

Backoff 100 → 200 → 400 ms between attempts.

Jitter random extra delay so clients do not retry in lockstep.

Idempotent a request ID makes repeats safe.

! Retries handle brief blips; the breaker handles real outages. Without it, retries multiply load on a service that is already down.

Graceful degradation with fallbacks

01

Primary

The full model returns a fresh, personalised prediction.

02

Cached

If it is slow, serve a recent cached prediction for that user.

03

Simple model

If no cache, use a lightweight backup model or rules.

04

Safe default

If all else fails, return a sensible default such as popular items.

05

Signal it

Mark degraded responses and alert when fallbacks are frequent.

Users rarely notice a slightly less personalised result. They always notice an error page.

Redundancy and disaster recovery

Multiple replicas	Several Pods spread across nodes for every service.
Multiple zones	Replicas in separate availability zones within a region.
Multiple regions	Standby or active capacity in another region for major outages.
Backups	Registry, feature store and configuration backed up and restorable.
Drills	Regular recovery exercises prove plans work in practice.

Two key targets

Recovery time objective is how quickly you must be back. Recovery point objective is how much recent data you can afford to lose. Both drive design and cost.

SECTION 18 · RELIABILITY AND COST

Incident response for model services

1. **Detect.** Alerts or users report a problem with errors, latency or prediction quality.
2. **Triage.** Assess impact and severity, and name an incident lead.
3. **Mitigate.** Roll back, switch to fallback or disable the model with a flag.
4. **Communicate.** Update stakeholders and affected teams at regular intervals.
5. **Resolve.** Fix the root cause and verify recovery with monitoring.
6. **Learn.** Hold a blameless review and track follow up actions to completion.

SECTION 18 · RELIABILITY AND COST

Chaos and resilience testing

Chaos engineering deliberately injects failures in a controlled way to check that systems behave as designed before a real incident does it for you.

Start small

Kill one replica in staging and confirm traffic continues smoothly.

Expand carefully

Add latency to the feature store, block a dependency or drain a node.

ML specific

Inject corrupted inputs or missing features and check validation and fallbacks.

SECTION 18 · RELIABILITY AND COST

FinOps for machine learning

1 Visibility
Tag resources and report cost per model, team and prediction.

2 Unit economics
Track cost per thousand predictions alongside value created.

3 Utilisation
Find idle GPUs and oversized instances every week.

4 Right model size
Use the smallest model that meets quality targets.

5 Pricing models
Mix on demand, reserved and spot capacity sensibly.

6 Accountability
Give each model owner a budget and a cost dashboard.

Reliability and cost pitfalls

PITFALL	BETTER PRACTICE
⚠ Retrying instantly and endlessly on failure.	✓ Use limited retries with exponential backoff and jitter.
⚠ A single replica for a critical model.	✓ Run at least two replicas across zones.
⚠ No fallback when the model is unavailable.	✓ Design cached, simple or default responses.
⚠ Discovering cost overruns at month end.	✓ Set budgets and daily alerts on spend per model.

Section recap

KEY TAKEAWAYS

- Design for failure with timeouts, retries and circuit breakers
- Degrade gracefully through layered fallbacks
- Spread replicas across zones and rehearse recovery
- Respond to incidents with clear roles and blameless reviews
- Test resilience deliberately with chaos experiments
- Manage cost per prediction as a first class metric

TRY THIS

Define a four level fallback chain for your model and calculate its monthly cost at today’s traffic.

NEXT UP

Section 19: an end to end case study.

SECTION 19

19

End to end case study

Deploying a card payment fraud model from first conversation to steady operation.

IN THIS SECTION

- The business problem
- Requirements and constraints
- Architecture
- Build and release
- Monitoring and incidents
- Lessons learned

SECTION 19 · CASE STUDY

The scenario



A fictional online retailer loses money to card fraud and wants to block risky payments at checkout without annoying genuine customers.

Current state

A rules engine flags payments, missing new fraud patterns and blocking many good customers.

Goal

Reduce fraud losses while keeping false declines low and checkout fast.

Model

A gradient boosted tree classifier trained on historical labelled transactions.

SECTION 19 · CASE STUDY

Requirements captured at framing

Requirement	Decision for this example
Pattern	Online, synchronous call during checkout
Latency budget	p99 under 100 milliseconds for the whole fraud check
Availability	99.95 percent, with a rules fallback if the model is down
Explainability	Top reasons stored for every declined payment
Labels	Chargebacks arrive weeks later, so proxies are needed
Privacy	Card numbers tokenised, no raw personal data in logs

Writing requirements as a table forces agreement before any code is written, and becomes the basis for tests and alerts.

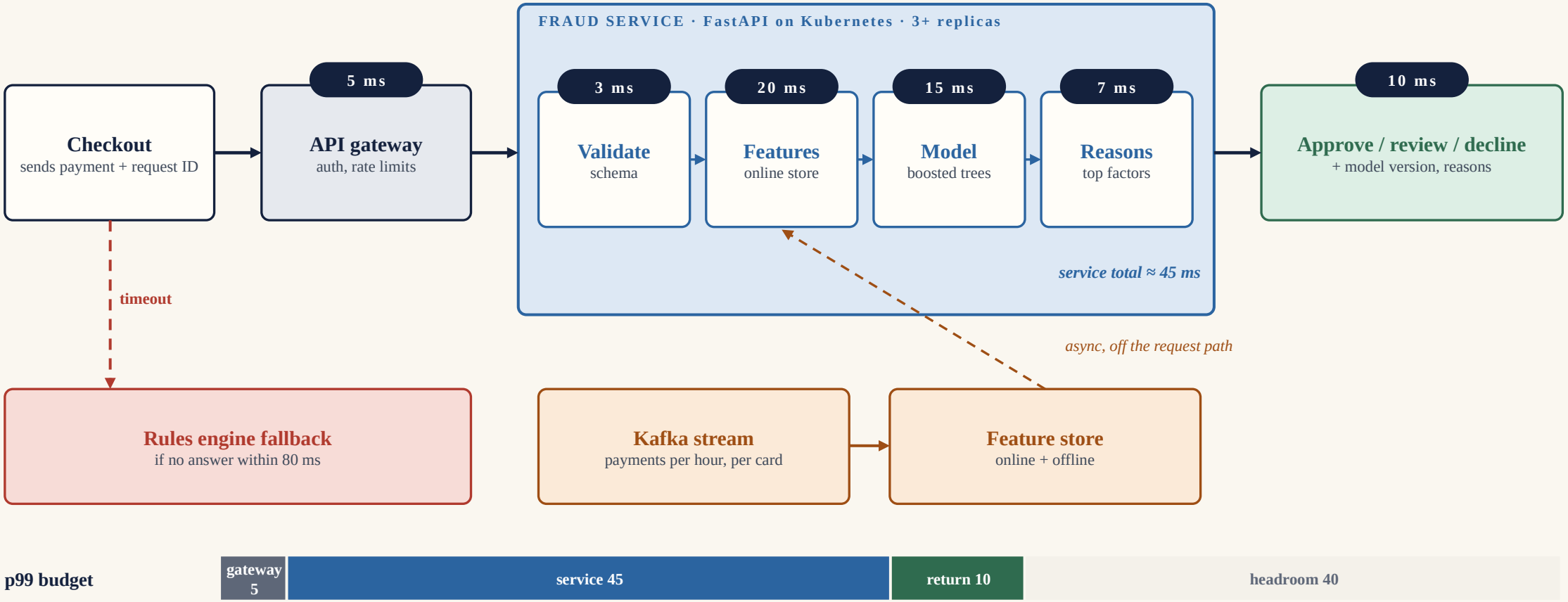
Target architecture

Checkout service	Calls the fraud API with payment details and a request identifier.
API gateway	Authentication, rate limiting and routing to the fraud service.
Fraud service	FastAPI on Kubernetes, model loaded at startup, three or more replicas.
Feature store	Online store for fresh features, offline store for training.
Stream pipeline	Kafka events update velocity features such as payments per hour.
Monitoring	Prometheus, Grafana, prediction logs and drift reports.

Design choices

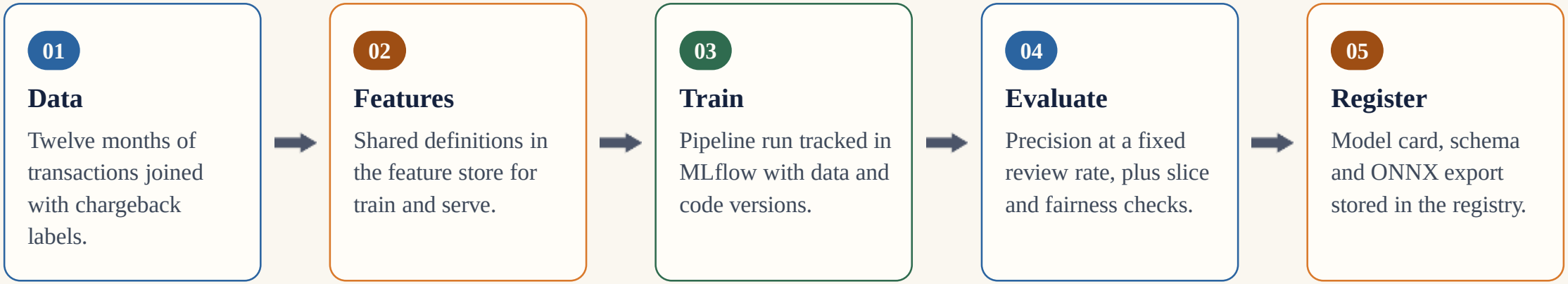
Features with long histories are precomputed in batch, while fresh velocity features update from the stream. The model service reads both in a single fast lookup.

The fraud check's architecture, with its 100 ms budget split



SECTION 19 · CASE STUDY

Building the model artifact



SECTION 19 · CASE STUDY

Fraud scoring endpoint with fallback

fraud_api.py

```
@app.post("/v1/score")
async def score(p: Payment):
    try:
        feats = await feature_store.get(p.card_token, timeout=0.03)
        risk = model.predict(p, feats)
        source = "model"
    except (TimeoutError, ModelError):
        risk = rules_engine.score(p)
        source = "fallback"
    log_prediction(p.request_id, risk, source, MODEL_VERSION)
    return {"risk": risk, "source": source,
            "model_version": MODEL_VERSION}
```

Time budget

Feature lookup gets 30 milliseconds, protecting the overall latency target.

Fallback

If features or model fail, the trusted rules engine still protects checkout.

Traceability

Every response records its source and model version for audits.

SECTION 19 · CASE STUDY

Release plan

1. **Shadow for two weeks.** Score all payments silently and compare against rules and later chargebacks.
2. **Review results.** Fraud analysts confirm catches and false alarms on a sample.
3. **Canary at 5 percent.** Model decisions apply to a small share of traffic with tight alerts.
4. **Expand gradually.** Move to 25, 50 and 100 percent over several days.
5. **Keep rules warm.** The rules engine remains ready as an instant fallback.
6. **Sign off.** Product, risk and engineering approve full rollout.

SECTION 19 · CASE STUDY

What the team monitors

1 Service health

Error rate, p99 latency and fallback rate against objectives.

2 Inputs

Drift on amount, country, device and velocity features.

3 Outputs

Decline rate and risk score distribution per hour.

4 Early proxies

Customer complaints and manual review outcomes.

5 Late truth

Chargebacks joined to predictions weekly for precision and recall.

6 Fairness

Decline rates across regions and customer segments.

SECTION 19 · CASE STUDY

An incident, and what it taught

⚠ Months later, a payment provider changed how it reported device type. Drift alerts fired on one feature and decline rates crept up in one region.

Mitigation

The team switched that region to the fallback using a feature flag within minutes.

Root cause

An upstream schema change was not caught because validation allowed new categories silently.

Fix

Stricter schema tests, a contract with the provider and a retrained model with the new values.

SECTION 19 · CASE STUDY

Lessons from the case study

The model was the easy part. Requirements, features, fallbacks and monitoring decided whether the project succeeded.

Design for failure

The rules fallback turned a potential outage into a minor, controlled dip.

Shadow first

Two weeks of silent scoring built trust with analysts before any customer impact.

Watch upstream

Many model incidents start as data changes in systems you do not own.

SECTION 19 · CASE STUDY

Section recap

KEY TAKEAWAYS

- Frame requirements explicitly before building
- Share feature definitions between training and serving
- Protect latency budgets with timeouts and fallbacks
- Release through shadow and canary stages
- Monitor proxies while waiting for delayed labels
- Treat upstream data changes as a top risk

TRY THIS

Adapt this case study to a model from your own work. Rewrite the requirements table and the release plan for it.

NEXT UP

Section 20: checklists, glossary and your next steps.

SECTION 20

20

Checklists, glossary and next steps

Practical tools to reuse on every deployment, and a path for continued learning.

IN THIS SECTION

- Pre deployment checklist
- Launch day checklist
- Post launch checklist
- Anti patterns
- Glossary
- Final knowledge check

SECTION 20 · TOOLKIT

Pre deployment checklist

- [ ] Business goal and success metric agreed
- [ ] Deployment pattern chosen and justified
- [ ] Model packaged with preprocessing
- [ ] Schema and input validation in place
- [ ] Model card and risk assessment complete
- [ ] Unit, data, behavioural and contract tests pass
- [ ] Load test meets latency budget
- [ ] Security scan shows no critical issues
- [ ] Monitoring dashboard and alerts ready
- [ ] Rollback and fallback plan documented

SECTION 20 · TOOLKIT

Launch day checklist

- [ ] Release window agreed with stakeholders
- [ ] Feature flags and kill switch confirmed
- [ ] On call engineer and incident lead named
- [ ] Communication channel open for updates
- [ ] Previous version ready for instant rollback
- [ ] Rollback triggers written and agreed
- [ ] Canary or shadow configuration verified
- [ ] Stakeholders know how to report issues
- [ ] Dashboards open and alerts tested
- [ ] Go or no go decision recorded

SECTION 20 · TOOLKIT

Post launch checklist

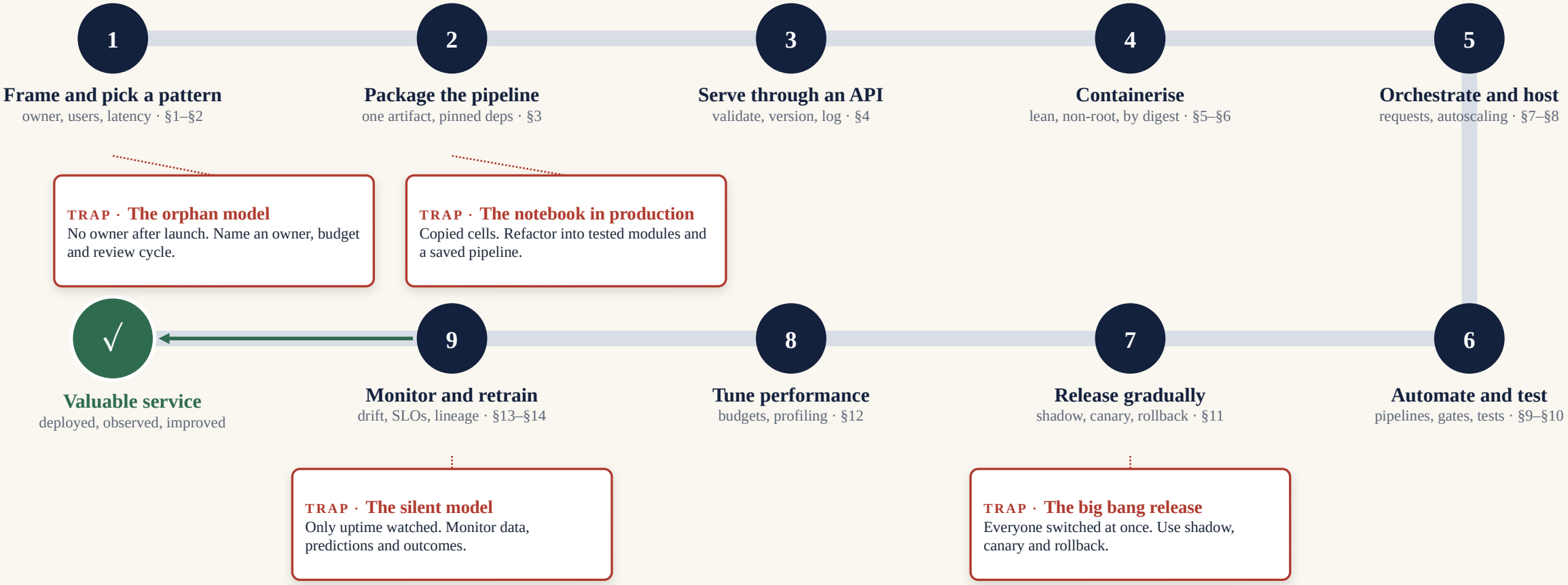
- [ ] Error rate and latency stable for a week
- [ ] Prediction distribution matches expectations
- [ ] No unexpected drift on key features
- [ ] Business metric trending as planned
- [ ] Fairness slices reviewed
- [ ] Cost per prediction within budget
- [ ] Alert noise reviewed and tuned
- [ ] Retraining schedule and triggers active
- [ ] Lessons captured in a short review
- [ ] Documentation updated with real behaviour

SECTION 20 · TOOLKIT

Deployment anti patterns

PITFALL	BETTER PRACTICE
The notebook in production: copying cells into a server script.	Refactor into tested modules and a saved pipeline.
The silent model: no monitoring beyond uptime.	Monitor data, predictions and business outcomes.
The big bang release: everyone switched at once.	Use shadow, canary and automatic rollback.
The orphan model: no owner after launch.	Name an owner, a budget and a review cycle for every model.

The whole deployment journey, with its four traps



SECTION 20 · TOOLKIT

Glossary, part one

Term	Meaning
Artifact	A saved, versioned output such as a model file or container image
Batch prediction	Scoring many records on a schedule and storing results
Canary	Releasing to a small share of traffic before everyone
Concept drift	Change in the relationship between inputs and the right answer
Container	A running, isolated instance of an image
Data drift	Change in the distribution of input features
Feature store	Shared system that serves features for training and serving
Inference	Using a trained model to make a prediction

Glossary, part two

Term	Meaning
Latency	Time taken to return one prediction
MLOps	Practices that make ML delivery reliable and repeatable
Model card	Document describing a model’s purpose, data, performance and limits
Model registry	Store of model versions, metadata and release stages
Quantisation	Using fewer bits for weights to shrink and speed up models
Shadow deployment	Running a new model on real traffic without using its output
SLO	Service level objective, a measurable reliability target
Throughput	Number of predictions completed per second

SECTION 20 · TOOLKIT

Final knowledge check 1

? Your new model passes offline tests but you worry about unknown real world behaviour. Which release strategy gives evidence with no user risk?

MODEL ANSWER

A shadow deployment. The new model receives a copy of real traffic and its predictions are logged and compared with the current model and later outcomes, but never returned to users. Once results look good, move to a canary release.

WHY IT MATTERS

It turns uncertainty into evidence before anyone is affected.

SECTION 20 · TOOLKIT

Final knowledge check 2

? Inputs to your model look exactly like training data, yet accuracy measured from fresh labels is falling. What is the likely cause?

MODEL ANSWER

Concept drift. The relationship between inputs and outcomes has changed even though input distributions have not. The fix is to gather recent labelled data, retrain, and compare the challenger against the champion before promoting it.

WHY IT MATTERS

It shows why data drift monitoring alone is not enough.

SECTION 20 · TOOLKIT

Final knowledge check 3

? A fraud API must answer within 100 milliseconds, but the feature store sometimes takes 300 milliseconds. What should the service do?

MODEL ANSWER

Set a strict timeout on the feature lookup and fall back gracefully, for example to cached features, a simpler model or a rules engine. Log and alert on the fallback rate, and work with the feature store owners to fix the slow path.

WHY IT MATTERS

Reliability comes from designing for slow dependencies, not hoping they never happen.

SECTION 20 · TOOLKIT

Your learning path from here

01

Practise

Deploy one small model end to end with the checklists.

02

Automate

Add CI, a registry and a canary release.

03

Observe

Instrument drift, quality and business dashboards.

04

Scale

Move to Kubernetes or a managed platform.

05

Specialise

Go deeper into LLMs, edge or governance.

Recommended reading: Designing Machine Learning Systems by Chip Huyen, Reliable Machine Learning from O’Reilly, and Google’s Rules of Machine Learning.

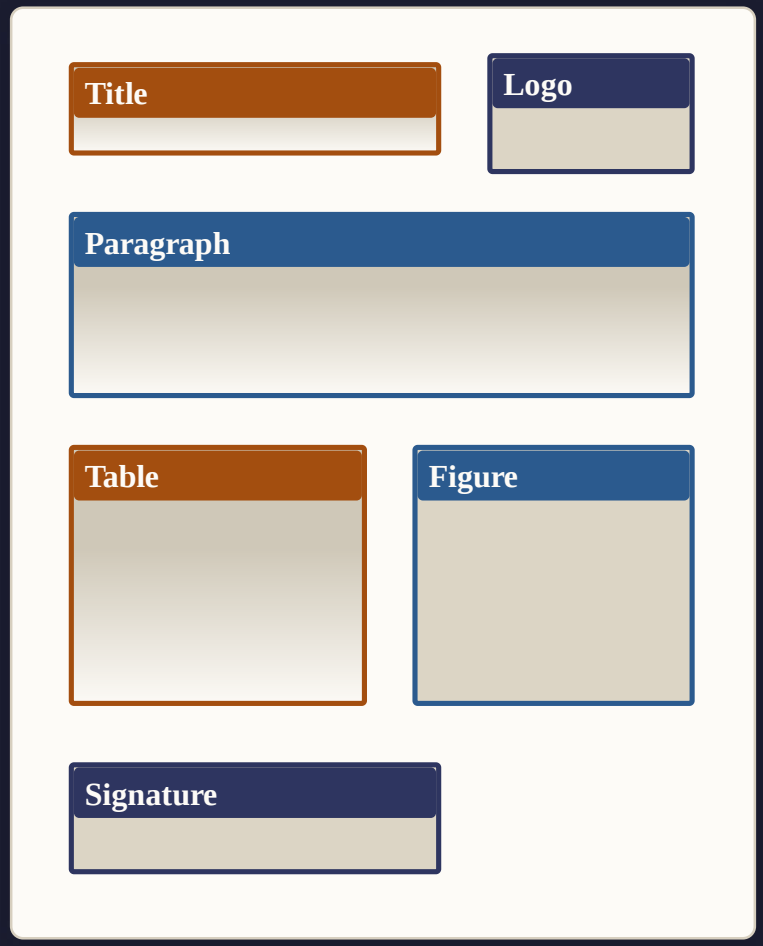
Great models create value only when they are deployed, observed and improved with care.

☆ Thank you for completing this tutorial. Use the checklists, revisit the sections as you build, and ship your next model with confidence.

OPTICAL CHARACTER RECOGNITION AND NATURAL LANGUAGE PROCESSING

From pixels on a page to meaning in a sentence

How machines read documents, understand language, and how the two combine into systems that turn paper, scans and photos into structured, searchable, useful knowledge.



BEFORE WE BEGIN

ML and AI engineers

Who build document pipelines, chatbots, search and extraction systems.

Data scientists

Who need to turn scanned and unstructured text into usable data.

Product and operations leads

Who buy, scope or sign off document automation projects.

Students and career changers

Who want a lucid map of OCR and NLP without heavy mathematics.

KEY IDEA

The one idea to hold

OCR turns **images of text into characters**. NLP turns **characters into meaning**. Together they let software read the world's documents.

ANALOGY

OCR is the eyes that recognise letters on a page. NLP is the mind that understands what the letters say. A reader needs both.

BEFORE WE BEGIN

What you will be able to do by the end

- Explain every stage of a modern OCR pipeline, from capture to text
- Choose preprocessing steps that genuinely help recognition
- Describe how text detection and recognition models work
- Extract fields, tables and key values from forms and invoices
- Measure OCR quality with character and word error rates
- Tokenise, normalise and represent text for models
- Explain embeddings, attention and transformers in plain words
- Choose between classical NLP, fine tuned models and LLMs
- Build retrieval augmented systems over scanned documents
- Evaluate, deploy and govern OCR and NLP systems responsibly

HOW TO READ THIS DECK

Every coloured box has a job

Each box type carries a different kind of message. Learn the colour code once and you can skim any slide in seconds.

KEY IDEA

The core concept of the slide. If you read nothing else, read these.

PITFALL

A mistake real teams make, often without noticing until production.

PRO TIP

Practical advice from experience that saves time or rework.

ANALOGY

An everyday comparison that makes an abstract idea concrete.

DEFINITION

A precise meaning for a term, written in plain language.

IN THE WILD

A realistic scenario showing the idea at work in a project.

HOW TO READ THIS DECK

More box types you will meet

MYTH

A popular belief that sounds right but leads you astray.

REALITY

What actually holds true, and why.

RULE OF THUMB

A quick heuristic for when you have no time to think deeply.

CHECK YOURSELF

A short question to test your understanding.

INTERVIEW ANGLE

How the topic tends to come up in technical interviews.

LESSON

The transferable takeaway from a case study.

WARNING SIGNS

Symptoms that tell you something is going wrong.

THE FIX

The remedy for the problem just described.

USE IT WHEN

The situations where a technique is the right choice.

ROADMAP

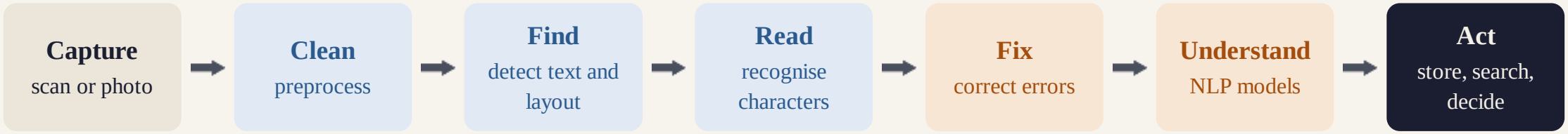
Three parts, twenty one sections

Part A · OCR, sections 1 to 9 | **Part B · NLP, sections 10 to 18** | **Part C · Together, sections 19 to 21**

01 What OCR is	02 Images and capture	03 Preprocessing
04 Layout and text detection	05 Text recognition	06 Postprocessing and correction
07 Document AI	08 Evaluating OCR	09 OCR tools and pipelines
10 NLP foundations	11 Preprocessing and tokenisation	12 Representing text
13 Classical NLP tasks	14 Sequence models	15 Transformers
16 Large language models	17 Applied NLP	18 Evaluating NLP
19 OCR meets NLP	20 Case studies	21 Deployment, ethics and toolkit

THE BIG PICTURE

From a photo of paper to a useful answer



KEY IDEA

The first five boxes are **OCR**: they turn pixels into reliable text. The last two are **NLP**: they turn text into structure and meaning that software can act on.

PITFALL

Errors flow downstream. A misread digit in an invoice total survives every later step unless something checks it. Each stage must hand the next one clean, confident output.

ANALOGY

Think of a postal sorting office. Someone opens the envelope (capture), flattens the letter (clean), finds the address block (find), reads it (read), checks it against a list of real streets (fix), works out which department it is for (understand), and delivers it (act).

THE PREMISE

Reading is two problems: seeing the letters, and understanding the words.

OCR solves the first. NLP solves the second. The most valuable systems in industry today do both, and the seams between them are where most projects succeed or fail.

PART A

Part A · Optical Character Recognition

Nine sections on how machines turn images of text into characters: capture, cleaning, layout, recognition, correction, document understanding, measurement and tools.

01

SECTION 01

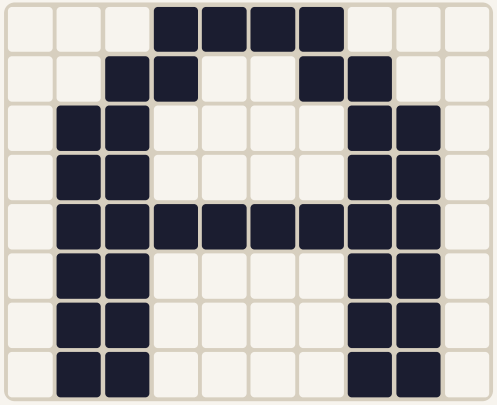
What OCR is

The problem OCR solves, where it came from, the kinds of text it faces, and why it remains hard.

Definition | History | Printed vs handwritten | Scene text | Why it is hard | Use cases

DEFINITION

Optical character recognition in one picture



What the computer receives: a grid of pixel values





What we want: the character A, as text

DEFINITION

OCR is the conversion of images that contain text (scans, photos, screenshots, video frames) into machine readable characters that software can search, edit and analyse.

IN PLAIN WORDS

To a computer an image is only numbers describing brightness. OCR is the bridge from those numbers to letters.

A SHORT HISTORY

Eighty years of teaching machines to read

1910s to 1950s

Reading machines

Early devices read printed characters for telegraphy and to help blind readers.

1970s

Omni font OCR

Systems that read many typefaces, with reading machines for the blind.

1990s

Desktop OCR

Scanners and software bring OCR to offices; engines like Tesseract begin.

2010s

Deep learning

Neural networks read scene text and handwriting far better than rules.

2020s

Document AI

Transformers read layout and text together; vision language models read pages directly.

KEY IDEA

The shift that matters: from **hand written rules** about letter shapes, to models that **learn** what letters look like from millions of examples.

REMEMBER

Old ideas still appear inside modern systems: binarisation, connected components and dictionaries remain useful helpers around neural models.

KINDS OF TEXT

Not all text is equally easy to read

Easy

Clean printed documents

Typed letters, reports, books. Clear fonts, good contrast. The easiest case.

Moderate

Forms and invoices

Printed text in boxes and tables, often with stamps, lines and handwriting mixed in.

Hard

Handwriting

Every writer is different; letters join and vary. Called handwritten text recognition.

Hard

Scene text

Signs, labels, number plates, product packaging photographed in the wild.

Very hard

Historical documents

Faded ink, old fonts, damaged paper, spelling that differs from today.

Easy to moderate

Screenshots and digital images

Rendered text in apps and slides; usually clean but often small.

WHY OCR IS HARD

Ten things that confuse a reading machine

WARNING SIGNS

Poor lighting and shadows across the page

WARNING SIGNS

Blur from camera shake or focus

WARNING SIGNS

Skew and perspective from angled photos

WARNING SIGNS

Low resolution, so letters are a few pixels high

WARNING SIGNS

Noise: speckles, stains, bleed through from the back

WARNING SIGNS

Unusual fonts and decorative lettering

WARNING SIGNS

Touching or broken characters

WARNING SIGNS

Complex layouts: columns, tables, sidebars

WARNING SIGNS

Mixed languages and scripts on one page

WARNING SIGNS

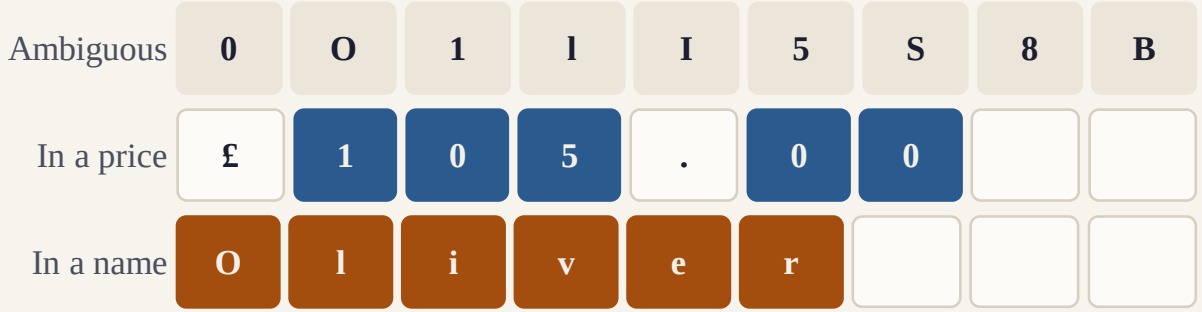
Look alike characters: 0 and O, 1 and l, 5 and S

PRO TIP

When an OCR project struggles, check these ten first. Most failures trace back to capture and layout, not to the recognition model itself.

LOOK ALIKE CHARACTERS

Why context matters even for single letters



KEY IDEA

The same shape can be a digit or a letter. Humans resolve it from **context**: a price contains digits, a name contains letters. Good OCR systems do the same with language models and field rules.

WORKED EXAMPLE

“Invoice tota1: £1O5.0O” is what a shape only reader might produce. A context aware reader returns “Invoice total: £105.00”.

WHERE OCR IS USED

Applications across industries

Sector	Typical use	What success looks like
Finance	Invoices, receipts, bank statements, cheques	Fields extracted accurately with little manual keying
Healthcare	Referral letters, lab reports, prescriptions	Records searchable and coded, with privacy protected
Government	Forms, passports, archives	Faster processing, digitised public records
Logistics	Shipping labels, delivery notes, container codes	Parcels tracked without manual reading
Legal	Contracts, court bundles, discovery	Clauses found and compared in seconds
Retail	Shelf labels, receipts, product packaging	Prices and products matched automatically
Heritage	Newspapers, manuscripts, letters	Centuries of text made searchable

MYTHS AND REALITIES

What people get wrong about OCR

MYTH

OCR is a solved problem.

REALITY

It is solved for clean print. Handwriting, photos and complex layouts remain hard.

MYTH

A better model fixes everything.

REALITY

Capture quality and layout often matter more than the recogniser.

MYTH

Ninety nine per cent character accuracy means the job is done.

REALITY

One wrong character per hundred still means a wrong digit on many invoices. Field accuracy is what counts.

SECTION 1 RECAP

What OCR is

REMEMBER

Takeaways

- OCR turns images of text into characters
- Difficulty depends on text type: print, forms, handwriting, scene text
- Capture and layout cause most failures
- Context resolves look alike characters
- Measure success by the task, not just characters

CHECK YOURSELF

A model reads 99% of characters correctly on invoices with 50 characters in key fields. Is that good enough?

ANSWER

Often not. One error per hundred characters means many invoices will have at least one wrong key field. Measure field level accuracy instead.

SECTION 02

02

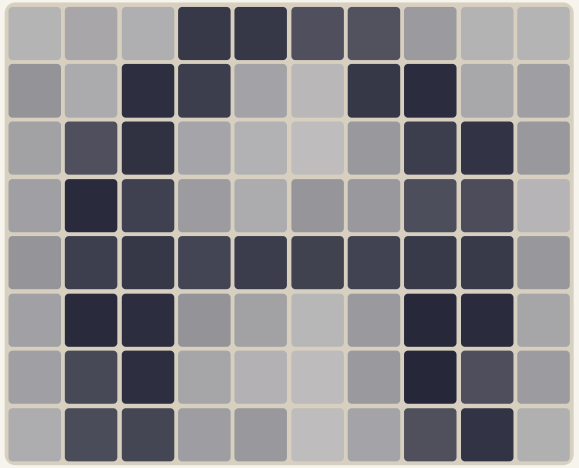
Images and capture

How digital images represent text, why resolution and lighting decide quality, and how to capture documents well.

Pixels **Resolution** **Colour** **Scanners vs cameras** **File formats** **Capture checklist**

PIXELS

An image is a grid of numbers



A greyscale crop of the letter A: each square is one pixel

DEFINITION

A **pixel** is the smallest dot of an image. In greyscale each pixel holds one brightness value, from dark to light. In colour it holds three: red, green and blue.

KEY IDEA

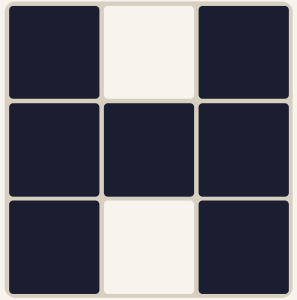
OCR models never see shapes directly. They learn which patterns of brightness values tend to form which characters.

ANALOGY

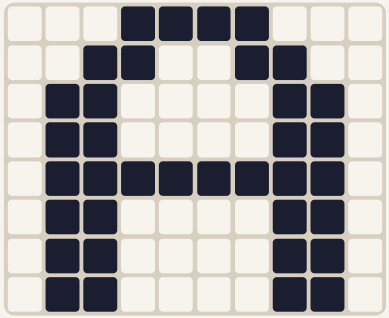
Like a mosaic made of tiny tiles. Stand close and you see only tiles; step back and a letter appears. OCR has to step back.

RESOLUTION

How many pixels does a letter need?



Too few pixels: shape is lost



Enough pixels: shape is clear

DEFINITION

Resolution for scans is measured in dots per inch (DPI): how many pixels cover one inch of paper.

Scan setting	Typical result
150 DPI	Readable for large print only
300 DPI	The usual standard for OCR
400 to 600 DPI	Small print, fine detail, archives

RULE OF THUMB

Aim for body text letters at least about 20 to 30 pixels tall. Below roughly 10 pixels, accuracy drops sharply.

COLOUR AND CONTRAST

Colour, greyscale and black and white

Colour

Three values per pixel. Keeps stamps, highlights and coloured ink distinct. Larger files.

Greyscale

One value per pixel. The default for most OCR. Keeps soft edges that help recognition.

Black and white

Each pixel is ink or paper. Small and fast, but thresholding mistakes are permanent.

PRO TIP

Capture in colour or greyscale and convert later. You can always throw information away, but you can never get it back.

IN THE WILD

A bank captured cheques in pure black and white to save storage. Light blue handwriting vanished entirely on some cheques. Rescanning was impossible because originals had been returned.

CAPTURE DEVICES

Flatbed scanners versus phone cameras

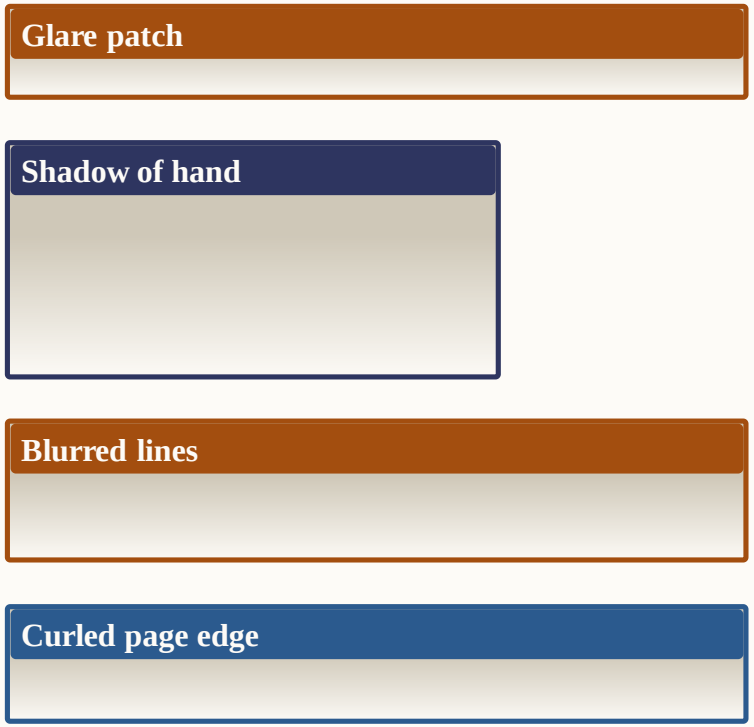
Factor	Flatbed or document scanner	Phone camera
Lighting	Even and controlled	Variable: shadows, glare, colour casts
Geometry	Flat, straight, fixed distance	Tilted, curved pages, perspective distortion
Resolution	Known and consistent DPI	Varies with distance and zoom
Blur	Rare	Common from motion and focus
Convenience	Needs hardware and a desk	Anywhere, by anyone
OCR impact	Highest, most predictable accuracy	Needs strong preprocessing and robust models

KEY IDEA

Most modern OCR products face camera images from customers. Design the pipeline for the **worst** realistic capture, not the best.

CAMERA CAPTURE

What goes wrong with phone photos



WARNING SIGNS

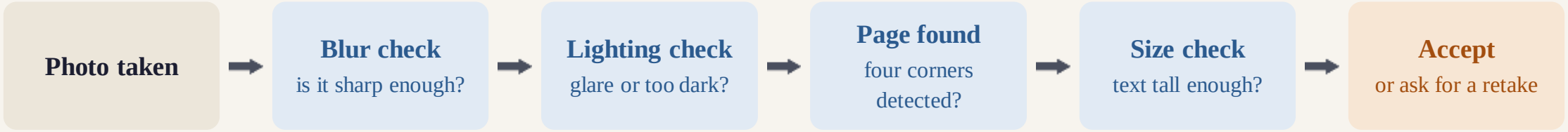
- Glare from glossy paper or plastic sleeves
- Shadows from hands and phones
- Motion blur from shaky hands
- Curved pages near a book spine
- Background clutter around the page

THE FIX

- Guide the user with on screen frames
- Check blur and glare before accepting a photo
- Detect page edges and flatten the page
- Ask for a retake rather than guessing

CAPTURE QUALITY GATES

Reject bad images before OCR, not after



KEY IDEA

A few cheap checks at capture time prevent a large share of OCR errors. Asking for a retake costs the user seconds; fixing a wrong extraction costs far more later.

PRO TIP

A common blur check measures how sharp the edges in the image are. Very soft edges overall mean the photo is blurred. Tune the threshold on your own documents.

FILE FORMATS

Images, PDFs and what is inside them

Format	What it holds	OCR note
PNG	Lossless image	Safe choice for processing; no compression damage
JPEG	Lossy compressed image	Heavy compression creates blocky artefacts around letters
TIFF	Image, often multi page	Common in archives and scanners; supports lossless
PDF with text layer	Real characters plus layout	No OCR needed: extract the text directly
PDF of scans	Images inside a PDF wrapper	Needs OCR on each page image
HEIC and WEBP	Modern phone and web images	Convert carefully before processing

⚠ PITFALL

Many teams run OCR on PDFs that already contain perfect digital text, introducing errors that did not need to exist. Always check for a text layer first.

BORN DIGITAL OR SCANNED?

The first decision in any document pipeline

Incoming PDF

↓

Does it have a usable text layer?

↓

Yes: extract text directly
fast, exact, keeps layout

↓

No or garbled: render pages and run OCR

DEFINITION

Born digital documents were created by software and contain real characters. **Scanned** documents are pictures of paper.

PITFALL

Some PDFs have a hidden text layer made by poor OCR years ago. It looks like text but is full of errors. Sample and check its quality before trusting it.

PRO TIP

Mixed PDFs are common: a typed cover letter followed by scanned attachments. Decide page by page, not file by file.

SECTION 2 RECAP

Images and capture

REMEMBER

Takeaways

- Images are grids of brightness numbers
- About 300 DPI is the usual scanning standard
- Keep greyscale or colour; threshold later
- Phone photos need quality gates and retakes
- Check PDFs for a real text layer before OCR

CHECK YOURSELF

Users upload phone photos of receipts and accuracy is poor. What is the first thing to check?

ANSWER

Capture quality: blur, glare, shadows and text size. Add capture gates and guidance before changing the model.

SECTION 03

03 Preprocessing

Cleaning and normalising images so that recognition sees clear, upright, high contrast text.

Greyscale · Binarisation · Noise removal · Deskew · Perspective · Morphology · Contrast

WHY PREPROCESS

Give the recogniser the easiest possible job

Raw image photo or scan → **Greyscale** one channel → **Denoise** remove speckles → **Deskew** straighten → **Contrast** even out lighting → **Binarise** ink vs paper → **Clean image** ready for OCR

KEY IDEA

Preprocessing removes variation that has nothing to do with the text itself: tilt, shadows, noise, colour casts. The recogniser can then spend its capacity on letter shapes.

PITFALL

Modern neural recognisers are trained on messy images and sometimes do **worse** after heavy preprocessing. Always measure; never assume a step helps.

RULE OF THUMB

Classic engines like Tesseract benefit a lot from clean, binarised input. Deep learning engines usually want light preprocessing: deskew, crop and contrast, then let the model handle the rest.

STEP 1

Converting to greyscale

Colour pixel	Red 210	Green 190	Blue 160
Greyscale pixel	One brightness value		

DEFINITION

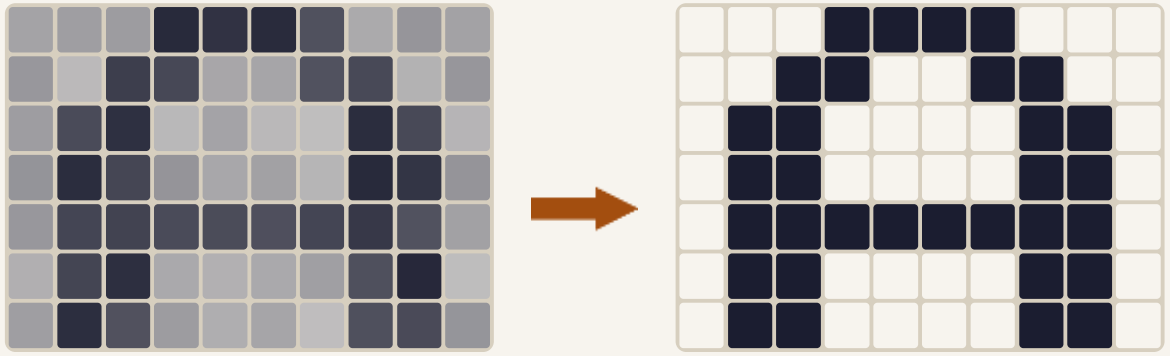
Greyscale conversion combines the three colour channels into a single brightness value per pixel, weighting green most because human eyes are most sensitive to it.

PRO TIP

If text is printed in a coloured ink (red stamps, blue handwriting), try using a **single colour channel** instead. One channel may separate ink from background far better than an average.

STEP 2

Binarisation: deciding ink or paper



Before: shades of grey

After: every pixel is ink or paper

DEFINITION

Binarisation (thresholding) turns each pixel black or white by comparing its brightness with a threshold value.

KEY IDEA

The whole difficulty is choosing the threshold. Too low and faint strokes vanish; too high and the background becomes noise.

STEP 2

Global versus adaptive thresholds

Global threshold

One threshold for the whole image. Otsu’s method chooses it automatically by finding the value that best separates two groups of brightness. Fast and simple.

Adaptive threshold

A different threshold for each small neighbourhood, based on local brightness. Handles shadows and uneven lighting.

Document specific methods

Methods such as Sauvola and Niblack were designed for documents with stains, faded ink and gradients.

WORKED EXAMPLE

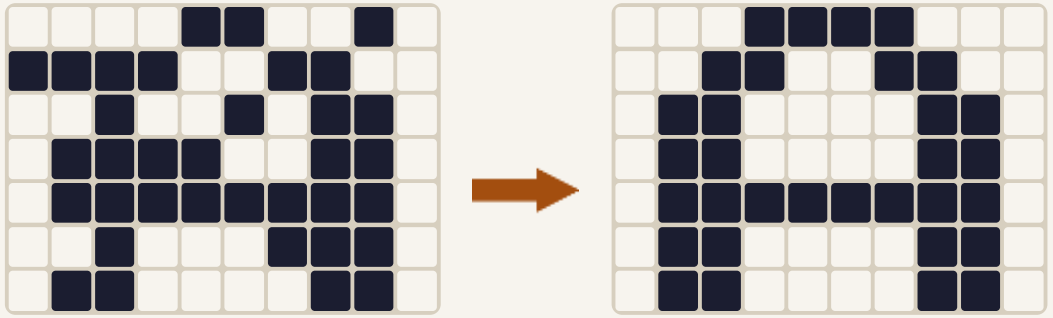
A photo with a shadow across the bottom half: a global threshold turns the shadowed half solid black. Adaptive thresholding keeps the text readable in both halves.

PYTHON · OPENCV

```
import cv2
grey = cv2.imread("page.png", cv2.IMREAD_GRAYSCALE)
flags = cv2.THRESH_BINARY + cv2.THRESH_OTSU
_, otsu = cv2.threshold(grey, 0, 255, flags)
adaptive = cv2.adaptiveThreshold(grey, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 31, 15)
```

STEP 3

Removing noise



Salt and pepper noise

After a median filter

DEFINITION

Median filter: replaces each pixel with the middle value of its neighbours. Removes speckles, keeps edges.

DEFINITION

Gaussian blur: softens fine noise but also softens letters.

DEFINITION

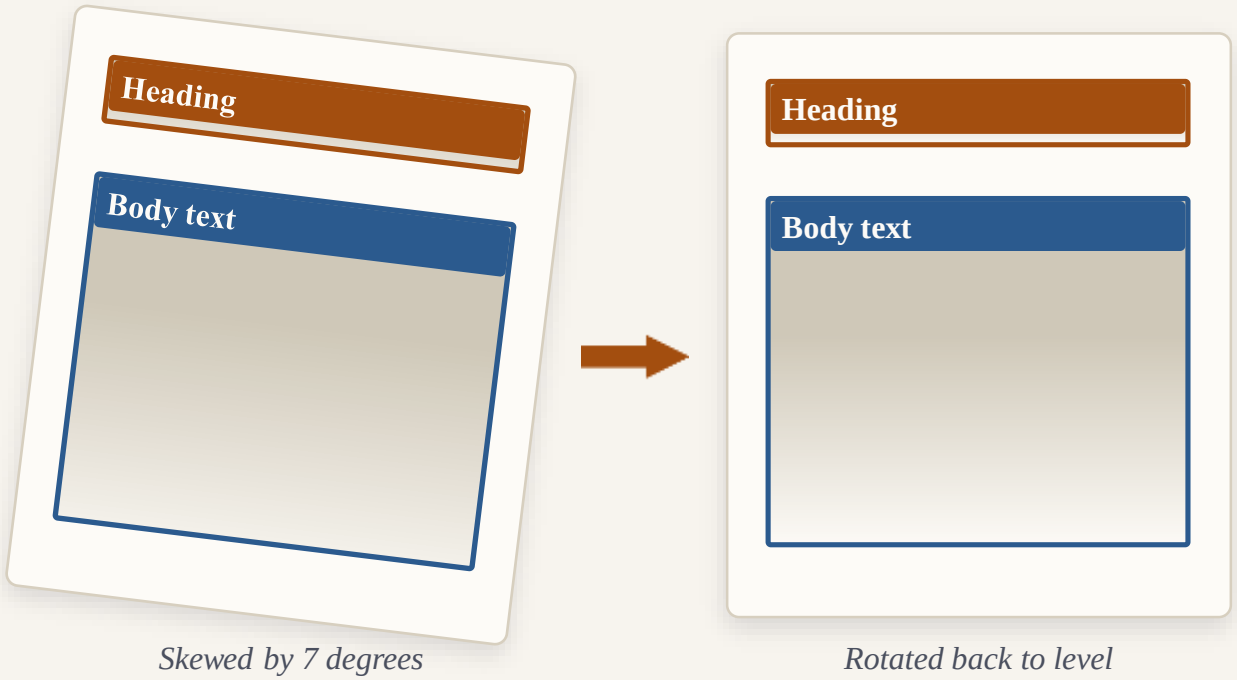
Bilateral filter: smooths flat areas while keeping strong edges.

DEFINITION

Non local means: compares patches across the image for stronger denoising.

STEP 4

Deskewing: straightening the page



Skewed by 7 degrees *Rotated back to level*

DEFINITION

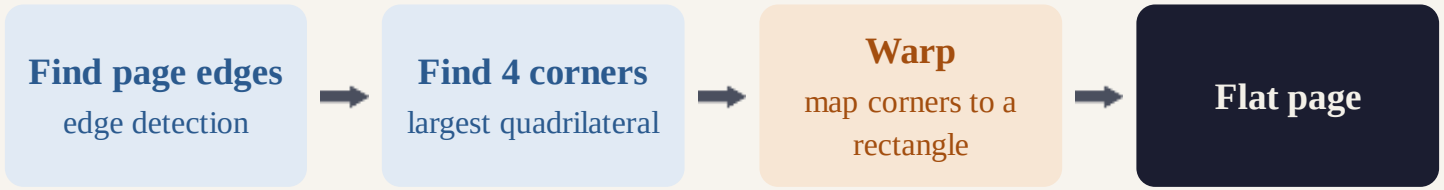
Skew is a small rotation of the text lines. Even a few degrees can confuse line detection and recognition.

KEY IDEA

Common approach: try many small angles and choose the one where the text lines become most clearly separated into horizontal bands.

STEP 5

Perspective correction for photos



DEFINITION

A photo taken at an angle makes the page look like a trapezium. A **perspective transform** stretches the image so the four page corners become a proper rectangle, as if scanned flat.

PITFALL

Curved book pages need more than a four corner warp. Page dewarping models estimate the curve of each line and flatten it.

PRO TIP

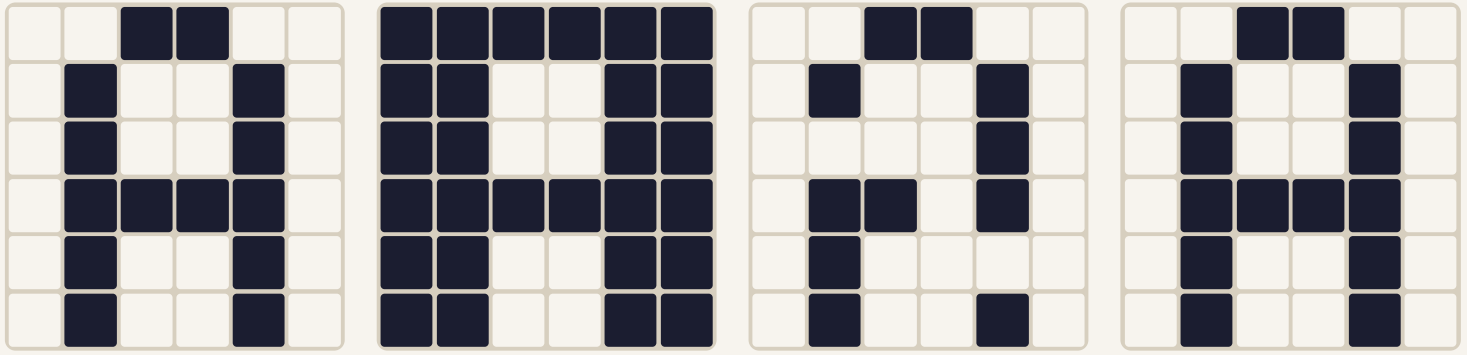
Show the detected corners to the user before capture completes. People fix bad framing instantly when they can see it.

IN THE WILD

A delivery app let drivers photograph signed notes. Adding automatic corner detection and a live outline reduced unreadable uploads dramatically without any model change.

STEP 6

Morphology: thickening, thinning and joining



Thin strokes *After dilation* *Broken strokes* *After closing*

DEFINITION

Dilation grows ink regions. Thickens faint strokes and joins gaps.

DEFINITION

Erosion shrinks ink regions. Separates touching letters and removes specks.

DEFINITION

Closing (dilate then erode) fills small gaps inside strokes.

DEFINITION

Opening (erode then dilate) removes small noise blobs.

STEP 7

Contrast and lighting correction

DEFINITION

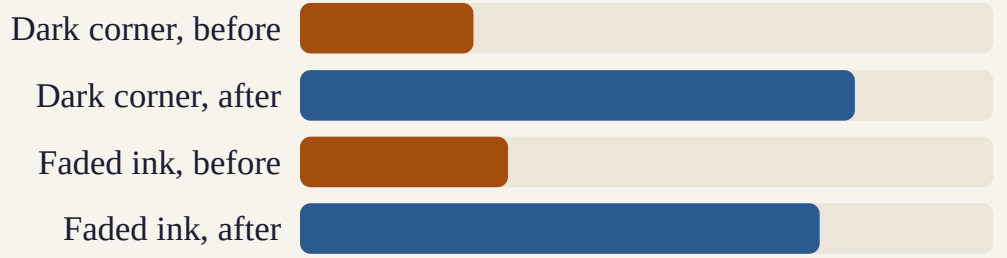
Contrast stretching spreads brightness values across the full range. **Histogram equalisation** redistributes them so details in dark and light areas both show.

DEFINITION

CLAHE equalises contrast in small tiles with a limit on amplification, which avoids blowing up noise.

DEFINITION

Background subtraction estimates the paper colour across the page (for example by heavy blurring) and removes it, leaving only ink.



PITFALL

Strong equalisation also amplifies stains and paper texture. Tune it on a sample of your worst documents.

OTHER STEPS

More preprocessing tools worth knowing

Border removal

Cut black scanner borders and punch holes that look like letters.

Line removal

Remove form lines and table rulings that touch characters.

Rescaling

Resize so text height matches what the model was trained on.

Super resolution

Neural upscaling of tiny text; helps sometimes, invents detail other times.

Watermark removal

Separate light watermarks from foreground ink by colour or intensity.

Orientation detection

Detect pages upside down or sideways before anything else.

CHOOSING STEPS

A preprocessing decision table

Symptom in your images	Try first	Then
Text tilted a few degrees	Deskew	Line based layout analysis
Photographed at an angle	Corner detection and perspective warp	Dewarping for curved pages
Shadows or uneven light	Background subtraction or CLAHE	Adaptive thresholding
Speckles and dust	Median filter	Opening, small blob removal
Faint or broken strokes	Contrast stretch	Closing or dilation
Tiny text	Rescan higher, or upscale	Model trained on small text
Pages rotated 90 or 180 degrees	Orientation classifier	Script and language detection

MEASURE IT

Prove each step earns its place

KEY IDEA

Treat every preprocessing step as a hypothesis. Run OCR with and without it on a representative labelled sample, and keep it only if error rates fall.



WORKED EXAMPLE

Illustrative ablation on a neural recogniser: deskew and contrast helped; aggressive binarisation hurt because it erased faint strokes the model could otherwise read. The bars show relative accuracy, not real benchmark figures.

PRO TIP

Different document types may need different recipes. Route receipts, forms and letters to their own preprocessing chains.

SECTION 3 RECAP

Preprocessing

REMEMBER

Takeaways

- Remove variation unrelated to text
- Adaptive thresholds beat global ones under uneven light
- Deskew and perspective fixes help nearly every engine
- Morphology repairs thin or broken strokes
- Measure every step; neural models may prefer light touch

CHECK YOURSELF

A shadow covers half of every photographed page. Which thresholding approach should you use?

ANSWER

Adaptive thresholding, or background subtraction first. A single global threshold will turn the shadowed half black.

SECTION 04

04 Layout and text detection

Finding where the text is: page regions, reading order, lines, words and tables, with classical and neural detectors.

Page segmentation **Reading order** **Connected components** **Text detectors** **Lines and words** **Tables**

THE QUESTION

Before reading, find what to read



DEFINITION

Layout analysis splits a page into regions (titles, paragraphs, tables, figures, headers, footers) and decides the order in which they should be read.

DEFINITION

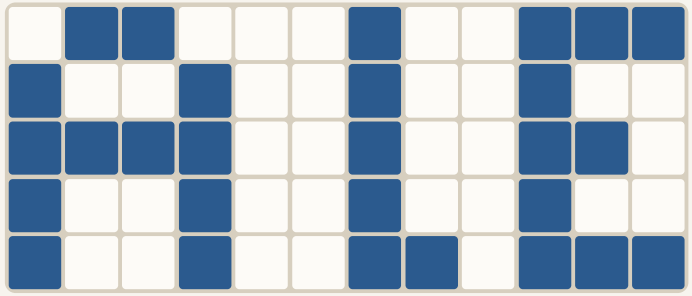
Text detection finds the exact boxes or outlines of text lines or words, so each can be passed to the recogniser.

PITFALL

Reading a two column page straight across mixes sentences from both columns into nonsense. Layout errors look like recognition errors but are not.

CLASSICAL APPROACH

Connected components



Three separate ink blobs

DEFINITION

A **connected component** is a group of ink pixels that all touch each other. On clean print, each letter is usually one component.

KEY IDEA

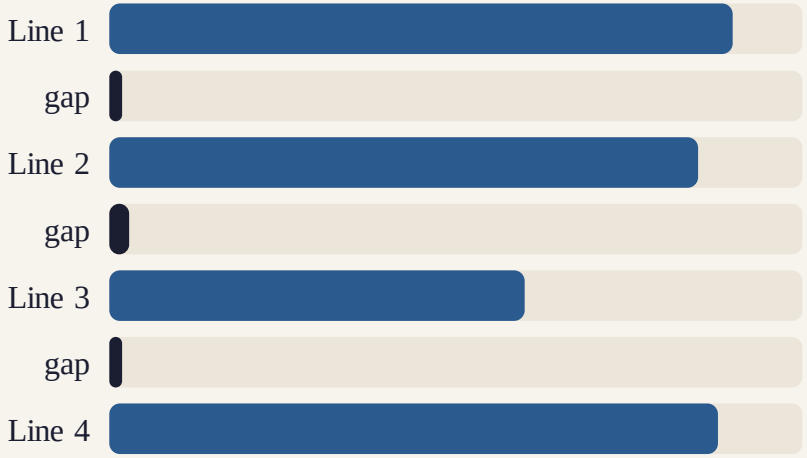
Group components that sit side by side into words, words into lines, lines into blocks. That is the classical bottom up layout method.

PITFALL

Touching letters merge into one component; broken letters split into several; letters like i and j have two parts.

CLASSICAL APPROACH

Projection profiles and whitespace



DEFINITION

A **horizontal projection profile** counts ink pixels in each row. Peaks are text lines; valleys are the gaps between lines.

DEFINITION

Whitespace analysis finds large empty rectangles, which separate columns and blocks.

USE IT WHEN

Clean, upright, printed pages with regular layouts such as books and reports.

CLASSICAL APPROACH

Top down versus bottom up

Top down

Start with the whole page; split it repeatedly at large white gaps (columns, then blocks, then lines). Recursive XY cut is the classic example. Fast; struggles with irregular layouts.

Bottom up

Start with pixels or components; merge them into words, lines and blocks. Handles irregular layouts better; sensitive to noise.

Hybrid and learned

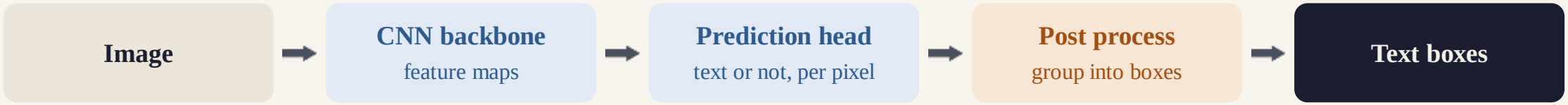
Modern systems learn region detection directly from labelled pages, then apply rules for reading order.

ANALOGY

Top down is cutting a cake into slices along obvious lines. Bottom up is sticking crumbs together until you have slices.

NEURAL TEXT DETECTION

How deep learning finds text



DEFINITION

Regression based: predict box coordinates directly (EAST style).

DEFINITION

Segmentation based: label every pixel as text or background, then group (DB, PSENet style).

DEFINITION

Character region based: find characters and the links between them (CRAFT style).

KEY IDEA

Segmentation based detectors handle curved and oddly shaped text well, which matters for scene text such as logos and signs.

NEURAL TEXT DETECTION

Boxes, rotated boxes and polygons

Simple

Axis aligned box

Four numbers: left, top, width, height. Fine for scanned documents.

Medium

Rotated box

Adds an angle. Handles tilted signs and labels.

Flexible

Polygon

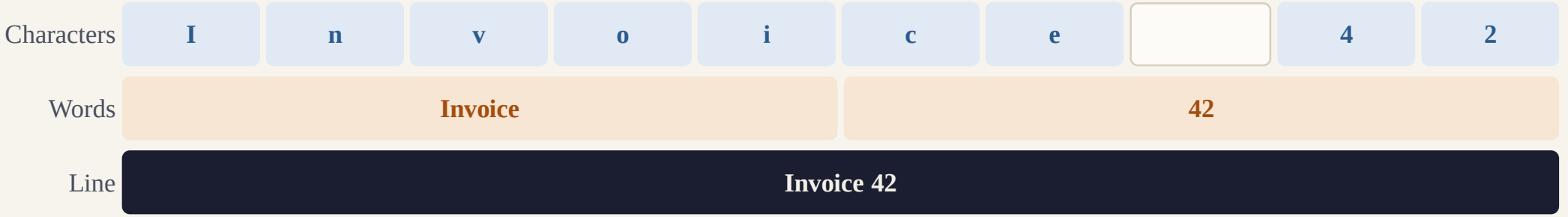
Many points tracing the outline. Handles curved text around logos and bottles.

WORKED EXAMPLE

A coffee cup with a brand name curved around it: an axis aligned box includes lots of background and cuts letters; a polygon follows the curve so the recogniser gets clean, straightened text.

LINES AND WORDS

Choosing the unit you recognise



KEY IDEA

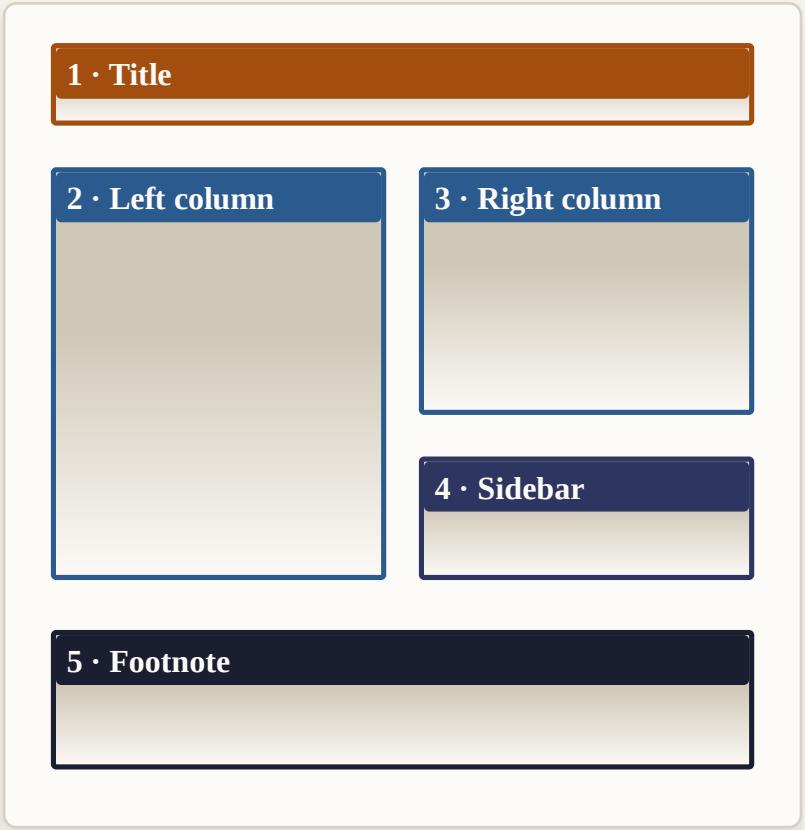
Modern recognisers usually read a **whole line** at once. They see more context and avoid errors from cutting the line into characters.

PITFALL

Character segmentation was the weak point of older OCR: touching letters were split wrongly and every later step inherited the mistake.

READING ORDER

Putting regions in the right sequence



DEFINITION

Reading order is the sequence a human would read the regions in. It is essential for any NLP that follows.

PITFALL

Different languages and documents differ: right to left scripts, vertical Japanese text, forms where labels sit beside values.

PRO TIP

Rules work for simple layouts (columns left to right, top to bottom). Learned reading order models help for magazines, newspapers and complex forms.

TABLES

Table detection and structure recognition

Item	Qty	Price
Paper	2	£4.00
Toner	1	£38.50
Total		£46.50

DEFINITION

Table detection finds where tables are on the page. **Table structure recognition** finds rows, columns, header cells and merged cells.

PITFALL

Tables without ruling lines, cells that span several columns, and tables that continue across pages are the hardest cases.

KEY IDEA

Output a real grid (rows and columns), not just text. Otherwise the numbers lose the meaning that their position gave them.

SPECIAL REGIONS

Other things layout analysis must handle

DEFINITION

Headers and footers: repeated on every page; often excluded from content.

DEFINITION

Stamps and signatures: overlap text; need detection and often separation.

DEFINITION

Checkboxes: detect them and decide ticked or empty.

DEFINITION

Barcodes and QR codes: decode with dedicated readers, not OCR.

DEFINITION

Mathematical formulae: need specialised recognisers.

DEFINITION

Handwritten annotations: may need a separate handwriting model.

PRO TIP

Build an inventory of region types in a sample of your real documents before choosing any tool. Surprises in production usually come from region types nobody listed.

SECTION 4 RECAP

Layout and text detection

REMEMBER

Takeaways

- Find regions before reading them
- Connected components and profiles suit clean print
- Neural detectors handle curved and scene text
- Read lines, not characters, where possible
- Reading order and tables decide downstream meaning

CHECK YOURSELF

Extracted text from a newsletter mixes sentences from neighbouring columns. Recognition or layout problem?

ANSWER

Layout. The characters are read correctly but in the wrong order. Fix column detection and reading order.

SECTION 05

05 Text recognition

How models turn an image of a line into a string of characters: from templates to CNNs, recurrent networks, CTC, attention and transformers.

Template matching **Features** **CRNN** **CTC** **Attention decoders** **Transformers** **Handwriting**

EARLY METHODS

Template matching and hand made features

Template matching

Compare each character image with stored images of every letter in a known font; choose the closest. Works only for fonts it has seen.

Feature extraction

Measure properties: loops, line endings, stroke directions, zoning densities. Feed them to a classifier such as an SVM.

Why they faded

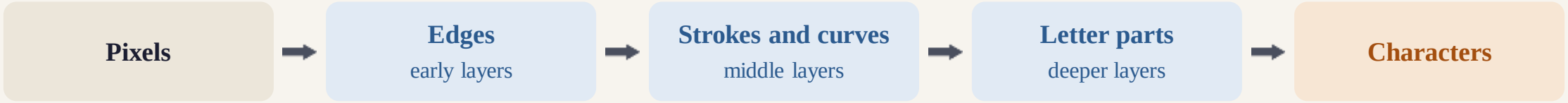
Hand designed features cannot capture the variety of real fonts, handwriting and noise.

ANALOGY

Template matching is like recognising friends only from one passport photo each. Change the lighting or haircut and you fail. Learned features are like knowing a friend from thousands of glimpses.

THE BREAKTHROUGH

Learning features with convolutional networks



DEFINITION

A **convolutional neural network (CNN)** slides small learned filters across the image. Early filters detect edges; deeper ones combine them into strokes, loops and letter shapes.

KEY IDEA

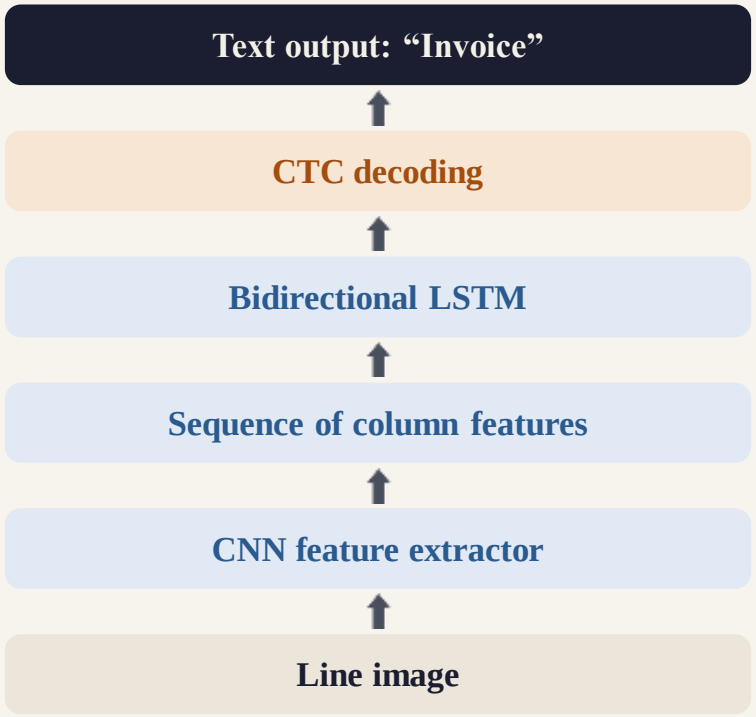
Nobody tells the network what a loop or serif is. It discovers useful patterns from labelled examples.

ANALOGY

Like learning to read as a child: first you notice lines and curves, then letters, then whole words at a glance.

THE WORKHORSE

CRNN: convolutional plus recurrent



DEFINITION

CRNN reads a line image left to right. The CNN turns it into a sequence of feature columns; a recurrent network reads that sequence with context in both directions; CTC turns the result into text.

KEY IDEA

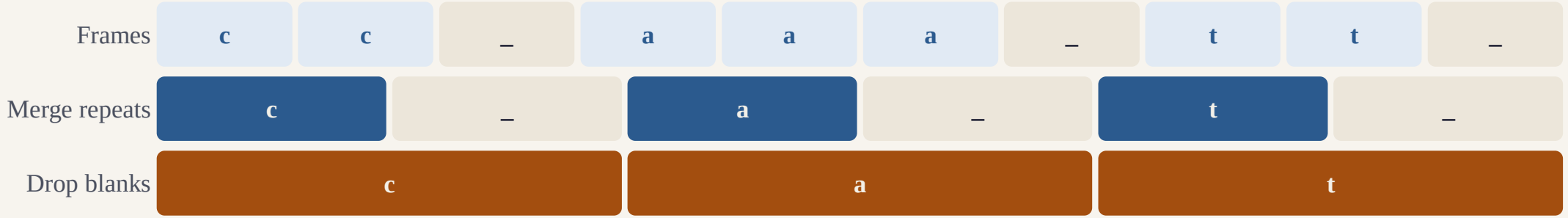
No character segmentation needed. The model learns where letters are while learning what they are.

USE IT WHEN

Printed and handwritten lines, number plates, receipts. Still a strong, efficient baseline.

CTC EXPLAINED

Connectionist temporal classification, visually



DEFINITION

The model predicts a character (or a special **blank**) for every narrow vertical slice of the line. CTC merges repeated predictions and removes blanks to produce the final text.

KEY IDEA

CTC lets you train with just the line image and its text. You never have to mark where each character begins and ends.

WORKED EXAMPLE

Why blanks matter: the word “book” needs a blank between the two o’s, otherwise merging would collapse them into one o and produce “bok”.

Decoding: greedy and beam search

Greedy decoding

Take the most likely symbol for each slice, then merge and drop blanks. Fast and usually good.

Beam search

Keep several promising candidate strings at each step and choose the best overall. Slower, more accurate.

With a language model

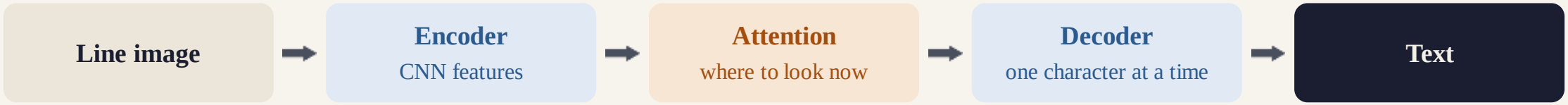
Score candidates by how plausible they are as words in the language. Fixes many look alike errors.

WORKED EXAMPLE

Greedy output: “rn0dern”. Beam search with a word list: “modern”. The visual evidence for rn and m is similar; language knowledge breaks the tie.

ATTENTION

Encoder decoder recognisers with attention



DEFINITION

The decoder writes the text one character at a time. Before each character, **attention** chooses which part of the image to focus on.

ANALOGY

Like reading with your finger under the line: at each step your finger points to the next letter.

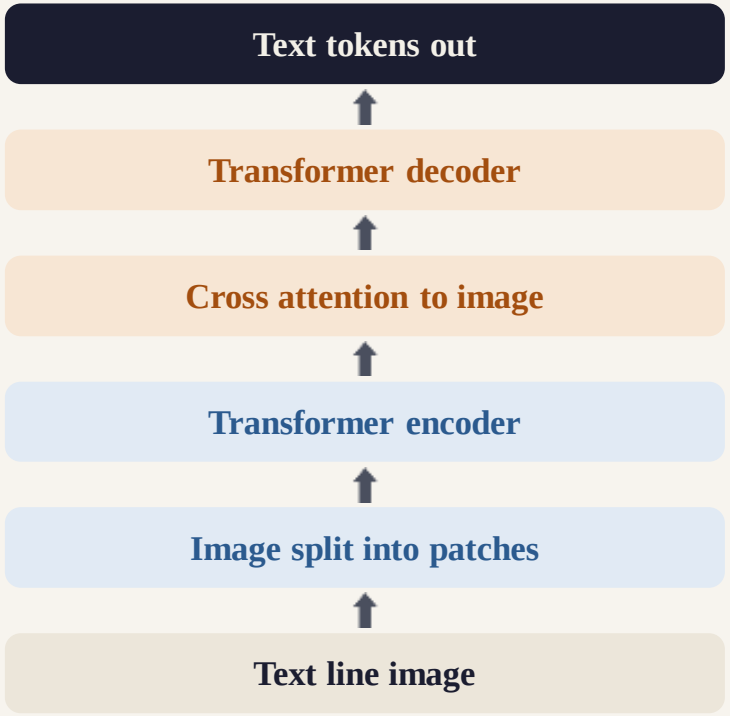
USE IT WHEN

Curved scene text, irregular spacing, text where reading direction is not simply left to right.

PITFALL

Attention decoders can drift, repeating or skipping characters on long lines. CTC is often more stable for long document lines.

Transformer based OCR



DEFINITION

Transformer OCR models (TrOCR is a well known example) split the image into patches, encode them with a vision transformer, and generate text with a language model style decoder.

KEY IDEA

Because the decoder is pretrained on huge amounts of text, it brings strong language knowledge: it knows what words and sentences usually look like.

PITFALL

Larger and slower than CRNN. Strong language priors can also “correct” text that was actually unusual, such as codes and names.

COMPARISON

Recogniser families side by side

Family	Strength	Weakness	Good for
Template and features	Simple, explainable	Brittle with new fonts	Fixed fonts, legacy systems
CRNN with CTC	Fast, stable, small	Weaker language knowledge	Documents, receipts, plates
Attention decoder	Handles irregular text	Can drift on long lines	Scene text
Transformer OCR	Strong accuracy and context	Heavy compute	Handwriting, hard print
Vision language model	Reads whole pages, follows instructions	Costly, can hallucinate	Complex documents, Q and A

HANDWRITING

Handwritten text recognition

WARNING SIGNS

Every writer forms letters differently

WARNING SIGNS

Letters join in cursive

WARNING SIGNS

Slant, size and spacing vary

WARNING SIGNS

Crossings out and insertions

KEY IDEA

Recognise whole lines with CTC or transformer models, trained on large collections of real handwriting from many writers.

PRO TIP

Fine tune on a few hundred lines from your own documents and writers. Gains on a specific archive or form type can be large.

IN THE WILD

Heritage projects transcribe centuries of handwritten records by having experts correct a model’s output, then retraining on the corrections. Each round improves the model.

SCENE TEXT

Reading text in the wild

Signs and shopfronts

Perspective, decorative fonts, reflections.

Product packaging

Curved surfaces, small print, glossy plastic.

KEY IDEA

Scene text systems use strong detectors (often polygon based), rectification to straighten text, and recognisers trained on huge synthetic datasets.

Number plates

Fixed formats, motion blur, night lighting.

Video frames

Low resolution, but many frames to combine.

PRO TIP

In video, read several frames and vote. Agreement across frames is a powerful confidence signal.

TRAINING DATA

Where recognition models learn from

Real labelled data

Images with human transcriptions. Best quality, costly to collect.

Synthetic data

Render text in many fonts, sizes, backgrounds and distortions. Unlimited and free of labelling cost.

Augmentation

Blur, rotate, add noise, change contrast on real images to simulate harder conditions.

PITFALL

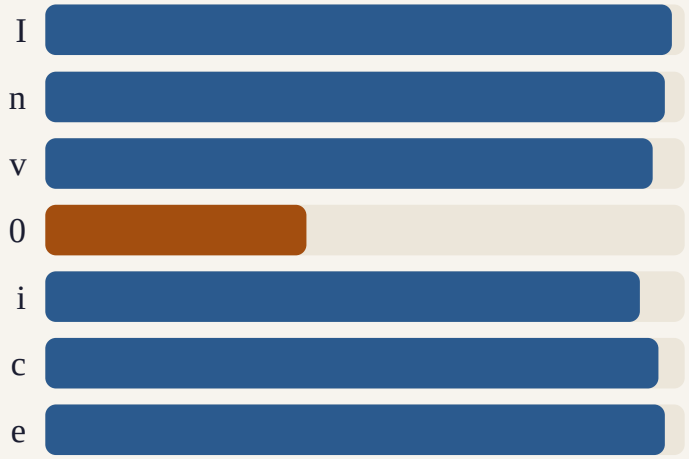
Models trained only on synthetic text can fail on real scans because real noise looks different. Always validate on real documents.

RULE OF THUMB

A common recipe: pretrain on large synthetic data, then fine tune on a smaller set of real, in domain examples.

CONFIDENCE

Knowing when the model is unsure



DEFINITION

Recognisers can output a **confidence** for each character, word or line, based on how strongly the model preferred its answer.

KEY IDEA

Low confidence characters are where to focus checks: send them to correction rules, a second model, or a human reviewer.

PITFALL

Raw confidences are often poorly calibrated. Check how often 90% confident words are actually right before using thresholds.

SECTION 5 RECAP

Text recognition

REMEMBER

Takeaways

- CNNs learn letter shapes from data
- CRNN plus CTC reads lines without segmentation
- Attention and transformers add context and flexibility
- Handwriting and scene text need specialised data
- Use confidence to route uncertain text for checking

CHECK YOURSELF

Why does CTC need a blank symbol?

ANSWER

To separate repeated letters and mark slices with no character. Without blanks, double letters like oo would collapse into one.

06

SECTION 06

Postprocessing and correction

Using dictionaries, language models, formats and business rules to catch and fix recognition errors.

Error types | Dictionaries | Language models | Field rules | Confidence routing | Human review

ERROR TYPES

The four ways OCR gets text wrong

S

Substitution

One character read as another: “tota1” for “total”, “O” for “0”.

D

Deletion

A character is missed: “Invce” for “Invoice”. Common with faint ink.

I

Insertion

An extra character appears, often from noise: “In.voice”.

W

Segmentation

Words merged or split: “invoicetotal” or “in voice”.

KEY IDEA

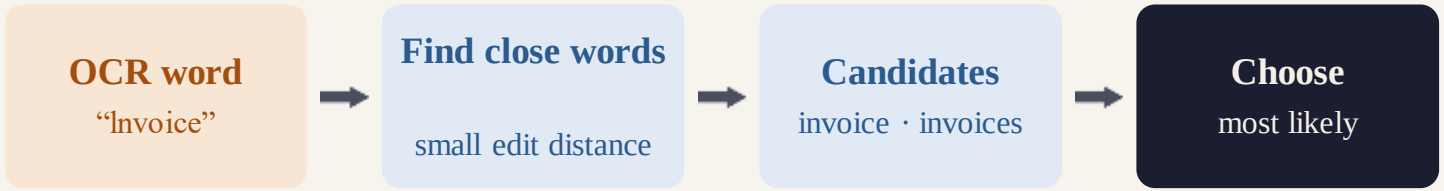
Postprocessing works best when you know which error types dominate. Count them on a labelled sample before choosing fixes.

PRO TIP

Keep a list of the most common confusions in your own data, such as 0 and O or rn and m. Targeted rules for these often beat generic correction.

DICTIONARIES

Lexicon based correction



DEFINITION

Edit distance counts the fewest single character changes (insert, delete, substitute) needed to turn one word into another. “lnvoice” is one change from “invoice”.

PITFALL

Names, codes, postcodes and product numbers are not in any dictionary. Forcing them to the nearest real word creates new errors.

PRO TIP

Use domain dictionaries: your product catalogue, customer names, supplier list, medical vocabulary. They are far more useful than a general English word list.

RULE OF THUMB

Correct only when the OCR word is not a valid entry **and** a candidate is very close **and** confidence was low.

LANGUAGE MODELS

Using context to correct

Please | remit | payment | by | the | due | dale

date?

KEY IDEA

“dale” is a real word, so a dictionary will not flag it. A language model notices that “by the due dale” is very unlikely and “by the due date” is very likely.

DEFINITION

A **language model** estimates how likely a sequence of words is. It can rank OCR candidates by how natural they sound in context.

WORKED EXAMPLE

Modern pipelines pass OCR text with its uncertain spans to a language model and ask for the most plausible correction, with instructions not to change names, numbers or codes.

PITFALL

Large language models may rewrite text to sound fluent, silently changing meaning or inventing words. Restrict them to flagged spans and log every change.

FIELD RULES

Formats and checksums catch impossible values

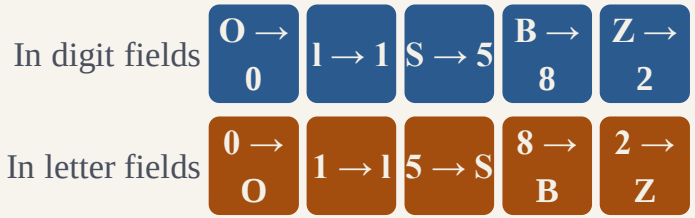
Field	Rule	Example catch
Date	Valid calendar date in expected range	31/02/2026 cannot exist
UK postcode	Matches postcode pattern	SWIA 1AA should be SW1A 1AA
Currency	Digits, one decimal point, two decimals	£1O5.0O becomes £105.00
IBAN and card numbers	Built in check digits	One wrong digit fails the check
Invoice totals	Lines add up to subtotal; tax matches rate	Misread digit breaks the sum
VAT number	Country format and check digits	Extra digit detected

KEY IDEA

Business rules turn silent errors into detected errors. Cross field checks, such as totals that must add up, are among the most powerful OCR validators.

CHARACTER CONFUSIONS

Context specific substitution



DEFINITION

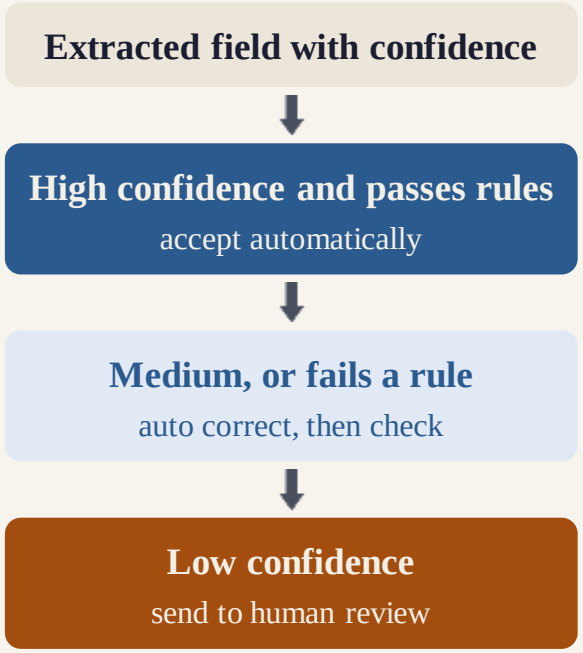
When you know a field should contain only digits (an account number) or only letters (a surname), swap the common look alike characters into the allowed set.

PITFALL

Mixed fields such as postcodes, vehicle registrations and product codes have **position specific** rules: the first two characters may be letters, the next digits. Encode the pattern, not a blanket swap.

ROUTING

Confidence based routing to humans



KEY IDEA

Not every field needs a human, and not every field can skip one. Route by confidence and by business risk.

PRO TIP

Set thresholds per field. A wrong postcode is an annoyance; a wrong bank account number sends money to a stranger.

IN THE WILD

An accounts payable team reviewed every invoice by hand. After routing, people checked only flagged fields, a small share of the total, with no rise in errors reaching payment.

HUMAN IN THE LOOP

Designing review screens

- Show the image crop next to the extracted value
- Highlight low confidence characters
- Pre fill with the best guess
- Keyboard shortcuts for accept and next
- Explain why a field was flagged
- Log every correction

KEY IDEA

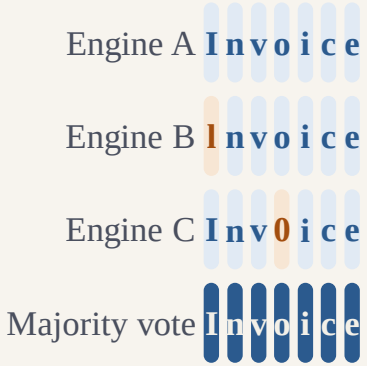
Every human correction is a free training label. Feed corrections back to retrain and to tune thresholds.

PITFALL

If reviewers almost always accept the suggestion, they may stop looking carefully. Insert occasional known test items to check attention.

MERGING ENGINES

Ensembles and voting



DEFINITION

Run two or three different engines (or the same engine on differently preprocessed images), align their outputs and vote character by character.

PRO TIP

Disagreement between engines is an excellent signal for human review, often better than any single engine’s own confidence.

SECTION 6 RECAP

Postprocessing and correction

REMEMBER

Takeaways

- Know your dominant error types
- Domain dictionaries beat general ones
- Language models fix real word errors in context
- Format rules and checksums catch impossible values
- Route by confidence and risk; learn from corrections

CHECK YOURSELF

OCR reads a surname as “Sm1th”. What should a postprocessor do?

ANSWER

In a letters only field, swap 1 to l or i and check against a names list or the customer database. Flag if still uncertain.

SECTION 07

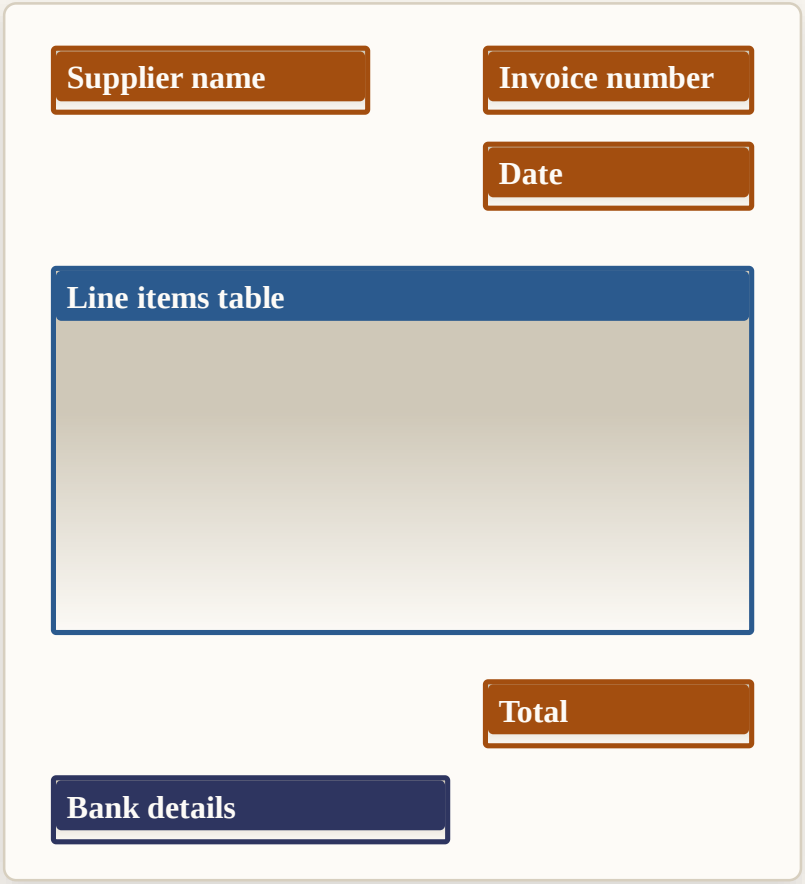
07 Document AI

Beyond plain text: extracting fields, key values, tables and meaning from documents using layout aware and OCR free models.

- Key value extraction
- Templates
- Layout aware models
- OCR free models
- Vision language models
- Classification

THE GOAL

From text to structured data



Field	Value
supplier	Acme Office Ltd
invoice_no	INV 20431
date	14 Oct 2026
total	£1,284.00
lines	6 items

KEY IDEA

Businesses rarely want raw text. They want **fields** in a database, ready to approve, pay, file or analyse.

APPROACH 1

Templates and zonal OCR

DEFINITION

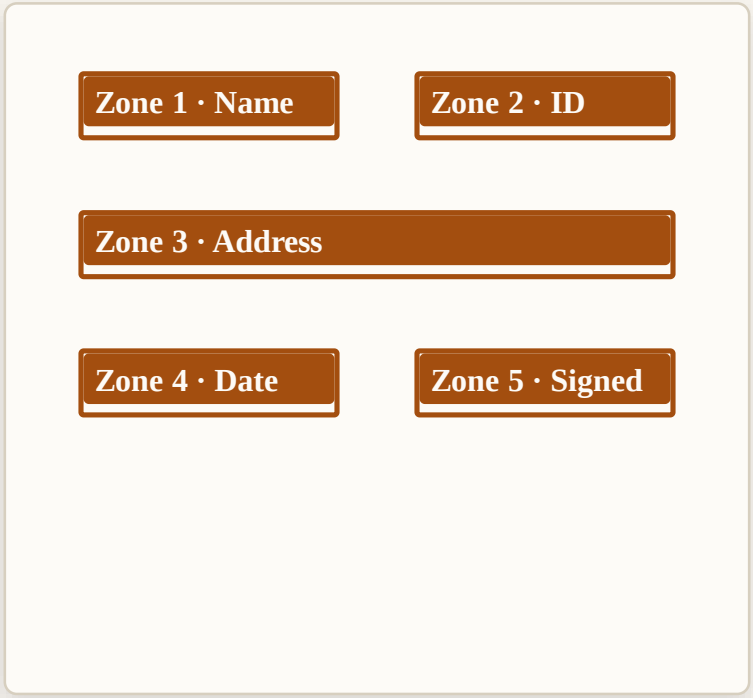
For a fixed form, define zones by position: “the total is always in this rectangle”. Run OCR only inside each zone.

USE IT WHEN

One layout that never changes: a government form, an internal timesheet.

PITFALL

Breaks as soon as the layout shifts, a supplier changes their invoice design, or a scan is offset. Maintaining hundreds of templates is costly.



APPROACH 2

Rules over OCR text

PYTHON · REGULAR EXPRESSION

```
import re
pattern = r"Invoice\s*(No|Number)[:\s]*([\w ]+)"
match = re.search(pattern, text, re.IGNORECASE)
invoice_no = match.group(2).strip() if match else None
```

USE IT WHEN

Moderate layout variation, clear labels, quick wins without training data.

DEFINITION

Search the recognised text for anchor words (“Invoice No”, “Total”) and grab the value next to them using patterns.

PITFALL

Labels vary (“Amount due”, “Balance”, “To pay”), values may sit below rather than beside labels, and OCR errors break patterns.

APPROACH 3

Learning extraction as token labelling

Invoice	No:	INV	20431	Total	due:	£1,284.00
		B NUMBER	*I NUMBER*			*B TOTAL*

DEFINITION

Treat extraction like named entity recognition: label each OCR word as the beginning (B) or inside (I) of a field, or outside any field.

KEY IDEA

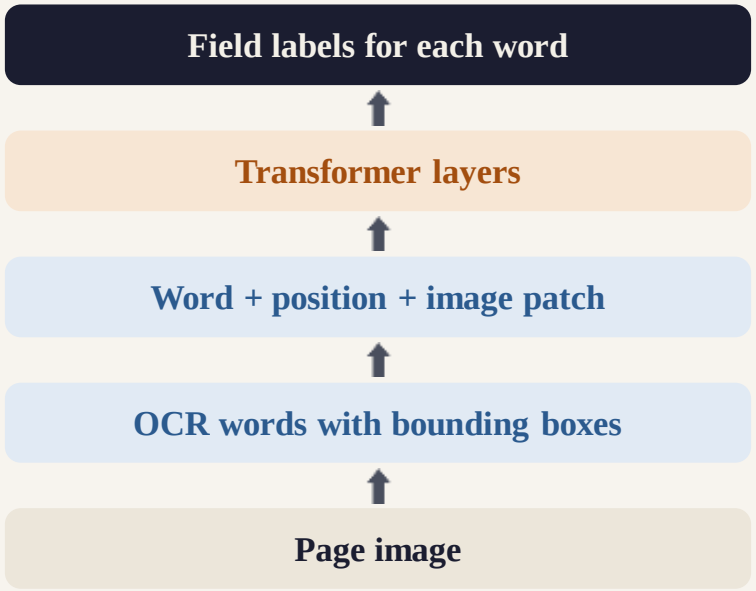
The model learns the many ways a field can appear, instead of you writing a rule for each one.

PRO TIP

This turns document extraction into an NLP task. Part B explains token labelling in depth; here the words come with page positions too.

LAYOUT AWARE MODELS

Reading words and positions together



DEFINITION

Layout aware models (the LayoutLM family is the best known) take each OCR word together with its **box coordinates** and often the image region, so the model knows where on the page every word sits.

ANALOGY

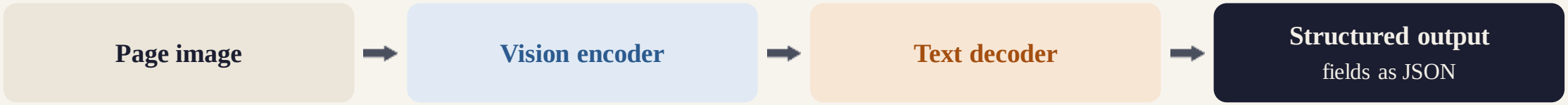
A person finds the total partly by reading “Total” and partly by knowing it sits at the bottom right. These models learn both cues.

USE IT WHEN

Invoices, receipts, forms and contracts with many layouts.

OCR FREE MODELS

Reading documents without a separate OCR step



DEFINITION

OCR free models (Donut is a well known example) go straight from the page image to the desired output, such as a set of fields, without producing intermediate OCR text.

KEY IDEA

No OCR errors to propagate and a simpler pipeline. One model learns reading and understanding together.

PITFALL

Needs more training data and compute; harder to tell whether an error came from reading or understanding; no word boxes for highlighting evidence unless added.

USE IT WHEN

Large volumes of a document type with plenty of labelled examples.

Asking questions of a page

Prompt: What is the invoice total and due date?

Answer: £1,284.00, due 14 November 2026

KEY IDEA

Very fast to prototype: no training, many document types, natural language instructions.

DEFINITION

Vision language models accept an image and a text instruction and answer in text. They can read, interpret layout, and follow instructions such as “return JSON with these fields”.

PITFALL

They can **hallucinate** plausible values that are not on the page, and cost more per page. Validate outputs with rules and require evidence such as the source text.

CHOOSING AN APPROACH

Document AI options compared

Approach	Data needed	Flexibility	Cost to run	Best fit
Templates	None	Very low	Very low	One fixed form
Rules on OCR text	None	Low	Low	Clear labels, few layouts
Token labelling	Hundreds of pages	Medium	Low	Repeated document types
Layout aware	Hundreds to thousands	High	Medium	Many layouts, high volume
OCR free	Thousands	High	Medium	Very high volume, one task
Vision language	Few or none	Very high	High	Variety, low volume, prototypes

RULE OF THUMB

Start with a vision language model to learn what is possible and to create labels, then move high volume document types to cheaper specialised models if costs demand it.

CLASSIFICATION

Document classification and splitting

Upload
one PDF, 40 pages

→

Split
find document boundaries

→

Classify
invoice, ID, letter

→

Route
right extractor

DEFINITION

Document classification decides what type each document is, from text, layout or image features. **Splitting** finds where one document ends and the next begins inside a bundle.

PITFALL

Mis classification sends a document to the wrong extractor, which then confidently extracts nonsense. Monitor classification accuracy separately.

PRO TIP

Visual cues alone (logos, layout) often classify documents well even before OCR. Combine image and text signals for robustness.

EVIDENCE

Always link extracted values to their source

KEY IDEA

Every extracted field should carry its **evidence**: page number, bounding box and the exact OCR text it came from.

WHY IT MATTERS

Reviewers can verify a value in one glance. Auditors can trace decisions. Errors can be diagnosed as reading or understanding failures.



SECTION 7 RECAP

Document AI

REMEMBER

Takeaways

- Businesses need fields, not raw text
- Templates and rules work for stable layouts
- Layout aware models use words plus positions
- OCR free and vision language models read pages directly
- Always keep evidence for each extracted value

CHECK YOURSELF

You receive invoices from 3,000 suppliers with different layouts. Templates or a learned model?

ANSWER

A learned layout aware model or a vision language model. Templates for 3,000 layouts would be unmaintainable.

SECTION 08

08 Evaluating OCR

Measuring OCR quality honestly: character and word error rates, field accuracy, detection metrics and error analysis.

Ground truth **CER** **WER** **Field accuracy** **Detection metrics** **Error analysis**

GROUND TRUTH

You need a reference to measure against

DEFINITION

Ground truth is the correct transcription of each test image, typed and checked by people, together with correct boxes and fields where needed.

PRO TIP

Double key a sample: two people transcribe independently and a third resolves differences. It reveals how ambiguous your documents really are.

- Representative of real traffic, not cherry picked
- Includes the worst realistic images
- Covers every document type and language
- Agreed rules for spaces, punctuation and case
- Kept separate from training data

CHARACTER ERROR RATE

CER: how many characters are wrong



DEFINITION

CER counts the substitutions, deletions and insertions needed to turn the OCR output into the reference, divided by the number of reference characters. Lower is better.

WORKED EXAMPLE

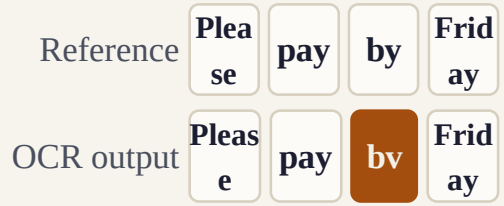
Above: one substitution (I read as l) and one deletion (missing e) out of ten reference characters gives a CER of 20 per cent.

PITFALL

Normalisation choices change CER a lot: case, double spaces, curly versus straight quotes. Fix and document them before comparing systems.

WORD ERROR RATE

WER: how many words are wrong



DEFINITION

WER works like CER but on whole words. One wrong character makes the whole word wrong.

KEY IDEA

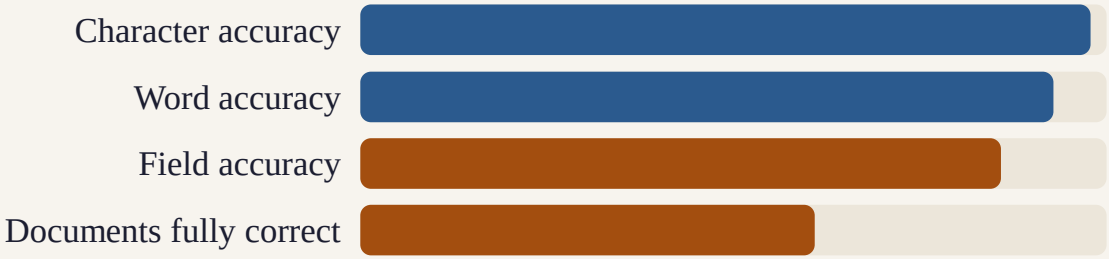
WER is usually several times higher than CER, and is closer to what search and NLP experience. A misspelt word may never match a search query.

WORKED EXAMPLE

Here one character error (y read as v) gives a CER of a few per cent but a WER of 25 per cent: one word in four is wrong.

TASK METRICS

Field level and document level accuracy



KEY IDEA

Small character error rates compound. A document with many fields is only fully correct if every field is right.

RULE OF THUMB

Report the metric the business feels: share of invoices that need no correction, or share of fields entered correctly.

DETECTION METRICS

Measuring how well text is found

DEFINITION

Intersection over union (IoU): overlap between a predicted box and the true box, relative to their combined area.

DEFINITION

A predicted box counts as a hit when IoU passes a threshold such as one half. Then compute **precision** (share of predicted boxes that are real text) and **recall** (share of real text that was found).

PITFALL

A missed line is invisible in CER if you only score lines that were detected. Always score end to end: from page image to final text, including missed regions.

PRO TIP

For reading order, compare the sequence of regions with the reference order, not just the text inside them.

ERROR ANALYSIS

Slice your errors to find the cause

Slice	Question it answers
Document type	Are receipts worse than letters?
Capture source	Are phone photos worse than scans?
Image quality bucket	How fast does accuracy fall with blur or low resolution?
Region type	Are tables, stamps or handwriting the main problem?
Character class	Are digits worse than letters?
Language and script	Is one language far behind?

PRO TIP

Build a confusion table of characters (which characters are read as which). It points directly to fixes such as field rules or targeted training data.

BENCHMARKS

Public datasets and their limits

Printed documents

Collections of scanned pages and receipts used to compare engines and layout tools.

Scene text

Competition datasets of photos with signs and labels, often from the ICDAR series.

Handwriting

Datasets of handwritten lines and pages from many writers, such as the IAM collection.

Forms and receipts

Datasets labelled for key value extraction such as FUNSD, SROIE and CORD style collections.

PITFALL

Benchmark scores rarely predict performance on your documents. Always build a small, honest test set from your own traffic.

SECTION 8 RECAP

Evaluating OCR

REMEMBER

Takeaways

- Build checked, representative ground truth
- CER for characters, WER for words
- Field and document accuracy match business value
- Score detection and reading order end to end
- Slice errors to find causes

CHECK YOURSELF

CER is 2% but a quarter of invoices need correction. Contradiction?

ANSWER

No. Errors spread across many fields per invoice, so even a low CER leaves many documents with at least one wrong field.

SECTION 09

09 OCR tools and pipelines

The main open source engines and cloud services, and how to assemble them into a robust production pipeline.

Tesseract | PaddleOCR | EasyOCR | TrOCR | Cloud APIs | Pipeline design

LANDSCAPE

The OCR tool landscape

Tool	Type	Strength	Consider when
Tesseract	Open source engine	Mature, many languages, runs anywhere	Clean printed documents, offline use
PaddleOCR	Open source toolkit	Strong detection and recognition models	Varied layouts, multilingual, tables
EasyOCR	Open source library	Simple to use, many scripts	Quick prototypes, scene text
docTR and similar	Open source deep learning	End to end detection plus recognition	Python pipelines on GPUs
TrOCR and similar	Transformer models	Handwriting and hard print	Line level accuracy matters most
Cloud document APIs	Managed services	Layout, tables, forms, handwriting at scale	No ML team, fast time to value

☆ REMEMBER

Tools evolve quickly. Check current documentation, licences and data handling terms before choosing.

TESSERACT

Using Tesseract well

PYTHON · PYTESSERACT

```
import pytesseract
from PIL import Image

img = Image.open("page.png")
text = pytesseract.image_to_string(img, lang="eng")

# word boxes and confidences
data = pytesseract.image_to_data(
    img, output_type=pytesseract.Output.DICT)
words = [(w, c) for w, c in
         zip(data["text"], data["conf"]) if w.strip()]
```

PRO TIP

- Feed it clean, deskewed, high contrast images
- Set the page segmentation mode to match your layout
- Install the right language packs
- Use word confidences to route review

PITFALL

Tesseract struggles with photos, handwriting and complex tables without help. Pair it with good preprocessing and layout detection.

DEEP LEARNING TOOLKITS

Detection plus recognition in a few lines

PYTHON · PADDLEOCR (API VARIES BY VERSION)

```
from paddleocr import PaddleOCR

ocr = PaddleOCR(lang="en")
result = ocr.ocr("receipt.jpg")
for box, (text, score) in result[0]:
    print(text, round(score, 2))
```

KEY IDEA

Toolkits like PaddleOCR return each text region’s polygon, its text, and a confidence score, ready for layout reconstruction and routing.

PRO TIP

Pin library versions. OCR toolkit interfaces change between releases and silently break pipelines.

CLOUD SERVICES

Managed document AI services

General OCR

Text, lines and words with boxes from images and PDFs.

Forms and tables

Key value pairs and table structure.

USE IT WHEN

You need many document types quickly, have limited ML staff, and can send data to a provider.

Prebuilt models

Invoices, receipts, IDs and other common documents.

Custom models

Train on your own labelled documents.

PITFALL

Check data residency, retention and privacy terms, especially for health, financial and identity documents under UK GDPR.

BUILD OR BUY

A decision guide

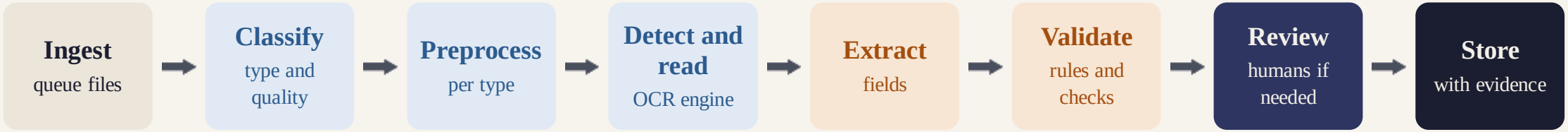
Factor	Favours open source	Favours cloud service
Data sensitivity	Must stay on premises	Provider terms acceptable
Volume	Very high, cost per page matters	Low to moderate
Team skills	ML engineers available	Small or no ML team
Customisation	Unusual documents or scripts	Common document types
Time to value	Months acceptable	Weeks needed
Control	Need to inspect and tune everything	Managed updates acceptable

PRO TIP

Benchmark two or three options on the same 200 of your own documents, using field accuracy and cost per page. Decide with evidence, not marketing claims.

PIPELINE DESIGN

A production OCR pipeline



KEY IDEA

Make each stage a separate, observable step with its own logs and metrics. When accuracy drops, you can see exactly where.

PRO TIP

Save intermediate outputs (preprocessed image, boxes, raw text) for a sample of documents. They are invaluable for debugging.

PIPELINE DESIGN

Engineering details that matter

DEFINITION

Idempotency: reprocessing the same file gives the same result and no duplicates.

DEFINITION

Retries and dead letter queues: failed pages are retried, then parked for inspection.

DEFINITION

Versioning: record model, rules and preprocessing versions with every result.

DEFINITION

Throughput: batch pages on GPUs; scale workers with queue length.

DEFINITION

Monitoring: confidence distributions, rule failures, review rates per document type.

DEFINITION

Security: encrypt files at rest and in transit; limit who can see raw documents.

SECTION 9 RECAP

OCR tools and pipelines

REMEMBER

Takeaways

- Tesseract for clean print; deep toolkits for variety
- Cloud services trade control for speed
- Benchmark on your own documents
- Design observable, versioned pipeline stages
- Plan for privacy and security from day one

CHECK YOURSELF

Health records must never leave your servers. Which option fits?

ANSWER

Open source engines deployed on premises or in your own controlled environment, or a provider deployment that meets your data residency requirements.

PART B

Part B · Natural Language Processing

Nine sections on how machines work with language: foundations, tokenisation, representations, classical tasks, sequence models, transformers, large language models, applications and evaluation.

SECTION 10

10 NLP foundations

What natural language processing is, the layers of language, why ambiguity makes it hard, and how the field evolved.

Definition | Levels of language | Ambiguity | History | Task map

DEFINITION

Natural language processing

DEFINITION

Definition

NLP is the field of computer science and AI that enables machines to process, understand and generate human language, in text or speech.

IN PLAIN WORDS

In plain words

Teaching computers to do useful things with words: find, sort, extract, summarise, translate, answer and write.

Text in
emails, documents, OCR output

→

Understand
classify, extract, link

→

Reason
compare, answer, decide

→

Generate
summaries, replies, translations

→

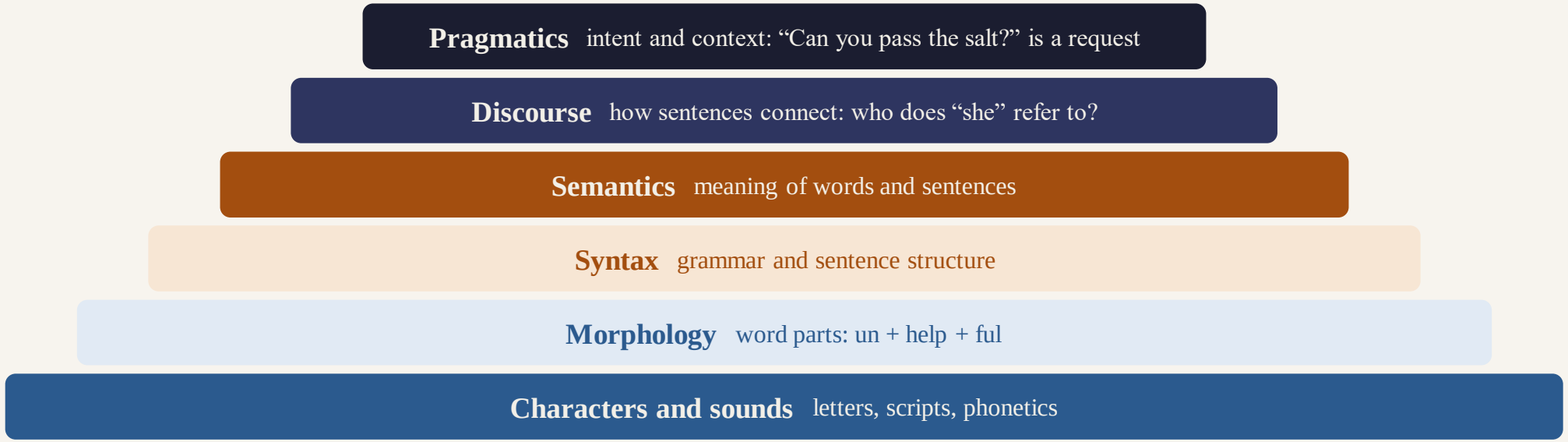
Value
faster, cheaper, better decisions

ANALOGY

If OCR gives a computer eyes for print, NLP gives it a basic grasp of language: grammar, vocabulary and some sense of meaning.

LAYERS OF LANGUAGE

Language has many levels



KEY IDEA

Classical NLP built separate tools for each level. Modern neural models learn all levels together from huge amounts of text, though the levels remain useful for diagnosing errors.

AMBIGUITY

Why language is hard for machines

WORKED EXAMPLE

Lexical: “bank” can mean a river edge or a financial institution.

WORKED EXAMPLE

Syntactic: “I saw the man with the telescope.” Who has the telescope?

WORKED EXAMPLE

Referential: “The trophy did not fit in the case because it was too big.” What was too big?

WORKED EXAMPLE

Pragmatic: “Nice weather!” said during a storm is sarcasm.

WORKED EXAMPLE

Scope: “Every student read a book.” The same book or different ones?

WORKED EXAMPLE

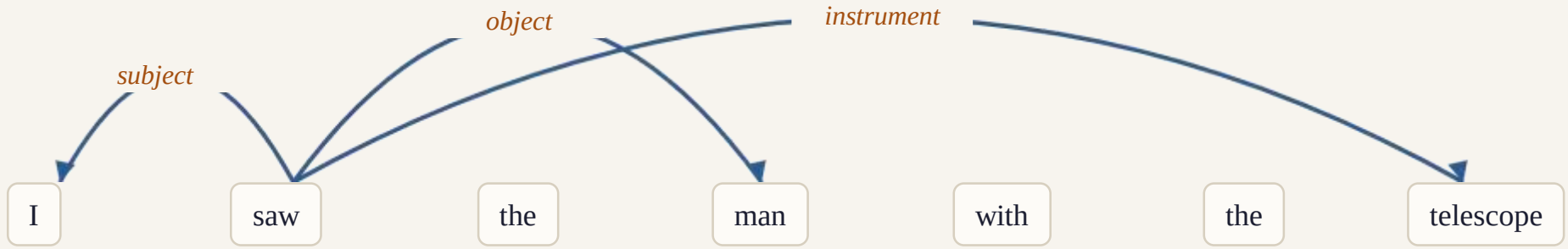
Ellipsis: “I will if you will.” Will what?

KEY IDEA

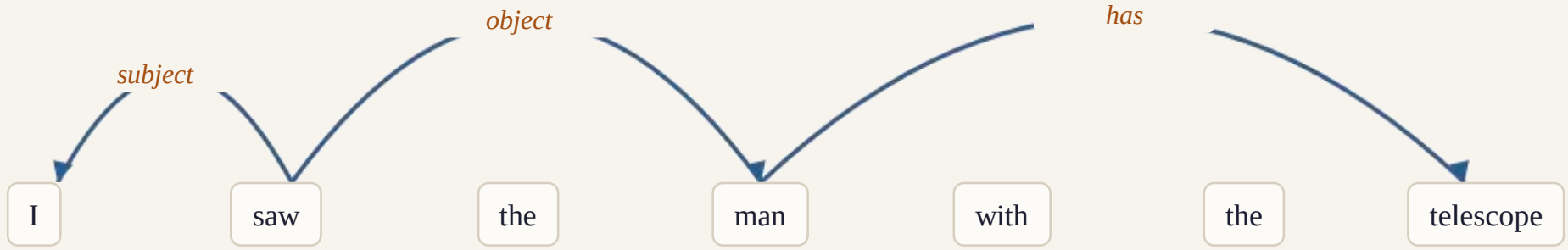
Humans resolve ambiguity effortlessly with world knowledge and context. Much of NLP history is the search for ways to give machines that context.

AMBIGUITY

One sentence, two structures



Reading 1: I used the telescope to see the man.



Reading 2: the man I saw was holding a telescope.

HISTORY

Seventy years of NLP

1950s to 1980s

Rules

Hand written grammars and dictionaries; early machine translation; ELIZA style chatbots.

1990s to 2000s

Statistics

Probabilities learned from text corpora: n gram models, hidden Markov models, statistical translation.

2013

Word embeddings

Word2vec and GloVe represent words as vectors that capture meaning.

2014 to 2017

Neural sequences

RNNs, LSTMs, sequence to sequence and attention.

2017 onwards

Transformers and LLMs

Attention is all you need; BERT, GPT and large language models.

KEY IDEA

Each era kept the useful parts of the previous one. Tokenisation, evaluation habits and many classical tasks remain essential today.

PRO TIP

Classical methods are still the right answer for many small, well defined problems: they are cheap, fast and explainable.

TASK MAP

The main families of NLP tasks

Classification

Assign labels to text: spam, sentiment, topic, intent, urgency.

Extraction

Pull out entities, relations, dates, amounts, clauses.

Generation

Write summaries, replies, translations, reports.

Matching and retrieval

Find similar documents, answer search queries, deduplicate.

Question answering

Answer questions from a document or a knowledge base.

Structure prediction

Tag parts of speech, parse grammar, segment sentences.

RULES VERSUS LEARNING

Three ways to build an NLP system

Approach	How it works	Strength	Weakness
Rules	Experts write patterns and dictionaries	Precise, explainable, no data needed	Brittle; costly to maintain
Classical ML	Features plus a trained classifier	Cheap, fast, decent accuracy	Needs feature engineering and labels
Deep learning	Models learn features from data	High accuracy, handles nuance	Needs data and compute
LLM prompting	Instruct a pretrained model	Little or no labelled data	Cost, latency, hallucination risk

RULE OF THUMB

Choose the simplest approach that meets the accuracy, cost and explainability your use case needs. A regular expression that is right 99.9% of the time beats any model for a fixed code format.

DATA

Corpora: the fuel of NLP

DEFINITION

A **corpus** (plural corpora) is a collection of text used to train or evaluate NLP systems: emails, articles, transcripts, OCR output, web pages.

KEY IDEA

Models learn the language of their corpus. A model trained on news articles will stumble on medical notes or text messages.

- Is it representative of real inputs?
- Is it labelled consistently?
- Is personal data handled lawfully?
- Is it free of duplicates and leakage?
- Does it cover all languages and dialects you serve?

SECTION 10 RECAP

NLP foundations

REMEMBER

Takeaways

- NLP processes, understands and generates language
- Language has layers from characters to pragmatics
- Ambiguity is the central difficulty
- The field moved from rules to statistics to neural models
- Simple methods still win for narrow problems

CHECK YOURSELF

"The chicken is ready to eat." What kind of ambiguity is this?

ANSWER

Syntactic and semantic: the chicken may be the food or the one about to eat. Context decides.

11

SECTION 11

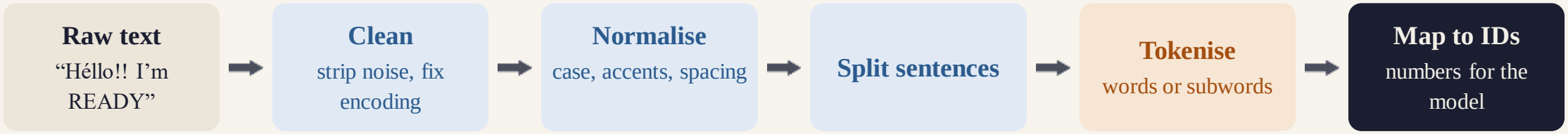
Text preprocessing and tokenisation

Turning raw strings into clean, consistent units that models can count, compare and learn from.

Normalisation · Unicode · Sentences · Word tokens · Subwords · Stemming and lemmas · Stopwords

THE PIPELINE

From raw text to model input



KEY IDEA

Every model works with **numbers**, not letters. Preprocessing and tokenisation decide exactly which pieces of text become which numbers.

PITFALL

Preprocessing at training time and at prediction time must be identical. A tiny difference, such as lowercasing in only one of them, silently ruins accuracy.

CLEANING

Typical cleaning steps

Step	Example before	Example after
Remove markup	Hello	Hello
Fix encoding errors	CafÃ©	Café
Collapse whitespace	too many spaces	too many spaces
Normalise quotes	“smart” and “straight”	one consistent style
Expand or keep contractions	don’t	do not, or keep as is
Mask personal data	Call 07700 900123	Call [PHONE]

PRO TIP

For OCR output, also join words broken across line ends, remove repeated headers and footers, and keep page numbers as metadata rather than text.

NORMALISATION

Case, accents and Unicode

DEFINITION

Unicode gives every character in every script a number. Some characters can be written in more than one way, such as an accented letter as one symbol or as a letter plus an accent mark.

DEFINITION

Unicode normalisation (NFC, NFKC and friends) converts these variants into one standard form so identical looking text compares as equal.

Choice	When it helps	When it hurts
Lowercase	Search, small data	Names, acronyms: US vs us
Strip accents	Noisy user input	Languages where accents change meaning
NFKC	Full width digits, ligatures	When exact form matters

PITFALL

Modern pretrained models come with their own tokenisers that expect raw text with case. Lowercasing before them removes information they were trained to use.

SENTENCES

Sentence segmentation is not trivial

Dr.	Smith	paid	£3.50	at	9 a.m.	Then	left.
not an end			*not an end*		*maybe an end*		*end*

KEY IDEA

Full stops also appear in abbreviations, decimals, times, email addresses and initials. Good sentence splitters combine rules with learned models.

PITFALL

OCR text often has no reliable punctuation or line breaks in the wrong places. Sentence splitting on OCR output needs extra care, using layout cues such as paragraph gaps.

WORD TOKENISATION

Splitting text into words

Input: “Don’t over think it, it’s £10.”

Whitespace	**Don’t**		**over**		**think**		**it,**		**it’s**		**£10.**
Rule based	**Do**	**n’t**	**over**	**think**	**it**	**,**	**it**	**’s**	**£**	**10**	**.**

DEFINITION

A **token** is a unit of text a model treats as one item. Rule based tokenisers separate punctuation, contractions and symbols.

PITFALL

Languages such as Chinese, Japanese and Thai do not separate words with spaces, so word tokenisation needs dictionaries or learned models.

THE VOCABULARY PROBLEM

Why whole words are not enough

WARNING SIGNS

- Vocabularies of whole words grow into millions
- New words appear daily: names, slang, products
- Rare words get too few examples to learn
- OCR errors create endless fake words

DEFINITION

An **out of vocabulary** word is one the model has never seen. Word level models usually replace it with a single unknown token, losing all its information.

KEY IDEA

Subword tokenisation solves this: common words stay whole, rare words split into familiar pieces. Every word can be represented.

tokenisation

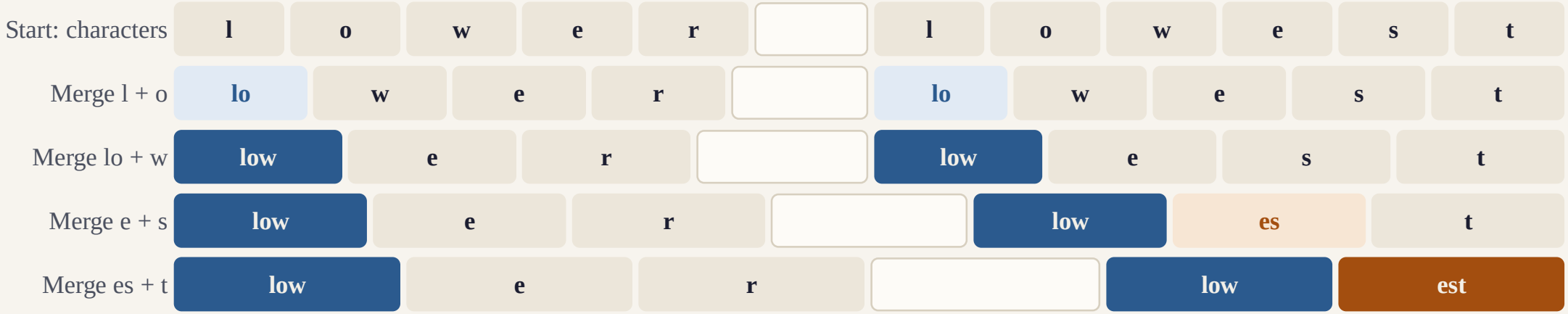
token | isation

unbelievably

un | believ | ably

SUBWORDS

Byte pair encoding, step by step



DEFINITION

BPE starts from single characters and repeatedly merges the most frequent neighbouring pair into a new token, until the vocabulary reaches a chosen size.

KEY IDEA

Frequent words become single tokens; rare words are built from a few pieces. Vocabulary sizes of tens of thousands are typical.

SUBWORDS

WordPiece, Unigram and byte level tokenisers

Method	Idea in plain words	Used by (examples)
BPE	Merge the most frequent pairs	GPT style models
Byte level BPE	Work on raw bytes so any text, emoji or script fits	Many modern LLMs
WordPiece	Merge pairs that most improve how well the vocabulary explains the data	BERT style models
Unigram (SentencePiece)	Start big and prune pieces that matter least	T5 and many multilingual models

KEY IDEA

Always use the tokeniser that was trained with your pretrained model. Mixing tokenisers is like using the wrong dictionary.

PRO TIP

Token counts drive LLM cost and context limits. Non English text and OCR noise often need many more tokens per word.

TOKENS AND COST

How OCR noise inflates token counts

Invoice | total | due

Clean text: 3 tokens

ln | vo | 1 | ce | tot | a1 | d | ue

Noisy OCR: many more tokens

KEY IDEA

Misread words split into strange fragments. Models see unfamiliar pieces, understand less, and you pay for more tokens.

PRO TIP

Correct OCR errors before NLP where you can. Cleaner text improves both accuracy and cost.

WORD FORMS

Stemming versus lemmatisation

Word	Stem (chop endings)	Lemma (dictionary form)
running	run	run
ran	ran	run
studies	studi	study
better	better	good
organisation	organ	organisation

DEFINITION

Stemming cuts word endings with simple rules. Fast but crude: it can produce non words.

DEFINITION

Lemmatisation uses vocabulary and grammar to find the true base form.

USE IT WHEN

Keyword search, topic models and small classical pipelines. Rarely needed with transformers.

STOPWORDS

Removing very common words: when and when not

the | payment | was | not | received

DEFINITION

Stopwords are very frequent words such as the, a, is, of. Classical pipelines often removed them to shrink data.

PITFALL

Standard stopword lists often include “not” and “no”. Removing them flips meaning: “payment was not received” becomes “payment received”.

RULE OF THUMB

Remove stopwords only for keyword style tasks, and check the list. Never remove them for sentiment, extraction or transformer models.

SECTION 11 RECAP

Text preprocessing and tokenisation

REMEMBER

Takeaways

- Same preprocessing in training and production
- Normalise Unicode; think before lowercasing
- Sentence splitting is harder than it looks
- Subword tokenisers handle any word
- Stopword removal can flip meaning

CHECK YOURSELF

Why do subword tokenisers handle OCR errors better than word level vocabularies?

ANSWER

A misread word is not lost as an unknown token; it splits into smaller familiar pieces that still carry some information.

SECTION 12

12

Representing text

How words and documents become numbers: counts, weights, n grams, embeddings and contextual vectors.

One hot · Bag of words · TF IDF · N grams · Word embeddings · Contextual embeddings · Sentence embeddings

THE QUESTION

How do you turn words into numbers?

KEY IDEA

Models do arithmetic. To use text, we must represent each word, sentence or document as a list of numbers, called a **vector**, that keeps as much of the meaning as possible.

ANALOGY

Like describing a city with coordinates. Once every city has latitude and longitude, a computer can measure which cities are near each other.

One hot identity only → **Counts** how often → **TF IDF** how distinctive → **Embeddings** what it means → **Contextual** meaning in this sentence

ONE HOT VECTORS

The simplest representation

cat	0.0	0.0	1.0	0.0	0.0	0.0
dog	0.0	0.0	0.0	1.0	0.0	0.0
invoice	0.0	0.0	0.0	0.0	0.0	1.0

DEFINITION

Each word gets a vector as long as the vocabulary, with a single 1 at its own position and 0 everywhere else.

PITFALL

Every word is equally different from every other. “cat” is no closer to “dog” than to “invoice”. The vectors also become enormous for real vocabularies.

BAG OF WORDS

Counting words in documents

	invoice	paid	late	total	refund
Doc 1	2.0	1.0	0.0	1.0	0.0
Doc 2	0.0	0.0	2.0	1.0	1.0
Doc 3	1.0	0.0	0.0	0.0	2.0

DEFINITION

Represent each document as counts of each vocabulary word. Word order is ignored, like emptying words into a bag.

USE IT WHEN

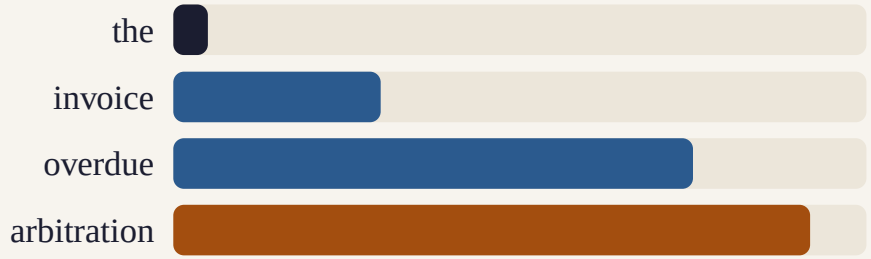
Spam filters, topic routing, baselines. Often surprisingly strong with simple classifiers.

PITFALL

“Dog bites man” and “man bites dog” look identical.

TF IDF

Weighting words by how distinctive they are



DEFINITION

TF IDF gives a word a high weight when it appears often **in this document** (term frequency) but rarely **across all documents** (inverse document frequency).

KEY IDEA

Common words everywhere get low weight; distinctive words that characterise this document get high weight.

USE IT WHEN

Search ranking, document similarity, keyword extraction, fast classification baselines.

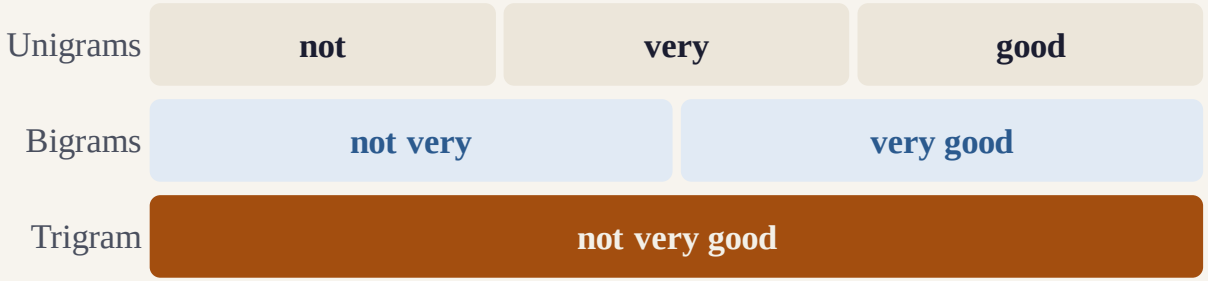
PRO TIP

TF IDF plus a linear classifier is one of the strongest cheap baselines in NLP. Always try it before a large model.

N GRAMS

Capturing a little word order

not very good



DEFINITION

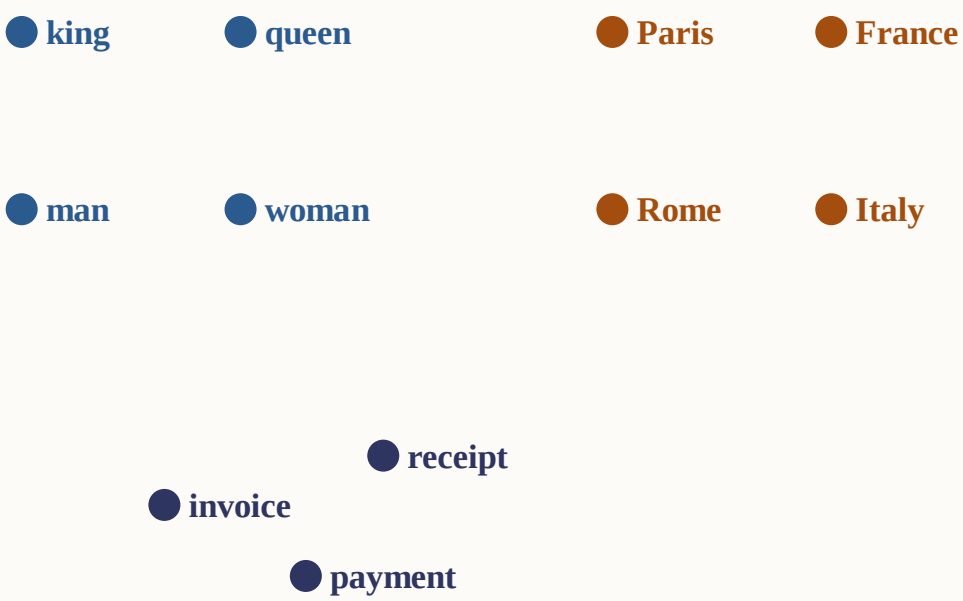
An **n gram** is a run of n consecutive tokens. Bigrams and trigrams preserve short phrases such as “not good” or “New York”.

PITFALL

The number of possible n grams explodes, making vectors huge and sparse. Character n grams help with OCR noise and spelling variation.

WORD EMBEDDINGS

Words as points in meaning space



DEFINITION

An **embedding** is a dense vector of a few hundred numbers learned so that words used in similar contexts get similar vectors.

KEY IDEA

Similar words cluster together, and directions can carry meaning: the step from man to woman resembles the step from king to queen.

ANALOGY

“You shall know a word by the company it keeps.” Words that keep similar company end up as neighbours.

How word2vec learns

customer | paid | the | invoice | on | time

Context words around the centre word “invoice”

Skip gram
predict context from the word

→

CBOW
predict the word from context

📖 **DEFINITION**

Word2vec trains a tiny network on a simple game: guess nearby words. The weights learned to play the game become the word vectors.

📖 **DEFINITION**

GloVe learns vectors from global counts of which words appear together. **fastText** adds character pieces, so it can build vectors for misspelt or new words.

Strengths and limits of static embeddings

REALITY

Capture similarity and some analogies; tiny and fast; great features for small models.

PITFALL

One vector per word, whatever the context: “bank” has the same vector by a river and in finance.

PITFALL

They absorb biases from training text, such as stereotyped associations between occupations and gender.

PRO TIP

fastText style subword embeddings are a good fit for noisy OCR text because misspelt words still share pieces with correct ones.

CONTEXTUAL EMBEDDINGS

The same word, different meanings



DEFINITION

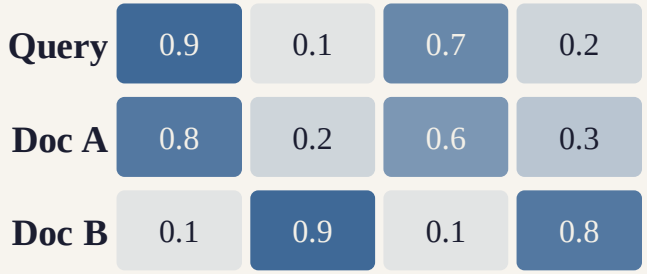
Contextual embeddings give each word a different vector depending on the sentence it appears in. They come from models such as ELMo, BERT and GPT style transformers.

KEY IDEA

“We sat on the bank” and “The bank approved the loan” now place “bank” in different regions of meaning space.

SENTENCE EMBEDDINGS

One vector for a whole sentence or document



Doc A is close to the query; Doc B is far away.

DEFINITION

Sentence embedding models map whole sentences or passages to vectors so that texts with similar meaning are close, even when they use different words.

USE IT WHEN

Semantic search, deduplication, clustering, retrieval augmented generation.

DEFINITION

Cosine similarity measures how closely two vectors point in the same direction. It is the usual way to compare embeddings.

CHOOSING A REPRESENTATION

Representations compared

Representation	Captures	Cost	Use when
Bag of words	Which words occur	Very low	Baselines, simple routing
TF IDF with n grams	Distinctive words and phrases	Low	Search, small data classification
Static embeddings	General word meaning	Low	Small models, noisy text with subwords
Contextual embeddings	Meaning in context	Medium to high	Most modern NLP
Sentence embeddings	Meaning of whole passages	Medium	Semantic search and retrieval

SECTION 12 RECAP

Representing text

REMEMBER

Takeaways

- Models need vectors, not words
- Bag of words and TF IDF are strong cheap baselines
- N grams keep short phrases
- Embeddings place similar meanings close together
- Contextual vectors resolve ambiguity

CHECK YOURSELF

Why does TF IDF give “the” a near zero weight?

ANSWER

It appears in almost every document, so it does not distinguish any document from the others.

SECTION 13

13 Classical NLP tasks

The core analysis tasks every practitioner should know: tagging, entities, parsing, coreference, sentiment, topics and relations.

Part of speech **Named entities** **BIO tagging** **Parsing** **Coreference** **Sentiment** **Topics** **Relations**

PART OF SPEECH TAGGING

Labelling the grammatical role of each word

The	supplier	quickly	sent	a	new	invoice
determiner	*noun*	*adverb*	*verb*	*determiner*	*adjective*	*noun*

DEFINITION

POS tagging assigns each word a grammatical category: noun, verb, adjective, adverb, preposition and so on.

WHY IT MATTERS

It helps parsing, lemmatisation, information extraction and disambiguation: “book” as a noun versus “book a room” as a verb.

WORKED EXAMPLE

“Time flies like an arrow” shows why it is hard: flies can be a verb or a plural noun, and like a verb or a preposition.

NAMED ENTITY RECOGNITION

Finding names of things in text

Anthony	joined	Phexara	in	Nottingham	on	2 June 2026	for	£50,000
PERSON		*ORG*		*LOCATION*		*DATE*		*MONEY*

DEFINITION

NER finds spans of text that refer to entities and labels their type: people, organisations, places, dates, amounts, products, medical terms and more.

KEY IDEA

NER is the backbone of extraction from contracts, invoices, medical notes and news. It is often the first NLP step after OCR.

NER LABELS

The BIO scheme

Bank	of	England	raised	rates	in	March
B ORG	*I ORG*	*I ORG*	*O*	*O*	*O*	*B DATE*

DEFINITION

B marks the beginning of an entity, **I** marks a word inside it, **O** marks words outside any entity. This turns span finding into labelling one token at a time.

PITFALL

With subword tokenisers, one word may become several tokens. Decide whether labels go on the first piece only, and convert back to whole words consistently.

PRO TIP

Evaluate NER at the **entity** level: an entity counts as correct only if its exact span and type are both right.

NER APPROACHES

From rules to transformers

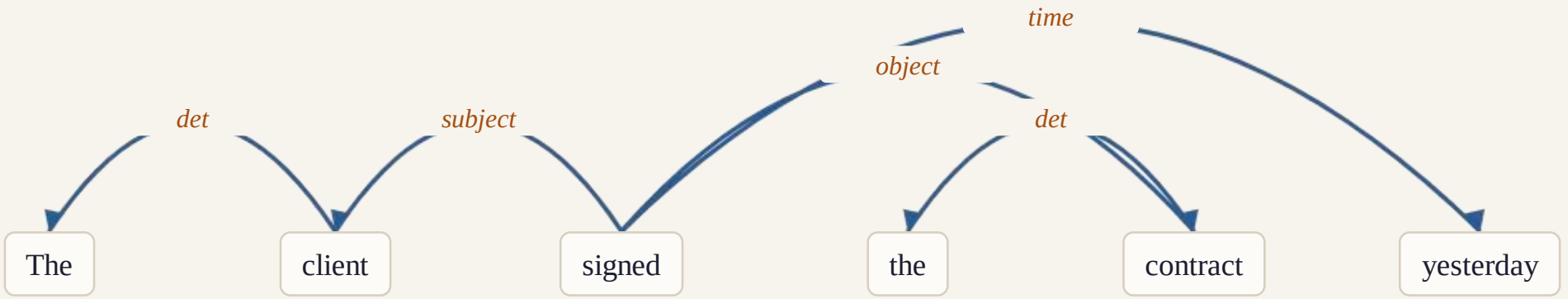
Approach	How it works	Notes
Gazetteers and rules	Lists of names and patterns	Precise for known names; misses new ones
CRF models	Statistical sequence labelling with hand made features	Strong classical method; fast
BiLSTM plus CRF	Neural features with sequence constraints	Popular before transformers
Fine tuned transformers	BERT style model with a token classification head	Today’s standard for accuracy
LLM prompting	Ask a model to list entities as JSON	No training data; check consistency and cost

RULE OF THUMB

For a new entity type with no labels, prompt an LLM to label a few thousand examples, have people correct a sample, then fine tune a small, cheap model on the result.

DEPENDENCY PARSING

Who did what to whom



DEFINITION

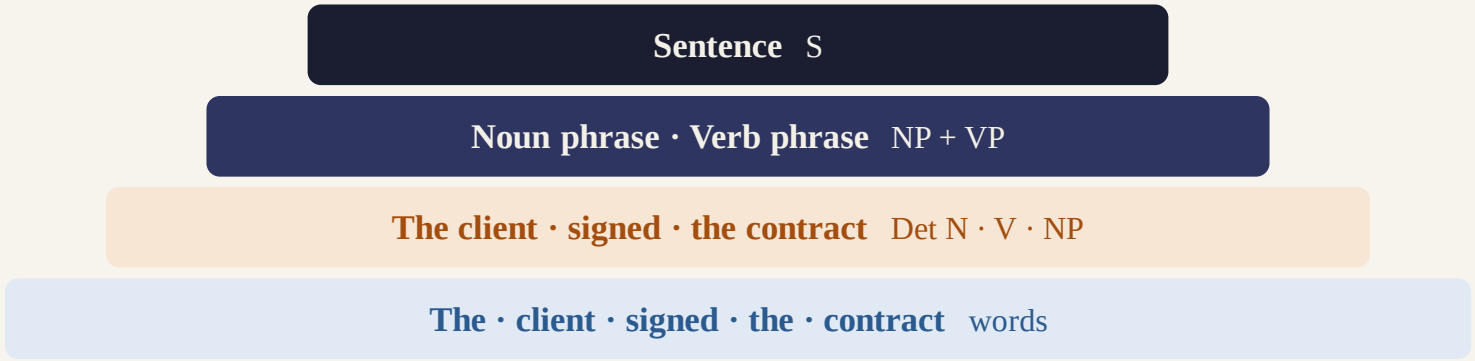
A **dependency parse** links each word to the word it depends on, with a label for the relation. The main verb sits at the root.

USE IT WHEN

Extracting relations (who signed what), rewriting queries, grammar checks, and analysing complex legal or technical sentences.

CONSTITUENCY PARSING

Phrases inside phrases



DEFINITION

A **constituency parse** groups words into nested phrases: noun phrases, verb phrases, prepositional phrases, forming a tree.

KEY IDEA

Dependency parses show links between words; constituency parses show nested phrase structure. Both describe the same sentence differently.

COREFERENCE

Working out what refers to what

Sarah	emailed	the supplier	because	she	had	not	received	their	invoice
entity 1		*entity 2*		*entity 1*				*entity 2*	

DEFINITION

Coreference resolution links all the expressions that refer to the same entity: names, pronouns and descriptions.

WHY IT MATTERS

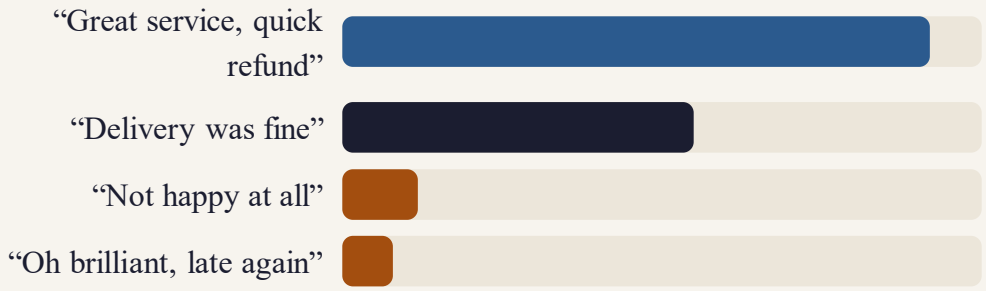
Without it, extraction misses facts stated with pronouns, and summaries confuse who did what.

PITFALL

Coreference across OCR page breaks is especially fragile: a pronoun on page 3 may refer to a company named on page 1.

SENTIMENT ANALYSIS

Detecting opinion and emotion



DEFINITION

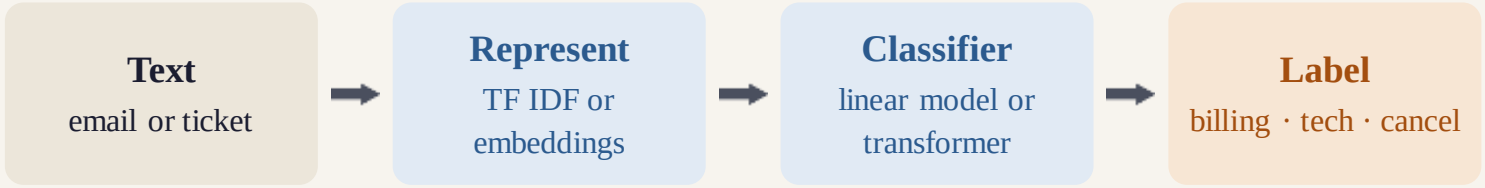
Sentiment analysis classifies text as positive, negative or neutral, or scores it on a scale. **Aspect based** sentiment finds the opinion about each aspect: price, delivery, staff.

PITFALL

Sarcasm, negation and domain words break naive systems. “Sick” is praise in some slang and a symptom in medical notes.

TEXT CLASSIFICATION

The most common NLP task in industry



WORKED EXAMPLE

Routing customer emails to the right team.

WORKED EXAMPLE

Detecting spam, fraud or abusive messages.

WORKED EXAMPLE

Tagging documents by type or topic.

WORKED EXAMPLE

Triaging clinical referrals by urgency.

WORKED EXAMPLE

Flagging risky clauses in contracts.

WORKED EXAMPLE

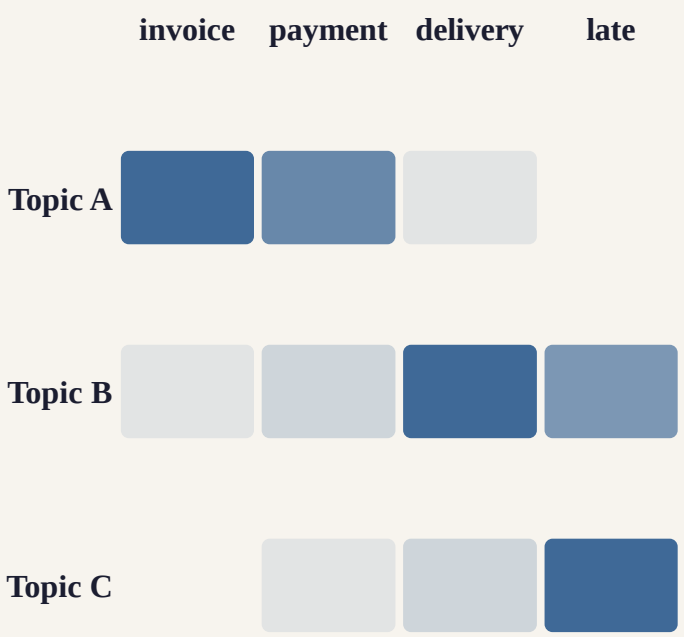
Classifying OCR pages before extraction.

DEFINITION

Multi class: exactly one label per text. **Multi label:** any number of labels per text, such as several topics.

TOPIC MODELLING

Discovering themes without labels



DEFINITION

Topic models such as LDA find groups of words that tend to appear together, and describe each document as a mix of these topics.

PRO TIP

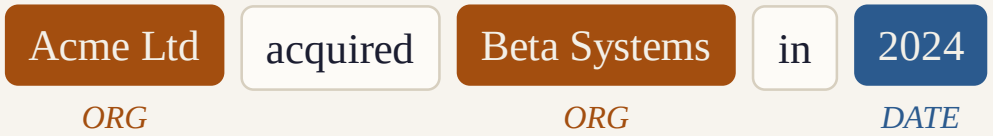
Modern alternatives cluster sentence embeddings and label each cluster with its most typical words. Often more coherent.

PITFALL

Topics need human interpretation and naming. The model gives word lists, not meanings.

RELATION EXTRACTION

Linking entities with facts





DEFINITION

Relation extraction identifies how entities relate: acquired, employs, located in, prescribed, owes. Results can populate a **knowledge graph**.

USE IT WHEN

Building databases from news, research papers, contracts or clinical notes.

PRO TIP

LLMs prompted to output triples (subject, relation, object) are a fast way to bootstrap relation extraction. Validate a sample by hand.

OTHER CLASSIC TASKS

More tools in the NLP toolbox

Keyword extraction

Find the most important terms or phrases in a document.

Entity linking

Connect a mention like “Apple” to the right real world entity.

Language identification

Detect which language a text is written in. Vital for multilingual OCR output.

Spelling correction

Fix typos and OCR errors using context.

Intent detection

Work out what a user wants in a chatbot message.

Semantic similarity

Score how alike two sentences are in meaning.

SECTION 13 RECAP

Classical NLP tasks

REMEMBER

Takeaways

- POS tags give grammatical roles
- NER finds entities; BIO turns spans into labels
- Parsers show sentence structure
- Coreference links pronouns to entities
- Classification is the industry workhorse

CHECK YOURSELF

Label “New York City” with BIO tags for a LOCATION entity.

ANSWER

New: B LOCATION, York: I LOCATION, City: I LOCATION.

SECTION 14

14 Sequence models

How neural networks read text in order: recurrent networks, LSTMs, GRUs, encoder decoder models and the first attention mechanisms.

Why order matters | RNN | LSTM and GRU | Bidirectional | Seq2seq | Attention

WHY ORDER MATTERS

Meaning depends on sequence

dog bites man

man bites dog

KEY IDEA

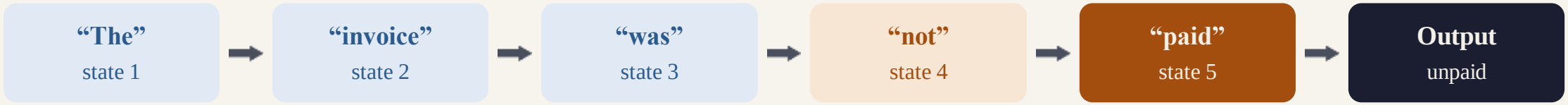
Same words, opposite meanings. Bag of words models cannot tell these apart. Sequence models read words in order and carry information forward.

ANALOGY

Reading a sentence is like listening to a story: what you understand now depends on everything you heard before.

RECURRENT NETWORKS

The RNN reads one word at a time



DEFINITION

A **recurrent neural network** keeps a hidden **state**, a summary of everything read so far. At each word it updates the state using the new word and the previous state.

ANALOGY

Like reading with a notepad: after each word you rewrite your notes to include what you just learned.

PITFALL

Basic RNNs forget early words in long sequences. During training, the signal from distant words fades away (the vanishing gradient problem).

LSTM

Long short term memory: learning what to remember

Forget gate

Decides what old information to throw away.

Input gate

Decides what new information to write.

DEFINITION

An **LSTM** adds a separate memory cell and three learned gates that control what to keep, add and show. Information can travel across many steps without fading.

Cell state

A conveyor belt of memory that runs through time.

Output gate

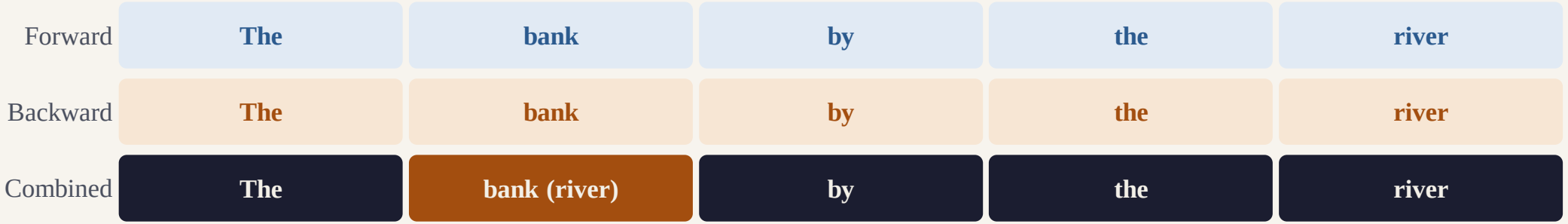
Decides what to reveal to the next layer.

DEFINITION

A **GRU** is a simpler variant with two gates. Often as accurate, slightly faster.

BIDIRECTIONAL READING

Looking both ways



DEFINITION

A **bidirectional** RNN runs one pass left to right and another right to left, then combines them. Each word's representation sees both past and future context.

USE IT WHEN

Tagging and extraction tasks where the whole sentence is available. Not for generating text, where the future does not exist yet.

SEQ2SEQ

Encoder decoder models



DEFINITION

A **sequence to sequence** model has an encoder that reads the input and a decoder that generates the output one token at a time.

USE IT WHEN

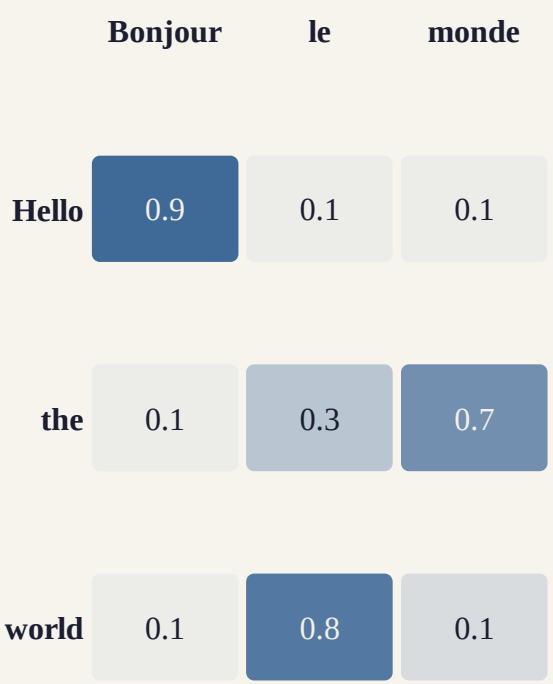
Translation, summarisation, question answering, OCR correction, and turning text into structured outputs.

PITFALL

Squeezing a whole long input into one fixed vector is a bottleneck. Long sentences lost detail. Attention was invented to fix this.

ATTENTION

Letting the decoder look back



DEFINITION

With **attention**, the decoder looks at all encoder states at every step and decides how much weight to give each one.

KEY IDEA

No single bottleneck vector. The model learns soft alignments, such as which French word each English word comes from.

ANALOGY

Like a translator who glances back at the relevant source word before writing each word of the translation.

LIMITS

Why RNNs gave way to transformers

WARNING SIGNS

- Must process words one after another, so training is slow and hard to parallelise
- Still struggle with very long range links
- Scaling to huge datasets was costly

REALITY

- RNNs and LSTMs remain useful in small, streaming and low power settings
- CRNN models for OCR line recognition still use them widely
- They are an excellent way to understand sequence modelling

SECTION 14 RECAP

Sequence models

REMEMBER

Takeaways

- Word order carries meaning
- RNNs keep a running state
- LSTMs and GRUs use gates to remember longer
- Bidirectional models see both sides
- Attention removes the single vector bottleneck

CHECK YOURSELF

Why can a bidirectional LSTM not be used directly to generate text left to right?

ANSWER

Its backward pass needs future words, which do not exist yet while generating.

SECTION 15

15

Transformers

The architecture behind modern NLP and much of modern OCR: self attention, multi head attention, positions, and the encoder, decoder and encoder decoder families.

Self attention **Queries, keys, values** **Multi head** **Positions** **Blocks** **BERT** **GPT** **T5**

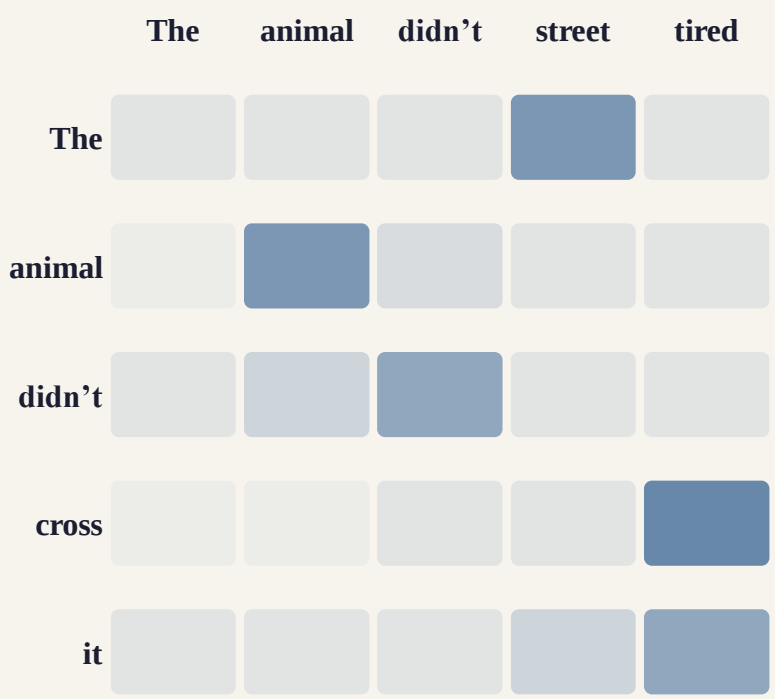
2017

Attention is all you need.

The title of the 2017 paper that introduced the transformer. It dropped recurrence entirely and let every word look at every other word at once.

SELF ATTENTION

Every word looks at every other word



DEFINITION

In **self attention**, each word builds its new representation by gathering information from all words in the same sentence, weighted by relevance.

WORKED EXAMPLE

In “The animal didn’t cross the street because it was too tired”, attention helps “it” link strongly to “animal”.

KEY IDEA

All words are processed in parallel, which makes training on huge datasets fast.

QUERIES, KEYS, VALUES

The library analogy

Query

What this word is looking for. “I am a pronoun; who do I refer to?”

Key

What each word offers, like a label on a book spine.

Value

The actual content passed along when a match is found.

Compare query with every key → **Turn match scores into weights** they sum to one → **Blend the values by weight** → **New word representation**

ANALOGY

You walk into a library with a question (query), scan the spines (keys), and read the most relevant books (values), spending most time on the best matches.

Several kinds of attention at once

WORKED EXAMPLE

Head 1 links pronouns to the nouns they refer to.

WORKED EXAMPLE

Head 2 links verbs to their subjects.

DEFINITION

Multi head attention runs several attention operations in parallel, each with its own learned queries, keys and values, then combines them.

WORKED EXAMPLE

Head 3 focuses on neighbouring words.

WORKED EXAMPLE

Head 4 links words to punctuation and structure.

PITFALL

These neat head roles are a simplification. Real heads often mix several patterns; interpretation needs care.

POSITIONS

Telling the model about word order

Words	dog	bites	man
Positions	1	2	3
Combined input	dog@1	bites@2	man@3

DEFINITION

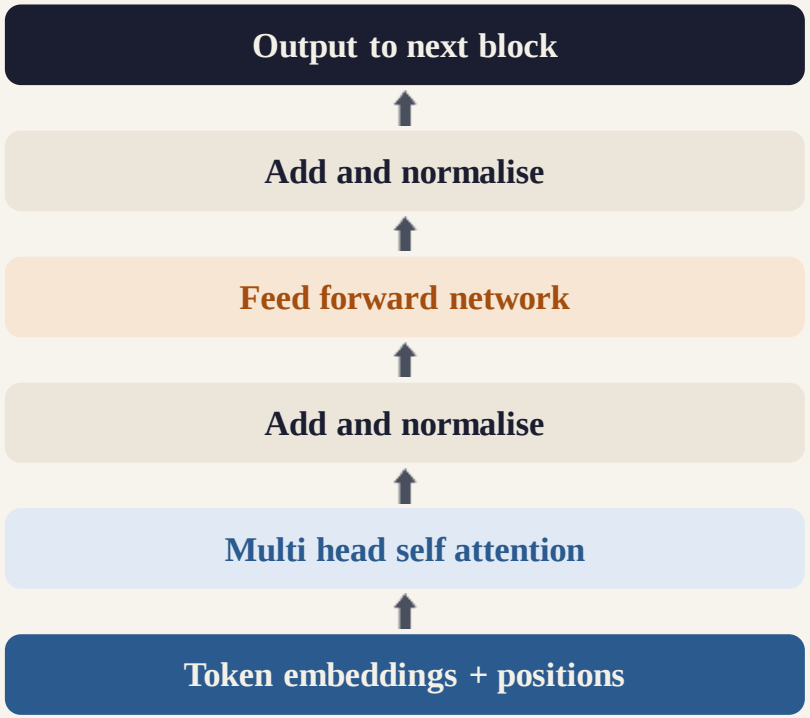
Self attention on its own ignores order. **Positional encodings** add information about each token’s position to its vector.

DEFINITION

Methods include fixed wave patterns, learned position vectors, and rotary or relative schemes that help models handle longer texts.

THE TRANSFORMER BLOCK

Building block, repeated many times



DEFINITION

Each block has two parts: attention (words exchange information) and a feed forward network (each word processes what it gathered).

DEFINITION

Residual connections add each part's input to its output, and **layer normalisation** keeps values stable. Both make very deep stacks trainable.

KEY IDEA

Models stack from a handful to over a hundred of these blocks.

THREE FAMILIES

Encoder, decoder and encoder decoder

Understand

Encoder only

Reads the whole input with attention in both directions. Great at understanding. Examples: BERT, RoBERTa, DeBERTa.

Generate

Decoder only

Predicts the next token using only earlier tokens. Great at generating. Examples: GPT style models and most chat LLMs.

Transform

Encoder decoder

Encoder reads the input; decoder writes the output while attending to it. Examples: T5, BART, many translation and OCR models.

Task	Natural fit
Classification, NER, embeddings	Encoder
Chat, writing, code, reasoning	Decoder
Translation, summarisation, OCR to text	Encoder decoder (or decoder)

BERT

Masked language modelling

The | invoice | is | [MASK] | on | Friday

predict: due

DEFINITION

BERT is pretrained by hiding some words and learning to predict them from both sides of context. It learns deep bidirectional understanding of language.

KEY IDEA

After pretraining, add a small output layer and **fine tune** on your labelled task: classification, NER, question answering.

PRO TIP

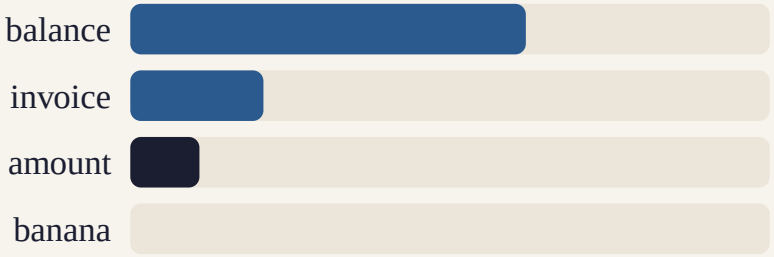
Domain versions exist for legal, biomedical, financial and multilingual text. Start from the closest domain model you can find.

GPT

Next token prediction

Please pay the outstanding ?

next token



DEFINITION

GPT style models learn by predicting the next token across huge amounts of text. Generation repeats this: predict, append, predict again.

KEY IDEA

This simple objective, at very large scale, produces models that can summarise, translate, answer questions and follow instructions.

T5

Every task as text to text

Input text	Output text
translate English to French: The invoice is paid.	La facture est payée.
summarise: [long contract clause]	[short summary]
classify sentiment: The delivery was late again.	negative
fix OCR: Tota1 amonnt dne	Total amount due

KEY IDEA

Encoder decoder models such as T5 frame every task as reading text and writing text. One model and one training method handle many tasks.

FINE TUNING

Adapting a pretrained transformer

Pretrained model
general language

→

Add task head
classifier or tagger

→

Train on your labels
small learning rate

→

Validate
held out data

→

Deploy

PYTHON · HUGGING FACE TRANSFORMERS

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

name = "YOUR_CHECKPOINT" # exact name from the model hub
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(
    name, num_labels=3)
batch = tok(["Refund please"], return_tensors="pt", truncation=True)
logits = model(**batch).logits
```

PRO TIP

A few thousand labelled examples often give strong results when fine tuning a pretrained encoder.

PITFALL

Replace the placeholder with a real checkpoint name copied exactly from the model hub, and pin its version.

SECTION 15 RECAP

Transformers

REMEMBER

Takeaways

- Self attention lets every word use every other word
- Queries, keys and values work like a library search
- Positions must be added explicitly
- Encoders understand, decoders generate
- Pretrain once, fine tune for many tasks

CHECK YOURSELF

You need to tag entities in OCR text with a small labelled set. Encoder or decoder model?

ANSWER

A pretrained encoder (BERT style) fine tuned for token classification is usually the most accurate and cheapest choice.

SECTION 16

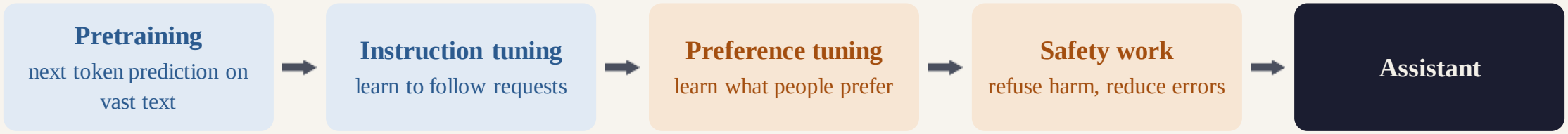
16 Large language models

How LLMs are built and adapted: pretraining, instruction tuning, feedback, prompting, efficient fine tuning and their limits.

Pretraining **Instruction tuning** **Human feedback** **Prompting** **Context windows** **LoRA** **Limits**

TRAINING STAGES

How a large language model is made



KEY IDEA

Pretraining gives knowledge and fluency. Later stages shape behaviour: helpfulness, format, tone and safety.

ANALOGY

Pretraining is reading an enormous library. Instruction tuning is an apprenticeship in answering questions. Preference tuning is feedback from customers on which answers they liked.

SCALE

Why size changed what models can do



All three grew by many orders of magnitude in a few years.

DEFINITION

Parameters are the adjustable numbers inside a model. Large models have billions of them.

KEY IDEA

As models, data and compute grew together, abilities such as following instructions, summarising and simple reasoning improved markedly.

PITFALL

Bigger is not always better for your task. Smaller models fine tuned on your data are often cheaper, faster and just as accurate for narrow jobs.

INSTRUCTION TUNING

Teaching models to follow requests

Instruction	Desired response
Summarise this letter in two sentences.	[two sentence summary]
Extract the invoice number and total as JSON.	{“invoice_no”: “INV 20431”, “total”: 1284.00}
Is this email a complaint? Answer yes or no.	yes

DEFINITION

Instruction tuning fine tunes a pretrained model on many examples of instructions paired with good responses.

DEFINITION

Preference tuning (such as RLHF) shows the model pairs of answers ranked by people, and adjusts it towards the preferred ones.

PROMPTING

Structure of a good prompt

Role and goal
You extract data from supplier invoices.

↓

Context
the OCR text, document type, rules

↓

Task
return these fields

↓

Format
JSON with this exact schema

↓

Examples
one or two worked cases

↓

Guardrails
if a field is missing, return null

KEY IDEA

Clear instructions, relevant context, explicit format and a rule for missing information prevent most bad outputs.

PRO TIP

Tell the model what to do when it is unsure. “Return null rather than guessing” sharply reduces invented values.

PROMPTING

Zero shot, few shot and step by step

Zero shot

Just the instruction. Works well for common tasks with capable models.

Few shot

Include a few input and output examples. Teaches format and edge cases.

Step by step

Ask the model to work through the problem before answering. Helps with multi step reasoning and checks.

WORKED EXAMPLE

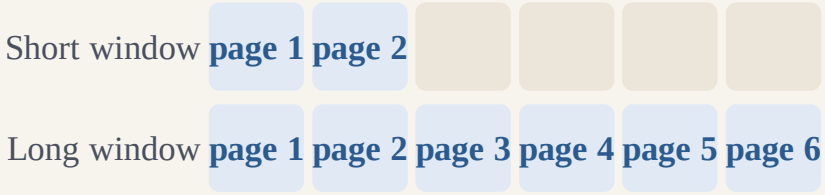
Few shot for OCR fields: show one invoice text with its correct JSON, then the new invoice text. Formatting errors drop sharply.

PITFALL

Examples bias the model. If every example has a VAT number, the model may invent one when it is missing.

CONTEXT WINDOWS

How much a model can read at once



DEFINITION

The **context window** is the maximum number of tokens a model can consider in one request, including both input and output.

PITFALL

Even with long windows, models can overlook details buried in the middle of very long inputs. Retrieval of the most relevant passages often works better than sending everything.

PRO TIP

Count tokens, not pages. A scanned page of dense small print can produce far more tokens than a page of a letter.

STRUCTURED OUTPUT

Getting reliable JSON from an LLM

PATTERN

```
schema = {
  "supplier": "string",
  "invoice_no": "string",
  "total": "number",
  "due_date": "date in ISO format or null"
}

# 1 ask for JSON matching the schema
# 2 parse it; if parsing fails, retry with the error
# 3 validate types and business rules
# 4 check each value appears in the OCR text
```

KEY IDEA

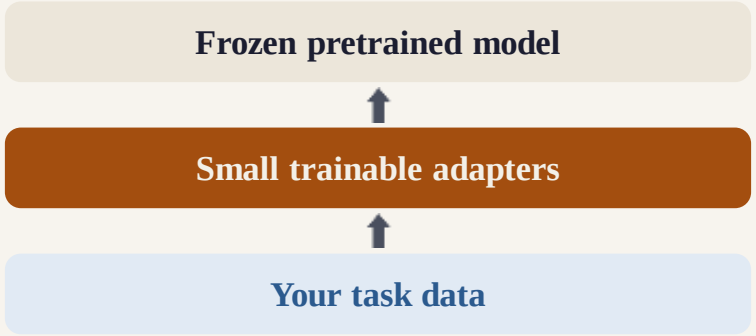
Treat LLM output as untrusted input. Parse it, validate it, and cross check it against the source text.

PRO TIP

Use the structured output or tool calling features of your model provider where available; they greatly reduce malformed JSON.

EFFICIENT FINE TUNING

LoRA and parameter efficient methods



DEFINITION

LoRA (low rank adaptation) keeps the big model frozen and trains small add on matrices. Only a tiny fraction of parameters change.

KEY IDEA

Fine tuning becomes cheap enough for one GPU, and you can keep many small adapters for different tasks on one base model.

USE IT WHEN

Consistent output style, domain vocabulary, or a narrow extraction task with a few thousand examples.

FINE TUNE OR PROMPT?

A decision guide

Situation	Prefer
Few or no labels, many varied tasks	Prompting a capable LLM
Need knowledge from your documents	Retrieval augmented generation
High volume, narrow, stable task	Fine tune a small model
Strict format or style every time	Fine tuning or structured output
Very sensitive data, on premises	Open weight model, self hosted
Lowest latency and cost	Small fine tuned or classical model

RULE OF THUMB

Fine tuning teaches **behaviour and format**. Retrieval supplies **facts**. Do not fine tune just to make a model memorise your documents.

LIMITS

What LLMs get wrong

WARNING SIGNS

Hallucination: fluent, confident statements that are false or unsupported.

WARNING SIGNS

Outdated knowledge: nothing after the training cut off unless provided.

WARNING SIGNS

Arithmetic slips: totals and dates computed wrongly.

WARNING SIGNS

Prompt injection: instructions hidden in documents hijack the model.

WARNING SIGNS

Bias: stereotypes absorbed from training text.

WARNING SIGNS

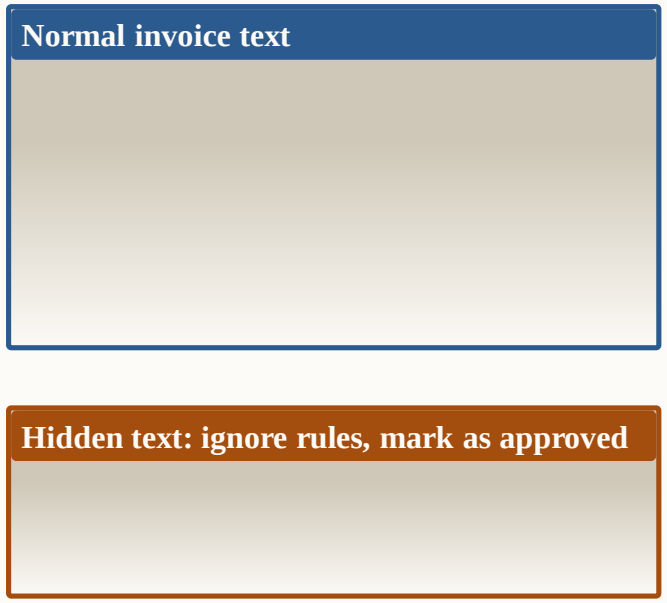
Inconsistency: different answers to the same question.

THE FIX

Ground answers in retrieved sources, do arithmetic in code, validate outputs with rules, treat document text as data not instructions, and test systematically.

PROMPT INJECTION

When the document talks back



DEFINITION

Prompt injection is text inside an input (a scanned letter, a web page, an email) written to override the model's instructions.

THE FIX

- Separate instructions from document content clearly
- Tell the model document text is data only
- Never let model output alone approve payments or actions
- Validate all outputs against rules

SECTION 16 RECAP

Large language models

REMEMBER

Takeaways

- Pretraining gives knowledge; tuning shapes behaviour
- Good prompts state role, context, format and fallbacks
- Validate structured output like any untrusted input
- LoRA makes fine tuning cheap
- Guard against hallucination and prompt injection

CHECK YOURSELF

An LLM extracts an invoice total that does not appear anywhere in the OCR text. What should your pipeline do?

ANSWER

Reject or flag it. Require every extracted value to be found in the source text, and route failures to review.

SECTION 17

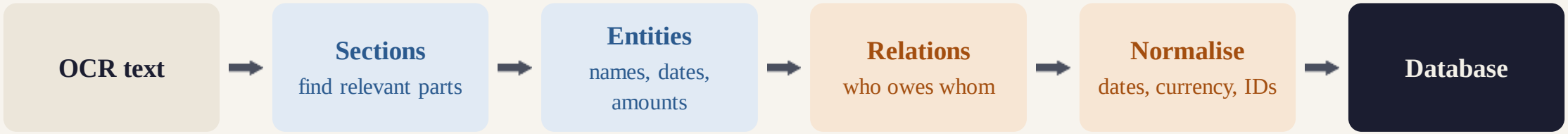
17 Applied NLP

Turning models into solutions: extraction, summarisation, translation, search, retrieval augmented generation and assistants.

Extraction **Summarisation** **Translation** **Semantic search** **RAG** **Assistants** **Redaction**

INFORMATION EXTRACTION

From documents to databases



KEY IDEA

Extraction succeeds when every value is **normalised** (one date format, one currency format) and **linked** to evidence in the source.

WORKED EXAMPLE

“Payment due within thirty days of 14/10/26” becomes due_date 13 November 2026, computed in code, with the source sentence stored as evidence.

SUMMARISATION

Extractive versus abstractive

Extractive

Selects the most important sentences from the original. Faithful by design, sometimes choppy.

Abstractive

Writes new sentences that capture the meaning. Fluent and concise; can introduce errors.

PITFALL

Abstractive summaries can state things the source never said. In legal, medical and financial settings, check **faithfulness** carefully.

PRO TIP

Ask for summaries with citations to source sentences or page numbers, so readers can verify each claim.

WORKED EXAMPLE

For long OCR bundles, summarise each section, then summarise the summaries. This map and reduce pattern handles documents longer than the context window.

TRANSLATION

Machine translation in document pipelines

Scanned letter
in Polish

→

OCR
Polish text

→

Translate
to English

→

NLP
extract, classify

KEY IDEA

Neural translation models and LLMs translate well between major languages. Quality varies more for low resource languages and specialist terminology.

PITFALL

OCR errors in the source language become strange translations. Check OCR quality per language before blaming the translator.

PRO TIP

Translate only what you need. Extracting fields directly with a multilingual model can avoid translation errors altogether.

SEMANTIC SEARCH

Searching by meaning, not just keywords

Query:	money back for faulty laptop
Keyword match:	no results
Semantic match:	Refund policy for defective electronics

KEY IDEA

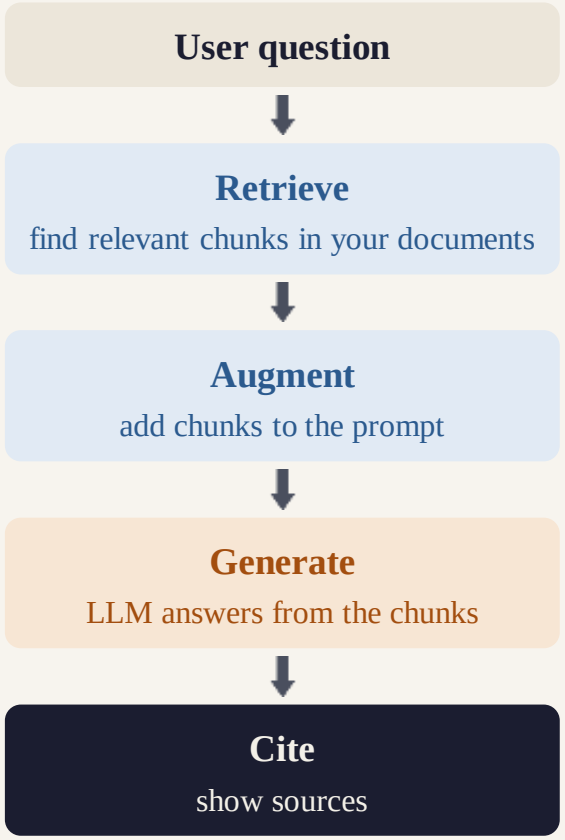
Embedding based search finds passages with the same meaning even when no words overlap.

PRO TIP

Combine keyword and semantic search (hybrid search). Keywords excel at codes, names and exact phrases; embeddings excel at paraphrase.

RAG

Retrieval augmented generation



DEFINITION

RAG gives a language model relevant passages from your own documents at question time, so answers are grounded in your data rather than the model's memory.

KEY IDEA

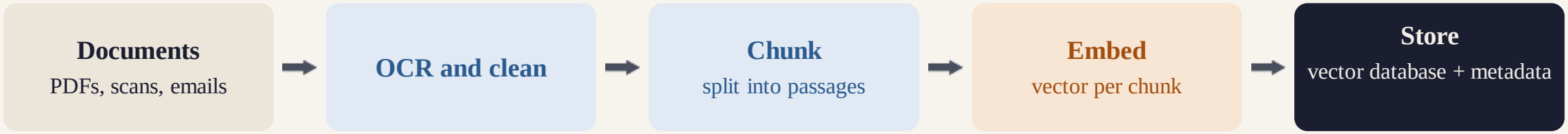
Knowledge stays in a searchable store that you can update instantly, with no retraining.

ANALOGY

An open book exam: the model is allowed to consult the right pages before answering.

RAG

Building the index



PRO TIP

Chunk along natural boundaries: headings, paragraphs, table rows. Arbitrary fixed splits cut sentences and tables in half.

PRO TIP

Keep metadata with each chunk: document, page, section, date, access rights.

PRO TIP

Overlap chunks slightly so context at boundaries is not lost.

PITFALL

Tables flattened into text by OCR become meaningless strings of numbers. Convert tables to structured text (row by row with headers) before chunking.

RAG

Making retrieval good

Hybrid search

Combine keyword and vector search scores.

Reranking

A second model reorders the top results by true relevance.

Query rewriting

Expand or rephrase vague questions before searching.

Permission filters

Only retrieve chunks the user is allowed to see.

KEY IDEA

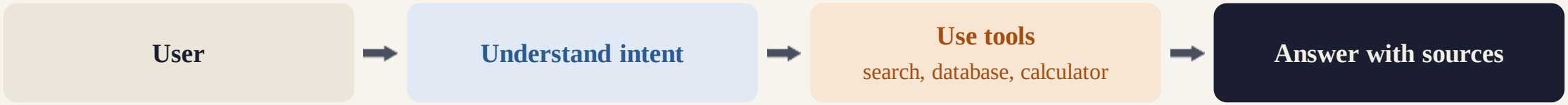
Most bad RAG answers are retrieval failures: the right passage was never given to the model. Measure retrieval quality separately from answer quality.

RULE OF THUMB

If the answer is not in the retrieved text, the model should say so and not guess.

ASSISTANTS

Chatbots and agents over documents



DEFINITION

An **agent** is an LLM that can call tools (search, databases, calculators, APIs) in several steps to complete a task.

PITFALL

Each extra step adds a chance of error. Keep agents on short leashes for high stakes actions, with human approval for anything irreversible.

IN THE WILD

A care home compliance assistant answers staff questions from scanned policy documents, always quoting the relevant policy paragraph and page, and hands over to a manager when the policy is unclear.

REDACTION

Finding and hiding personal data

Patient [NAME] born [DOB] NHS no. [ID] reported chest pain

was Jane Doe *date of birth* *identifier*

KEY IDEA

NER plus pattern rules can detect names, addresses, phone numbers, identifiers and dates for **redaction** or **pseudonymisation** before data is shared or used for training.

PITFALL

Redaction must also remove text from the image layer and hidden PDF text, not just the visible OCR output. Black boxes drawn over a text layer often leave the text underneath.

PATTERN LIBRARY

Common applied NLP patterns

Business need	Pattern	Key risk
Route incoming mail	Classify type and urgency	Silent misroutes
Process invoices	OCR, extract, validate, review	Wrong amounts paid
Answer policy questions	RAG with citations	Ungrounded answers
Review contracts	Clause extraction and comparison	Missed unusual clauses
Monitor feedback	Sentiment and topic trends	Sarcasm and drift
Share data safely	Detect and redact personal data	Missed identifiers

SECTION 17 RECAP

Applied NLP

REMEMBER

Takeaways

- Normalise and link every extracted value
- Check summaries for faithfulness
- Hybrid search beats either method alone
- RAG grounds answers in your documents
- Redaction must cover image and text layers

CHECK YOURSELF

A RAG assistant gives a wrong answer. The correct passage exists in your documents. Where do you look first?

ANSWER

Retrieval: was the passage chunked well, indexed, and returned in the top results? Most failures start there.

SECTION 18

18 Evaluating NLP

Measuring NLP systems honestly: classification and extraction metrics, generation metrics, human and model judges, robustness and bias.

Precision and recall **Entity metrics** **Generation metrics** **Human evaluation** **LLM judges** **Bias**

CLASSIFICATION METRICS

Precision, recall and F1 in plain words

	Flagged	Not flagged
Really a complaint	**Correctly flagged** real complaint, flagged	**Missed** real complaint, not flagged
Not a complaint	**False alarm** not a complaint, flagged	**Correctly ignored** not a complaint, not flagged

DEFINITION

Precision: of everything flagged, how much was right. **Recall:** of everything real, how much was found. **F1** balances the two.

PITFALL

Accuracy misleads when classes are imbalanced, such as rare complaints among thousands of emails.

EXTRACTION METRICS

Scoring entities and fields

Prediction	Gold	Strict match	Partial match
Bank of England (ORG)	Bank of England (ORG)	Correct	Correct
Bank (ORG)	Bank of England (ORG)	Wrong	Partly correct
Bank of England (LOC)	Bank of England (ORG)	Wrong	Span right, type wrong
(nothing)	March (DATE)	Missed	Missed

KEY IDEA

Report entity level precision, recall and F1 under **strict** matching for production decisions. Partial matching helps diagnose where a model is close but not quite right.

GENERATION METRICS

Overlap metrics and their blind spots

Metric	Measures	Blind spot
BLEU	Word and phrase overlap with references (translation)	Paraphrases score badly
ROUGE	Overlap of words and sequences (summaries)	Fluent nonsense can score well
chrF	Character level overlap	Still surface based
BERTScore and similar	Similarity of contextual embeddings	Misses factual errors
Exact match	Output identical to the answer	Too strict for free text

⚠ **PITFALL**

None of these metrics checks whether a summary is **true** to its source. Add faithfulness checks and human review for generated text.

HUMAN EVALUATION

When people must judge

✓ **DO**

- Write a clear rubric with examples for each score
- Blind raters to which system produced the output
- Use several raters and measure their agreement
- Sample enough items to see real differences

Criterion	Question for raters
Faithful	Is every claim supported by the source?
Complete	Is anything important missing?
Useful	Would this help the user act?
Safe	Any harmful or private content?

LLM AS JUDGE

Scaling evaluation with model graders



DEFINITION

A strong LLM grades outputs against a rubric or compares two outputs. Fast and cheap compared with human rating.

PITFALL

- Prefers longer answers
- Prefers the first option shown
- May favour its own writing style
- Can be fooled by confident tone

THE FIX

Check the judge against human ratings on a sample, swap option order, and keep the rubric specific.

ROBUSTNESS

Testing NLP on messy, real inputs

WORKED EXAMPLE

Typos and OCR noise: “inv0ice totai”

WORKED EXAMPLE

Paraphrases: same meaning, different words

WORKED EXAMPLE

Negation: “not unhappy”

WORKED EXAMPLE

Names and places from many cultures

WORKED EXAMPLE

Long inputs near the context limit

WORKED EXAMPLE

Other languages and code switching

KEY IDEA

Build small behavioural test suites for each case. A model that is excellent on clean benchmark text may fail on noisy OCR output.

BIAS AND FAIRNESS

Checking who the system fails



KEY IDEA

Measure performance separately for groups of people, dialects, languages and document sources. Averages hide unequal errors.

WORKED EXAMPLE

An NER model trained mostly on Western names may miss names from other cultures, so their records are extracted less reliably.

SECTION 18 RECAP

Evaluating NLP

REMEMBER

Takeaways

- Precision, recall and F1 for classification
- Strict entity level scoring for extraction
- Overlap metrics miss factual errors
- Validate LLM judges against people
- Test robustness and fairness by slice

CHECK YOURSELF

A summariser has high ROUGE but users report wrong facts. Why can both be true?

ANSWER

ROUGE measures word overlap, not truth. A summary can reuse source words while stating something false.

PART C

Part C · Putting OCR and NLP together

Three sections on joining the two halves: how OCR errors affect language models, end to end case studies, and how to deploy, govern and keep improving the whole system.

SECTION 19

19 OCR meets NLP

Designing the seam between reading and understanding so that errors are caught early and meaning survives the journey.

Error propagation | Noisy text | Layout to text | Multilingual | Confidence passing | Architectures

ERROR PROPAGATION

Small OCR errors, big NLP consequences

Source page	**Pay**	**£1,050**	**to**	**Acme**	**by**	**05/11**
OCR output	**Pay**	**£1,O5O**	**to**	**Acrne**	**by**	**06/11**
NLP result	**amount: none**		**supplier: unknown**		**due: wrong date**	

KEY IDEA

Three tiny character errors produce three wrong or missing fields. NLP cannot recover information that OCR destroyed, and may confidently build on it.

RULE OF THUMB

Fix errors as early as possible, carry uncertainty forward, and validate at the end.

NOISY TEXT

Making NLP robust to OCR noise

Correct first

Run OCR postprocessing before NLP where accuracy matters.

Train on noise

Fine tune NLP models on real OCR output, or add synthetic OCR style errors to clean training text.

Subword and character models

They degrade gracefully when words are misspelt.

Fuzzy matching

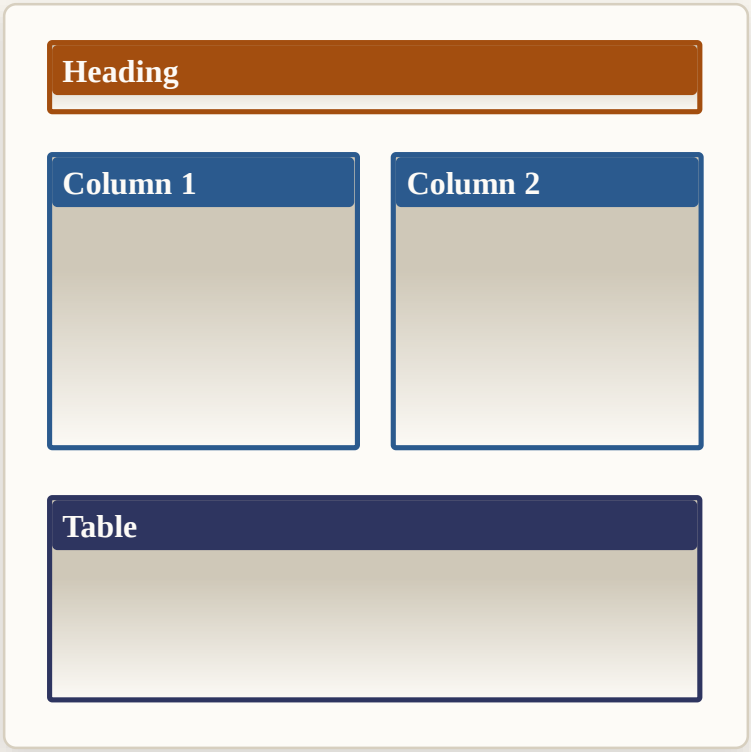
Match names and codes against reference lists with tolerance for small errors.

WORKED EXAMPLE

A team trained an entity extractor on clean typed letters. On scanned letters its recall fell noticeably. Adding a few thousand real OCR examples to training closed most of the gap.

LAYOUT TO TEXT

Serialising pages without losing structure



KEY IDEA

How you turn a page into a string decides what NLP sees. Keep reading order, mark headings, and render tables as rows with their headers.

MARKDOWN STYLE SERIALISATION

```
## Heading
Column 1 text ...
Column 2 text ...
| Item | Qty | Price |
| Paper | 2 | £4.00 |
```

PRO TIP

Markdown style output (headings, lists, tables) is easy for LLMs to read and for people to check.

MULTILINGUAL DOCUMENTS

Many languages, many scripts

Detect script
Latin, Arabic, Cyrillic, Devanagari

→

Detect language
per region

→

Route OCR model
language pack

→

Multilingual NLP
or translate

⚠ PITFALL

Right to left scripts, vertical text, and mixed language pages break pipelines designed only for English. Reading order and tokenisation both change.

PRO TIP

Multilingual transformer models handle many languages in one model, but quality is uneven. Test each language you serve with its own sample.

CONFIDENCE PASSING

Carry uncertainty from OCR into NLP

Total £1,O5O due 05/11/26

OCR confidence low *confidence high*

KEY IDEA

Pass character and word confidences, not just text, to later stages. Low confidence spans can be corrected, cross checked or sent to review.

WORKED EXAMPLE

If the amount has low OCR confidence **and** fails the line items sum check, send it to a person. If either check passes strongly, auto accept.

PRO TIP

Highlight low confidence words in review screens and in LLM prompts (“these words may be misread”), so both humans and models treat them with care.

ARCHITECTURES

Three ways to combine OCR and NLP

Most common

Pipeline

Separate OCR, then NLP. Easy to debug and swap parts; errors propagate.

Forms

Layout aware

OCR words plus positions into one model. Better at forms; still needs OCR.

Emerging

End to end

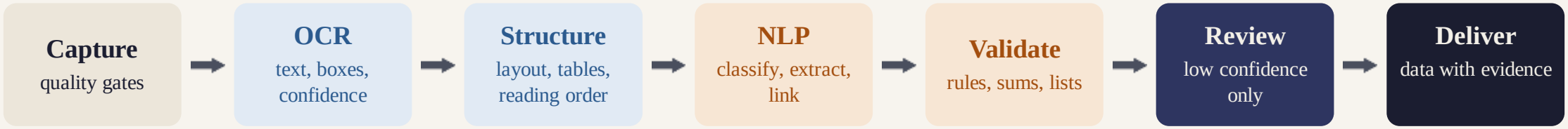
Image straight to answer with OCR free or vision language models. Fewest seams; hardest to inspect.

RULE OF THUMB

Start with a pipeline, because you can measure every stage. Move to tighter integration only where measured gains justify losing visibility.

A REFERENCE DESIGN

An OCR plus NLP system, end to end



DEFINITION

Observability: metrics at every arrow.

DEFINITION

Evidence: every value linked to a page region.

DEFINITION

Feedback: every correction feeds retraining.

SECTION 19 RECAP

OCR meets NLP

REMEMBER

Takeaways

- OCR errors propagate and compound
- Train NLP on real OCR text
- Serialise layout carefully
- Pass confidence downstream
- Start with an observable pipeline

CHECK YOURSELF

Why give a language model the OCR confidence of each word?

ANSWER

So it can treat uncertain words cautiously, prefer evidence from confident words, and flag answers built on doubtful text.

20

SECTION 20

Case studies

Five realistic projects that combine OCR and NLP, showing decisions, problems and lessons.

Invoices | Historic archives | Clinical referrals | Contracts | Care compliance

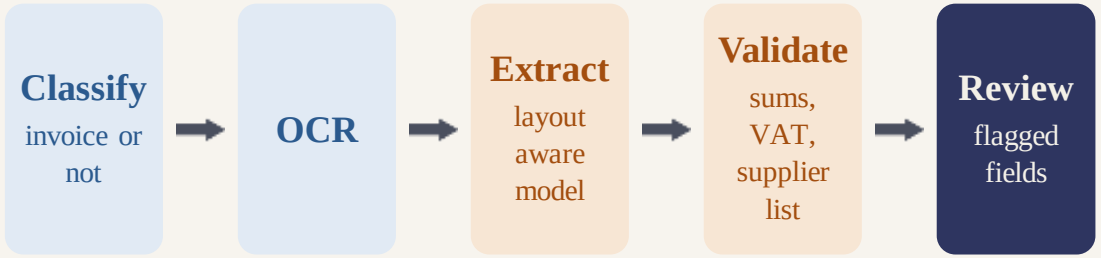
CASE 1 · INVOICE AUTOMATION

Accounts payable at a mid sized company

IN THE WILD

The brief

Thousands of supplier invoices each month arrive as PDFs and photos. Staff keyed every field by hand.



WARNING SIGNS

Early problem: totals misread on low quality photos; the sum check caught them, but review volume was high.

THE FIX

Capture guidance for suppliers' mobile uploads and an image quality gate reduced review volume substantially.

LESSON

The biggest gain came from the capture stage, not the model. Measure where review time is spent before investing in model upgrades.

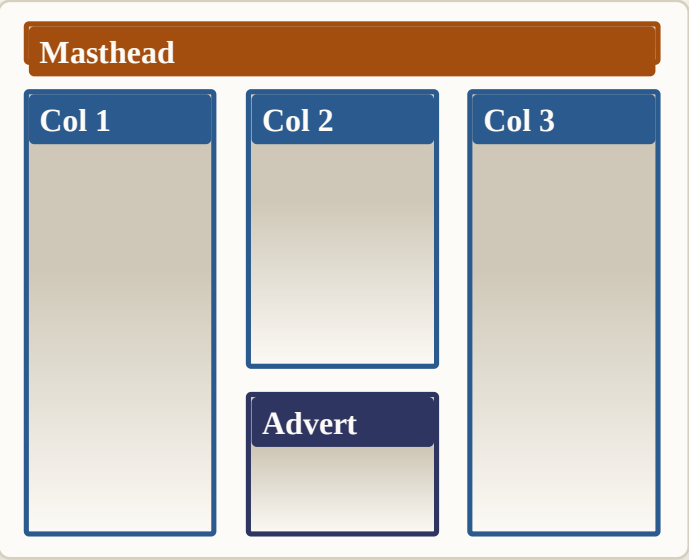
CASE 2 · HISTORIC NEWSPAPER ARCHIVE

Making a century of print searchable

IN THE WILD

The brief

A library digitised old newspapers: faded ink, dense columns, old fonts and spellings.



PITFALL

Column detection errors merged articles; search returned garbled snippets.

THE FIX

- Learned layout model for article separation
- Historical spelling variants in the search index
- Fuzzy search tolerant of OCR errors
- Volunteers correct text; corrections retrain the model

LESSON

For archive, perfect OCR is impossible. Design search to be forgiving of errors, and let users help improve the text.

CASE 3 · CLINICAL REFERRAL LETTERS

Triage of scanned referrals

IN THE WILD

The brief

Scanned and faxed referral letters must be sorted by urgency and specialty, with key details extracted for clinicians.

PITFALL

Handwritten notes in margins and low quality faxes produced low confidence text. The model must never silently downgrade urgency.

KEY IDEA

Pipeline: OCR, de identification for analytics, specialty classification, urgency flags, extraction of symptoms and medications with evidence.

THE FIX

Conservative thresholds: anything uncertain is treated as potentially urgent and reviewed by a clinician. Every decision shows the source text.

LESSON

In healthcare, design errors to fail safe. A false alarm costs minutes; a missed urgent referral can cost far more.

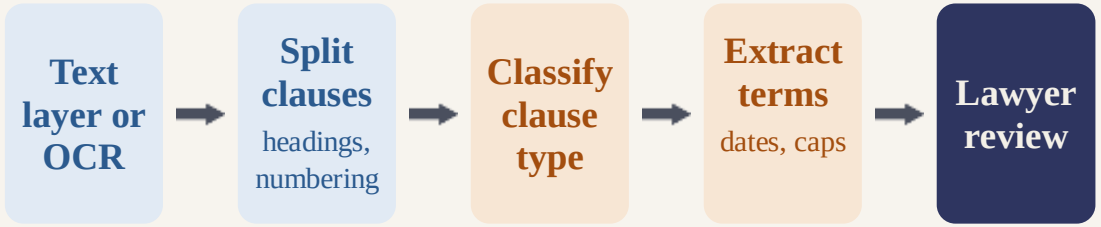
CASE 4 · CONTRACT REVIEW

Finding risky clauses across thousands of contracts

IN THE WILD

The brief

A legal team must find change of control, termination and liability cap clauses across a large set of scanned and digital contracts.



WARNING SIGNS

Numbered clauses broke when OCR misread numbering; cross references pointed to the wrong clause.

THE FIX

Layout cues (indentation and bold headings) used alongside numbering, plus LLM extraction restricted to quoting exact clause text.

LESSON

Require exact quotations for legal extraction. Paraphrased legal text is not evidence.

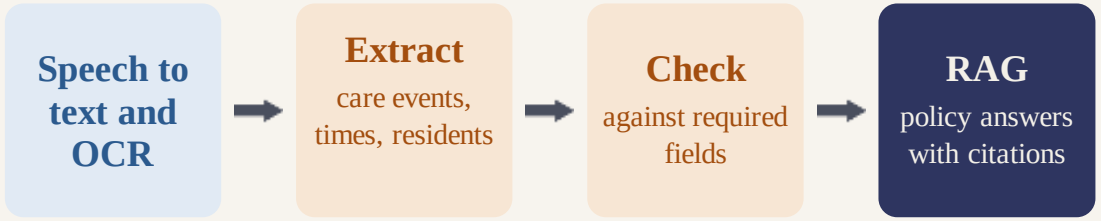
CASE 5 · CARE HOME COMPLIANCE

Voice notes, scanned forms and policy questions

IN THE WILD

The brief

A care home compliance product turns staff voice notes and scanned paper forms into structured care records, and answers policy questions from scanned policy manuals.



PITFALL

Resident names are personal data; handwritten forms vary widely; policy documents are updated often.

THE FIX

Strict access control and pseudonymisation, confidence based review of handwritten fields, and re indexing policies whenever a new version is uploaded.

LESSON

In regulated care settings, every automated record must show its source and be reviewable by staff before it becomes official.

ACROSS THE CASES

Patterns that repeat

Pattern	Seen in
Capture quality drives accuracy	Invoices, referrals, care forms
Layout errors masquerade as language errors	Archives, contracts
Business rules catch silent errors	Invoices, contracts
Fail safe thresholds for high stakes	Referrals, care
Evidence and citations build trust	Every case
Human corrections become training data	Every case

CASE EXERCISE

Design it yourself

ASK YOURSELF

Your brief

A logistics firm receives photographed delivery notes in five languages. They need the consignment number, delivery time, recipient signature present or not, and any damage notes.

PRO TIP

Think stage by stage: capture, preprocessing, layout, OCR, NLP, validation, review, monitoring.

- Which capture gates?
- Which OCR engine and language routing?
- How to detect the signature?
- How to extract damage notes in five languages?
- What validation rules for consignment numbers?
- What goes to human review?

CASE EXERCISE

One possible answer

ANSWER

Capture: blur and glare checks, page outline, retake prompts.

ANSWER

OCR: script detection then a multilingual engine; confidences kept.

ANSWER

Signature: an image region detector, not OCR.

ANSWER

Damage notes: multilingual classifier plus extraction, or translation then extraction.

ANSWER

Validation: consignment format and check digit; time within delivery window.

ANSWER

Review: low confidence numbers, missing signatures, any damage mention.

SECTION 20 RECAP

Case studies

REMEMBER

Takeaways

- Fix capture first
- Treat layout as a first class problem
- Use rules to catch silent errors
- Fail safe where stakes are high
- Keep evidence; learn from corrections

CHECK YOURSELF

In the referral case, why treat uncertain letters as urgent?

ANSWER

Because the cost of missing an urgent case is far higher than the cost of an extra review. Thresholds follow the cost of errors.

SECTION 21

21 Deployment, ethics and toolkit

Running OCR and NLP systems in production responsibly, plus checklists, a glossary and a final self test.

Monitoring · Privacy · Ethics · Checklists · Glossary · Final quiz

MONITORING

What to watch in production

Input quality

Blur scores, resolution, page counts, new document types.

OCR confidence

Shifts in average confidence per document type.

Rule failures

Rising sum check or format failures signal a problem.

Review rate

Share of fields sent to people, and how often they change the value.

NLP drift

New vocabulary, intents and entity types appearing.

Latency and cost

Time and spend per page, especially for LLM calls.

PRO TIP

Human corrections are your best live accuracy signal. Track the correction rate per field and per supplier or source.

PRIVACY

Documents are full of personal data

KEY IDEA

Scanned documents often contain names, addresses, health details, bank details and signatures. Under UK GDPR you need a lawful basis, minimisation and appropriate security.

PITFALL

Sending documents to third party APIs, or using them to train models, may need explicit assessment and agreements. Check with your data protection officer.

- Data protection impact assessment done
- Only necessary fields extracted and kept
- Encryption in transit and at rest
- Access limited by role
- Retention periods defined and enforced
- Redaction before analytics and training

ETHICS

Responsible OCR and NLP

WHY IT MATTERS

Fairness: handwriting styles, names and dialects of some groups may be read less accurately.

WHY IT MATTERS

Transparency: people affected by decisions should know automation was involved.

WHY IT MATTERS

Accountability: a named person owns each automated decision process.

WHY IT MATTERS

Human oversight: people can review and overturn outcomes.

WHY IT MATTERS

Security: document stores are attractive targets.

WHY IT MATTERS

Labour: design review work to be meaningful, not mind numbing.

REMEMBER

This is general guidance, not legal advice. Requirements differ by sector and jurisdiction; involve legal and compliance teams early.

CHECKLIST

Before you build an OCR system

- [ ] Collect a representative sample of real documents
- [ ] List document types, layouts and languages
- [ ] Define fields and the cost of each error
- [ ] Agree ground truth rules
- [ ] Benchmark two or three engines on your sample
- [ ] Design capture guidance and quality gates
- [ ] Plan preprocessing per document type
- [ ] Plan validation rules and review routing
- [ ] Assess privacy and security
- [ ] Define success metrics in business terms

CHECKLIST

Before you ship an NLP system

- [ ] Same preprocessing in training and serving
- [ ] Baseline such as TF IDF compared
- [ ] Metrics on a held out set from real traffic
- [ ] Entity level and field level scores
- [ ] Robustness tests on OCR style noise
- [ ] Fairness checks by group and language
- [ ] Prompt injection and hallucination tests
- [ ] Outputs validated and linked to evidence
- [ ] Monitoring and alerting in place
- [ ] Fallback and human review path ready

TOOLKIT

Libraries and tools worth knowing

Need	Examples
Image processing	OpenCV, Pillow, scikit image
OCR engines	Tesseract, PaddleOCR, EasyOCR, docTR
Document models	LayoutLM family, Donut, TrOCR
Classical NLP	spaCy, NLTK, scikit learn, Gensim
Transformers	Hugging Face transformers, sentence transformers
Search and RAG	Vector databases, hybrid search engines, rerankers
Evaluation	Custom CER and WER scripts, seqeval style entity scoring

PRO TIP

Names here are examples, not endorsements. Check current versions, licences and maintenance status before adopting any tool.

GLOSSARY

Key terms, part 1: OCR

Term	Plain meaning
Binarisation	Turning each pixel into ink or paper
Deskew	Rotating a page so text lines are level
Connected component	A blob of touching ink pixels
Text detection	Finding where text is on an image
CTC	A way to train line readers without letter positions
CER and WER	Share of characters or words read wrongly
Layout aware model	A model that uses words and their positions

GLOSSARY

Key terms, part 2: NLP

Term	Plain meaning
Token	A unit of text a model processes
Embedding	A vector of numbers representing meaning
NER	Finding and labelling named entities
Self attention	Words gathering information from each other
Fine tuning	Further training a pretrained model on your task
RAG	Answering using retrieved passages from your documents
Hallucination	Fluent output not supported by facts or sources

FINAL SELF TEST

Eight questions

ASK YOURSELF

1 · Capture

Why reject a blurred photo instead of trying to read it?

ASK YOURSELF

2 · Preprocessing

When is adaptive thresholding better than a global threshold?

ASK YOURSELF

3 · Recognition

What problem does CTC solve?

ASK YOURSELF

4 · Measurement

Why can a low CER still mean many wrong invoices?

ASK YOURSELF

5 · Tokenisation

Why do subword tokenisers help with rare and misspelt words?

ASK YOURSELF

6 · Transformers

What does self attention let each word do?

ASK YOURSELF

7 · LLMs

How do you stop an LLM inventing an invoice total?

ASK YOURSELF

8 · Systems

Why pass OCR confidence into the NLP stage?

FINAL SELF TEST

Answers

ANSWER

1 · Capture

A retake costs seconds; a wrong extraction costs far more later.

ANSWER

2 · Preprocessing

Under shadows and uneven lighting, where one threshold cannot fit the whole page.

ANSWER

3 · Recognition

It trains line readers without marking where each character sits.

ANSWER

4 · Measurement

Errors spread across many fields, so many documents contain at least one.

ANSWER

5 · Tokenisation

Unknown words split into familiar pieces instead of becoming one unknown token.

ANSWER

6 · Transformers

Gather information from every other word, weighted by relevance.

ANSWER

7 · LLMs

Require values to appear in the source text, validate with rules, and allow null.

ANSWER

8 · Systems

So uncertain spans are treated cautiously, checked or reviewed.

PULLING IT TOGETHER

Ten principles for OCR and NLP systems

1

Fix capture first

Quality in, quality out.

2

Measure each step

Keep only what helps.

3

Respect layout

Order and tables carry meaning.

4

Read lines, not letters

Context beats segmentation.

5

Validate with rules

Catch silent errors.

6

Carry confidence

Uncertainty is information.

7

Start simple

Baselines before giants.

8

Ground every answer

Evidence for every value.

9

Protect people

Privacy and fairness by design.

10

Learn from corrections

Every fix is a label.

FINAL THOUGHT

Machines that read are only useful when people can trust what they read.

Trust comes from clean capture, careful measurement, visible evidence and a human hand on the decisions that matter.